



Extended Coding Conventions for the Java™ Programming Language

**Coding and Formatting Guidelines, Recommendations and
Best Practices**

Thomas Thrien
(thomas.thrien@tquadrat.org)

2026-06-28

For all the absent family and friends:

Ursula und Rudolf Thrien

Rasa Kuodienė

Klaus Wichmann

Thomas Irländer

Sabine Bahr

Johann Raffetseder

Claire Lavelle

Kerstin Barion

Klaus-Dieter Busch

Heinrich Müller

Jürgen Schön

Heinrich Schwakopf



Contents

1. Introduction	15
1.1. The Goals	16
1.1.1. Conclusion	21
1.2. About this Document	21
1.2.1. The Document Structure	21
1.2.2. Sample Code	22
1.2.3. References	23
1.3. History and Implementation	23
1.4. Other Programming Languages	24
1.5. Code Generators	26
1.6. Coding for Sustaining	26
1.7. Tool Support	27
1.8. How to use this Document	27
1.9. How to implement these 'Coding Conventions' in your Project?	29
1.10. About the Author	31
1.11. The Basic Rule	31
 I. Formatting the Source Code	 35
2. Lines and Spaces	39
2.1. Indentation	39
2.1.1. Indentation Style	39
2.1.2. Indentation Size	40
2.1.3. Indentation Character	41
2.2. White Space	41
2.2.1. Blank Lines	41
2.2.2. Round Brackets	42
2.2.3. Blank Spaces	45



2.3. Line Length	47
2.3.1. Wrapping Lines	48
3. Java Source File Structure	55
3.1. File Encoding	55
3.2. Source File Size	56
3.3. The File Structure	57
3.3.1. Beginning Comment	57
3.3.2. Package Statement	60
3.3.3. Import Statements	60
3.3.4. Class and Interface Declarations	62
3.3.5. Closing Comment	71
4. Declarations	75
4.1. General Guidelines	75
4.1.1. Multiple Declarations per Line	75
4.1.2. Placement of Declarations	76
4.1.3. Array Declarations	79
4.2. Class Declarations	80
4.3. Method Declarations	83
5. Statements	87
5.1. Simple Statements	87
5.2. Compound Statements	87
5.2.1. 'if', 'if-else', 'if-else if-else' Statements	89
5.3. Method Chaining	91
5.4. 'return' and 'yield' Statements	93
5.4.1. 'if', 'if-else', 'if-else if-else' Statements	95
5.4.2. 'switch' Statements	95
5.4.3. 'for' Statements	100
5.4.4. 'while' Statements	102
5.4.5. 'do-while' Statements	103
5.4.6. 'try-catch' and 'try-catch-finally' Statements	103
5.4.7. 'synchronized' Statements	107
5.4.8. Labels and 'break' Statements	109
5.5. Special Files	111
5.5.1. The Module Definition	111



5.5.2. The Package Documentation	112
6. Naming Conventions	115
6.1. General Accords	116
6.1.1. Language	116
6.1.2. UPPERCASE, lowercase, CamelCase	117
6.1.3. Length of Names and Use of Abbreviations	118
6.1.4. 'Special' Characters	119
6.1.5. 'test' as Part of Names	119
6.1.6. Names forced by 3 rd Party Components	119
6.2. Files	120
6.3. Modules	120
6.4. Packages	121
6.4.1. The Packages 'internal' and 'spi'	122
6.5. Classes	123
6.5.1. Names for 'real' Classes	123
6.5.2. Names for Interfaces	124
6.5.3. Names for enum Classes	124
6.5.4. Names for Record Classes	125
6.5.5. Names for Annotations	125
6.6. Constructors	125
6.7. Methods	126
6.8. Local Variables	128
6.9. Parameters	133
6.9.1. Names for formal Parameters of Methods and Constructors	133
6.9.2. Names for Lambda Parameters	133
6.10. Fields	135
6.11. Constants	136
6.12. enum Values	138
6.13. Type Arguments	140
6.14. Projects	141
6.15. Library Files	141
7. Writing proper Comments	143
7.1. Documentation Comments	146
7.1.1. Structure and Contents	148



7.1.2. The JavaDoc Tags	161
7.2. Implementation Comment Formats	170
7.2.1. Structuring Comments	170
7.2.2. Block Comments	171
7.2.3. Single-Line Comments	173
7.2.4. Trailing or End-Of-Line Comments	175
7.3. Maintenance Comments	181
7.4. Special Comments	183
7.5. Commenting out Code	184
7.6. Comments when?	185
7.6.1. Empty Code Blocks are forbidden	187
7.6.2. Description of the used Algorithm	188
7.6.3. Broken Rules	188
7.6.4. Swallowed Exceptions	190
7.6.5. Suppressed Warnings	191
7.6.6. Comments vs. Annotations	191
7.7. Updating Comments	192
8. Coding Guidelines	193
8.1. Swap Logic Errors for Compiler Errors	197
8.2. Error Handling	200
8.2.1. The Basics	201
8.2.2. Throwing Exceptions	204
8.2.3. General Exception Handling	207
8.2.4. Exceptions in Threads	212
8.2.5. Reporting Errors	220
8.2.6. Logging	222
8.3. Checking Method Parameters and Return Values	250
8.3.1. How to implement the Validation	251
8.3.2. Delegating the Validation	260
8.3.3. Foreign Code	261
8.3.4. Assertions	263
8.4. Extending Classes, Overriding Methods	265
8.4.1. 'final' for Classes and Methods	266
8.4.2. Non-final Classes	266
8.4.3. Non-final Methods	267
8.4.4. Adapter Classes	277



8.4.5. Alternatives to Extending a Class	281
8.5. Method Signatures	285
8.5.1. Mutable Types for Arguments	287
8.5.2. Abstract vs. concrete Argument Types	289
8.5.3. boolean Arguments	293
8.5.4. Multiple Parameters with same Type	297
8.5.5. Generic Parameter Types	297
8.5.6. Arrays and varargs as Arguments	300
8.6. Types for Return Values	302
8.6.1. Abstract vs. concrete Return Types	303
8.6.2. Mutable Return Types	304
8.6.3. Special Return Values	308
8.7. Implementing Immutable Types	311
8.7.1. The final Keyword in Java	312
8.7.2. Property Types, Constructor Arguments and Return Values for Accessors	314
8.7.3. Mutators for Immutable Types	315
8.7.4. <i>Apparently</i> immutable Types	316
8.7.5. Changes through Reflection	318
8.8. Local Variable Type Inference	321
8.8.1. Naming	323
8.8.2. Scope	325
8.8.3. Initialisers	326
8.8.4. var and “Programming to the Interface”	327
8.8.5. var with Literals	328
8.8.6. Using var with the ‘Diamond Operator’ or Generic Methods	330
8.8.7. Examples	331
8.8.8. Conclusion	334
8.9. Access to Properties	335
8.10. Value Types	336
8.11. Accessing Fields or Methods using Reflection	343
8.11.1. Callback Methods	345
8.11.2. Factory Methods with Reflection	350
8.11.3. Using Reflection to write Unit Tests	352
8.11.4. Annotations and Reflection	355



8.12. Implementing the Methods from 'java.lang.Object'	357
8.12.1. <code>equals()</code> and <code>hashCode()</code>	357
8.12.2. <code>toString()</code>	360
8.12.3. <code>clone()</code>	363
8.13. String Concatenation	371
8.13.1. The Basics	371
8.13.2. Concatenating String Constants	373
8.13.3. Concatenating String Variables	374
8.13.4. Concatenating Strings in Loops	375
8.13.5. Composing Structured Strings	376
8.13.6. Conclusion	377
8.14. try-with-resources	378
8.14.1. Basics	379
8.14.2. Error Handling	381
8.14.3. Execution Sequence	382
8.14.4. When to use?	383
8.15. Terminating a Method and returning Values	388
8.15.1. Lambda Results	394
8.15.2. 'case' Results	396
8.16. Generics	398
8.17. ————— Proceed from here!	399
8.18. Internationalisation	399
8.19. ————— Proceed from here!	400
8.19.1. ————— Proceed from here!	400
8.20. The Annotation <code>@API</code>	400
8.21. ————— Proceed from here!	401
8.21.1. ————— Proceed from here!	401
8.22. ' <i>Convention over Configuration</i> '	401
8.23. ————— Proceed from here!	403
8.23.1. ————— Proceed from here!	403
8.24. Compiler Warnings and Errors	403
8.25. ————— Proceed from here!	404
8.26. The Ternary Operator '?'	406
8.27. ————— Proceed from here!	406
8.27.1. ————— Proceed from here!	406
8.28. Programming to the Interface	406



8.29. —————	Proceed from here!	407
8.29.1. —————	Proceed from here!	407
8.30. The "switch" Statement		407
8.30.1. Advantages of the new Syntax		409
8.30.2. 'switch' Expressions		411
8.31. —————	Proceed from here!	412
8.32. —————	Proceed from here!	413
8.32.1. More Enhancements		413
8.33. —————	Proceed from here!	414
8.33.1. 'switch' Statements		414
8.34. Encapsulation		420
8.35. —————	Proceed from here!	420
8.35.1. Encapsulation with Modules		420
8.35.2. —————	Proceed from here!	420
8.36. Lambdas		421
8.37. —————	Proceed from here!	421
8.37.1. Functional Interfaces		421
8.37.2. —————	Proceed from here!	421
8.38. The Interface <code>java.util.Formatter</code>		422
8.39. —————	Proceed from here!	422
8.40. The Interface <code>java.lang.Comparable</code>		422
8.41. —————	Proceed from here!	422
8.42. Utility Classes		423
8.43. —————	Proceed from here!	423
8.44. Date and Time Values		423
8.45. —————	Proceed from here!	423
8.46. Unique Ids		424
8.47. Utilising JMX		425
8.48. —————	Proceed from here!	425
8.49. Finalisation		426
8.50. —————	Proceed from here!	426
8.51. Deprecation of Elements		426
8.52. —————	Proceed from here!	426
8.53. Multi-Threading and Synchronisation		427
8.54. —————	Proceed from here!	427
8.55. —————	Proceed from here!	428
8.56. Inner Classes		428



8.57. Chaining Constructors	429
8.57.1. The “Primary Constructor” Pattern	429
8.57.2. The “Common Constructor” Pattern	431
8.57.3. Constructor Helpers	434
8.58. ————— Proceed from here!	436
8.59. Miscellaneous	436
8.60. ————— Proceed from here!	448
9. Appendices	449
9.1. The Naming Dictionary	449
9.1.1. Verbs	449
9.1.2. Suffixes for Class Names	462
9.2. Configurable Errors and Warnings	463
9.2.1. Eclipse	463
9.2.2. JetBrains IntelliJ IDEA	463
9.3. IDE Configuration	463
9.3.1. Eclipse	463
9.3.2. JetBrains IntelliJ IDEA	466
9.4. Embedded Code	467
9.4.1. Formatting SQL inside Java	467
9.4.2. Formatting XML inside Java	467
9.4.3. Formatting HTML inside Java	467
9.5. The Reason why Prefix Notation should be preferred over Post- fix Notation	467
9.6. Examples	468
9.6.1. AutoLock	468
9.6.2. Illegal Argument Exceptions	471
9.6.3. Lazy	477
9.6.4. Load JDK Logging Configuration	485
9.6.5. MountPoint	487
9.6.6. Patch Identification	488
9.6.7. PostProcessor	492
9.6.8. ThreadGroup	493
9.6.9. UnsupportedOperationException	497
Tables	500



Listings	501
References	504
Index	545



Contents



1. Introduction

To get your “Coding Guardrails for Java™ in a Nutshell”, you can take them from below:¹

1. Beautiful is better than ugly.
2. Explicit is better than implicit, and verbosity is your friend.
3. Do not repeat yourself.
4. Simple is better than complex.
5. Complex is better than complicated.
6. Flat is better than nested.
7. Sparse is better than dense.
8. Readability counts.
9. Special cases aren't special enough to break the rules ...
10. ... although practicality beats purity.
11. Errors should never pass silently and Exceptions may never be swallowed ...
12. ... unless *explicitly* silenced.
13. *Either* log *or* throw, neither both.
14. In the face of ambiguity, refuse the temptation to guess.
15. There should be one – and preferably only one – obvious way to do it ...
16. ... although that way may not be obvious at first.
17. Now is better than never ...
18. ... although never is often better than *right now*.
19. If the implementation is hard to explain, it's a bad idea.
20. If the implementation is easy to explain, it may be a good idea.
21. Think about your future self and add a comment.
22. Without the unit tests, the code is not complete ...
23. ... neither it will be complete without documentation.
24. Always do it right in the first place!

¹obviously stolen from the “Zen of Python”[372, 322] and slightly amended ...



25. Have Fun!

1.1. The Goals

The main goal of coding conventions in general is to help you writing better code.

So what is *good code*?

Good Code does not have bugs and is running fast, without wasting valuable resources.

“Does not have bugs” also means that the code is doing what it is intended to do. We all know that it is nearly impossible to produce completely bug free code once the program is not damn stupid simple, for several reasons. One important reason is that already the specification, used as the base for the implementation, is often faulty. So the code is doing what it should do *according to the specification*, but it is not doing what it is *really* meant to do ...

However, we should not forget that there is still the possibility for *real* bugs.

And often you see the situation that your code is getting slower and slower, or is eating up more CPU cycles and/or memory than available. That may be due to higher data volumes or more users or lots of other reasons.

But as a result of all these scenarios, you will have to modify your code – although, in most cases, it will be someone else who have to dig into *your* code. Or you are the unlucky one who needs to revise someone else’s code. And if the code is only old enough, it does not really matter if you wrote it or someone else – refer to the “Coding Guardrails for Java in a Nutshell”, bullet point 21.

So what you want is code that helps you with being modified – you want to have *maintainable code*.



Maintainable Code supports its modification in various ways.

From a magazine dealing with software development tools, I found the following statement²:

“If you compare software development with the Apollo program, most programmers would be very successful bringing man to the moon, but never will get them back alive because of their common incapability to deliver software that can be maintained over a longer period of time with reasonable costs.”

Unknown Source[1]

“Maintainable Code” neither means “Reusable Code” nor “Generic Code”. Of course, reusing code (isolate it as a method or function) instead of repeating it (copy and paste to another location) can support maintainability, so can it help to generalise an implementation. But first both alone is not sufficient for maintainable code, and second, especially generalisation can be contraproductive.

Larger applications are usually splitted into parts that are organised as libraries or modules. Following the principle of “Separation of concerns”[327] when designing these parts is helpful to get maintainable software. But some additional thought should be spent on how generic and reusable these libraries need to be. On the other hand, if the library code is really maintainable, it should not be a big deal to add additional features without breaking existing code.

Maintainability has a lot of facets, and I will discuss several of them in the run of this document. But before you can even think of modifying the code – either for fixing a bug, optimising the memory consumption, or adding new functionality – you need to identify the location.

For that you have to read the already existing code, and of course, you have to understand what you are reading. So a crucial quality of maintainable code is that it is easy to read and *basically* understandable³.

²I noted down the statement, but forgot to note the source – so if you know that source, I would appreciate if you would share it with me.

³Ok, the requirement should be ‘easy to understand’, but that is a relative term. If something is ‘easily’ understandable depends on the experience level of the reader, and here



No matter whether your code will meet all the other criteria that would make it maintainable: if it is not readable, it cannot really be maintainable.

Readable Code can be easily consumed by a human.

“[...], programs must be written for people to read, and only incidentally for machines to execute.”

Herold Abelson et. al.: *Structure and Interpretation of Computer Programs* [3]

“Everybody can write code that can be read by a computer, but only good developers will write code that can be read by humans.”⁴

Martin G. Fowler: *Refactoring: Improving the Design of Existing Code*[126]

“Reading code is more important than writing code. Code is read much more often than it is written. Further, when writing code, we usually have the whole context in our head, and take our time; when reading code, we are often context-switching, and may be in more of a hurry. Whether and how particular language features are used ought to be determined by their impact on future readers of the program, not its original author. Shorter programs can be preferable to longer ones, but shortening a program too much can omit information that’s useful for understanding the program. The central issue here is to find the right size for the program such that understandability is maximized.

We are specifically unconcerned here with the amount of keyboarding that’s necessary to input or to edit a program. While concision may be a nice bonus for the author, focusing on it misses the main goal, which is to improve the understandability of the resulting program.”

we talk not only about their coding experience, but also about their familiarity with the problem domain. And you cannot always code for the rookies.

⁴I agree in general, but also the converse is true: only developers that will write code that can be read by humans are good developers.



Stuart W. Marks: *Local Variable Type Inference – Style Guidelines*[246]

And from my own experience I would confirm that verbosity is your friend!⁵

An external documentation (if existing at all ...) can be helpful to understand the code, in particular regarding the aspects of the problem domain the code is dealing with. But the flow of reading is significantly disturbed when you need to consult that external documentation every second line, same as when you have to look into a dictionary all the time when reading a book in a foreign language. So that external documentation is no substitute for readable code.

Several guidelines and recommendations in this document, in particular regarding how to apply comments and how to write comments, but also those about to write a method, do require significant additional typing. Java™ in general has the reputation to be too verbose, and this coding conventions will even add to that verbosity – but as said: that’s not generally bad!

But if you are afraid of the keyboarding, you should learn typewriting! For my understanding, someone who does not reach at least 20 WPM⁶ should look for a job outside of software development!⁷

Writing code is not the only area where you would benefit from mastering that important skill; it will also help you to write all the other stuff you have to deliver in addition to code (documentation, meeting notes, emails, specification documents, ...).

From a site that offers typing courses[24]:

⁵Refer to the “Coding Guardrails for Java in a Nutshell”, bullet point 2.

⁶WPM = “Words per Minute”

⁷Just for your comparison: average values for a computer typist are around 41 WPM, the world record is 216 WPM, set by Stella Pajunas in 1946, using an IBM electric typewriter. See also [24].

According to Google, for the English language WPM roughly translates to CPM (= “Characters per Minute” or “Anschläge pro Minute” in German) by multiplying it by 5. The average secretary scores at least 200 CPM, peak values are in the 500 to 900 CPM, with a world record an 955 CPM.



“WHEN TO START LEARNING

Developed typing skills can help young people get better study results in school or college, and get better job offers once they have finished school. 65% of people who learn how to touch type faster are under the age of 24. The main goal of learning to touch type or improving touch typing skills, after age 25 is to become more successful in current job.

MOTIVATION

Improving touch typing skills is a must for every person under 18, but everyone can benefit from some time spent practicing. Average typing speed is one of the key skills listed on your resume. Spend some more time practicing and get better results [...].”

That they do marketing to sell their product does not make their statements wrong.

Also, modern IDEs do a lot of the typing for you, as long as you take the trouble to configure them appropriately. Refer to chapter “1.7 Tool Support”.

“Code readability shouldn’t depend on IDEs. Code is often written and read within an IDE, so it’s tempting to rely heavily on code analysis features of IDEs. [Why not relying on that?]

There are two reasons. The first is that code is often read outside an IDE. Code appears in many places where IDE facilities aren’t available, such as snippets within a document, browsing a repository on the internet, or in a patch file. It is counterproductive to have to import code into an IDE simply to understand what the code does.

The second reason is that even when one is reading code within an IDE, explicit actions are often necessary to query the IDE for further information [...] This might take only a moment, but it disrupts the flow of reading.

Code should be self-revealing. It should be understandable on its face, without the need for assistance from tools.”



Stuart W. Marks: *Local Variable Type Inference – Style Guidelines*[247]

1.1.1. Conclusion

To summarize from above, the goal for these coding conventions is to help you to write ...

- ... good, ...
- ... maintainable, ...
- ... readable ...

... Java™ source code.

As a side effect, all source code looks familiar to all members of the team. That conformity further supports readability, but such a common look also identifies the source code as written by the team, as its trademark.

1.2. About this Document

This document was compiled using $\text{T}_\text{E}\text{X}/\text{L}_\text{A}\text{T}_\text{E}\text{X}$ ⁸, and its source can be found on GitHub[368]. The diagrams were generated with Graphviz[167] and GreenLightning’s ‘Nassi-Shneiderman Diagram Generator’[315, 314, 313].

1.2.1. The Document Structure

This document itself consists of four major parts, aside of this “Introduction” and the “Appendices” (containing additional information, references and examples).

⁸In case it is relevant for you, I used ‘TexMaker’ as the editor for the $\text{L}_\text{A}\text{T}_\text{E}\text{X}$ sources. I run it on Mac and Linux, but the software is also available for Windows; it can be downloaded from this location: <https://www.xmlmath.net/texmaker/download.html>.



“Formatting the Source Code” deals with proper formatting of the Java source files and additional files that are part of your project.

“Naming Conventions” covers the naming of the program elements (modules, packages, classes, methods, fields, constants, local variables, formal parameters).

“Writing proper Comments” discusses some guidelines for writing proper comments, also for the generation of the documentation of the code.

“Coding Guidelines” is the biggest part and provides guidelines on how to use various language features, based on best practices. The chapters here are in no particular order.

1.2.2. Sample Code

The code examples in this document underline some particular aspect or demonstrate a single guideline or recommendation; to stay focused on that purpose, and to keep the samples at a reasonable length, they may often hurt other guidelines or ignore other recommendations. For instance, in most cases, methods are lacking the required comments, the names are not really meaningful, or otherwise mandatory error handling is just missing. Often the code as given will not compile, and it may also be not particularly useful for other purposes than demonstrating a feature.

At the opposite, the source code examples in the chapter “9.6 Examples” on page 468 obey all rules, follow all guidelines and implement all recommendations.

All the rules, guidelines and recommendations in this document assume that you use at least Java 25 – see the language specification in [133] – to write and to compile your code. Of course this version was also used for the examples.



1.2.3. References

Instead of adding URLs directly into the document, I provided them in the “Bibliography” section. In the text, you will find references to it like this: [133]. If you read the electronic version of the document, the reference is clickable and brings you to the respective entry. There you can click on the URL.

This allows me to update the URLs on one single location if they will change.

1.3. History and Implementation

Around the year 2000 I was asked to compile a set of coding guardrails for a team of developers that had been new to Java (and most were new to programming, too). That was the very first version of this document, based on and extending the “Code Conventions for the Java™ Programming Language”[108], originally released by SUN and therefore often referred to as the “SUN Java Coding Conventions”. That first version reflected the way I wrote Java™ code at that time as well as several best practices from various sources.

Later versions of this document were created for other developer teams, and several software projects had been delivered successfully working with these coding conventions.

Some of the code from these projects is still alive (in 2026), proving the initial statement that coding conventions do help to create maintainable software.

I wrote a lot of code using the rules, guidelines and recommendations from this document (and its predecessors, of course), including some libraries that I published on GitHub. Some of the code from these libraries is mentioned in chapter “8 Coding Guidelines”, starting on page 193. I also added some stuff as examples in chapter “9.6 Examples” in the “Appendices” part of the document. If you are interested in the whole package, you should have a look to [367].



1.4. Other Programming Languages

Of course coding conventions like those described by this document are not only useful for Java™ programs, but for code in any other programming language also.⁹ But each language has its unique features and specialities, meaning that coding conventions written for one programming language do not necessarily match the requirements for another.

Obviously there are some common rules that are valid for every programming language (e.g. the request to use meaningful names), but usually the differences will outweigh the similarities. So it is not a good idea to do something in language A just and only it is done that way in programming language B. On the other side, it should always be proved whether something that worked fine for one programming language would not be a great idea to be applied to code written in another language, too.

In this sense, I stole the “Zen of Python”[372, 322]¹⁰ and used it for this document. Below the original as published by Tim Peters on the Python mailing list in June 1999:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren’t special enough to break the rules ...
- ... although practicality beats purity.
- Errors should never pass silently ...
- ... unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one – and preferably only one – obvious way to do it ...
- ... although that way may not be obvious at first unless you’re Dutch.

⁹Even for descriptive languages like XML and HTML coding conventions make sense, also for documents written in T_EX/L^AT_EX.

¹⁰I added the ellipsis.



- Now is better than never ...
- ... although never is often better than *right now*.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea – let's do more of those!

Only the last one (that one regarding “Namespaces”) does not work for code written in JavaTM, unless you translate “Namespace” into “Package” (or “Module” ...). All other fits for the JavaTM Programming Language, too – and also for most other programming languages.

A good example for the other way round – other programming languages adopting a feature from JavaTM – is the idea to add documentation comments to the source code and to use a tool like Javadoc to externalise and to publish them as the API documentation. It was adopted for various other languages, including so different specimen as C/C++, JavaScript, go and PL/SQL.¹¹ Obviously, if you want to benefit from that adopted feature, you have to write your C++ (JavaScript, PL/SQL, ...) code (more precise, your documentation comments) according to coding conventions similar to those defined here.

A sample for code conventions for JavaScript can be found in the web at [117].

Coding conventions for C# are part of Microsoft's documentation for C#[30], another source can be found on GitHub[5].

The authors of the go programming language provided a code convention document on their website[122]; although meanwhile outdated, it is still sufficient for the core language. A more recent document, maintained by Google, can be found on GitHub[129]

The number of documents dealing with coding standards, naming conventions and style guides for C++ is close to infinite. A good starting point could be the ISO page[31], another one would be Google's C++ Style Guide[131], again available on GitHub.

¹¹Although I have to confess that the commenting style does not get part of the specification of these languages, instead it is only supported by external tools; one of these tools is doxygen[170].



1.5. Code Generators

Java™ code that is automatically generated by another piece of software should implement this coding conventions in the same way as “hand crafted” code. Most generators are highly configurable and/or allow to use code templates that are customisable.¹²

Especially if base classes will be generated it is important that proper comments are generated at least for all API elements; this covers all **public** and **protected** elements, and in some cases all package-local elements, too.

Java 6 introduced an annotation **javax.annotation.Generated** (**@Generated**) that should be used to mark generated code. Refer to [12] for the details.¹³

1.6. Coding for Sustaining

And what to do with “legacy” code that has to be fixed, amended or extended?

Most probably that code will not follow *these* coding conventions (if any at all ...), and it is rarely a good idea to reformat or completely rewrite the existing source code just to align it with the current recommendations and guidelines. Instead a fix should be limited to that area in the source code that is broken.

But if possible, your fix should incorporate the standards defined by this document. Be creative how to implement these standards. So if a method has to be reimplemented completely, format it as described here and add the appropriate Javadoc comments¹⁴, if missing. Or if a new field has to be added to a class, add the “m_” prefix to its name (see chapter “6.10 Fields”).

¹²Perhaps you want to have a look to [357].

¹³In case you still have to use Java 5 as the source code version, you should consider to create your own **@Generated** annotation along the lines given by that existing implementation in Java 6 and later.

¹⁴Even when there are no other Javadoc comments in the whole file ...



Obviously, completely new interfaces and classes should be implemented fully compliant to this conventions.

In this context, see also chapter “7.3 Maintenance Comments” on page 181.

1.7. Tool Support

Applying a number of the rules, guidelines, and recommendations in this document can be significantly facilitated by appropriately configuring the programming environment being used, especially those from chapter “1 Formatting the Source Code” that deals with the formatting of the source code.

The chapter “9.3 IDE Configuration” provides some configuration samples for Eclipse and JetBrains IntelliJ IDEA.

I strongly discourage the use of tools that reformat or even reorganise the code to match with the coding guidelines. In order to be useful for the developer’s daily work, they must have internalised this document. Using a tool to adjust their work to the agreed standard does the opposite from what should be achieved with the introduction of the coding guidelines.

To use a linting tool that verifies whether no rules were hurt is a good idea, on the other hand.

Same to feed this document into your preferred AI companion, just to tell it how generated code has to look like.

1.8. How to use this Document

First, you can use this document as is! It should work for you, as its predecessors already worked for a bunch of project teams before.

But you can also shape it closer to your particular needs, if you want! Feel free to strip it from parts you do not like, add stuff you think its missing, or change areas you don’t like! If you fix errors, I would appreciate some feed back ...



As said, you find the sources for this document on GitHub. Clone the repository on GitHub [368], replace the examples and the references to my libraries by some that fits better to your project and/or your company, then adjust the other stuff accordingly.

In particular, for your project, you should elaborate on the following topics:

- Chapters “6.14 Projects” on page 141, “6.3 Modules” on page 120, “6.4 Packages” on page 121 and “6.15 Library Files” on page 141
In no particular order:
 - Specify the name of your project.
 - Specify a format for the names of the library file(s) produced by your project.
 - Specify the names for the modules and packages of your project.
- Chapters “8.2.6 Logging” on page 222 and “8.2.6 Logger Configuration” on page 241:
 - Specify the type of the logging framework you use in your product – if any – or if you want to code against SLF4J (or if you do not want to log at all).
 - Specify the names of the loggers.
 - Specify the format for the log messages, in particular for the log levels DEBUG and TRACE (or FINE, FINER and FINEST, if you are using JDK Logging).
 - Provide a (default) logging configuration.
 - Optionally, you may remove all references to the logging frameworks and APIs that you do *not* use in your project.
- Chapter “9.1 The Naming Dictionary” on page 449:
 - Add verbs that are specific to your project, remove obsolete/unwanted ones.
 - Adjust the description for verbs where appropriate.



- Do the same for the class name suffixes.
- Add a Glossary that explains special terms used in the context of your project (the “Domain Specific Terminology”).
- Chapter “9.3 IDE Configuration” on page 463:
 - Remove the configuration setting samples for the IDEs that you are not using.
 - Adjust the configuration setting sample according to the changes you made elsewhere.
 - Add matching configuration setting samples for additional IDEs used in your environment.
- Provide code snippets for recurring patterns that are specific to your project.
- Remove this chapter “1.8 How to use this Document” and the chapter “1.9 How to implement these ‘Coding Conventions’ in your Project?”.

The repository contains also a build script `build.sh` that generates the final PDF.

1.9. How to implement these ‘Coding Conventions’ in your Project?

First any coding conventions are better than none! So if your team is already adhering to some coding conventions, the first hurdle had been taken already.

Adjust this document that it will incorporate the already accepted guidelines and recommendations; this may require that you remove conflicting recommendations and guidelines – don’t care, just do it!

The biggest blocker in this case could be that too much code needs to be modified to match with the coding conventions, meaning tedious work no



one likes to do. In this regard, you should also have a look to the chapter “1.6 Coding for Sustaining” on page 26.

But the most simple case is a new team starting over with a new project from scratch. Just present them the document as is ...

Experienced programmers know about the benefits of having detailed guardrails they can follow – in particular when they know that the other team members will also follow the same guardrails.

Nevertheless, you may have one or the other hacker in your team that complains that this kind of guardrails are suffocating their creativity.

Perhaps telling them this helps: I was always interested in fotography, and therefore I attended some classes about the topic. There the question was raised what are bad pictures, good pictures and exceptionally artistic pictures, and the professor explained it like this: *Fotography has a whole bunch of rules on how to compose a good picture, and you do not even need to know one of these rules to distinguish a bad shot from a good fotography: the bad ones will break these rules.*

But the really artistic pictures always break *one* or *the other* rule, but *with intention*. The fotographer’s creativity is to choose the rule to break and how – but this first means that they have to know each of the rules and how to follow them (usually referred to as ‘knowing the craft’), and second, that there is a visible intention.

But in most cases the client just wants to get only *good* pictures, they pay for the craft, not for the art.

Same in software development: only few clients will pay for a piece of art; instead they expect good craftsmanship.

If this explanation does not help, and you are in the proper position: replace the hacker!



1.10. About the Author

My name is Thomas Thrien, I was born and raised in Germany, and I still live there, so my primary language is German, not English. You may notice that one or the other time when you read this text: although I have to speak English most of time on the job, I too often fall back to Denglish¹⁵ ...

I am in the IT business since 1982 now, mainly in roles dealing with support, field service, and sustaining, but over the years I also did some development work, and even some IT consulting. I did the ground work with my fingers on the keyboard as well as leading teams of various size, even running whole departments.

In support and sustaining, you are mainly on the “receiving end” of the development process, suffering from the output of development teams that had never heard about the basics, or don’t care about them (or both). This experience was my main motivation to keep it up to date after I initially compiled it for a client in 2000.

If you have questions or comments regarding this document, please send me an email to thomas.thrien@tquadrat.org. Or leave it at <https://tquadrat.github.io/>. Of course I am also interested in your suggestions for improvements or in the bug and mistakes you found.

1.11. The Basic Rule

In engineering, it is common lore that nothing lives as long as a temporary solution. This is true for software development, too. And the IT business has also a specific variant of this: “Nothing lives as long as a PoC!”

This leads us to the one rule that should be followed before any other rule or recommendation given in this document, that one listed as bullet point 24 in the nutshell version of the “Coding Guardrails for Java in a Nutshell”:¹⁶

¹⁵Same as Spanglish, just the German, not the Spanish variant ...

¹⁶And it does not contradict with bullet points 17 and 18!



Always do it right in the first place!

Experiences with lots of programming projects have shown that programmers seldom touch their code again, once it is written.¹⁷ As a result, missing comments will never be added, shady programming patterns would not be fixed – with the consequence that in case of a bug the maintainer is lost in a poorly documented chaos of badly formatted source code.

And this is exactly what we want to avoid!

This means there is never ever any excuse for omitting comments, using ‘temporary’ names or doing other ‘funny’ things. It is not an excuse that it is only a temporary solution, an emergency fix or last minute Proof of Concept.

We all know about all the quick and dirty PoCs that made it into a product without significant changes to the code ... – it means that you must treat a PoC in the same way as you would write product code.¹⁸

Of course you can interpret this also as “Don’t make mistakes”, and you are not far away from the truth. But we all know that humans make mistakes, and that it is unavoidable that our code will contain one or the other bug. So it means that you should *do your very best* to *avoid* mistakes, and the rules, guidelines and recommendation from this document will help you with that – if you follow them!

So again:

Always do it right in the first place!

But the most important rule from the “Coding Guardrails for Java in a Nutshell” is that with the bullet point number 25:

¹⁷Usually, they will return to the code only when it is broken – and then they want to fix the bug as fast as possible, not adding missing comments or reformatting the code ...

¹⁸I was inclined to add “There is nothing like a *disposable prototype*.” to the “Coding Guardrails for Java in a Nutshell” ...



Have fun!



1. Introduction



Part I.

Formatting the Source Code



As I said already in the introduction, coding conventions are primarily guidelines that should help making code better to maintain. Usually, this starts with making the code better to read.

In this regard, a source file is like any other text document: it benefits from having a clear structure. And that structure is supported by the formatting of the text.

An article may have sections with headlines in bold face, the paragraphs are separated by blank lines and the first lines of them are indented by 0.3 cm, and there are rules regarding line length, hyphenation, and how to treat widows and orphans.

This part of the document deals with the proper formatting of Java™ source code files and their structure with the goal to create human readable *documents*.

“[...], we want to establish the idea that a computer language is not just a way of getting a computer to perform operations but rather that it is a novel formal medium for expressing ideas about methodology.”

Herold Abelson et. al.: *Structure and Interpretation of Computer Programs* [3]

Principles

- P1. The structure of the program text (the source code) must support the flow of reading.

As said in the “Coding Guardrails for Java in a Nutshell”, bullet point 8: “Readability counts”.

- P2. The formatting of the program text has to be consistent.

Permanent changes in the way things are presented will confuse the reader.



P3. Navigation through the source file should be easy.

When loaded inside an IDE, navigating inside a Java source file is easy: click on the name of the respective program element and the cursor is moved to the corresponding line in the file.

But when using a plain text editor, the search function is often the only tool you have. A consistent structure with headlines and a defined order would help here, too.

P4. It does not harm when the source code also looks beautiful.

Bullet point 1 from the “Coding Guardrails for Java in a Nutshell” is also about the visual appearance of the source code. We can discuss how useful ASCII art in a source file is, but not that it can contribute to the beauty of the program text.



2. Lines and Spaces

2.1. Indentation

Indentation, together with empty lines, is the most important tool to structure any source code. For the Python programming language, the indentation is even part of the syntax[325].

2.1.1. Indentation Style

The indentation of code blocks, based on their context, increases the readability of the code.

Obviously, “readability” is a relative term, depending from one’s habits. So some people are more familiar with the Kernighan-Ritchie (K&R) indentation style

```
public final void myMethod (){
    if (flag) {
        ...
    }
} // myMethod()
```

as it is also shown in the “Code Conventions for the Java™ Programming Language”[108], other prefer to have the curly braces on a line of their own (sometimes referred to as GNU or BSD style¹):

¹All, BSD, GNU and K&R styles, will define more than just how to place the curly braces. In addition, all three are originally part of code conventions for programming in C and/or C++ that cannot be applied to Java without modifications. See chapter “1.4 Other Programming Languages” on page 24 on this topic.



```
public final void myMethod()  
{  
    if( flag )  
    {  
        ...  
    }  
} // myMethod()
```

Both styles will work fine, but they look different. So as a lot of other guidelines from a code conventions document, too, this is a matter of taste. But as conformity supports readability, a main purpose of these guidelines is to ensure that all source code looks similar, and we have to determine the style to use.

I prefer the so-called GNU or BSD style and recommend it to you as well, because it makes it easier to detect the begin and the end of a code block than the Kernighan-Ritchie style. It demands to have the opening and closing curly braces on a line of their own, like below. This let the corresponding opening and closing braces appear on the same column:

```
public final class MyClass  
{  
    public final void myMethod()  
    {  
        if( flag )  
        {  
            /* do something */  
        }  
  
        for( final var i = 0; i < max; ++i )  
        {  
            /* do something else */  
        }  
    } // myMethod()  
} // class MyClass
```

2.1.2. Indentation Size

Again, at the very end, the size for the indentation is a matter of taste. But using four (4) spaces has been established as a de facto standard for the



base unit of indentation in Java™ source code, also because the original “Code Conventions for the Java™ Programming Language”[111] by SUN stated it that way. Therefore I recommend that you also use four spaces (‘’, ASCII 32 or 0x20) as the base unit of indentation.

When you write code *snippets* for a publication (for an article, as part of a program documentation, or for an answer on StackOverflow[331] ...), sometimes using a base indentation unit of two spaces would be better, as it spares some room. But reformatting existing code for this use case is in most cases something that a text editor can do for you.

2.1.3. Indentation Character

As said, you should use four *blanks*; do *not* use tabs (‘`\t`’, ASCII 9 or 0x09)! Tabs are *not* allowed! This is different from the “Code Conventions for the Java™ Programming Language”[111] that do not explicitly specify whether to use blanks or tabs.

Usually both, the indentation character and the indentation base unit can be configured in your IDE; in Eclipse this is part of the *Formatter* definition (Window|Preferences|Java|Code Style|Formatter).

To configure this in IntelliJ IDEA, it is part of the Code Style configuration for Java (File|Settings|Editor|Code Style|Java|Tabs and Indents).

2.2. White Space

This chapter covers how to use white space other than the indentation blanks for the formatting of your Java™ source code.

2.2.1. Blank Lines

Blank lines improve readability by setting off sections of code that are logically related. These are the principles on how to use blank lines.



Principles

- P1. Each single line comment or block comment should be preceded by a blank line, unless it is not preceded by another comment
- P2. Use a blank line between the various parts of a class declaration
- P3. A blank line separates multiple class and interface definitions
- P4. Methods are also separated by single blank lines
- P5. Logical sections inside a method can be separated through a blank line to improve readability
- P6. Blank lines can be used to group code lines logically
- P7. More than one blank line should be avoided
- P8. If it seems feasible for a particular purpose, feel free to use ASCII art to express the separation of parts in the source file

This is again different from what the “Code Conventions for the Java™ Programming Language”[113] stated regarding blank lines.

2.2.2. Round Brackets

Round Brackets (or Parentheses) are of course no whitespace characters – in the opposite, they are important syntax elements. But they trigger the use of whitespace, in particular blank spaces, and therefore, they are mentioned in this chapter.

We distinguish between *arithmetical* round brackets and *parameter* round brackets; the first is used in (mathematical or logical) expressions to order the terms, the latter is used with method or constructor calls or in lambdas.

The “Code Conventions for the Java™ Programming Language”[114] does not make this distinction, and it handles them differently than described here (basically in the same way as described here as arithmetical round brackets).



Arithmetical Round Brackets An opening arithmetical round bracket is always *preceded* by a blank or another opening round bracket and *never* followed by a blank. A closing arithmetical round bracket is *never preceded* by a blank, but *always followed* either by another closing round bracket, a blank, or a newline (or a semicolon, in case the statement ends).

This is a sample for arithmetical round brackets: `((5 + 7)* 9 + 3)/ 4.`

Parameter Round Brackets An opening parameter round bracket is *never preceded* by a blank and *always followed* by one, while a closing parameter parenthesis is *always preceded* by a blank. The only exception is for empty arguments lists where the opening parenthesis is immediately followed by the closing parenthesis.

Of course this is also valid for the declaration of methods and constructors.

Regarding to this rule, `if`, `for`, `while`, `try` and `switch` are treated like they are functions.²

Applying these rules can improve the readability of complex terms drastically, compared with the format suggested by the “Code Conventions for the Java™ Programming Language”[108], as the examples below will illustrate.

²And also the keywords `catch` and `synchronized`, but they do not allow complex terms inside their ‘argument list’.



2. Lines and Spaces

<code>final double d = sin((a + b) / c);</code>	<code>final double d = sin ((a + b) / c);</code>
<code>System.out.println("Finished!");</code>	<code>System.out.println ("Finished!");</code>
<code>set(get());</code>	<code>set (get ());</code>
<code>System.out.println(sin(((a + b) * (c + 4) + c)) / d));</code>	<code>System.out.println (sin (((a + b) * (c + 4) + c)) / d));</code>
<code>if(a < b) c = (d + e) / sin(f);</code>	<code>if (a < b) c = (d + e) / sin (f);</code>
<code>while(((line = reader.readLine()) != null) && (++i < capacity)) processLine(line);</code>	<code>while (((line = reader.readLine()) != null)~&& (++i < capacity)) processLine (line);</code>

A special case are round brackets used with a cast:

```
final Object o = "Text";
final var s = (String) o;
```

Here an opening round bracker is never followed by a blank, and a closing one is never preceded by a blank.

Lambda Expressions

According to the rules above, a lambda expression with more than one argument should be written like this:

```
final BiFunction f1 = ( a, b ) -> (a + b) * PI;
final BiFunction f2 = ( a, _ ) -> a * PI;
final BiFunction f3 = ( _, _ ) -> PI;
```

But it was found that it is easier to read without the blanks in the parameter list, at least for single-letter parameter names or the unnamed variable (the underscore that is provided instead of the parameter):

```
final BiFunction f1 = (a,b) -> (a + b) * PI;
final BiFunction f2 = (a,_) -> a * PI;
final BiFunction f3 = (_,_) -> PI;
```



So I suggest this style for lambda expressions.

2.2.3. Blank Spaces

The principles below are in place regarding the use of blank spaces, above and beyond to what was said already.

Principles

- P1. For a keyword like `if`, `while`, `for` etc., followed by a opening round bracket, there is no blank space between the keyword and the round bracket, but between that bracket and the argument, and another is between the argument and the closing round bracket.

As a short it can be said that the round brackets for these keywords will be treated like parameter round brackets, as already stated in “2.2.2 Parameter Round Brackets”.³

Examples:

```
while( true ) { ... }

for( int i = 0; i < 10; ++i ) { ... }

try
{
    ...
}
catch( Exception e )
{
    e.printStackTrace();
}
```

³This is in opposite to the “Code Conventions for the Java™ Programming Language”[108]; there it is recommended to have a blank between the keyword and the opening round bracket while not using it between the method name and the opening round bracket here, just to distinguish between keywords and methods calls. I think that it is more important to distinguish between round brackets in arithmetical expressions and those in method calls.



2. Lines and Spaces

```
if( list.isEmpty() ) { ... }
```

- P2. A blank space has to appear after commas in argument lists. The exception are generics where the comma is not followed by a blank space in case there is more than one type.

Examples:

```
Map<String,String> map1 = new HashMap<String,String>();  
Map<String,String> map2 = new HashMap<>(); // using the diamond  
operator  
map.put( key, value );
```

- P3. All binary operators except “.” has to be separated from their operands by spaces.

Example:

```
a += c + d;  
a = (a + b) / (c * d);  
printSize( "size_is" + foo + "\n" );
```

- P4. Blank spaces should never separate unary operators such as unary minus, increment (“++”), and decrement (“--”) from their operands.

Example:

```
while( d-- > s++ )  
{  
    ++n;  
}
```

- P5. The expressions in a **for** statement has to be separated by blank spaces after the semicolon.

Example:



```
for( expr1; expr2; expr3 )
{
    ...
}
```

For an extended for loop, blank spaces has to be placed around the colon, too:

```
for( String s : stringArray ) System.out.println( s );
```

- P6. For ternary statements, the question mark (“?”) and the colon (“:”) has to be surrounded by blank spaces:

Example:

```
int signum = d < 0 ? -1 : d > 0 ? 1 : 0;
```

In this sample, some round bracket would be helpful to increase readability.

- P7. Casts should be followed by a blank space.

Examples:

```
myMethod( (byte) aNum, (Object) x);
myMethod( (int) (cp + 5), ((int) (i + 3)) + 1 );
```

2.3. Line Length

Several sources (including the original “Code Conventions for the Java™ Programming Language”[108]) recommend to avoid code lines longer than 80 characters, because they were not handled well by many terminals and tools.



This had been true at the time those documents were written⁴, but today's graphical screens are much wider (sometimes even 200 and more characters can be put into one line), and modern tools do not even care about line length, so we do not want to recommend a fixed line length for source code.

Note: Sample code for the use in documentations, articles or alike *should* have a limited line length – generally not more than 70 characters.

Different to lines with program code, *comment lines* are limited to 80 characters – or more precise, they have to end at column 80. All lines with “ASCII graphics” are extended to that column. This is pertaining to the beginning comments (chapter “3.3.1 Beginning Comment” on page 57), the structuring comments (chapter “7.2.1 Structuring Comments” on page 170) and most single-line comments (chapter “7.2.3 Single-Line Comments”).

2.3.1. Wrapping Lines

As we have an unlimited line length, it will not happen that an expression does not fit into a single line. Nevertheless you might consider to break long lines to improve the readability of the code. In this case, you should follow these general principles:

Principles

- P1. Only one statement per line!
- P2. Break after a comma.
- P3. Break before an operator.
- P4. Break before a dot.
- P5. Break after an opening parenthesis and before a closing one.
- P6. Prefer higher-level breaks to lower-level breaks.

⁴The original “Code Conventions for the Java™ Programming Language”[108]were published by SUN in 1999 ...



- P7. Align the new line with the beginning of the expression at the same level on the previous line.
- P8. If the above rules lead to confusing code or to code that's squished up against the right margin, consider to break up the code into multiple parts, by introducing additional temporary variables.

Line Wrapping for Method Calls

Here are some examples on how to break long method calls:

```
1 // USUAL
2 final var result = myMethod( longExpression1, longExpression2,
3     longExpression3, longExpression4, longExpression5 );
4
5 // SINGLE LINE FOR EACH ARGUMENT
6 myProcedure
7 (
8     longExpression1,
9     longExpression2,
10    longExpression3,
11    longExpression4, // some comment
12    longExpression5
13 );
14
15 final var result = myFunction
16 (
17     longExpression1,
18     longExpression2,
19     longExpression3,
20     longExpression4, // some comment
21     longExpression5
22 );
23
24 // OTHER VARIANTS
25 final var result = myMethod( longExpression1,
26     someMethod( longExpression2,
27         longExpression3 ) );
28
29 final var result = myMethod1( longExpression1,
30     someMethod( longExpression2,
31         longExpression3 ),
32     longExpression4,
```



2. Lines and Spaces

```
33 otherMethod( longExpression5 ) );
```

Using a single line for each argument makes especially sense in cases where one or more expressions are really long, or if a comment should be added to the argument.

Next there are two examples of breaking an arithmetic expression. The first is preferred, since the break occurs outside the parenthesized expression, which is at a higher level.

```
1 // PREFERRED
2 final var longName1 = longName2 * (longName3 + longName4 - longName5)
3   + 4 * longname6;
4
5 longName1 = longName2 * (longName3 + longName4 - longName5)
6   + 4 * longname6;
7
8 // AVOID!!!
9 longName1 = longName2 * (longName3 + longName4
10   - longName5) + 4 * longname6;
```

Then we have a few examples for the indentation on long method declarations. First the conventional case. Next, a `throws` clause that will be either appended on the same line as the closing parentheses, or it can be on its own line with an indentation of four blanks.

The second example would shift the second and third lines to the far right if it uses conventional indentation, so instead it indents only 8 spaces. The final example uses a single line for each argument.

```
1 // CONVENTIONAL INDENTATION
2 public final void myMethod( final int anArg, final Object anotherArg,
3   final String yetAnotherArg, final Object andStillAnother )
4 {
5   ...
6 } // myMethod()
7
8 private final synchronized void horkingLongMethodName( final int arg,
9   final Object anotherArg, final String yetAnotherArg,
10  final Object andStillAnotherArg )
11 {
12   ...
13 } // horkingLongMethodName()
```



```
14
15 public final void myMethod( final int anArg, final Object anotherArg,
16     final String yetAnotherArg, final Object andStillAnother,
17     final Instant andTheFinalArg ) throws IllegalArgumentException
18 {
19     ...
20 } // myMethod()
21
22 // ADDITIONAL INDENTATION BECAUSE OF THE throws CLAUSE
23 public final void myMethod( final int anArg, final Object anotherArg,
24     final String yetAnotherArg, final Object andStillAnother )
25     throws IllegalArgumentException, IOException
26 {
27     ...
28 } // myMethod()
29
30 // SINGLE LINE FOR EACH ARGUMENT
31 private static final synchronized anotherHorkingLongMethodLine
32 (
33     final int anArg,
34     final Object anotherArg, // a comment ...
35     final String yetAnotherArg,
36     final Object andStillAnother
37 )
38 {
39     ...
40 } // anotherHorkingLongMethodLine()
41
42 private static synchronized anotherHorkingLongMethodLineThrows
43 (
44     int anArg,
45     Object anotherArg,
46     @MyAnnotation String yetAnotherArg,
47     Object andStillAnother
48 ) throws IllegalArgumentException, IOException
49 {
50     ...
51 } // anotherHorkingLongMethodLineThrows()
```

For a method declaration, a comment for a formal argument (like in line 34) is usually obsolete, as the description for the parameters is a mandatory part of the documentation comments (see chapter “7.1.1 The Method Comment” on page 157 for the details).



2. Lines and Spaces

But you should think about having each formal argument in a line of its own if you have annotations on parameters, as shown in line 46. Especially when the annotation will have its own arguments.

Here are the acceptable ways to format ternary expressions⁵:

```
1 alpha = (aLongBooleanExpression) ? beta : gamma;
2 alpha = (aLongBooleanExpression) ? beta
3                                     : gamma;
4 alpha = (aLongBooleanExpression)
5     ? beta
6     : gamma;
7
8 final var alpha = (aLongBooleanExpression) ? beta : gamma;
9 final var alpha = (aLongBooleanExpression) ? beta
10                                                         : gamma;
11 final var alpha = (aLongBooleanExpression)
12     ? beta
13     : gamma;
```

And finally a bad example; the formula below calculates the date for Easter sunday for a given year, based on Gauss' Easter algorithm[118, 128].

```
1 /**
2  * <p>{@summary This method calculates the date of Easter Sunday
3  * for the given year.} The year has to be in the range from 1583
4  * to 3900 (included).</p>
5  * <p>The resulting date is for the Gregorian calendar.</p>
6  * <p>The algorithm itself is not the original one published by Carl
7  * Friedrich Gauß first in 1800 (corrected version in 1816), but
8  * that one published by Heiner Lichtenberg in 1997.
9  *
10 * @thanks Carl Friedrich Gauß
11 * @thanks Heiner Lichtenberg
12 *
13 * @param year    The year for which the date of Easter Sunday is
14 *                wanted for.
15 * @return The date of Easter Sunday in the given year.
16 *
17 * @see <a
18 *      href="https://de.wikipedia.org/wiki/Gau%C3%9Fsche_Osterformel">Gaußsche
19 *      Osterformel</a>
```

⁵Read more about the ternary operator ? in chapter “8.26 The Ternary Operator ‘?’” on page 406.



```

18  */
19  public static final LocalDate calcEasterDate( final Year year )
20  {
21      //---* Check the arguments *-----
22      if( requireNonNullArgument( year, "year" ).isBefore( Year.of( 1583 ) )
23      )
24      {
25          throw new IllegalArgumentException( "This method will work only
26          for years greater than or equal to 1583" );
27      }
28      if( year.isAfter( Year.of( 3900 ) ) )
29      {
30          throw new IllegalArgumentException( "This method will work only
31          for years less than or equal to 3900" );
32      }
33      /*
34      * Initialise the return value with the last day of February and
35      * add the calculated number of days.
36      */
37      final var retValue = LocalDate.of( year.getValue(), MARCH, 1 )
38          .minusDays( 1 )
39          .plusDays( 21 + ((19 * (year.getValue() % 19) + (15 + (3 *
40              (year.getValue() / 100) + 3) / 4 - (8 * (year.getValue() /
41              100) + 13) / 25)) % 30) - (((19 * (year.getValue() % 19) + (15
42              + (3 * (year.getValue() / 100) + 3) / 4 - (8 *
43              (year.getValue() / 100) + 13) / 25)) % 30) + (year.getValue()
44              % 19) / 11) / 29) + (7 - ((21 + ((19 * (year.getValue() % 19)
45              + (15 + (3 * (year.getValue() / 100) + 3) / 4 - (8 *
46              (year.getValue() / 100) + 13) / 25)) % 30) - (((19 *
47              (year.getValue() % 19) + (15 + (3 * (year.getValue() / 100) +
48              3) / 4 - (8 * (year.getValue() / 100) + 13) / 25)) % 30) +
49              (year.getValue() % 19) / 11) / 29) - (7 - (year.getValue() +
50              year.getValue() / 4 + (2 - (3 * (year.getValue() / 100) + 3) %
51              4)) % 7)) % 7) );
52
53      //---* Done *-----
54      return retValue;
55  } // calcEasterDate()

```

Even proper breaking of the term in line 37 will not help much to understand what's going on there. A much better formatted (and therefore more readable) version of the method `calcEasterDate()` can be found in the listing 7.2 on page 176, showing what is meant with "breaking up the code into



2. *Lines and Spaces*

multiple parts by introducing additional temporary variables”.



3. Java Source File Structure

Basically, what we refer to as a “Java Source File” is defined by the Java™ Programming Language as a “Compilation Unit”[154].

The Java Language Specification distinguishes between “Ordinary Compilation Units”, “Compact Compilation Units” and “Modular Compilation Units”. Here, we are only interested in the “Ordinary Compilation Units”.

A “Modular Compilation Unit” defines the module structure of an application; this is described in “5.5.1 The Module Definition” on page 111.

“Compact Compilation Units” or “compact source files” were introduced with Java 25 and are mainly meant to support writing small scripts in Java, without the overhead of a full-fledged Java program. They should never be used in the context of a bigger Java program or application.

3.1. File Encoding

The `javac` Java compiler expects UTF-8[336, 370] encoded source files. Obviously, ASCII[23] encoded files will work, too, as well as most ISO-8859 encodings, as long as only the ASCII characters are used.

A resource bundle or property file[100] is expected to be encoded with ISO-8859-1[212], although this can be changed by setting a system property. Refer to [107] for the details.

The encodings of other source files depend on their format and usage.



3.2. Source File Size

Above a certain number of lines, source code files become unwieldy, but it is difficult to specify a concrete value for the maximum size of a Java source file. First, generated source files tend to have lots of lines because it is often much easier to use boiler-plate code here than to generate methods and call them instead. And in particular when dynamically generated code is used¹, it would not even hurt the DRY principle: a change to the repeated code can be performed on one single place, the generator.

Second, classes that have lots of attributes usually end up with lots of lines, too: the formatting style and coding guidelines proposed in this document require a least 5 lines for each attribute, 6 lines for each getter method and 11 lines for each setter method, always including all comments and blank lines. This is about 22 lines for each attribute. One hundred attributes seems to be a lot, but I have seen more than one real life application where database tables had more than that number of columns that each needed to be reflected as an attribute of the class that should reflect the table. I do not think that this is a good design anyway, but if your new Java class has to reflect the legacy data model, it has to have that number of attributes, bringing you to about 2200 lines of code without any methods other than the already mentioned getters and setters.

I found a limit of about 2000 lines to be feasible; if a class becomes larger than this number of lines, you should consider to move some of the code into other classes. This could be a utility class, a parent class (probably abstract), or the functionality would be better placed with another entity in general. Also externalising some functionality by implementing the command pattern^[127] might be an idea.

Principles

- P1. A Java source file should not exceed the limit of 2000 lines. As shorter it is, as better.

¹Code that is regenerated for each build.



- P2. Generated source files may have significant more lines.
- P3. Other formatting rules may not be sacrificed only to reduced the number of lines of a source file.

3.3. The File Structure

According to the Java Language Specification[154] does each (ordinary) compilation unit consist of three parts, each of which is optional:

- A package declaration, giving the fully qualified name of the package to which the compilation unit belongs.

A compilation unit that has no package declaration is part of an unnamed package.

- `import` declarations that allow classes and interfaces from other packages, and static members of classes and interfaces, to be referred to by using their simple names.
- Top level declarations of classes and interfaces.

Based on that, I define a Java source code file having *five* parts, in the given order:

1. Beginning Comment
2. Package Statement
3. Import Statements
4. Class and Interface Declarations
5. Closing Comment

3.3.1. Beginning Comment

A Java project will not consist from Java source files only; aside the already mentioned module definition files(see 5.5.1), the package documentation files (refer to “5.5.2 The Package Documentation” on page 112), there can



3. Java Source File Structure

be various build files, scripts, resource and property files, and even source files other programming languages.

All source files of a project – not only the Java sources – have to start with a comment that gives the copyright notice and the license information², if possible at all: it should be obvious that it is sometimes difficult to add this kind of information to a binary resource like an image or a sound file.

For a Java file (a file with a name ending on *.java), that beginning comment looks like this:

Listing 3.1: Beginning Comment for a Java file

```
1  /*
2  * =====
3  *  Copyright © <Year> <Copyright Notice>
4  *  =====
5  *
6  *  <License Notice>
7  */
```

The same comment block can be used for Groovy source files, Kotlin source files, Gradle build files (**build.gradle**) and generally for all file formats that uses the pattern /*...*/ for comments.

A *resource* file[100] can have XML format or it may contain simple key-value pairs. In the latter form, comments are prepended with the hash symbol (“#”), and the beginning comment has the following form:

Listing 3.2: Beginning Comment for a Resource or a Script file

```
1  #
2  # =====
3  #  Copyright © <Year> <Copyright Notice>
4  #  =====
5  #
6  #  <License Notice>
7  #
```

²This is different from the Sun coding conventions[110] that recommends to put also the class name and the date into the beginning comment. I omitted the name of the class because this is obvious from the file name in case of a **public** class, and it would confuse the reader if the file contains more than one class. The date is considered obsolete.



Obviously, this can be used for all other file formats that use the same comment style, like Unix shell scripts.

For a resource file in XML format³ (or any other XML file that is part of the sources), as well as for HTML files, the beginning comment would look like this:

Listing 3.3: Beginning Comment for XML and HTML files

```

5 <!--
6 =====
7 Copyright © <Year> <Copyright Notice>
8 =====
9
10 <License Notice>
11 -->

```

T_EX/L_AT_EX uses the per cent sign (“%”) to indicate a comment. So the beginning comment for a T_EX/L_AT_EX source file would be like this:

Listing 3.4: Beginning Comment for a T_EX/L_AT_EX file

```

1 %
2 % =====
3 % Copyright © <Year> <Copyright Notice>
4 % =====
5 %
6 % <License Notice>
7 %

```

<Year>, <Copyright Notice> and <License Notice> has to be replaced by the appropriate texts. The special character “©” is used not only because it looks better; it tells you immediately whether your environment is capable to deal with UTF-8 encoded files.

The lines with the equals signs (“====”) will end at column 80; refer to chapter “2.3 Line Length” on page 47.⁴

³For an XML file, the beginning comments follows the initial <?xml ...?>, so it is not starting on line 1. That’s similar for HTML files.

⁴Keep in mind that in this document the line length is set to 70 columns; this means that you cannot just copy and paste the comment blocks into your development environment.



Principles

- P1. Each file that is part of the sources for the program/application to build should have a beginning comment.
- P2. That beginning comment must contain the copyright notice with the year and the copyright holder.
- P3. It should contain a license notice (either the license itself or a reference to it).
- P4. No other information should be added to this comment block.

3.3.2. Package Statement

The first non-comment line of a Java source file is the `package` statement. According to the Java Language Specification[155], it is optional (resulting in an ‘unnamed’ or default package), but this is never used for production code. So you must have a `package` statement in each and every Java source file.

For the names of packages, see chapter “6.4 Packages” on page 121.

Principles

- P1. Each Java source file has a `package` statement.

3.3.3. Import Statements

Usually several `import` statements will follow the `package` statement. Only very basic code will not import other stuff than that from the `java.lang` package. But interfaces with only one method (a so-called “functional interface”[9]) may not need any imports at all, same for “marker interfaces” that does not declare any methods at all. Sometimes you may have also



(abstract) parent classes without methods that do not need any `import` statements, too.

But in general, that part of a source file would look like this:

```
import static java.lang.String.format;
import static java.lang.System.out;
...

import java.awt.peer.CanvasPeer;
import java.io.InputStream;
...
```

Static imports (for methods and constants) have to be placed before the imports for classes. Inside these blocks, the imports should be ordered alphabetically.

Wildcard imports like

```
import static java.lang.String.*;
...

import java.util.*;
...
```

are not allowed and should not even be used for throwaway code.

Eclipse users can use the function `Source | Organize Imports` from the menu, or the short-key `Shift+Ctrl+O`. This will reorder the import statements, explode the wildcards and remove unused imports; it will even add currently missing imports – given that this is properly configured in the preferences at `Java | Code Style | Organize Imports`.

At IntelliJ IDEA, the function `Code | Optimize Imports` from the menu does something similar; the short key would be `Ctrl+Alt+O`.

Java 25 introduced a new feature, called *Module Import Declarations*[\[238\]](#). It allows to import all public classes and interfaces of a module with a single statement:

```
...
import module java.base;
...
```



As it can cause ambiguities, module imports should be avoided.

Principles

- P1. Place **static** imports before non-static ones.
- P2. Sort the **import** statements alphabetically.
- P3. Do never use wildcard imports.
- P4. Don't use module imports.
- P5. Remove unused imports. If an import is only required because of a Javadoc reference, change the Javadoc reference to use the fully qualified class name and remove the import.

3.3.4. Class and Interface Declarations

Each Java source file⁵ contains at least one top level class or interface declaration; in most cases, it is just one. And it is strongly discouraged that it would be more than one!

Only one of these top level classes or interfaces can be **public**, the others have to be package-local (meaning the modifiers **private** and **protected** are not allowed). If you really have more than one class in a single source file, the **public** one should be the first entry, all others will follow, sorted alphabetically by their names.

With the JPMS (Java Platform Module System) the need for non-public top-level classes got nearly obsolete and they should be avoided.

If you need classes or interfaces that are closer related to a top-level class, you should consider to define these as *inner* classes or interfaces. Inner classes can be **public**, **private** or **protected**. An inner class is part of the top level class, but when defined as **static**, it can be used independently from the enclosing class.

⁵... or compilation unit



At least since Java 5, there are not only classes and interfaces that are declared in a Java source file. Today, we have

- classes (keyword `class`)
- interfaces (keyword `interface`)
- enums (keyword `enum`)
- records (keyword `record`)
- annotations (keyword `@interface`)

Technically, records and enums are special types of classes, while an annotation is somehow a special kind of an interface.

Each of these allows different sections in their declarations; the following tables describe the various sections and the order they should appear in. Each section has a header comment, and inside each section, the elements are ordered alphabetically.

Class

A skeleton for a Java `class` may look like this:

Listing 3.5: Class Skeleton

```

1 public class MyClass
2 {
3     /*-----*\
4     ===** Inner Classes **=====
5     \*-----*/
6     ...
7
8     /*-----*\
9     ===** Constants **=====
10    \*-----*/
11    ...
12
13    /*-----*\
14    ===** Attributes **=====
15    \*-----*/
16    ...
17
18    /*-----*\
19    ===** Static Initialisations **=====

```



3. Java Source File Structure

```
20      /*-----*/
21      ...
22
23      /*-----*\
24      ===** Constructors **=====
25      /*-----*/
26      ...
27
28      /*-----*\
29      ===** Methods **=====
30      /*-----*/
31      ...
32  }
33  // class MyClass
```

Table 3.1.: Parts of a `class` declaration

Part	Description
Inner Classes (optional)	Given the class defines inner classes or interfaces, either private or public , they will come first. Internally, they will follow the same rules as a top-level class.
Constants (optional)	This part takes all constants defined by this class (meaning public final and public static final fields). All values for the constants are known at compile time, otherwise you should place the respective field to the <i>Static Initialisations</i> part.
Attributes (optional)	Here go the attributes for the class; technically, this part is in fact optional for a class, but usually a class without any attributes does not make much sense. Attributes are private (under some exceptional circumstances, they can be protected) and usually, they are not static .

Continued on next page



Table 3.1 – Continued from previous page

Part	Description
Static Initialisations (optional)	If a class declares static fields that are not mere constants or their initialisation is more complex than just assigning a compile time constant, these fields go into this part. The fields should be initialised in a static {...} block. This is also the right place for the <code>serialVersionUID</code> of a serialisable class.
Constructors (optional)	All the constructors for the class go into this part. It is optional in case the class will have only an empty public default constructor, but usually this will be coded, too.
Methods (optional)	All methods of the class are in this part. Again, this part is technically optional, but classes without methods are barely useful.

Interface

A skeleton for an **interface** may look like this:

Listing 3.6: Interface Skeleton

```

1 public interface MyInterface
2 {
3     /*-----*|
4     ===** Inner Classes **=====
5     /*-----*/
6     ...
7
8     /*-----*|
9     ===** Constants **=====
10    /*-----*/
11    ...
12

```



3. Java Source File Structure

```
13      /*-----*\
14      ===** Methods **=====
15      \*-----*/
16      ...
17
18  }
19  // interface MyInterface
```

Table 3.2.: Parts of an **interface** declaration

Part	Description
Inner Classes (optional)	Defining inner classes for an interface is discouraged, although there are some valid use cases for this. If the interface defines inner classes or interfaces, they have to be public - classes have to be static , too, and they will be the first part. Internally, they will follow the same rules as for top-level classes.
Constants (optional)	This part takes all constants defined by this class (meaning public final and public static final fields). All values for the constants are known at compile time.
Methods (optional)	All methods of the interface are in this part. Again, this part is technically optional, it can be omitted for so-called <i>marker interfaces</i> .

enum

An **enum** is a special case of a Java class. Usually it only contains the enumeration values, but it can have more components. A skeleton for a **enum** class may look like this:

Listing 3.7: enum Skeleton

```
1 public enum MyEnum
2 {
3     /*-----*\
4     ===** Enum Definitions **=====
```



```

5      \*-----*/
6      ...
7
8      /*-----*\
9      ===** Constants **=====
10     \*-----*/
11     ...
12
13     /*-----*\
14     ===** Attributes **=====
15     \*-----*/
16     ...
17
18     /*-----*\
19     ===** Static Initialisations **=====
20     \*-----*/
21     ...
22
23     /*-----*\
24     ===** Constructors **=====
25     \*-----*/
26     ...
27
28     /*-----*\
29     ===** Methods **=====
30     \*-----*/
31     ...
32
33 }
34 // enum MyEnum

```

Table 3.3.: Parts of an **enum** declaration

Part	Description
Enum Definitions	This part is mandatory. Each enum is an instance of the enum class.

Continued on next page



Table 3.3 – Continued from previous page

Part	Description
Constants (optional)	This part takes all additional constants defined by this enum (meaning public final and public static final fields). All values for the constants are known at compile time, otherwise you should place the respective field to the <i>Static Initialisations</i> part.
Attributes (optional)	Here goes the attributes for the enum, if any. Each attribute has to be private final and will be initialised by the constructor.
Static Initialisations (optional)	If an enum declares static final fields with an initialisation that is more complex than just assigning a compile time constant, these fields go into this part. The fields should be initialised in a static { ... } block.
Constructors (optional)	All the constructors for the enum (rarely there is more than one) go into this part, if any. A constructor has to private .
Methods (optional)	All methods of the class are in this part.

Technically, an **enum** can also define inner classes, but this is strongly discouraged, and I am not aware of any use case for this.

Record

Records had been introduced to Java with version 14 and were finalised with version 16[219, 158, 29]. Like an **enum**, a record is a restricted form of a Java class. It's ideal for "plain data carrier" classes that contain data not meant to be altered. Usually it has only the most fundamental methods, such as constructors and accessors.



A skeleton for a Java record class may look like this, with all parts being optional:

Listing 3.8: Record Skeleton

```
1 public record MyRecord( ... )
2 {
3     /*-----*\
4     ===** Constants **=====
5     \*-----*/
6     ...
7
8     /*-----*\
9     ===** Attributes **=====
10    \*-----*/
11    ...
12
13    /*-----*\
14    ===** Static Initialisations **=====
15    \*-----*/
16    ...
17
18    /*-----*\
19    ===** Constructors **=====
20    \*-----*/
21    ...
22
23    /*-----*\
24    ===** Methods **=====
25    \*-----*/
26    ...
27 }
28 // record MyRecord
```



Table 3.4.: Parts of a **record** declaration

Part	Description
Constants (optional)	This part takes all constants defined by this record (meaning <code>public final</code> and <code>public static final</code> fields). All values for the constants are known at compile time, otherwise you should place the respective field to the <i>Static Initialisations</i> part.
Attributes (optional)	Here go the attributes for the class; this part is optional, because usually, all attributes are defined as arguments to the record definition.
Static Initialisations (optional)	If a record declares <code>static</code> fields that are not mere constants or their initialisation is more complex than just assigning a compile time constant, these fields go into this part. The fields should be initialised in a <code>static {...}</code> block.
Constructors (optional)	If the record needs additional constructors, these go into this part. It is optional, because in most cases no additional constructor is need.
Methods (optional)	All additional methods of the record are in this part.

Same as **enums** can records have inner classes, but this is similarly discouraged, and there is no use case for it.

Annotation

Annotations had been introduced with Java 5[162]. A skeleton for an annotation interface may look like this:



Listing 3.9: Annotation Skeleton

```

1 public @interface MyAnnotation
2 {
3     /*-----*\
4     ===** Attributes **=====
5     \*-----*/
6     ...
7
8 }
9 // @interface MyAnnotation

```

Table 3.5.: Parts of an annotation declaration

Part	Description
Attributes (optional)	The attributes for the annotation go here, if any.

Principles

- P1. Only one public top-level class (interface, record, enum, ...) per compilation unit. Avoid non-public top-level classes.
- P2. Group the parts of the class as shown above. Remove the comments for unused sections.

3.3.5. Closing Comment

A closing comment like this, for a Java source file,

Listing 3.10: Closing Comment for a Java file

```

1 /*
2  * End of File
3  */

```

at the end of the source file is helpful to detect whether a source file was corrupted: if missing, there is a good chance that the file is incomplete.



3. Java Source File Structure

I confess that this kind of error has got very unlikely with modern hard-drives and SSD, but it can still happen when sending around sources over the net.

Below samples of closing comments for some of the most common file types that appear as sources within a Java project, in addition to *.java files; just be creative for other file types.



For a resource file, the closing comment has the following form:

Listing 3.11: Closing Comment for a resource or a script file

```
1 #  
2 # End of File  
3 #
```

And for an XML or HTML file:

Listing 3.12: Closing Comment for a HTML or an XML file

```
1 <!--  
2 End of File  
3 -->
```

Principles

P1. Each source file must have a closing comment, where feasible.



3. Java Source File Structure



4. Declarations

This chapter describes how to format declarations of various kinds. See chapter “2.3.1 Wrapping Lines” on page 48 for information on how to break up the lines for long declarations.

4.1. General Guidelines

4.1.1. Multiple Declarations per Line

For local variables, it is possible to have multiple declarations in one single line, and the compiler does also not complain when you do this for attributes, too, but there it is even worse than for local variables.

So you should avoid the following:

```
// AVOID!!!  
int level, size;
```

It is strongly encouraged to have only one declaration per line, as it is also comforting to add some comments to the declaration. In other words,

```
int level; // indentation level  
int size; // size of table
```

is preferred over the sample above.

And by no means, you may put different types on the same line, despite the compiler does not complain about it. Example:

```
// REALLY BAD!!!  
int foo, fooarray [];
```



Adding comments to local variables should not be necessary if you name them appropriately (refer to “6.8 Local Variables” on page 128). For attributes, the respective Javadoc comment (see “7.1 Documentation Comments” on page 146) is mandatory.

Principles

- P1. One declaration per line.
- P2. Comments for local variables are optional.
- P3. Documentation comments for attributes and constants are mandatory.

4.1.2. Placement of Declarations

A declaration should be placed as close to the location where it is needed. This is mainly relevant for local variables, It means that you should wait with the declaration of variables until their first use, in particular when you need the result of an calculation for the initialisation. The “Code Conventions for the Java™ Programming Language”[112] recommends to put the declaration only to the beginning of a block.

For small blocks (this includes small method bodies), there is not much difference. Nevertheless, I find the Sun Code Conventions recommendation impractical, and I cannot follow the reasoning for that recommendation (to wait until first use “[...] can confuse the unwary programmer and hamper code portability within the scope”). Even when “code portability” should mean “cut-and-paste” having the declaration close by would be an advantage.

So *do not* declare the local variables like this:

```
// AVOID!!
public final void myComplexMethod( final String arg )
{
    final String a1;
    final String a2;
    final int a3;
```



```
final double a4;
final Map<String,String> a5;

if( isNull( arg ) ) throw new IllegalArgumentException( "arg_is_
    null" );

// 50 lines of code before a1, a2, and a3 are initialised...
...
// 20 lines of code before a2 is used and a3 is initialised...
...
// 30 lines of code before a3 and a4 are used...
...
// 20 lines of code before a5 is initialised and used...
...
}
```

The preferred pattern is similar to this:

```
// AVOID!!
public final void myComplexMethod( final String arg )
{
    if( isNull( arg ) ) throw new IllegalArgumentException( "arg_is_
        null" );

    // 50 lines of code before a1, a2, and a3 are initialised...
    final var a1 = getInitialValueForA1();
    final var a2 = getInitialValueForA2();
    final var a3 = 0;

    ...
    // 20 lines of code before a2 is used and a3 is initialised...
    final var a4 = getValueForA4( a2 );

    ...
    // 30 lines of code before a3 and a4 are used...
    ...
    // 20 lines of code before a5 is initialised and used...
    final Map<String,String> a5 = new HashMap<>()
    ...
}
```

Placing the declaration close to its first use would also allow to limit the scope of a variable. This can help to prevent the pollution of the names-



4. Declarations

pace with too many local variables as this could happen quickly when using lambda expressions:

```
// AVOID!!
var i = 0;

final var i1 = ++i;
final IntFunction f1 = arg -> arg + i1;

final var i2 = ++i;
final IntFunction f2 = arg -> arg + i2;

final var i3 = ++i;
final IntFunction f3 = arg -> arg + i3;
```

should be written like this instead:

```
var i = 0;

final IntFunction f1;
{
    final var x = ++i;
    f1 = arg -> arg + x;
}

final IntFunction f2;
{
    final var x = ++i;
    f2 = arg -> arg + x;
}

final IntFunction f3;
{
    final var x = ++i;
    f3 = arg -> arg + x;
}
```

Declaring and initialising a local variable inside a loop is even perfectly ok, as long as the initial value is different for each loop cycle:

```
for( var i = 0; i < limit; ++i )
{
    final var a = getValue( i );
    ...
}
```



Only the repeated *initialisation* of a local variable inside a loop should be avoided, when the value does not change:

```
1 // AVOID!  
2 for( var i = 0; i < limit; ++i )  
3 {  
4     final var a = new VeryComplexImmutableType();  
5     ...  
6 }
```

In this case, line 4 should be moved before the loop header (before line 2):

```
final var a = new VeryComplexImmutableType();  
for( var i = 0; i < limit; ++i )  
{  
    ...  
}
```

And yes, `final var a = ...` inside the loop body is valid!

Principles

- P1. Declarations should be placed as close to the location where it is first used.
- P2. Limit the scope as much as possible.
- P3. Declaring a local variable inside a loop does not harm, but repeated initialising it with the *same* value would do.

4.1.3. Array Declarations

The brackets for an array declaration always have to be put to the type, not to the name. So it should read

```
int [] foo;  
String [] bar;  
java.awt.Button [] buttons;
```



and not

```
// AVOID!!!  
int foo [];  
String bar [];  
java.awt.Button buttons [];
```

This is the same way as an array has to be declared as the return type for a method:

```
public final static String [] getTexts() { return m_Texts; }
```

Principles

P1. The brackets for an array declaration must be put to the type.

4.2. Class Declarations

In “3.3.4 Class and Interface Declarations” on page 62 we already described the general structure of declarations for Java™ classes, interfaces, enums, records, and annotations, while the use of blanks, round brackets, curly braces and linefeeds was discussed in “2.2 White Space” on page 41.

Based on that, the declaration for a class, interface or enum starts with the respective Javadoc comment for the type, followed by the annotations for it, if any, and the line setting the name for the type, as well as the base class and implemented interfaces.

If the type is sealed, the **permits** clause is placed on the following line, with the default indentation, before finally the line with the opening curly brace follows.

Basically the declaration for a record looks the same, but because it declares the attributes also on that first line, that could be longer and may require to be wrapped.



Each type ends with a comment after the closing curly brace, followed by a comment line repeating the name and kind of the type.

Some samples¹:

```
public final class MySimpleClass
{
    ...
}
// class MySimpleClass
```

```
public interface MyInterface<T extends CharSequence>
{
    ...
}
// interface MyInterface
```

```
public enum MyEnum
{
    ...
}
// enum MyEnum
```

```
public final class MyRecord( String name, int age, boolean
    retirementStatus )
{
}
// record MyRecord
```

```
public final class MyClass extends MyBaseClass implements
    MySealedInterface, Serializable
{
    ...
}
// class MyClass
```

```
public sealed class MySealedInterface extends List<String>
    permits MyClass
{
    ...
}
// interface MySealedInterface
```

¹I omitted the Javadoc comments from the samples for brevity.



```
@FunctionalInterface
public interface MyFunctionalInterface<T>
{
    ...
}
// interface MyFunctionalInterface
```

```
@Documented
@Retention(RUNTIME)
@Target(ANNOTATION_TYPE)
public @interface MyAnnotation
{}
// @interface MyAnnotation
```

Inner classes² will follow the same pattern, but with an additional level of indentation. Inner classes are discussed in detail in chapter 8.56 on page 428.

The elements of the class (inner classes, constants, attributes, constructors, methods, ...) are all separated from each other by a single blank line (see chapter “2.2.1 Blank Lines” on page 41).

Principles

- P1. Only one public top-level class (interface, record, enum, ...) per compilation unit. Avoid non-public top-level classes.
- P2. Group the parts of the class as shown in 3.3.4 on page 62. Remove the comments for unused sections.
- P3. Place each class annotation on a line of its own.
- P4. The **permits** gets a line of its own, too.
- P5. No blank before the opening angle bracket for a type variable. No blanks after a comma inside the angle brackets.

²..., interfaces, enums, records, and annotations – although the latter are rarely defined as inner classes



- P6. The opening curly brace is again on a line of its own.
- P7. The closing curly brace is followed by a line with a comment repeating type and name of the class.
- P8. A single blank line separates the elements of a class (see chapter “2.2.1 Blank Lines”).

4.3. Method Declarations

The declaration of methods is described in the “The Java™ Language Specification”[133], and there at two locations: [159] for classes and [161] for interfaces.

The formatting is quite simple and straight forward: a method³ starts with the Javadoc comment followed by the line with the method’s signature.

If the method is not abstract, the body follows. If that body is short or empty, it can be placed on the same line, otherwise the opening curly braces follows on the next line, with the same indentation level as the line with the signature. The next line(s) have the code of the body, indented by one level, and finally the closing curly braces with a comment repeating the name of the method, with the same indentation level as the opening curly brace.

As said, short methods may be written in one single line. In this case, “one single line” means exactly that: *one single* line! Any line break requires the full form.

A method with an empty body could be treated as very short methods, but usually you provide a comment instead of code.

Some samples⁴:

³All this is also valid for constructors; in regard of formatting, they are just a special kind of a method

⁴I omitted the Javadoc comments from the samples for brevity.



4. Declarations

```
public static final void main( final String... args )
{
    // Do something ...
} // main()

public MyClass( final String arg )
{
    m_Arg = arg;
} // MyClass()
```

For an interface:

```
// For an interface:
public void myAbstractMethod( final String... args );

// For a class:
public abstract void myAbstractMethod( final String... args );
```

```
public String myString() { return m_String; }
```

```
public String myMountPoint( final String arg ) {}

// BETTER:
public String myMountPoint( final String arg )
{
    /* This implementation does nothing! */
} // myMountPoint()
```

According to what we said already in “2.2 White Space” on page 41, there is no space between a method or constructor name and the opening round bracket (“(”) starting its formal parameters list. Basically the parenthesis in the declaration are “Parameter Round Brackets” and should be treated along the same lines as for method and constructor *calls*; refer to 2.2.2 on page 43.

Some modifiers are obsolete and/or redundant under some circumstances, but I recommend to add them anyway⁵

⁵Remember “Coding Guardrails for Java in a Nutshell”bullet point 2: Verbosity is your friend.



- All methods of an **interface** are public (add **public** anyway). If not static or default, they are all abstract, but that modifier is not allowed in an interface.
- A private method is automatically final, but add **final** nevertheless. Same for static methods.
- Of course all methods of a final class are final, but you will add **final** to each method.
- A package-private method does not have an explicit modifier⁶ Add a comment instead:

```
/*package-private*/ String myPackagePrivateMethod() { ... }
```

The method's parameters are described in the respective Javadoc comment for the method. Adding comments to the parameter in the signature does not make much sense.

Principles

- P1. No blanks between the method's name and the opening round bracket. Refer to "2.2.2 Parameter Round Brackets" on page 43 on how to place the white space.
- P2. *Really short* methods can be written in *one single line*. Otherwise, the opening and the closing curly braces are on lines of their own, with the same indentation as the signature.
- P3. The method body is indented one level relative to the curly braces.
- P4. A single blank line separates the methods and constructors from each other (see chapter "2.2.1 Blank Lines").
- P5. Keep redundant modifiers for verbosity.
- P6. If a method has more than one line, there is always a comment repeating the method's or constructor's name after the closing curly brace.

⁶It is not private, not public and not protected.



4. *Declarations*



5. Statements

This chapter deals with the formatting of the various types of statements.

5.1. Simple Statements

Each line may contain at most one single statement. Or, the other way round, each (simple) statement has to be written to its own line.

Example:

```
// AVOID!!!  
++argv; --argc;  
  
// CORRECT  
++argv;  
--argc;
```

5.2. Compound Statements

Compound statements are statements that contain lists of statements enclosed in curly braces { <statement, ...> }. A compound statement (also called a “code block” or “block statement”) typically appears as the body of another statement, such as the `if` or `for` statements, but it may appear also on its own¹.

¹In Java, a block also indicates a scope; a local variable lives only in the scope it was declared in, and all inner scopes contained in it.



5. Statements

The block's opening curly brace “{” appears alone at a line by itself with the same indentation as the statement it belongs to and the corresponding closing curly brace “}” is placed at a new line with the same indentation. All code inside such a block is indented one level more than the curly braces.

Large compound statements (long code blocks) should be marked with a label and should get an end comment, like in the examples below:

```
ForeverLoop: while( true )
{
    ... // Lots of lines
} // ForeverLoop:

CountrySelector: switch( countryCode )
{
    case DE: ...
    case EN: ...
    ... // All the other countries
    default: ...
} // CountrySelector:

ProcessLoop: for( final var entry : entries )
{
    ... // Lots of lines
} // ProcessLoop:
```

You may have seen something like this in existing code:

```
// NOT RECOMMENDED!
if( thisConditionIsTrue )
{
    // Lots of lines
    ...
} // if( thisConditionIsTrue )
```

As long as the `if` clause (or whatever statement starts the block) will not change, this pattern serves the same purpose as my variant with the label. But when you change the `if`, you have to modify the end comment accordingly – something that is forgotten much too often!

As there is rarely a reason to change the label (the purpose of the block remains the same, no matter what the condition will be), it will not happen that the end comment will not match the start ‘comment’ of the block.



See the chapters “5.4.8 Labels and ‘break’ Statements”, “7.2.4 Trailing or End-Of-Line Comments”, and “7.6 Comments when?” for some additional details.

For more examples regarding the formatting of compound statements refer in particular to the chapters “5.2.1 ‘if’, ‘if-else’, ‘if-else if-else’ Statements”, “5.4.2 ‘switch’ Statements”, “5.4.3 ‘for’ Statements”, and “5.4.4 ‘while’ Statements”.

5.2.1. ‘if’, ‘if-else’, ‘if-else if-else’ Statements

The base forms for the `if` statement are the following ones:

- This is the simplest form that does not have an `else` clause:

```
if( <condition> )
{
    <statements>;
}
```

It allows to omit the curly braces (“{}”) when there is a single statement on the same line as the `if` clause and there is no `else` clause:

```
if( <condition> ) <singleStatement>;
```

- With an `else` clause, the curly braces (“{}”) are always mandatory:

```
if( <condition> )
{
    <statements>;
}
else
{
    <statements>;
}
```

Even if may fit into a single line, do not write an `if-else` statement like below:

```
// STRONGLY DISCOURAGED!!!
if( <condition> ) <singleStatement>; else <otherStatement>;
```



5. Statements

- In case there are more conditions to check, but a `switch` statement cannot be used due to the data types involved or the logic for the conditions, use this form:

```
if( <condition1> )
{
    <statements>;
}
else if( <condition2> )
{
    <statements>;
}
```

- Same as before, but with a final `else` clause:

```
if( <condition1> )
{
    <statements>;
}
else if( <condition2> )
{
    <statements>;
}
else
{
    <statements>;
}
```

Of course you can repeat the `else if` as often as you need, but at some point the readability suffers seriously, and you should think about another logic.

- Avoid an empty code block:

```
// AVOID!!!
if( <condition> )
{ /* Empty! */ }
else
{
    <statements>;
}
```

Instead negate the condition and place the statements to the “then” block:



```
// BETTER
if( !<condition> )
{
    <statements>;
}
```

As said already above, an `if` statement does not need the curly braces (“{ }”) only if there is just a single statement on the same line as the `if` clause, and if there is no `else` clause:

```
if( <condition> ) <singleStatement>;
```

In any other case the curly braces *are always required*, to avoid the following error-prone forms:

```
// AVOID! MISSING CURLY BRACES {}!
if( <condition> )
    <statement>;

// AVOID!!! EVEN WORSE THAN ABOVE!!
if( <condition> )
    <statement>;
else
    <statement>;
```

If the `if` statement takes more than one single line, curly braces have to be used around all statements, even if there is just one. This makes it easier to add statements without accidentally introducing bugs due to forget adding braces.

Please refer also to chapter “8.26 The Ternary Operator ‘?’” on page 406 that discusses the ternary “?” operator.

5.3. Method Chaining

Some APIs are designed to allow “method chaining”, sometimes also referred as “Call Chaining”; you may find this in particular for implementations of the “builder” pattern, but it appears elsewhere, too. A prominent ex-



5. Statements

ample is the class `java.lang.StringBuilder`[68] (obviously not a builder), an example for a builder class would be the class `java.time.format.DateTimeFormatterBuilder`[81]

A class (or interface) is designed for method chaining when its methods return `this` (the containing object instance), but this is not mandatory for the use of method chaining, as shown in the code snippet below²:

```
return strings.stream()
    .collect( groupingBy( s -> s, counting() ) ) // returns java.util.Map
    .entrySet()
    .stream()
    .max( Map.Entry.comparingByValue() ) // returns java.util.Optional
    .map( Map.Entry::getKey );
```

Method chaining allows you to write

```
writer.append( "Caption\t:␣" ).append( value );
```

instead of

```
writer.append( "Caption\t:␣" );
writer.append( value );
```

If the “chain” does not fit completely into a single line, each method call has to be written into a single line of its own.

In fact, it is recommended to do this all the time:

```
// Acceptable
writer.append( "Caption\t:␣" ).append( value );

// RECOMMENDED
writer.append( "Caption\t:␣" )
    .append( value );
```

The dot has to be written in front of the method’s name and the methods will be indented regularly.

The streams that had been introduced with Java 8 will be formatted in the same way:

²The example will be explained in detail in chapter 8.8



```
final var names = customers.stream()
    .filter( c -> c.getCountry().equals( GERMANY ) )
    .filter( c -> c.getTurnover() > limit )
    .map( Customer::getName )
    .sorted()
    .toArray( String []::new );
```

5.4. 'return' and 'yield' Statements

The `return` statement has to be the last statement of a method, while the `yield` statement has to be the last statement of a `case` block; refer to the chapter “8.15 Terminating a Method and returning Values” on page 388 for more details on this.

When `return` is always the last statement of a method, it means that the simple form

```
return;
```

is obsolete and will be omitted. This implies that there is only one `return` statement per method allowed, and that short-cutting a method is a forbidden practice.

`yield` alone (without a return value) in a `case` block will always cause a compiler error. Some more details can be found at [143].

The use of parentheses with the `return` or `yield` statement is not required unless they make the return value more obvious and better readable in some way.

The recommended forms are shown in lines 15 and 22/23, below:

```
1 public final String myMethod( final InputType inputType, final String...
   input )
2 {
3     InputSwitch: final var retValue = switch( inputType )
4     {
5         case null -> throw new NullPointerException( "inputType" );
6         case FILE ->
```



5. Statements

```
7      {
8          var result = stream( input )
9              .filter( String::isBlank )
10             .filter( s -> !s.startsWith( "_" ) )
11             .collect( joining( "\n" ) );
12          result = result.length() > 80
13              ? abbreviate( result, 80 )
14              : result;
15          yield result;
16      }
17      case REGULAR -> abbreviate( input [0], 80 );
18      case UNKNOWN -> "Unknown_" + input;
19      default -> throw new UnsupportedOperationException( inputType );
20  } // InputSwitch:
21
22  //---* Done *-----
23  return retValue;
24 } // myMethod()
```



5.4.1. 'if', 'if-else', 'if-else if-else' Statements

5.4.2. 'switch' Statements

This chapter describes how to format a `switch` statement together with the related `case` and `default` statements.

Beginning with the first preview in Java 12, an alternative syntax for `switch` was introduced, so that we have now two (in fact, three) different forms of that construct. When to use which form and how is covered by chapter "8.30 The "switch" Statement" on page 407.

The traditional Form of 'switch'

Originally, a `switch` statement in Java looked like this:

```
1 switch( <selector> )
2 {
3     case <switchlabel1>:
4         <statements1>;
5         // Falls through!
6
7     case <switchlabel2>:
8         <statements2>;
9         break;
10
11    case <switchlabel3>: <singleStatement>; break;
12
13    case <switchlabel4>:
14    {
15        <statements>;
16        break;
17    }
18
19    case <switchlabel5>: // Also a fall-through
20    case <switchlabel6>:
21    {
22        <statements>;
23        break;
24    }
25
```



5. Statements

```
26     default:
27         <statements>;
28         break;
29 }
```

First, these coding conventions make the blank line above each **case** (except the very first one) and above **default** mandatory!

If a **case** falls through, like in lines 3 to 9, a comment “*// Falls through!*” as in line 5 of the sample is required! But basically, such constructs should be avoided when possible, because it also forces that the sequence of the **case** clauses cannot be changed, and that fact requires an additional comment to the **switch** itself. In fact here you should consider to ignore the “DRY Principle” – we will see this later – and repeat the code for the second **case** on the first one:

```
// BETTER
switch( <selector> )
{
    case <switchlabel1>:
        <statements1>;
        <statements2>;
        break;

    case <switchlabel2>:
        <statements2>;
        break;

    default: ...
}
```

Or you put the common code into a **private** method and call that from both branches.

Different to the case shown in lines 3 to 9, no comment is required in the case shown in lines 19 and 20: here we have two **case** clauses that triggers *exactly the same* action, and we can even omit the blank line between them (and also the comment from line 19).



For longer `switch` statements it is highly recommended to use a label with the `break` statement³; this can look like this:

```
final Color color;
ColorSelector: switch( colorIndex )
{
    case 1: color = RED;
           break ColorSelector;

    case 2: color = BLUE;
           break ColorSelector;

    case 3: color = YELLOW;
           break ColorSelector;

    case 4: color = GREEN;
           break ColorSelector;

    default: color = NONE;
            break ColorSelector;
} // ColorSelector:
```

The `break` in the `default` branch is redundant, but it prevents a fall-through error if later another `case` branch is added – although it should be assured that the `default` branch is always the last branch in the `switch` statement. Of course no `break` is required when the branch is left by throwing an exception.⁴

The new Form of 'switch'

The alternative syntax for `switch` comes in two flavours (as *statement* and *expression*) and looks like this:

```
1 // switch statement
2 switch( <selector> )
3 {
```

³In fact, I recommend to always use a label with `break`; see chapter “5.4.8 Labels and ‘break’ Statements” on this topic.

⁴A `break` would be also obsolete when the branch is left through a `return` statement, but in that case, the `return` would not be the last statement in the method, and this is another requirement!



5. Statements

```
4     case <switchlabel1> -> <singleStatement>;
5
6     case <switchlabel2>, <switchlabel3> -> <singleStatement>;
7
8     case <switchlabel4> ->
9     {
10         <statements>;
11     }
12
13     case <switchlabel4>, <switchlabel5> ->
14     {
15         <statements>;
16     }
17
18     default -> <singleStatement>;
19 }
20
21 // switch expression
22 var result = switch( <selector> )
23 {
24     case <switchlabel1> -> <expression>;
25
26     case <switchlabel2>, <switchlabel3> -> <expression>;
27
28     case <switchlabel4> ->
29     {
30         <expressions>;
31         yield <value>;
32     }
33
34     case <switchlabel4>, <switchlabel5> ->
35     {
36         <expressions>;
37         yield <value>;
38     }
39
40     default -> <expression>;
41 }
```

Of course the **default** in line 18 can be followed by a statement block as the **case** in line 8, and the **default** in line 40 can have a complex expression like the **case** in line 28.

As this format does not allow a fall-through in cases where the branches for



two or more labels are doing the same stuff, it is possible here to have more than one switch label per **case**, as shown in lines 6, 13, 26 and 34.

If used with pattern matching (refer to [220]), a **switch** looks like this:

```
1 var selector = <anObject>
2 // switch statement
3 switch( selector )
4 {
5     case null -> <singleStatement>;
6
7     case <class1> <name1> -> <singleStatement>;
8
9     case <class2> <name2> ->
10    {
11        <statements>;
12    }
13
14    default -> <singleStatement>;
15 }
16
17 // switch statement
18 var result = switch( selector )
19 {
20     case null -> <expression>;
21
22     case <class1> <name1> -> <expression>;
23
24     case <class2> <name2> ->
25     {
26         <expressions>;
27         yield <value>;
28     }
29
30     default -> <expression>;
31 }
```

Same as for the **default** branch, the **case null** branch can be a statement block or a complex expression.

And an expression can also be always a **throw**.



General formatting Rules for 'switch'

- Every `switch` statement should include a `default` branch, even when the `case` labels are exhaustive.
- If really all possible values for the `case` labels are covered, the `default` branch should throw an exception about an unknown/unexpected value.⁵ Refer to chapter “8.30 The “switch” Statement” on page 407 for additional details.
- The `default` branch should be always the last branch.
- A `switch` with pattern matching should always have a `case null` branch as it first branch.
- Syntactically, the use of curly braces in the branches is optional, even for multi-line statements, but here it is set as mandatory. One reason is that it allows to declare additional variables for a given branch that are local to the given branch.
- As already mentioned before, the blank line separating the branches is mandatory.

5.4.3. 'for' Statements

A `for` statement should have one of the following forms:

```
1 // Classic for loop:
2 for( <initialization>; <condition>; <update> )
3 {
4     <statements>;
5 }
6 for( <initialization>; <condition>; <update> ) <singleStatement>;
7
8 // Enhanced for loop:
9 for( <declaration> : <iterable> )
10 {
11     <statements>;
12 }
```

⁵Refer to chapter “9.6.9 `UnsupportedEnumError`”[351] for a sample.



```
13 for( <declaration> : <iterable> ) <singleStatement>;
```

This means that curly braces can be omitted only when there is just a single simple statement to be executed in the loop. In addition, this single statement has to be written completely into the same line as the `for` itself. In any other case, the curly braces are required.

Examples:

```
1 // Ok
2 for( var i = 0; i < max; ++i ) sum += i;
3 for( final var s : texts ) System.out.println( s );
4
5 // DISCOURAGED
6 for( var i = 0; i < max; ++i ) if( v [i] > 0 ) sum += v [i];
7 for( final var s : texts ) if( !s.empty() ) out.println( s );
8
9 // AVOID!!!
10 for( var i = 0; i < max; ++i ) if( v [i] > 0 )
11 {
12     sum += v [i];
13 }
14 else
15 {
16     sum -= v [i];
17 }
18
19 for( final var s : texts ) if( !s.empty() )
20 {
21     System.out.print( "Value:_" );
22     System.out.println( s );
23 }
```

The samples in lines 6 and 7 are not correct according to this rule, because the `if` statement is not a simple statement..

If you use an iterator with your `for` loop, it should look like this:

```
for( final var i = source.iterator(); i.hasNext(); )
{
    ...
}
```



5. Statements

An empty **for**-loop (one in which all the work is done in the initialization, condition, and update clauses) should have the following form:⁶

```
for( <initialization>; <condition>; <update> );
```

When using the comma operator in the initialization or update clause of a **for** statement, avoid the complexity of using more than three variables. If needed, use separate statements before the **for**-loop (for the initialization clause) or at the end of the loop (for the update clause).

An example:

```
for( var i = 0, j = target.length; i < source.length && j >= 0; ++i, j++ )
{
    target [j] = source [i];
}

var proceed = true;
var i = source.iterator();
for( var j = 0; proceed; )
{
    target [j++] = i.next();
    proceed = i.hasNext() && j < target.length;
}
```

5.4.4. 'while' Statements

A **while** statement should have one of the following forms:

```
1 while( <condition> )
2 {
3     <statements>;
4 }
5 while( <condition> ) <singleStatement>
```

⁶An *enhanced* **for**-loop without body does not make that much sense (that mentioned above is a classical **for**-loop). It may be possible to write an implementation of **java.lang.Iterable** with an iterator that does something as a side effect of **hasNext()** or **next()**, but this would break the contract of the interface **java.lang.Iterator**.



Again, the curly braces can only be omitted in case there is only a single statement, written on the same line than the `while` statement that has to be executed in the loop.

An empty `while`-loop⁷ should have the following form:

```
while( <condition> );
```

Longer `while`-loops should use a label:

```
LoopLabel: while( proceed )
{
    // Lots of code lines here ...
} // LoopLabel:
```

5.4.5. 'do-while' Statements

A `do-while` statement should look like below:

```
do
{
    <statements>;
}
while( <condition> );
```

5.4.6. 'try-catch' and 'try-catch-finally' Statements

Basically, there are two different forms of `try-catch` statements: the simple one and the `try-with-resources` that was introduced with Java 7 (also refer to chapter "8.14 `try-with-resources`" on page 378).

⁷An empty `while`-loop usually makes no sense as the compiler optimises it away. Only when the condition causes side effects, it will be executed. But such an implementation is difficult to understand and should be avoided.



The simple Form

A simple try-catch statement has the following format:

```
try
{
    <statements>;
}
catch( <ExceptionClass> e )
{
    <statements>;
}
```

For this simple form, the `catch` block is mandatory. Additional `catch` blocks and a `finally` block may follow – see below.

‘try-with-resources’

A try-with-resources allows to allocate resources that will be automatically released when the `try` block is left. Something similar could be achieved also by adding a `finally` block, but try-with-resources is easier and more secure.

It looks like this:

```
1 try( final var resource = new <ResourceClass>() )
2 {
3     <statements>;
4 }
5
6 final var resource = new <ResourceClass>();
7 try( final var r = resource )
8 {
9     <statements>;
10 }
```

`<ResourceClass>` must implement the interface `java.lang.AutoCloseable`^[182].

It is possible to allocate more than one resource in a single `try` statement:



```
try
(
    final var resource1 = new <ResourceClass1>();
    final var resource2 = new <ResourceClass2>()
)
{
    <statements>;
}
```

One or more **catch** blocks are optional for try-with-resources, as well as a **finally** block.

The chapter “8.14 try-with-resources” on page 378 handles try-with-resources further.

General formatting Rules for 'try-catch'

- The curly braces (“{}”) around the **try**, the **catch** and **finally** blocks are requested by the Java syntax; their placement follows the rules for any other code block.
- In case the **try** block can issue more than one exception type, these can be combined into one **catch** block, like this:

```
...
catch( <ExceptionClass1> | <ExceptionClass2> e )
{
    <statements>;
}
```

If all exceptions are handled by the same set of statements, or like this, when the exceptions are handled by different code blocks:

```
...
catch( <ExceptionClass1> e )
{
    <statements1>;
}
catch( <ExceptionClass2> e )
{
    <statements2>;
}
```



5. Statements

```
}
```

Of course both can be combined:

```
...  
catch( <ExceptionClass1> | <ExceptionClass2> e )  
{  
    <statements1>;  
}  
catch( <ExceptionClass3> e )  
{  
    <statements2>;  
}
```

- Each try-catch statement can also be followed by a **finally** block, which executes regardless of whether or not the **try** block has completed successfully.

```
try  
{  
    <statements>;  
}  
catch( <ExceptionClass> e )  
{  
    <statements>;  
}  
finally  
{  
    <statements>; // Will always be executed!  
}
```

In case a **finally** block is present, the **catch** block is optional even for a simple try-catch statement, meaning that no exception is caught:

```
try  
{  
    <statements>;  
}  
finally  
{  
    <statements>;  
}
```



- An *empty* `catch` block is unacceptable under all circumstances! Refer to chapter “8.2.3 General Exception Handling” on page 207 how to handle exceptions. See also chapter 7.2.3 on page 173 about empty blocks and comments.
- An empty `finally` block is completely obsolete and must be removed.

5.4.7. 'synchronized' Statements

A `synchronized` statement acquires a mutual-exclusion lock on behalf of the executing thread, executes a block, then releases the lock. While the executing thread owns the lock, no other thread may acquire the lock[139] – meaning no other thread can enter the block and execute the code in there.

This may look like this:

```
synchronized( <expression> )
{
    <statements>
}
```

The curly braces (“{}”) are requested by the Java syntax; their placement follows the rules for any other code block. A single-line `synchronized` is therefore not possible.

The Java Language Specification says, that the type of `<expression>` has to be a reference type[139]. That means that

```
// DOES NOT WORK!!
final int i = 5;
synchronized( i )
{
    ...
}
```

will not work⁸. Unfortunately, the compiler will not complain about something like this:

⁸In fact, it will cause an error at compile time.



5. Statements

```
// DO NOT DO THIS!!
final StringBuilder sb = new StringBuilder();
synchronized( sb )
{
    ...
}

final String s = null;
synchronized( s )
{
    ...
}
```

Ok, for the second one, you get a **NullPointerException** at runtime, but it may take a lot of time before you spot the error in the first one. Even worse to debug would be this pattern, although it could be handy under some circumstances:

```
// TRY TO AVOID
synchronized( provideLockObject() )
{
    ...
}
```

In this case you have to rely on the method **provideLockObject()** to return the proper guard object.

Therefore the rule set by this code conventions is that **<expression>** has to be a **final** attribute of a reference type, a constant reference⁹, **this** or the **class** object. In case the respective field exist only to provide a monitor, it should be named as 'Guard'.

```
1 public final MyClass
2 {
3     private final List<String> m_StringList = new ArrayList<>();
4     private static final Object m_MyClassGuard = new Object();
5
6     public final void myMethod()
7     {
8         synchronized( m_StringList )
```

⁹A **final** class variable of a reference type.



```
9      {
10         ...
11     }
12
13     synchronized( m_MyClassGuard )
14     {
15         ...
16     }
17
18     synchronized( MyClass.class )
19     {
20         ...
21     }
22
23     synchronized( this )
24     {
25         ...
26     }
27 } // myMethod()
28
29 // class MyClass
```

Of course, a method with that many `synchronized` blocks does not make any sense ...

The chapter “8.53 Multi-Threading and Synchronisation” on page 427 discusses this topic further.

5.4.8. Labels and 'break' Statements

In Java, labels are mainly used in conjunction with `break` and `continue` statements, and therefore, they will be assigned mostly to `switch`, `for`, `while` and `do-while` statements, but they can be placed before any code block.

The `break` statement is used with the traditional `switch` statement (refer to chapter “5.4.2 'switch' Statements”), and – together with the `continue` statement – with loops of all kind.¹⁰

¹⁰The use of `break` and `continue` with loops is usually discouraged, but in many cases it makes the logic of an algorithm easier and more understandable.



5. Statements

I recommend to always use a meaningful label together with **break** and **continue**. This would make it more obvious what is intended with the **break** or **continue**, and it assures that the right block will be terminated. If assigned, the label should be used also as the end comment for a code block, as described in chapter “7.2.4 Trailing or End-Of-Line Comments” and chapter “7.6 Comments when?”.

Some samples:

```
1 // BAD!!!
2 for( var line = 0; line < maxLines; ++line )
3 {
4     for( var column = 0; column < maxColumns; ++column )
5     {
6         if( !isValid( field [line] [column] ) break; // What??
7         ...
8     }
9
10    processLine( field [line] );
11 }
12
13 // RECOMMENDED
14 LineLoop: for( var line = 0; line < maxLines; ++line )
15 {
16     ColumnLoop: for( var column = 0; column < maxColumns; ++column )
17     {
18         if( !isValid( field [line] [column] ) break ColumnLoop;
19         ...
20     } // ColumnLoop:
21     processLine( field [line] );
22 } // LineLoop:
23
24 // RECOMMENDED
25 FilterLoop: while( input.hasNext() )
26 {
27     final var value = input.next();
28     if( !value.isValid() ) continue FilterLoop; // Skip invalid
29     ...
30 } // FilterLoop:
31
32 // RECOMMENDED
33 DirectionSwitch: switch( direction )
34 {
35     case LEFT: goLeft();
36     break DirectionSwitch;
```



```
37
38     case RIGHT: goRight();
39         break DirectionSwitch;
40
41     default:
42         throw new IllegalArgumentException( direction.toString() );
43 } // DirectionSwitch:
```

The name of the label is repeated in a comment at the end of the code block, as shown in lines 20, 22, 30, and 43. The colon at the end of the label name is mandatory for this kind of comment.

5.5. Special Files

Beside the files that contain the Java source code itself (with the extension `.java`), a project can contain several other files. Two (types) of these files are discussed here – interestingly, both have the `.java` extension without being really valid Java source files.

5.5.1. The Module Definition

Modules have been introduced to Java with version 9, as a result of the Jigsaw project[324]. A module will be defined in the file `module-info.java` that is located in the root of the source tree. Details can be found in [156].

Basically, a `module-info.java` file will look like this:

Listing 5.1: `module-info.java`

```
1  /*
2  * =====
3  * Copyright © <Year> <Copyright Notice>
4  * =====
5  *
6  * <License Notice>
7  */
8
9  /**
```



```
10  * <module description>
11  */
12  module <modulename>
13  {
14      requires <name_of_required_module_1>;
15      requires <name_of_required_module_2>;
16
17      requires transitive <name_of_required_module_3>;
18
19      exports <name_of_exported_package_1>;
20      exports <name_of_exported_package_2> to <name_of_target_module_1>;
21
22      opens <name_of_opened_package_1>;
23      opens <name_of_opened_package_1> to <name_of_target_module_1>,
24          <name_of_target_module_2>;
25
26      uses <name_of_provider_interface>;
27      provides <name_of_provider_interface> with
28          <name_of_provider_implementation>;
29  }
30  // module <modulename>
31  /*
32  * End of File
33  */
```

How to define the name of the module (<modulename>) is covered in chapter “6.3 Modules” on page 120.

The <module description> will be discussed in chapter “7.1.1 The ‘module’ Comment” on page 152.

5.5.2. The Package Documentation

Different from the module, there is no special source file that defines a package in Java. Instead all source files that are located in the same folder on the directory tree for the source code are in the same **package**.

Nevertheless there is a need for package-level documentation, and it is even possible to set annotations to packages.



For the documentation of a package, originally a file named `package.html` was used, placed into the package folder. Although this still works, you should use the new `package-info.java` file instead, because only this allows annotations on package level; it looks like this:

Listing 5.2: package-info.java

```
1  /*
2   * =====
3   *  Copyright © <Year> <Copyright Notice>
4   *  =====
5   *
6   *  <License Notice>
7   */
8
9  /**
10   *  <package description>
11   */
12  @<AnAnnotation>
13  package <packagename>;
14
15  import <AnAnnotation>
16
17  /*
18   *  End of File
19   */
```

The `<package description>` is a standard JavaDoc comment (refer to the chapter “7.1 Documentation Comments” on page 146) and can be as lengthy as necessary. It will be discussed in detail in chapter 7.1.1.

If the package is annotated¹¹, the annotations have to be placed directly before the `package ...` line, and the imports for the annotations follow below.

The chapter “6.4 Packages” on page 121 discusses how to define the name of the package (`<packagename>`).

¹¹I recommend to use the `@API` annotation even on the package level (refer to chapter “8.20 The Annotation `@API`” on page 400 for details).



5. *Statements*



6. Naming Conventions

While the primary goal of the formatting rules is to make it easier to *read* the source code, are the naming conventions targeted to make a program easier to *understand*.

Principles

- P1. The selected name for a program element has to be meaningful.

But “meaningful” is a relative term, depending from context and experience. So the abbreviation “TSU” for a class name is just a cryptic three-letter-acronym for someone in the banking business while for people from the warehouse & logistics domain no further explanation is required¹. So we want to enhance this principle as we say that the name of a program element has to be meaningful for as many people as possible – even for someone outside the current problem domain.

- P2. Names, in particular those for methods, provide an implicit contract² between the original author of a program and its users/maintainers.

This means that we expect that a method named `read()` will *read* something from somewhere. When it will perform a *write* operation instead, the implicit contract is broken. To fix that, at least an elaborate comment is required, explaining why that `read()` method *writes* in fact.

But because people may understand words differently, I have added a dictionary of common verbs and their implicit contracts to this document, together with a list of suffixes for class names. Refer to chapter

¹“TSU” stands for “Transport and Storage Unit” and is something like an abstract container of goods for a Warehouse Management System (WMS).

²If it is not a binding contract, so it is at least a kind of commitment.



“9.1 The Naming Dictionary” on page 449 in the appendices for those lists.

P3. The naming has to be consistent.

This is particularly important for the names of local variables and for the formal parameters of a method or a constructor; it means that the same thing should always get the same name.

P4. Correct spelling is not an unnecessary luxury.

Names should be spelled correctly, as typos are disturbing and interfere with the readability. Not to mention that they are quite often ugly.

P5. Upper and lower case also have a meaning.

Names for different program elements use upper and lower case characters in different ways.

6.1. General Accords

The details on how names and identifiers are composed in Java, and where they are used can be taken from the respective chapter of the Java Language Specification [153].

But however, names like `MyClass`, `mypackage` or `myMethod` may be syntactically correct and do look good in some sample code (like in the sample listings in this document ...), but they are definitely invalid for real code (even for quick-shot proof-of-concept code) – of course mainly because they are not meaningful! And the first rule about the naming of program elements is to use *meaningful names*.

6.1.1. Language

The identifier names have to be taken from English language, and with correct spelling. I am conscious about the fact that there are differences between British and American English (at least between these two ...) and



their spelling. “Correct spelling” means “correctly spelled according to at least one variant”. Decide for one spelling scheme and stuck with it. I personally use American English for the names. Consult a dictionary if in doubt of spelling and/or meaning for a word you want to use as a name.

The reason to use English is that this is the common language for most international teams.

But even if you are working in an environment where English is a foreign language, solely with people speaking all the local language, using English still has a big advantage: you can distinguish just by the language used if someone talks about the ‘real world’ object or its representation in the program.³

6.1.2. UPPERCASE, lowercase, CamelCase

Use *CamelCase*, starting with a capital letter, for the identifiers of all kinds of classes: **String**, **LocalDateTime**, **AccountOwner**, ...

For the identifiers of local variables, methods and formal parameters, use *camelCase* starting with a lowercase character: **i**, **proceed**, **isValid**, **lastOwner**, **indexOfLastRecord**, ...

Fields (attributes or properties) are a bit special⁴: their identifiers are prefixed with “m_”, followed by the concrete name in *CamelCase* (with a leading capital letter): **m_IsValid**, **m_Name**, **m_IndexOfLastRecord**, ...

For the identifiers of packages and modules, all lowercase is preferred, although this is not mandatory: **java.lang**, **org.tquadrat.foundation**, ... – Nevertheless, you should avoid a package name starting with a capital letter, as this could be mixed up easily with a class name.

³From a specification document: “Ein Konto wird repräsentiert durch ein Objekt der Klasse **Account**, der Kontoinhaber entsprechend durch ein Object der Klasse **AccountOwner**.” – If you do not understand German, the translation is like this: “An account will be represented through an object of the class **Account**, and the account owner by an object of the class **AccountOwner**, respectively.” Or short: an account is an **Account**, and an account owner is an **AccountOwner**; in writing, this looks good, but in oral communications, misunderstandings are easy.

⁴see chapter “6.10 Fields”



The Java coding convention [108] demands to use all uppercase for constants and separating words with underscores, and usually this is implicitly expanded to enum values, too⁵. I prefer a more relaxed approach for that – see chapters “6.11 Constants” on page 136 and “6.12 enum Values” on page 138. Samples for constants are: `EOF`, `EMPTY_STRING`, `XML_ATTRIBUTE_LastAccessed`, ...

Examples for an enum named `LightStatus` could be `LIGHT_STATUS_Unknown`, `LIGHT_STATUS_Off`, `LIGHT_STATUS_On` and so on.

Details are discussed in the chapters about the respective code elements.

6.1.3. Length of Names and Use of Abbreviations

In general, you should avoid abbreviations and acronyms as identifiers for program elements if those are not *very well known* – refer to P1.

The length of a name has very little to no influence on the size and speed of the compiled code. And with the code completion features provided by modern IDEs, the programmer may not even have to type long names ...

This means that you will never sacrifice the meaningfulness of a long name for typing speed – never ever!

If you think you will be more productive when using short names (because you can type them faster ...), you are wrong! When you have the feeling that you have to write code faster, learn typewriting⁶. It will also help you to write all the other stuff you have to deliver in addition to your code (documentation, meeting notes, emails, specification documents, ...).

But of course Java programs do have a maximum length for identifiers⁷, although it is higher than any practical use case. I think that a name for a

⁵The “Code Conventions for the Java™ Programming Language” [108] were last updated in 1999, enums had been introduced with Java 5, and that was released at 2004-09-29 ...

⁶For my understanding, someone who does not reach at least 100 CPM should look for a job outside of software development (average values for a trained user are around 200 CPM, peak values are at 500 to 900 CPM)

⁷The Java Language Specification does not impose such a limit: “An *identifier* is an unlimited-length sequence of *Java letters* and *Java digits*, the first of which must be a *Java letter*.”[150], but the compiled code may not exceed a given length.



module, package, class, method, field, constant, argument, or local variable that is longer than 100 or perhaps 150 characters is definitely too long to be useful.

6.1.4. 'Special' Characters

Java allows to compose an identifier from all Unicode letters and digits, plus the underscore “_” and the dollar symbol “\$”. This, too, is a reason for using English only for the names and identifiers: usually, you can limit yourself to the ASCII character set, and for the rest, avoid dollars and underscores in identifiers; the only exceptions are underscores for the names of constants and `enum` values (see the chapters “6.11 Constants” on page 136 and “6.12 enum Values” on page 138), for the “m_” prefix for field names (see chapter “6.10 Fields” on page 135) and a single dollar symbol (“\$”) in lambdas (see “8.36 Lambdas” on page 421).

6.1.5. 'test' as Part of Names

Avoid “test” as the part of the name of packages, classes, interfaces and methods that belong to the production code. For testers and for members of the test suite packages, it is allowed or even required, depending on the testing framework used. Also it may be recommended or even required if you write your own testing framework or extensions to an existing one.

6.1.6. Names forced by 3rd Party Components

Most Java programs do not exist in a vacuum, they have links to other components (like databases, messaging systems, or the real world). These components may have different naming conventions, whether due to technical or historical reasons, or just because the names are as they are. In such case, the foreign names should get hidden through aliases, like constants or getter methods, whose names are following the naming conventions imposed by this document. This is even made easier as those external names are usually used as Strings in the program.



Most Java libraries follow with their namings the naming conventions given in chapter 9 of the “Code Conventions for the Java™ Programming Language”[115], so usually you should not get in trouble here.

But if you encounter a library with significantly deviating namings, you should hide such a library behind a facade⁸.

6.2. Files

The names for the files containing Java source code are determined by the names of the (**public**) class such file defines: the file name has to be the name of that class with the extension `.java`. It is recommended to name the source code files for non-**public** (package-private) classes in the same way.

The path for the file is determined by the package the class resides in.

Some examples:

Java Class	Fully qualified Filename
<code>java.lang.String</code>	<code><srcroot>/java/lang/String.java</code>
<code>javax.xml.XMLConstants</code>	<code><srcroot>/javax/xml/XMLConstants.java</code>
<code>org.junit.Test</code>	<code><srcroot>/org/junit/Test.java</code>

For the naming of classes, refer to chapter 6.5, for the naming of packages, see chapter 6.4.

6.3. Modules

A module is defined in the file `module-info.java` that is located in the root of the source tree for that module; see chapter “5.5.1 The Module Definition” on page 111 and [156] for the details.

⁸Refer to [127]



Each module has a main package, and this should be also the name of that module. If the module will export any packages at all⁹, it has to export at least this main package.

So the names for modules will follow basically the same rules as those for packages.

6.4. Packages

According to the “Code Conventions for the Java™ Programming Language” [108], the prefix of a unique package name is always written in all-lower-case ASCII letters and should be one of the top-level domain names (TLDs) (like e.g. “com”, “net” or “org”)¹⁰, or one of the two-letter codes identifying countries as specified in ISO Standard 3166, 1981.[115]

Usually the second component is the company’s or organisation’s name as it appears in its website address, again in all lower case. This helps to ensure that packages will have a unique name across separate organisations.

Subsequent components of the package name vary according to an organization’s own internal naming conventions. Such conventions might specify that certain directory name components be division, department, project, machine, or login names. Here also only lower case should be used.

Valid samples are:

- `org.tquadrat`
- `com.ibm`
- `com.sun`
- `de.jug`
- `uk.gov.hmrc`
- `name.thrien.thomas`

⁹A module for a program is not required to export anything.

¹⁰There exist some more TLDs that are not country codes, but these are quite unlikely to be used in a package name. But of course, if you own a domain under one of these top-level domains, you can use its name as well.



6. Naming Conventions

Of course *you* should use *your* domain name! Those from the samples are already used by the respective companies or organisations (or persons).

Subsequent components of the package name vary according to an organisation's own internal naming conventions. Such conventions might specify that certain directory name components be division, department, project, machine, or login names. Here also only lower case should be used.

Next, a package name is also a folder name: the file with the source code for the class `com.foo.bar.internal.MyImpl` is located in the folder `com/foo/bar/internal` in the source code directory tree¹¹.

Finally the package name should reflect somehow the function of that package.

6.4.1. The Packages 'internal' and 'spi'

The modularisation that was introduced with Java 9 allows a much clearer separation of an API from its implementation. Let's assume that your module exports the package `com.foobar.library`, then this package contains the interfaces of the API, plus some public helper classes, while the package `com.foobar.library.internal` contains the implementation classes and internal helpers, and it is not exported.

If the API is extensible, an additional package `com.foobar.library.spi` contains the stuff that is needed to extend the API. It can be exported globally or only to some other named modules.

For the details, refer to chapter "8.35.1 Encapsulation with Modules" on page 420. The important message here is, that a package with name `internal` is never be exported, and that the exports for a package with name `spi` can be restricted to some named modules.

¹¹see also chapter 6.2 on page 120



6.5. Classes

In Java, classes, interfaces, enums, records and annotations are all “classes”, and the names for a class will be written in mixed case (also known as “camel case”) with the first letter capitalized (**MyClass** instead of **myClass**)¹². Try to keep your class names simple and descriptive. Use whole words, mainly nouns, and avoid acronyms and abbreviations, unless the abbreviation is much more widely used than the long form, such as URL or HTML.

According to the Java syntax specification, several special characters are allowed also for class names, but this coding convention forbids them – even the underscore (“_”)¹³

You should also avoid numerical characters in class names.

6.5.1. Names for ‘real’ Classes

The term ‘real’ class means that this Java class is declared with the keyword **class**. Records and enum classes are also Java classes, to some extent even interfaces and annotations, but you use a different keyword when you declare them.

Some parts of a class name will indicate a special position of that class in the class hierarchy: the suffix ‘Impl’ for the class name **FooImpl** shows that the class is the default implementation of an interface or an **abstract** class named **Foo**, either being the only one, or providing useful default implementations of the interface methods¹⁴.

‘Adapter’ as the suffix is similar, but here the methods are usually empty, as it is assumed that only a some of the methods of the interface are used by a particular implementation; various samples for this could be found in the

¹²refer to chapter 6.1.2 on page 117

¹³Especially the dollar sign (“\$”) can cause serious issues as this is used internally for the names of inner classes and anonymous classes, as well as for the classes that will be generated for lambdas.

¹⁴Although in the latter case, the name is more likely to be **FooBase**.



Swing packages. According to this coding conventions, an **Adapter** class is always **abstract** even when it does not contain any **abstract** methods.

A suffix 'Base' indicates that the class has to be extended; usually that class is **abstract**. It may or may not implement a specific interface.

Other suffixes indicate special usage of the class. Well known and obvious are the suffixes 'Error' and 'Exception' for error and exception classes. Others are 'Visitor' and 'Listener' for the implementations of the respective patterns, 'DAO', or 'Entity'. A more complete list can be found in chapter "9.1.2 Suffixes for Class Names" on page 462 in the appendices. Most of these suffixes are defined by the design patterns that are implemented by the respective classes.

6.5.2. Names for Interfaces

An interface declares the public API for an object that represents a *real world entity*, and it should be named according to that entity.

According to the Java Language Specification[152], the name of an interface should be basically a descriptive noun or noun phrase, which is appropriate when an interface is used as if it is an abstract superclass, such as the interfaces `java.io.DataInput` and `java.io.DataOutput`; or it may be an adjective describing a behavior, as for the interfaces `java.lang.Runnable`[188] and `java.lang.Cloneable`[184].

Some coding conventions (particularly in the Windows realm) require the prefix 'I' for interfaces, but it does not make much sense to mark interfaces in such way. Therefore it is not recommended to use such a prefix for the names of interfaces.

6.5.3. Names for enum Classes

An enum is a special class that will be declared with the keyword **enum**.



The name of enum class should be somehow the category of the enumerated entities, but plurals should be avoided; so it should read **Type** and not **Types**, and **DayOfWeek** and not **DaysOfWeek**.

Although an enum class can implement an interface, this should not be reflected in the name. This means that **TypeImpl** would be an invalid name for an enum class.

For the naming of the enum values refer to chapter “6.12 enum Values”.

6.5.4. Names for Record Classes

Basically, a record class is like a ‘real’ Java class, and should be named in the same ways. So it can implement an interface, like a regular class, meaning that the suffix ‘Impl’ would be valid in such case. But as record classes are implicitly **final**, ‘Base’ is not permitted, as well as any other suffix indicating a class that should or can be extended.

Also a record class should not be named ‘Record’.

6.5.5. Names for Annotations

Annotations are a special type of an interface, distinguished by the “@” as the first character of their name. The remaining part of the name is determined by the rules for regular interfaces. Samples for valid names are **@Text**, **@Translation**, **@Generated**¹⁵, or **@Deprecated**¹⁶.

6.6. Constructors

The name of a constructor is the name of the class it belongs to.

¹⁵see [12]

¹⁶see [8]



6.7. Methods

A method name has to be a verb plus one or more nouns indicating the object of the operation performed by the respective method, eventually combined with adjectives or additional verbs. The name will be in mixed or camel case with the first letter in lower case, and with the first letter of each internal word capitalized¹⁷.

The name of a method needs to describe what that method does; that is that contract between developer and maintainer mentioned in P2.

That means that names like `myMethod()` or `doSomething()` are syntactically correct, but are discouraged or even invalid¹⁸.

Chapter 9.1 beginning on page 449 contains “The Naming Dictionary” for method names; that is a list of verbs together with their implicit contract. It is highly recommended to consult this list when searching a name for a new method.

Using digits in method names is acceptable were it makes sense; so using the digit ‘4’ as replacement for the word “for”, and ‘2’ for the word “to” is acceptable, but not really recommended. If you have different versions of a method, you should not number them (`toString1()`, `toString2()`, ...), but instead describe the difference. Remember that digits are not allowed as first characters of a method name.

Regarding to “Data Objects” there are two different schools: one supports the JavaBean¹⁹ approach, using explicit getter and setter methods:

Listing 6.1: JavaBean

```
1 class JavaBeanD0
2 {
3     private Object m_Attribute;
4
5     public final Object getAttribute() { return m_Attribute; }
```

¹⁷see chapter 6.1.2 on page 117

¹⁸Nevertheless, both are used throughout this document as sample names for methods, but just as a *place holder for a meaningful method name*, and definitely *not* as an example for the naming of a method.

¹⁹see [227]



```
6
7 public final void setAttribute( final Object value )
8 {
9     if( isNull( value ) )
10    {
11        throw new NullPointerException( "value" );
12    }
13    m_Attribute = value;
14 } // setAttribute()
15 }
16 // class JavaBeanD0
```

The other one is in favour of the Property concept, using two methods with the attribute name, but with different signatures as implicit getter and setter methods (here usually named as 'accessor' and 'mutator'):²⁰

Listing 6.2: POJO

```
1 public class PropertyD0
2 {
3     private Object m_Attribute;
4
5     public final Object attribute() { return m_Attribute; }
6
7     public final void attribute( final Object value )
8     {
9         if( isNull( value ) )
10        {
11            throw new NullPointerException( "value" );
12        }
13        m_Attribute = value;
14    } // attribute()
15 }
16 // class PropertyD0
```

Both approaches do have their merits and pitfalls, and at the end of days both will do the job. Even in the Java API itself you may find samples for both approaches.

²⁰I use the term *POJO* ("Plain Old Java Object") here only to distinguish this approach from a *JavaBean*; usually, *POJOs* do not make any assumption about the naming of their methods. This means that a real *POJO* can have getters and setters, too.



In the Java world, the JavaBean concept is more widely accepted, and it is better supported with tools, but if seen over all languages, the property concept is more popular.

6.8. Local Variables

The identifiers for local variables have to be in mixed case, with a lowercase first letter. Internal words start with capital letters²¹. The names should contain neither the underscore (“_”) nor the dollar sign (“\$”) characters, even though both are allowed by the language specification; they also should not start with those characters. Digits are forbidden as first characters already by the language specification, but perfectly allowed as additional characters. But they should be used with care and only where necessary.

Variable names should be short yet meaningful – with a clear priority on being meaningful. The choice of a variable name should be mnemonic – that is, designed to indicate the intent of its use to the casual observer. As for class names, abbreviations and acronyms should be avoided. Using the digit ‘4’ as replacement for the word “for”, and ‘2’ for the word “to” is acceptable, yet not really recommended.

In case the first part of the variable name has to be an acronym, it will be written with all lower case:

```
String htmlHeader; // OK
```

instead of

```
String hTMLHeader; // AVOID!
```

Single-letter variable names should be avoided except for temporary “throw-away” variables. Some of these are widely used and have already a fixed meaning; so is **i** very common for the run value of a classical **for** loop:

²¹see chapter 6.1.2 on page 117



```
for( var i = 0; i < max; ++i )
{
    ...
}
```

e is the common name for the exception in a **catch** block:

```
try
{
    ...
}
catch( final IOException e )
{
    //---* Handle the exception *-----
    ...
}
```

For temporary strings, **s** is common, as **o** is for (temporary) objects of an unspecified type.

Some names are fixed: the name of the value that is returned from a method has to be **retValue** if the method does not return a field or attribute or **this**:

```
// OK
public final int getValue() { return m_Value; }

// AVOID!! Use 'retValue' here instead of 'i'!!
public final int signum( double d )
{
    final var i = (d < 0.0) ? -1 : ((d > 0.0) ? 1 : 0);

    //---* Done *-----
    return i;
} // signum()

// ACCEPTABLE (BARELY)
public final int signum( double d )
{
    return (d < 0.0) ? -1 : ((d > 0.0) ? 1 : 0);
} // signum()

// RECOMMENDED
public final int signum( double d )
{
```



6. Naming Conventions

```
final var retValue = 0;
if( d < 0.0 )
{
    retValue = -1
}
else
{
    retValue = 1;
}

//---* Done *-----
return retValue;
} // signum()

public final int signum( double d )
{
    final var retValue = (d < 0.0) ? -1 : ((d > 0.0) ? 1 : 0);

    //---* Done *-----
    return retValue;
} // signum()
```

In the same way, **result** is reserved as the return value for lambdas (see chapter “8.15.1 Lambda Results” on page 394) and for the results of **case** blocks (see chapter “8.15.2 ‘case’ Results” on page 396). The name **result** should not be used for a local variable in any other context.

A temporary buffer is named **buffer**, no matter if it is a **java.lang.StringBuilder** or **java.lang.StringBuffer**, an implementation of **java.util.List**, another collection type, or any other type that can be used as a buffer.

Do not use the names **in**, **out**, and **err** for your local variables. Although these names are not reserved words, they should be treated as such, as that allows to import the default streams from the class **java.lang.System**[69] statically.

Usually you would use the singular form for a variable name, but for collections or arrays, the plural form can be perfectly correct:

```
final Collection<Component> values = m_Components.values();
for( final Component value : values )
{
    ...
}
```



Using the class name (with a lower case first letter, of course) for a variable name is completely acceptable in case this is sufficient to explain the function of the variable; but the prefix “my” or “the” has to be avoided.

```
// RECOMMENDED
public final String retrieveMessage( final String messageKey,
    final String... additionalInfo )
{
    final MessageProvider messageProvider = getMessageProvider();
    final String rawMessage =
        messageProvider.getMessage( messageKey );
    final var retValue = String.format( rawMessage, additionalInfo );

    //---* Done *-----
    return retValue;
} // retrieveMessage()

// AVOID!!!! Don't use "my" or "the" prefix!!!
public final String retrieveMessage( final String string,
    final String... strings )
{
    final MessageProvider theMessageProvider = getMessageProvider();
    final String myString = theMessageProvider.getMessage( string );
    final var retValue = String.format( myString, strings );

    //---* Done *-----
    return retValue;
} // retrieveMessage()
```

Sometimes it is also a good idea to use the class name of the type as a suffix or prefix for the name, especially if the current block declares more than one variable of that type. In such a case, the rest of the name could be used to indicate how variables belong together.

```
// RECOMMENDED
public final ResultData callSpecialService()
{
    SpecialClientAgent clientAgent = obtainClientAgent();
    ...
} // callSpecialService()

// AVOID!!!! Don't use "my..." prefix!!! And don't abbreviate!
public final ResultData callSpecialService()
{
```



6. Naming Conventions

```
    SpecialClientAgent myCA = obtainClientAgent();
    ...
}    // callSpecialService()

// RECOMMENDED
public final ResultData loadData( Connection connection, ... )
{
    ...
    final Statement customerStatement = ...
    final Statement orderStatement = ...

    ...

    final ResultSet customerResultSet = customerStatement.execute();
    ...

    final ResultSet orderResultSet = orderStatement.execute();
    ...
}    // loadData()
```

Do not (never!) use a prefix to indicate the type of a variable (Hungarian Notation[173, 213])! Java is a strongly typed language and the compiler will take care that variables are only used according to their types.²² The use of the **var** keyword²³ in Java programs does not weaken the strong typing of Java in any way – refer to chapter “8.8 Local Variable Type Inference” on page 321.

Local variables inside lambdas are just like any other local variable and their naming follows the same rules. The only exception is that a lambda does not use **returnValue** for a return value, but **result** – refer to chapter “8.15.1 Lambda Results” on page 394 for the details.

²²The use of the Hungarian Notation – in particular the ‘Systems Hungarian’ variant – for the naming of variables is discouraged for strongly typed languages like Java or C++, because it does not provide any benefit.

For languages like PHP, C, JavaScript or also Groovy, this may be different: for these languages the type prefix may help the programmers to assign the semantically correct type to a variable when syntactically any (or most) types are correct.

²³**var** is in fact not a keyword, but a special type.



6.9. Parameters

The identifiers for the formal parameters of methods and constructors are different from those for lambdas.

6.9.1. Names for formal Parameters of Methods and Constructors

The names for the formal parameters of a method or a constructors follow the same rules as those for local variables. Especially they do *not* have any prefix.

The parameter that is used to initialise a field should have the same name as that field, but without the “m_” prefix:

```
public final class MyClass
{
    private String m_Name;

    public MyClass( final String name )
    {
        m_Name = name;
    }    // MyClass()

    public final void setName( final String name )
    {
        m_Name = name;
    }    // setName()
}
// class MyClass
```

6.9.2. Names for Lambda Parameters

A typical lambda in Java looks like this:

```
Predicate<String> filter = s -> !s.contains( "invalid" );
```



6. Naming Conventions

`s` is the parameter of that lambda, and that these parameters have just single letter names is common practice. As lambdas are usually quite short, this works sufficiently in most cases. In addition the meaning of the parameters is well explained through the documentation of the functional interface that is implemented by the lambda.

But if in doubt, you can use longer names with more meaningful names for the parameters of a lambda. The only constraint is that it should be clear what the parameter is, and what is injected to the lambda from the context:

```
// WILL NOT WORK AT ALL!!
final var s = "invalid";
Predicate<String> filter = s -> !s.contains( s ); // Hä?

// Still not good...
final var s = "invalid";
Predicate<String> filter = p -> !p.contains( s );

// Better...
final var s = "invalid";
Predicate<String> filter = toCheck -> !toCheck.contains( s );

// Recommended
final var criterion = "invalid";
Predicate<String> filter = s -> !s.contains( criterion );
```

It does not make much difference if you number the parameters, or if you use different names for them:

```
BiPredicate<String,String> filter = (s1,s2) -> s1.contains( s2 );
BiPredicate<String,String> filter = (a,b) -> a.contains( b );
```

Nevertheless, an accepted rule of thumb is to use different names if the parameters cannot be exchanged by each other (as in the sample above), while numbering the parameters is used when the parameters are interchangeable, like in an implementation of `java.util.Comparator`^[195]:

```
Comparator<String> comparator = (s1,s2) -> s1.compareToIgnoreCase( s2 );
```



6.10. Fields

The identifier for a field (or for an ‘attribute’ or a ‘property’) has to be prefixed with “m_”; the first letter after the underscore has to be a capital letter:

```
private String m_AString;  
private boolean m_IsValid;  
private String m_HTMLHeader;  
private MessageProvider m_MessageProvider;
```

This ensures that the names of local variables and formal parameters of methods or constructors will be always distinct from those of fields, and therefore neither local variables nor parameters can collide with or hide fields. Another consequence is that it is not necessary to use `this` when accessing a field.

Aside this, anything else that was said about the naming of local variables in chapter 6.8 is also valid for the naming of fields.

Please be aware that a bean property name does not carry the prefix. So the getters for the samples above are

```
1 public final String getAString() { return m_AString; }  
2 public final boolean getIsValid() { return m_IsValid; }  
3 /* Alternatively: */ public final boolean isValid() { return m_IsValid; }  
4 public final String getHTMLHeader() { return m_HTMLHeader; }  
5 public final MessageProvider getMessageProvider() { return  
    m_MessageProvider; }
```

Regarding the alternative getter in line 3 refer to [229].

Eclipse knows a configuration setting for this prefix (Window|Preferences|Java|Code Style). Setting the prefix list there to “m_” makes sure that the generation of getters and setters will work as expected.

For IntelliJ IDEA, a similar setting can be found at File|Settings, where you select under Editor|Code Style|Java the tab Code Generation.



6.11. Constants

Constants in Java are fields of primitive or immutable types that are declared as `public static final`. According to the Sun coding conventions, their names “should be all uppercase with words separated by underscores (‘_’)”[115].

Our definition for a constant is even more restrictive: that `public static final` field has to be initialised with a compile-time constant value. This basically limits constants to “magic numbers”, static configuration values and texts.

Some samples for valid constants:

```
public static final double PI = 3.1415;
public static final int ANSWER_TO_ALL_QUESTIONS = 42;
public static final String ISO_DATE_FORMAT = "yyyyMMdd'T'hhmmss";
public static final String [] EMPTY_STRING_ARRAY = new String [0];
```

Real constants go to the part of the class headlined with “Constants” (refer to chapter 7.2.1 on page 170).

The field `PATTERN` below is not a constant in the sense of this definition, and therefore it goes to the “Static Initialisations” part:

```
1  /*-----*\
2  ===** Static Initialisations **=====
3  \*-----*/
4  public static final Pattern PATTERN = Pattern.compile( ".*" );
5
6  // Better:
7  /*-----*\
8  ===** Static Initialisations **=====
9  \*-----*/
10 public static final Pattern PATTERN;
11
12 static
13 {
14     try
15     {
16         PATTERN = Pattern.compile( ".*" );
17     }
18     catch( final PatternSyntaxException e )
19     {
```




```

20         throw new ExceptionInInitializerError( e );
21     }
22 }
23
24 // Even better:
25 /*-----*\
26 ===** Constants **=====
27 \*-----*/
28 public static final String PATTERN_SOURCE = ".*";
29 ...
30
31 /*-----*\
32 ===** Static Initialisations **=====
33 \*-----*/
34 private static final Pattern m_Pattern;
35
36 static
37 {
38     try
39     {
40         m_Pattern = Pattern.compile( PATTERN_SOURCE );
41     }
42     catch( final PatternSyntaxException e )
43     {
44         throw new ExceptionInInitializerError( e );
45     }
46 }

```

No matter which variant you go for, the declaration and initialisation have to go to the part “Static Initialisations”.

static final references to *mutable* objects are no constants (no matter which definition you use), therefore they should never be **public** – with the final consequence that they are just fields – so refer to chapter 6.10 for their naming.

Samples for fields that are not feasible as **public** constants are (as the objects are not constant at all):

```

1 // AVOID!! Make it private!
2 public static final Map<String,File> m_Files = new TreeMap<String,File>();
3
4 // AVOID!! Make it private!
5 public static final Date BEGIN_OF_EPOCHE = new Date( 0 );

```



```
6
7 // AVOID!! Make it private!
8 public static final SimpleDateFormat ISO_DATE_FORMATTER = new
    SimpleDateFormat( ISO_DATE_FORMAT );
```

If you wonder why the lines 5 and 8 belong to this group of bad samples, refer to [86, 80]: instances of `java.util.Date` and `java.text.SimpleDateFormat` are not immutable!²⁴

In case you have a constant value (a magic number, a message string or alike) that should not be `public` for some reason, you can name that either as a regular field (with the “m_” prefix) or as a regular constant as described above. For a local constant (an alias for a magic number or a message string ...), use the naming pattern for constants.

Sometimes you may have classes, groups, families or categories of constants (e.g. the tags in an XML document, the column names of a database table, or message texts). It has proved that it supports the readability of the code if this is reflected in the names for those constants. For the column names of the table `PERSON`, the constants may be defined as shown in this example:

```
public final static String PERSON_COL_FirstName = "first_name";
public final static String PERSON_COL_LastName = "last_name";
public final static String PERSON_COL_MaidenName = "maiden_name";
public final static String PERSON_COL_Title = "title";
...
```

The camelCased suffix is the ‘real’ name, while the part that follows the stronger rules for a constant name is just indicating the “constant category”, `PERSON_COL` in this case.

6.12. enum Values

When the “Code Conventions for the Java™ Programming Language”[108] had been written in 1999, enums did not yet exist in Java. But because

²⁴In addition, the class `java.util.Date` and the tools around it were superseded by the `java.time` package (see [318]) and should not be used in new software anymore.



enum values are special cases of constants, the usual assumption is that their names have to follow the same rules as for normal constants: the name should be all upper case with words separated by underscores ("_").

Basically, this works fine, but when using `static` imports, you may get name clashes for common terms that can be used in different contexts. So it will make sense to add a prefix to the name of an enum, similar to what we did with the categories or groups for regular constants (see above). This may look like this:

```
enum Direction
{
    DIR_LEFT,
    DIR_RIGHT;
} // enum Direction

enum Alignment
{
    ALIGN_CENTER,
    ALIGN_LEFT,
    ALIGN_RIGHT;
} // enum Alignment
```

Using the prefix would allow now to use the values with static imports of their classes; otherwise, they have to be used with their class names as prefix.

Alternatively, you can use this:

```
enum Direction
{
    DIR_Left,
    DIR_Right;
} // enum Direction

enum Alignment
{
    ALIGN_Center,
    ALIGN_Left,
    ALIGN_Right;
} // enum Alignment
```



Again the camelCased suffix is the “name” itself, while the “proper” typed prefix is just the category. This is the recommended approach if the suffix consists of multiple words that otherwise require additional underscores.

6.13. Type Arguments

Type arguments belong to the definition of parameterised types [151] or “generics”. So in

```
public interface Function<T,R> { ... }
```

T and R are the type arguments.

It is common practice to name a single type argument as **T**. If there are more than one type arguments, the name **R** is usually used for the return type. Also often used are **V** as the name for a value type and **K** for the name of the type for a key, **E** as the name for the type of an entry to a list or set or for an exception type.

Aside that, no further rules exist. Even more than one character is possible for the name of a type argument:

```
public interface Map<Key,Value> { ... }
```

is absolutely valid, although not common.

Even numbers as part of the name are possible and sometimes helpful:

```
// OK
public interface Operation<T,U,V,R> { ... }

// BETTER in this case
public interface Operation<T1,T2,T3,R> { ... }
```

But because the type arguments denotes classes, their names have to start with a capital letter.



6.14. Projects

You can name your project however you want, although some IDEs have some constraints about the exact format.

I recommend to name the project after the module.

6.15. Library Files

Usually, the library files of a project are named after the project, appended by a version number and/or a release date. They have to be valid file names.



6. *Naming Conventions*



7. Writing proper Comments

Comments are crucial for the understanding of source code, in any programming language. Source code without any comments is not maintainable, meaning it is worthless in the long run.

Principles

P1. Comments have to support the readability of source code.

They do this by structuring the source code into sections and by providing hints on how to interpret the code.

P2. Comments should be used to give an overview on the code and to provide additional information that is not readily available in the code itself.

They should contain only information that is relevant for reading and understanding the program. For example, information about how the corresponding package is built or in what directory it resides should not be included as a comment to a class.

The discussion of non-trivial or non-obvious design decisions is appropriate, but the duplication of information that is present in (and clear from) the code must be avoided. It is too easy for redundant comments to get out of date. In general, any comments that are likely to get out of date as the code evolves should be avoided.

P3. For Java source code, comments are part of or the source for the external documentation of that code.



7. Writing proper Comments

The Java Development Kit (JDK) provides a tool that allows to externalize program comments, so that they can be used as the external documentation; the name of this tool is *JavaDoc*. The “Javadoc Guide”[230] provides an overview of the tool¹, the “Documentation Comment Specification for the Standard Doclet”[121] explains how to write the comments for a proper documentation generated with the JavaDoc tool.²

P4. The frequency of comments reflects poor quality of code.

Unfortunately, code can be “under-commented” or “over-commented”, meaning there is a “frequency band” for comments that has to be hit for good quality code.

One often heard advice is: “When you feel compelled to add a comment, consider rewriting the code to make it clearer.” But clearer to whom?

With increasing programming experience, things get more and more obvious to the programmers, so they write lesser comments – with the result, that newbies do not have any help to understand the code written by the experts.

Obviously, the advice: “Even if you don’t think, a comment might be necessary, add it nevertheless” is the other extrema – and equally bad! This means that writing proper comments remains a complex art, but it follows some rules, and for the rest, this document will give some advice: refer to chapter “7.6 Comments when?” on page 185 where this is discussed further.

P5. Comments should be in full sentence and using a clear language.

All comments has to be in English language; they should be grammatically correct and without typos.³ This improves readability and helps to avoid misunderstandings.

¹see [334] on how to invoke the tool on your source code

²JavaDoc is not the only tool for this purpose; another well known tool is Doxygen[170] that was created primarily to generate the documentation for annotated C++ code, but it works also for Java, C and several other programming languages. But for Java sources, JavaDoc is the preferred tool.

³... but it is still much more important that there is at least *some* comment than a correctly spelled one.



Java source code can generally have three kinds of comments:

- documentation comments
- implementation comments
- maintenance comments

In Java, the *documentation comments* (also known as “doc comments” or “the JavaDoc”) are delimited by “`/**...*/`” and cannot be placed everywhere; they will be externalised for the generation of the program/library documentation by the JavaDoc tool. Obviously, *implementation comments* are the other comments, that are not externalized and published.

Roughly, the documentation comments describe how to use the code (the API), unrelated to the implementation, while the implementation comments describe what the code is doing and why.

Maintenance comments are technically a special form of implementation comments, but as they have a special function, they are covered separately in chapter “7.3 Maintenance Comments” on page 181.

Some general rules are:

- Comments should not be enclosed in large boxes drawn with asterisks or other characters, with the exception of structuring comments as described below.
- Comments should never include special characters such as tabulator, form-feed and backspace or alike. In JavaDoc comments only (see below), most other non-ASCII characters should be escaped with their HTML equivalent.
- While code lines can have any length, comment lines will always end in or before column 80, except when their contents cannot be wrapped (like URLs for references to additional information). See also chapter “2.3 Line Length” on page 47.



7.1. Documentation Comments

It is a well known fact that most programmers are poor technical writers. That's one of the reasons why programmers rarely write the public documentation for their product.

But that is no excuse why programmers do not write proper documentation comments into their source code. They are the only people that could write these comments because technical writers usually do neither have the time nor the required skills to analyse the code to extract the information from it that is necessary for the documentation.⁴

In general there are (at least) two different target groups for the documentation that is generated from the documentation comments. The first group are the maintenance programmers, the second are programmers writing code interfacing with this one, using the public APIs. This means that each program element that can have a documentation comment must have a documentation comment! No exception! No excuse for missing documentation comments! The programmer must provide a documentation comment wherever it is possible.

Eclipse can be configured in a way that it will issue warnings or even errors for missing documentation comments: see `Window|Preferences|Java|Compiler|Java`

To achieve the same for IntelliJ IDEA, you go to `File|Settings`, and there you select `Editor|Inspections>Java>Javadoc`.

Documentation comments describe Java modules, packages, classes⁵, constructors, methods, and fields⁶. Each documentation comment is set inside the comment delimiters `/**...*/`, with one comment per module, package⁷, class, or member. This comment has to appear just before the declaration:

⁴In addition, the technical writers often do not have (write) access to the source code and are therefore not able to add the documentation comments.

⁵All types of *classes*, including *interfaces*, *enums*, *records* and *annotations*

⁶All types of fields: *attributes*, *constants*, but also the *enum values*

⁷The documentation comment for a Java package is special; refer to chapter "5.5.2 The Package Documentation" on page 112 for the details



```
/**
 * The {@code Example} class provides ...
 */
public class Example
{
    /**
     * The inner class provides ...
     */
    private static class InnerClass
    {
        ...
    }
    // class InnerClass

    ...

    /**
     * This flag ...
     */
    private boolean m_Flag;
    ...

    /**
     * Method that performs some action ...
     *
     * @param arg The argument.
     */
    public final void method( int arg )
    {
        ...
    } // main()
} // class Example
```

The first line of the documentation comments (“/**”) for top level classes is not indented; subsequent lines for the documentation comment have one space of indentation (to vertically align the asterisks). All members, including inner classes, have 4 spaces for the first documentation comment line and 5 spaces thereafter (this is congruent for inner classes).

If you need to give information about a class, interface, variable, or method that isn’t appropriate for the public documentation, use an implementation comment immediately after the declaration. For example, internal details



7. Writing proper Comments

about the implementation of a class should go in such an implementation block comment following the class statement, not in the class documentation comment.

Documentation comments should not be positioned inside a method or constructor definition block, because Java associates documentation comments with the first declaration after the comment.

7.1.1. Structure and Contents

Latest since the introduction of Java 9, the JavaDoc tool produces (more or less) correct HTML 5 documents from the JavaDoc comments in the source code. Therefore it is strongly recommended to use correct HTML 5 syntax inside the documentation comments itself. This means that tags has to be closed properly, empty tags like `<br>` and `<img>` are not closed, and so on.

If the comment has more than one single paragraph, use the `<p>` tag; do not use the `<br>` tag:

```
/**
 * Returns the status for this operation.
 *
 * ...
 */

/**
 * <p>Returns the status for this operation.</p>
 * <p>Possible return values are ..</p>
 *
 * ...
 */
```

The first sentence of each JavaDoc comment is taken to be placed on an overview. Per default, that sentence is defined as everything from the beginning until the first full stop followed by a blank (" . ") or other whitespace, or the first not-inline HTML tag.

This means that a comment like this



```
// AVOID!!  
/**  
 * <b>Returns the status for this operation.</b>  
 *  
 * ...  
 */
```

may cause some strange output (writing the comment like that - with the `<b>...</b>` tag - should be avoided anyway). Java 10 introduced the JavaDoc tag `@summary` to address issues like this; it allows the programmer to explicitly specify what portion of the JavaDoc comment appears in the overview rather than relying on JavaDoc's default behaviour to determine the summary portion of the comment. Refer to [121, 249] for the details and additional samples.

The `@summary` tag has to be used always when a documentation comment has more than one sentence:

```
// OK -- as single sentence  
/**  
 * Returns the status for this operation.  
 *  
 * ...  
 */  
  
// AVOID!! -- Two sentences.  
/**  
 * <p>Returns the status for this operation. The return value will  
 * never be {@code null}</p>  
 * <p>Possible return values are ...</p>  
 *  
 * ...  
 */  
  
/**  
 * <p>Returns the status for this operation.</p>  
 * <p>Possible return values are ...</p>  
 *  
 * ...  
 */  
  
// RECOMMENDED  
/**
```



7. Writing proper Comments

```
* <p>{@summary Returns the status for this operation.} The return
* value will never be {@code null}</p>
* <p>Possible return values are ...</p>
*
* ...
*/

/**
 * <p>{@summary Returns the status for this operation.}</p>
 * <p>Possible return values are ...</p>
 *
 * ...
 */
```

The documentation comments for modules, packages and classes may get longer, so that you want to structure it by giving headlines to sections. Usually this is done through the HTML tag `<h#>`, with `#` being a number in the range from 1 to 6. You can use these tags in the documentation comments, too, but the tags `<h1>` and `<h2>` are already used by Javadoc itself; that means that you should only use `<h3>` to `<h6>` in your documentation comments. Only in the overview comment (refer to chapter “7.1.1 The Overview Comment” on page 151), you can make use of the all the `<h#>` tags to structure that comment.

When a class, method, constant, field is mentioned the first time in a documentation comment, the documentation for that element should be linked, using the `@link` or `@linkplain` tags. This is not necessary if the type is used for a formal parameter or the return value. In these cases, Javadoc generates these links automatically.

Each `@link` or `@linkplain` tag has to be placed into a line of its own.

Some examples:

```
/**
 * <p>{@summary Searches the given key in the list and returns the
 * associated data.} If the key is
 * {@linkplain String#isBlank() blank},
 * the method will throw a
 * {@link BlankArgumentException},
 * while an empty will just not return a result.</p>
 *
```



```
* @param key The key.  
* @returns An instance of  
*      {@link Optional}  
*      that holds the search result.  
* @throws IllegalArgumentException The key is somehow invalid.  
*  
public final Optional<Data> searchData( final String key ) { ... }
```

JavaDoc creates links to the documentation of `java.lang.IllegalArgumentException`, `java.lang.String` and `Data` automatically; it also creates a link to `java.util.Optional` but it is recommended to use the pattern shown here, even when this means that the comment holds two links to the `Optional` class documentation.

The names of classes, methods, constant, fields, parameters etc. as well as `null`, `true`, and `false` have to be written in a monotype font. This can be achieved by encapsulating them in `<code>...</code>` HTML tags or placing them inside the JavaDoc `@code` tag. The monotype font is used automatically for everything inside a `@link` tag.

Each comment, including each text for a `@param`, `@return`, and `@throws` tag, ends with a full stop.

The document “How to Write Doc Comments for the Javadoc Tool”[172] is already a little bit older and therefore outdated in parts, but it still provides some useful hints on how to write proper documentation comments that should be processed by the JavaDoc tool.

The Overview Comment

When the JavaDoc tool is called with the option `-overview <filename>` (see [335]), an ‘Overview’ comment is added to the generated documentation. `<filename>` is the (fully-qualified) filename of a valid HTML 5 document (The recommended name is `overview.html`) containing general information about the project.

You can put nearly everything here, from the project’s history to manual on how to use the program or library, but you should not reproduce information that is given in the module, package, or class documentation comments.



Several JavaDoc tags can be also used in the overview comment; for details refer to the chapter “Where Tags Can Be Used” in [121].

The ‘module’ Comment

The `<module description>` (refer to chapter “5.5.1 The Module Definition”) describes the current module, its dependencies and what it provides. See the JavaDoc tags `@provides` and `@uses` for details.

If the project has just one module, the module comment can replace the overview comment.

The ‘package’ Comment

Each and every Java package has to have a file named `package-info.java`; the structure of that file was already discussed in chapter “5.5.2 The Package Documentation” on page 112.

The `<package description>` provides information about the package. So it describes the purpose of the classes in this package. It lists conventions that are common for all contained classes, it should specify their prerequisites.

If the package defines a single API, it can describe the usage of that API, too.

The package comment can list the authors of the code, using the `@author` tag⁸, the version with the `@version` tag, when the package was created or with which version it was integrated with the `@since` tag, and other things.

It should not repeat details that are written already in the documentation of a class in that package, instead it should reference that class documentation. If those details are important for the whole package, it should be considered to move them from the class comment to the package description and place a reference into the class documentation instead.

⁸Or the tag `@extauthor`, refer to chapter “7.1.2 Custom Tags for JavaDoc” on page 168)



The package comment for the main package of a project can replace the overview comment (if not the module comment is used for this⁹).

The ‘class’ Comment

The documentation comment for a class, an interface, an enum, a record or an annotation (the ‘class comment’) describes that class, its purpose and its usage. If the class is not `final`, the comment should provide some hints what the mount points¹⁰ are and how to utilise them.

Then the class comment should list the authors of the class, using the `@author`¹¹ tag, the class version (using the `@version` tag), and when the class was added to the project with the `@since` tag, although this can be omitted if this information is already given within the package.

In case of a parametrised type (a ‘generic’), it contains a `@param` tag for each formal parameter.

Here a real life sample for an interface:

```
/**
 * <p>{@summary This is the basic interface for any kind of DAO
 * (Data Access Object).} It is based on sample code from the book
 * &quot;Java Persistence with Hibernate&quot;;.</p>
 *
 * @param <T> The entity type for the DAO.
 * @param <I> The type of the entity id.
 *
 * @author Thomas Thrien - thomas.thrien@tquadrat.org
 * @version <version information>
 * @since 1.2.3
 */
public interface GenericDAO<T,I>
{
    ...
}
```

⁹Not all projects will produce modules, so it is possible that your project does not have a module definition file at all.

¹⁰Another term for “mount point” is “extension point”, but I do not like this expression as we do not always “extend” a class on these points. Most often we replace existing behaviour to customise the class to our needs.

¹¹Or the tag `@extauthor`, refer to chapter “7.1.2 Custom Tags for JavaDoc” on page 168)



7. Writing proper Comments

```
} // interface GenericDAO
```

The `<version information>` should be a reference to the version in the SCCS; if you are using Subversion, that line would look like this:

```
/**
 * ...
 * @version $Id:$
 * ...
 */
```

I also recommend to use the custom tags provided by the “Foundation JavaDoc” project[358] (see chapter “7.1.2 Custom Tags for JavaDoc” on page 168); then the same class documentation comment would look this:

```
/**
 * This is the basic interface for any kind of DAO (Data Access
 * Object).
 *
 * @inspired "Java Persistence with Hibernate"
 *
 * @param <T> The entity type for the DAO.
 * @param <I> The type of the entity id.
 *
 * @extauthor Thomas Thrien - thomas.thrien@tquadrat.org
 * @version $Id:$
 * @since 1.2.3
 *
 * @UMLGraph.link
 */
public interface GenericDAO<T,I>
{
    ...
} // interface GenericDAO
```

The tag `@extauthor` is an enhanced replacement for the `@author` tag, and the tag `@UMLGraph.link` places an UML diagram for the current class to the generated documentation.



The 'record' Comment

The class comment for a record is a bit special as it has to incorporate the documentation for the fields of the record as well as for the arguments for the constructor (as both are the same).

The declaration for a record may look like this:

Listing 7.1: A tuple class

```

1  /**
2   * <p>{@summary A tuple.} The first entry, usually referred to as
3   * the &quot;key&quot; identifies the object, the second one is the
4   * &quot;value&quot;.</p>
5   * <p>The sort order is determined only through the keys, therefore
6   * {@link #compareTo(Tuple)}
7   * is not consistent with
8   * {@link #equals(Object)}.</p>
9   *
10  * @param <K> The type of the key.
11  * @param <V> The type of the value.
12  *
13  * @param key The key of the tuple; can be {@code null}.
14  * @param value The value of the tuple; can be {@code null}.
15  *
16  * @extauthor Thomas Thrien - thomas.thrien@tquadrat.org
17  * @version $Id:$
18  * @since 1.2.3
19  *
20  * @UMLGraph.link
21  */
22  public record Tuple<K extends Comparable<K>,V>( K key, V value )
23    implements Comparable<Tuple<K,V>
24  {
25      /*-----*\
26      ===** Methods **=====
27      \*-----*/
28      /**
29       * {@inheritDoc}
30       */
31      @Override
32      public final int compareTo( final Tuple<K,V> o )
33      {
34          final var retValue = Comparator.comparing( Tuple<K,V>::key )
35            .compare( this, o );

```



7. Writing proper Comments

```
35
36      //---* Done *-----
37      return retValue;
38  }    // compareTo()
39  }
40  // record Tuple
```

Although not explicitly declared, **Tuple** has ...

- ... a constructor "**Tuple(final K key, final V value)**"
- ... two attributes "**private final K key**" and "**private final V value**"
- ... the two accessor methods "**K key()**" and "**V value()**"
- ... and implementations for "**equals()**", "**hashCode()**" and "**toString()**"

The documentation for these elements is generated either based on the arguments for the `@param` tags (see lines 13 and 14) or from standard texts.

The Field Comment

Attributes/properties, constants and enum values are all summarised under 'field' here.

Each and every field will have a comment, the 'field comment', describing it. For **private** fields it can be sufficient to refer to the related getter method instead of writing a lengthy comment into the field comment itself; do not use the `@see` tag instead of the `@link` tag:

```
/**
 * Refer to
 * {@link #getValue()}.
 */
private final Value m_Value;

// AVOID!!
/**
 * @see #getOtherValue()}.
 */
private final Value m_OtherValue;
```



If there is no getter method for a field, or it is not `private`, a proper description is mandatory. Usually one sentence might be sufficient, although `public` constants may require a full fledged usage description if that is not given elsewhere (for example, in the class comment or the package comment) and a reference to that description could be placed here.

It is always a good idea to describe the valid values for the field, its default value, and whether `null` is a possible value for a reference. This is a must for non-`final` `public` or `protected` fields – in particular because there should never be any non-`final` non-`private` fields at all.

For a serialisable class, the fields that will be serialised should be tagged with `@serial` and the appropriate description.

Constants (`public static final` fields) that are initialised with a literal have to use the `@value` tag in there description. Also `private static final` or `protected static final` fields that are initialised with a literal should use the `@value`.

This looks like this:

```
/**
 * The vested system property for the file encoding used by the JVM:
 * {@value}.
 */
public static final String PROPERTY_FILE_ENCODING = "file.encoding";
```

The comments for enum values are nothing else than field comments for constants – in fact, an enum value is exactly that: a `public static final` field initialised with an instance of the enum type.

The Method Comment

The documentation comment for a method describes its usage and its function within the class, together with its arguments, the return value, and any exception it may throw.

For each method parameter there have to be a `@param` tag that describes it in detail (if not already described in the main text of the method comment; in that case, a short sentence should be sufficient). The description has to



7. Writing proper Comments

cover the function of the parameter, its value range, and whether the parameter can be `null`. Usually, `null` is an invalid parameter value per default, so it has to be mentioned in the respective comment if it is allowed.

It is not enough to only give the type of the parameter in the comment; in fact, this is obsolete as it can be easily taken from the method's signature.

Usually, the return value (given the method is not of type `void`) will be described in the comment for the `@return` tag; the tag is mandatory, and with some text, even if the return value is described already in the method description itself. The description for the return value should provide the possible values and their meanings, whether `null` is a valid return value, and so on.

In particular, the comment for the `@return` tag has to describe which return values indicate special or error conditions.

It is obsolete to give the type of the return value here; it can already be seen from the method's declaration.

Next there has to be a `@throws` tag for each checked exception that may be thrown by the method, describing the condition that may trigger that exception. It is also possible to add `@throws` clauses for unchecked exceptions, but not required. Refer to chapter "8.2.3 General Exception Handling" on page 207 for additional details on exception handling.

A method that implements an interface method or that overrides a method from a base class may be commented with the `@inheritDoc` tag instead of writing a full comment. The tag can be combined with additional text, too.

The documentation comment for a non-`final` `public` or `protected` method has to provide detailed information when and how it has to be overwritten; especially if the overriding method has to call the super implementation and when. Usually you should avoid the requirement for calling the super implementation, but that is not always appropriate or possible; refer to chapter "8.4.3 Non-`final` Methods" on page 267 about some more details regarding this topic.



A comment for that case may look like this:¹²

```
/**
 * ...
 *
 * @note Call this implementation {before|after} your code, to make
 *       sure that the initialisations provided here are performed.
 *
 * ...
 */
```

For more details on this refer to chapter “8.4 Extending Classes, Overriding Methods” on page 265.

Usually, the documentation comment for a method does not reveal details about the method’s implementation, but in case of empty “place holder methods” or mount points, a sentence like below does not harm.

```
/**
 * ...
 *
 * @note This implementation does nothing.
 *
 * ...
 */
```

If such a method has a dummy or default return value, there has to be an appropriate `@return` tag, specifying that value:

```
/**
 * ...
 *
 * @note This implementation does nothing.
 *
 * ...
 * @return Always <the default value>.
 */
```

Finally it is absolutely crucial that the documentation comment provides all information about possible side effects of a call to the method, even

¹²The tag `@note` is a custom tag; refer to chapter “7.1.2 Custom Tags for JavaDoc” on page 168.



more when these side effects are unexpected. This includes the modification of an argument; refer to chapter “8.5.1 Mutable Types for Arguments” on page 287 for more details.

The Constructor Comment

Basically, a constructor is a special kind of a method, so the same rules are valid for the documentation comment for a constructor than for the documentation comment for a method, as given in chapter 7.1.1.

For an (empty) default constructor, the constructor comment may be as simple as this, no matter if it is **public**, **private**, or **protected**:

```
/**
 * Creates a new instance of {@code MyClass}.
 */
public MyClass() { /* Does nothing */ }
```

or

```
/**
 * Default constructor for class {@code MyClass}.
 */
private MyClass() { /* Does nothing */ }
```

with the first alternative being the preferred one, because this comment can be used for every constructor, not only for the default ones.

Of course, this comment do not say very much, but the constructor of a class is also not doing that much, and what it does is very obvious. But if the constructor has side effects, these should be described properly.

A class that does have only **static** methods¹³ should have a **private** constructor like this:¹⁴

¹³Such a class is called a “Utility Class”; refer to “8.42 Utility Classes” on page 423 for more details.

¹⁴The class for the **Error** used in the **throw** is described in [349].



```
/**
 * No instance is allowed for class {@code MyUtilityClass}.
 */
private MyUtilityClass()
{
    throw new PrivateConstructorForStaticClassCalledError(
        MyUtilityClass.class );
} // MyUtilityClass()
```

If a constructor takes parameters, there has to be a `@param` tag for each of them, exactly like for a method.

Although constructors should not throw (checked) exceptions, sometimes it could not be avoided without overcomplicating the API of a class. In such case, all exceptions has to be listed with the `@throws` tag and a description of the conditions for the particular exception – as far as it is possible or make sense. Refer also to chapter “8.2.3 General Exception Handling” on page 207 for some more details on exception handling.

Same as for a method, the documentation comment for a constructor has to describe all the side-effects caused by it; the modification of an argument is such a side-effect – refer to chapter “8.5.1 Mutable Types for Arguments” on page 287 for more details. But in general, a constructor should not have any (visible) side-effects at all.

7.1.2. The JavaDoc Tags

The next two chapters describe the tags that should be used in your documentation comments where appropriate.

One general rule for all JavaDoc tags: Do not insert line breaks between the parameters of a tag:

```
// WRONG!!
/*
 * ...
 * @param args
 *      The command line arguments
 * @param otherArg
```



7. Writing proper Comments

```
*      This parameter needs a very long explanatory comment that
*      requires a line break.
*      @throws IOException
*      Reading the file failed.
*      ...
*/

// CORRECT:
/*
*      ...
*      @param args    The command line arguments
*      @param otherArg This parameter needs a very long explanatory
*                      comment that requires a line break.
*      @throws IOException Reading the file failed.
*      ...
*/
```

The Standard Doclet Tags

Most of the contents of this chapter was taken from the document “Documentation Comment Specification for the Standard Doclet”[121].

The output of the JavaDoc tool is determined by the implementation of the interface `jdk.javadoc.doclet.Doclet`¹⁵ that is specified on the command line for javadoc. The `Doclet` implementation is also responsible for the interpretation of the content of a documentation comment.

If no explicit `Doclet` class is specified, the standard doclet (provided through the class `jdk.javadoc.doclet.StandardDoclet`[106]) is used; it understands the JavaDoc tags described below, and it provides an API for additional tags.

Alternative implementations for the `Doclet` interface may accept the same syntax as the standard doclet, they may handle the tags completely different or they can ignore them completely. However, due to the support by many tools, the syntax supported by the standard doclet has become a *de facto* standard.

¹⁵Refer to [208].



@author Usage: **@author** <name-text>

The tag adds an “Author” entry with the specified name text to the generated documents when the `-author` option is specified on the command line. A documentation comment can contain multiple **@author** tags. Consider to use the alternative **@extauthor** tag instead.

@code Usage: **{@code** <text>**}**

This is equivalent to `<code>{@literal text}</code>`.

It displays text in the code font without interpreting the text as HTML markup or nested JavaDoc tags. This enables you to use regular angle brackets (“<” and “>”) instead of the HTML entities (< and >) in documentation comments, such as in parameter types (`<Object>`), inequalities (`3 < 4`), or arrows (`->`).

If you want the same functionality without the code font, then use the **@literal** tag.

@deprecated Usage: **{@deprecated** <text>**}**

This tag is used in conjunction with the **@Deprecated**[8] annotation to indicate that this API should no longer be used (even though it may continue to work).

The first sentence of the text should tell the user when the API was deprecated and what to use as a replacement. Subsequent sentences can also explain why it was deprecated.

A **@link** tag that points to the replacement API should be added where feasible.



7. Writing proper Comments

@docRoot Usage: {@docRoot}

Represents the relative path to the generated document's (destination) root directory from any generated page. This tag is useful when you want to include a file, such as a copyright page or company logo, that you want to reference from all generated pages.

@exception This is a synonym for **@throws**; it should not be used.

@hidden Usage: @hidden

Hides a program element from the generated API documentation. This tag may be used when it is not otherwise possible to design the API in a way that such items do not appear at all.

@index Usage: {@index <word> <description>} or {@index "<phrase>"<description>}

Declares that a word or phrase, together with an optional short description, should appear in the index files generated by the standard doclet. The index entry will be linked to the word or phrase that will appear at this point in the generated documentation. The description may be used when the word or phrase to be indexed is not clear by itself, such as for an acronym.

@inheritDoc Usage: {@inheritDoc}

Inherits (copies) the documentation comment from the nearest inheritable class or implementable interface into the current documentation comment at this tag's location. This enables you to write more general comments higher up the inheritance tree and to write around the copied text.



@link Usage: `{@link <module/package.class#member> <label>}`

Inserts an inline link with a visible text label that points to the documentation for the specified module, package, class, or member name of a referenced class.

This tag is similar to the `@see` tag. Both tags require the same references and accept the same syntax for `<module/package.class#member>` and the label. The main difference is that the `{@link}` tag generates an inline link rather than placing the link in the “See Also” section. The `{@link}` tag begins and ends with curly braces to separate it from the rest of the inline text. If you need to use the right curly brace (“}”) inside the label, then use the HTML entity notation `}`.

@linkplain Usage: `{@linkplain <module/package.class#member> <label>}`

Behaves the same as the `@link` tag, except the link label is displayed in plain text rather than code font. Useful when the label is plain text.

@literal Usage: `{@literal <text>}`

Same as the `@code` tag, but the text is shown as plain text and not in the code font.

@param Usage: `@param <parameter-name> <description>`

Adds a parameter with the specified parameter name followed by the specified description to the “Parameters” section. The parameter name can be the name of a parameter in a method or constructor, or the name of a type parameter of a class, method, or constructor. Use angle brackets (“<...>”) around such a parameter name to indicate the use of a type parameter.



7. Writing proper Comments

@provides Usage: **@provides** <service-type> <description>

This tag may only appear in the documentation comment inside a `module-info.java` file. It serves to document an implementation of a service that is provided by the module. The description may be used to specify how to obtain an instance of this service provider, and any important characteristics of the provider itself.

@return Usage: **@return** <description>

Adds a “Returns” section with the description text to the documentation comment of a method.

@see Adds a “See Also” heading with a link or text entry that points to a reference. The **@see** tag has three variations; see [121] for the details.

@serial Used in the documentation comment for a default serializable field. See “Documenting Serializable Fields and Data for a Class”[225].

@since Usage: **@since** <since-text>

Adds a “Since” heading with the specified <since-text> value to the generated documentation. The text has no special internal structure. This tag that this change or feature has existed since the software release specified by the <since-text> value, for example: **@since** 1.5.

Although it would possible to provide a date or something else, it is recommended to always use a version number with the **@since** tag.



@summary Usage: {@summary <text>}

Identifies the summary of an API description, as an alternative to the default policy to identify and use the first sentence of the API description. The tag only has significance when used at the beginning of a description. In all cases, the tag is rendered by simply rendering its content.

The {@summary} tag has to be used always when a documentation comment has more than one sentence; see also chapter 7.1.1 on page 148.

@throws Usage: @throws <class-name> <description>

The @throws tag adds a “Throws” subheading to the generated documentation, with the <class-name> and the description text. The class name is the name of the exception that might be thrown by the method, and the description provides information about the conditions for that exception to be thrown.

@uses Usage: @uses <service-type> <description>

This tag may only appear in the documentation comment inside a module-info.java file. It serves to document that a service may be used by the module. The description may be used to specify the characteristics of the service that may be required, and what the module will do if no provider for the service is available.

@value Usage: {@value} or {@value <module/package.class#field>}

This tag is used to display the values of constant in the generated documentation. When the {@value} tag is used without an argument in the documentation comment of a `static final` field, it displays the value of that constant:

```
/**
 * The value of this constant is {@value}.
 */
public static final String SCRIPT_START = "<script>"
```



7. Writing proper Comments

When used with the argument `<module.package.class#field>` in any documentation comment, the `{@value}` tag displays the value of the specified constant:

```
/**
 * Evaluates the script starting with {@value #SCRIPT_START}.
 */
public final String evalScript( String script ) { ... }
```

The argument `<module.package.class#field>` takes a form similar to that of the `@link`, the `@linkplain`, or the `@see` tag argument, except that the member always must be a `static final` field.

@version Usage: `@version <version-text>`

Adds a “Version” subheading with the specified `<version-text>` value to the generated documents when the `-version` option is specified on the command line. This tag is intended to hold the current release number of the software that this code is part of, as opposed to the `@since` tag, which holds the release number where this code was introduced. The `<version-text>` value has no special internal structure.

Custom Tags for Javadoc

You can define your own Javadoc tags; simple tags can be defined on the command line for the Javadoc tool (see the option `-tag` in [335]) or by implementing the interface `jdk.javadoc.doclet.Taglet`[209].

I created a set of custom tags that I use regularly, and that I also recommend for your documentation comments. The respective library can be found at [358].

@anchor Usage: `{@anchor #<anchor-name> <text>}`

This tag allows to add an HTML anchor to the generated documentation, where `<anchor-name>` is the name of the anchor to the given text. The hash symbol (`#`) before the name of the anchor is mandatory!



@extauthor Usage: **@extauthor** <name-text> - <email-address>

This is a replacement for the **@author** tag that renders the given email address as a `mailto:` link in the generated documentation. It does not regard the option `-author` on the command line¹⁶.

@href Usage: **{@href** <url> <text>} or **{@href** <url>}

With this tag, an HTML hyperlink can be added to the generated documentation; it will be placed inside the description text, but different from the **@link** and **@linkplain** tags, it allows to refer to arbitrary external resources, not only to other documented elements. Obviously, <url> is the target URL, while <text> is the clickable text. If the latter is omitted, the URL itself will be used instead.

@inspired Usage: **@inspired** <text>

Sometimes a piece of code was inspired by a document of some kind, a description of an algorithm, a product white paper, or whatever. This tag allows you to add a reference to that source of inspiration.

@modified Usage: **@modified** <name-text> - <email-address>

This is a variant of the **@extauthor** tag. It is meant to provide the name of the developer that modified the respective element without claiming to be an author.

@note Usage: **@note** <text>

With this tag, it is easy to add important notes to the generated documentation for an element. All notes will be added to a bullet list placed immediately beneath the documentation text. The text for the **@note** tag is somehow limited as it does not allow other JavaDoc tags.

¹⁶This is valid for version 0.1.0 of the library; it may have changed for a later version.



7. Writing proper Comments

@thanks Usage: **@thanks** <name-text> - <email-address>

Use this tag to mention someone who provided input to the respective element without being an author; that person might have wrote an article about the algorithm that was implemented by this element, or they may have reported a bug.

Same as for the **@extauthor** and the **@modified** tags, the email address will be rendered to a `mailto:` link in the generated documentation.

@UMLGraph.link Usage: **@UMLGraph.link**

This adds an UML graph for the current class to the generated documentation.

7.2. Implementation Comment Formats

A Java source code file can have four styles of implementation comments that are described in detail in the following chapters:

- structuring comments
- block comments
- single-line comments
- trailing or end-of-line comments

7.2.1. Structuring Comments

Structuring comments are the most simple comments: they are used to separate the parts of a class (as defined in chapter “3.3.4 Class and Interface Declarations” on page 62) from each other. They have the form

```
3  /*-----*\
   ===** Enum Declaration **=====
   /*-----*/
   /*-----*\
6  ===** Inner Classes **=====
   /*-----*/
```



```

9  /*-----*\
   ===** Constants **=====
   /*-----*/
12 /*-----*\
   ===** Attributes **=====
   /*-----*/
15 /*-----*\
   ===** Static Initialisations **=====
   /*-----*/
18 /*-----*\
   ===** Constructors **=====
   /*-----*/
21 /*-----*\
   ===** Methods **=====
   /*-----*/

```

with the lines ending at column 80 (the samples here are shorter, to avoid a line break).

If a class or interface does not have a particular part, the assigned structuring comment must be omitted.

I recommend to create “Building Blocks” with these comments. Eclipse provides the Snippet facility for this purpose¹⁷. Refer to chapter “9.3.1 Structuring Comments” on page 464 for the snippet code for the structuring comments.

7.2.2. Block Comments

Block comments are used to provide detailed descriptions of files, methods, data structures and algorithms – meaning that the text of the comment is longer than just one single line. Block comments may be used at the beginning of each block after the opening brace. They can also be used in other places, such as within methods. Block comments inside a function or method should be indented to the same level as the code they describe.

¹⁷Storing the comments as code templates is not recommended as a template would be reformatted on insert, having some funny effects on the ASCII art.



7. Writing proper Comments

A block comment should be preceded by a blank line to set it apart from the rest of the code. If the block comment does not directly refer to the code line immediately after it, it should be followed by another blank line.

Next, the first line of the comment block has to remain empty, and the closing of the comment block has to be placed on a line of its own.

Some samples:

```
{
    /*
     * Here is a sample of a block comment. Block comments are used
     * to provide detailed information about code internals.
     */
    Result value = retrieveResult( parameter );
    ...

    /*
     * Here is another sample of block comment, somewhere in the
     * middle of a code block. Please note the blank line above!
     */
    ...
}

// AVOID!
/*
 * This block comment is outside the code block it refers to. Block
 * comments should be placed after the opening curly brace of the
 * block.
 */
{
    Result value = retrieveResult( parameter );
    ...

    /* The first line of the block comment should be left empty and the
     * comment should be indented in the same way as the code in the
     * block.
     */
    processResult( value, parameter );
    /*
     * Here the empty line above the comment is missing...
     * ...and the closing tag should be on a line of its own. */
}
```



7.2.3. Single-Line Comments

Short comments can appear on a single line; they will be indented also to the level of the code that follows. Usually it should be written in the form of a headline:

```
//---* Handle the condition *-----
```

with the dashes ending on column 80.¹⁸

If a comment cannot be written in a single line, it should have the block comment format (see chapter 7.2.2). A single-line comment should be separated from the preceding code by a blank line. Only when the preceding line contains only the opening curly brace as in the **if-then-else** sample below (lines 3 and 8), this blank line should be omitted.

Here are some examples of single-line comments in Java code:

```
1 if( cache.contains( key ) )
2 {
3     //---* Take the data from the cache *-----
4     ...
5 }
6 else
7 {
8     //---* Load the data from its original source *-----
9     ...
10 }
11 ...
12 ResultData resultData = executeService();
13
14 //---* Format the output for the UI *-----
15 formatResult( resultData );
16 ...
```

Another form of the single line comment is the empty block comment:

```
public interface Marker
{ /* No methods */ }
// interface Marker
```

¹⁸A quick reminder: the sample code in this document uses a line length of 70, so the dashes in the comment line above ends an column 70.



7. Writing proper Comments

```
public class Extension extends Base
{ /* No implementation */ }
// class Extension

private Constructor() { /* Just exists */ }

public void adapterMethod() { /* Does nothing */ }

public final void method()
{
    ...

    try
    {
        ...
    }
    catch( final MyException e ) { /* Exception deliberately swallowed */ }

    ...
} // method()
```

And finally, there are the “class termination comments” that repeats the name of the class after the closing curly brace of the class definition, as you may have noticed already in several other examples:

```
public final class MyClass
{
    ...
}
// class MyClass

public final interface MyInterface
{
    public record InnerRecord( final int number )
    {
        ...
    }
    // record InnerRecord

    ...
}
// interface MyInterface
```



7.2.4. Trailing or End-Of-Line Comments

Very short comments can appear on the same line as the code they describe, but should be shifted right far enough to separate them from the statements – and then that comment should still end at or before column 80. If more than one short comment appears in a chunk of code, they should all be indented to the same tab setting.

Here’s an example of a trailing comment in Java source code:¹⁹

```
final boolean retValue;
if( a == 2 )
{
    retValue = true;           /* special case */
}
else
{
    retValue = isPrime( a );   /* never true for even a */
}
```

But it is more common to use “//” instead of “/*...*/” for these trailing comments:

```
final boolean retValue;
if( a == 2 )
{
    retValue = true;           // special case
}
else
{
    retValue = isPrime( a );   // never true for even a
}
```

You will use trailing comments to provide the documentation for local variables²⁰:

¹⁹Of course that can be written easier as “`final var retValue = (a == 2) || isPrime(a );`”, but then it would be difficult to place the comments as shown. Again, another form of comment would be still possible ...

²⁰Usually, the name of that local variable should be sufficient (refer to chapter “6.10 Fields” on page 135), or the meaning of that variable is obvious from the context, but sometimes it still make sense to provide that kind of additional information.



7. Writing proper Comments

Listing 7.2: Gauss' Easter algorithm[118, 128]

```
1  /**
2   * <p>{@summary This method calculates the date of Easter Sunday
3   * for the given year.} The year has to be in the range from 1583
4   * to 3900 (included).</p>
5   * <p>The resulting date is for the Gregorian calendar.</p>
6   * <p>The algorithm itself is not the original one published by Carl
7   * Friedrich Gauß first in 1800 (corrected version in 1816), but
8   * that one published by Heiner Lichtenberg in 1997.
9   *
10  * @thanks Carl Friedrich Gauß
11  * @thanks Heiner Lichtenberg
12  *
13  * @param year    The year for which the date of Easter Sunday is
14  *                wanted for.
15  * @return The date of Easter Sunday in the given year.
16  *
17  * @see <a
18  *       href="https://de.wikipedia.org/wiki/Gau%C3%9Fsche_Osterformel">Gaußsche
19  *       Osterformel</a>
20  */
21 public static final LocalDate calcEasterDate( final Year year )
22 {
23     /*
24     * The explanation for the variables was taken from the German
25     * Wikipedia article and translated by me. The original terms
26     * are given in parenthesis.
27     */
28     final int x; // The year
29     final int k; // The secular number (die Säkularzahl)
30     final int m; // The secular moon shift (die säkulare Mondschaltung)
31     final int s; // The secular moon shift (die säkulare Sonnenschaltung)
32     final int a; // The moon parameter (der Mondparameter)
33     final int d; // The seed for the first full moon in spring (der Keim
34     // für den ersten Vollmond im Frühling)
35     final int r; // The calendar adjustment (die kalendarische
36     // Korrekturgröße)
37     final int og; // The Easter limit (die Ostergrenze)
38     final int sz; // The first Sunday in March (der erste Sonntag im März)
39     final int oe; // The distance of Easter Sunday from the Easter limit
40     // -- Easter distance in days (die Entfernung des Ostersonntags
41     // von der Ostergrenze -- Osterentfernung in Tagen)
42     final int os; // The date of Easter Sunday as a March date with
43     // March 32 as April 1 etc. (das Datum des Ostersonntags als
44     // Märzdatum -- 32. März = 1. April usw.)
```




```

43
44 //---* Check the arguments *-----
45 if( requireNonNullArgument( year, "year" ).isBefore( Year.of( 1583 ) )
46     {
47         throw new IllegalArgumentException( "This method will work only
48             for years greater than or equal to 1583" );
49     }
49 if( year.isAfter( Year.of( 3900 ) ) )
50     {
51         throw new IllegalArgumentException( "This method will work only
52             for years less than or equal to 3900" );
53     }
54 //---* Lichtenberg's Easter formula *-----
55 x = year.getValue();
56 k = x / 100;
57 m = 15 + (3 * k + 3) / 4 - (8 * k + 13) / 25;
58 s = 2 - (3 * k + 3) % 4;
59 a = x % 19;
60 d = (19 * a + m) % 30;
61 r = (d + a / 11) / 29;
62 og = 21 + d - r;
63 sz = 7 - (x + x / 4 + s) % 7;
64 oe = 7 - (og - sz) % 7;
65 os = og + oe;
66
67 /*
68  * Initialise the return value with the last day of February and
69  * add the calculated number of days.
70  */
71 final var retValue = LocalDate.of( x, MARCH, 1 )
72     .minusDays( 1 )
73     .plusDays( os );
74
75 //---* Done *-----
76 return retValue;
77 } // calcEasterDate()

```

As you can see, for this use case it is acceptable to continue a long comment in the following line (see lines 31 and 32, for example) without using a block comment. But writing

```

30 ...

```



7. Writing proper Comments

```
31     final int d; /* The seed for the first full moon in spring (der Keim
32         für den ersten Vollmond im Frühling) */
33     final int r; /* The calendar adjustment (die kalendarische
34         Korrekturgröße) */
35     final int og; // The Easter limit (die Ostergrenze)
36     final int sz; // The first Sunday in March (der erste Sonntag im März)
37     final int oe; /* The distance of Easter Sunday from the Easter limit
38         -- Easter distance in days (die Entfernung des Ostersonntags von
39         der Ostergrenze -- Osterentfernung in Tagen) */
40     final int os; /* The date of Easter Sunday as a March date with
41         March 32 as April 1 etc. (das Datum des Ostersonntags als
42         Märzdatum -- 32. März = 1. April usw.) */
43     ...
```

can be considered also as valid. But in general, lengthy trailing comments like these should be avoided.

Another use case for these kind of comments is to provide information about the arguments of a method call; usually, you should avoid method signatures where this is required (the formal parameter of the method should be sufficient to explain the argument), but sometimes it makes still sense:

```
...
drawCircle(
    x, y, // The center of the circle
    d/2.0 // The radius of the circle, calculated from the diameter
);
...
```

Again, you should avoid lengthy comments, but if really required, you can break the lines as shown above.

The end comment for a method is also of this type; I used lots of these in the sample code already:

```
public final void method()
{
    ...
} // method()
```



Also a long code block²¹ can be commented like this:

```
{
    //---* Calculate the result *-----
    // Lots of code comes here ...
    ...
} // End of result calculation
```

But if the comment should only mark begin and end of the code block, without providing further information, you should consider to use a label, as explained below.

The end comments for the bodies of **for**, **while**, **switch** and even **if-then-else** blocks are a special case, as those blocks should be introduced by a label, and this label should be repeated as the end comment:

```
ScanLoop: for( final var s : lines )
{
    // Lots of code comes here ...
    ...
} // ScanLoop:

ForeverLoop: while( true )
{
    // Lots of code comes here ...
    ...
} // ForeverLoop:

TypeSwitch: switch( type )
{
    case TYPE_1 -> ...
    ...
    case TYPE_n -> ...
    default -> throw new IllegalArgumentException()
} // TypeSwitch:

SpecialCaseTurnout: if( isSpecialCase() )
    // Lots of code comes here ...
    ...
} // SpecialCaseTurnout:
else
{
```

²¹But if it seems really necessary to add such a comment to a linear code block, you should consider to re-organise your code.



7. Writing proper Comments

```
// Not so much code here
...
}
```

Note the colon at the end of each of the comments!

Do not use the introducing code line for the end comment, although you can find this pattern in many books and even in real world code! That introducing code line may change at some point in time and then the end comment does not have a corresponding starting line anymore.²²

Avoid this:

```
// AVOID!!!
for( final var s : lines )
{
    // Lots of code comes here ...
    ...
} // for( final var s : lines )

while( true )
{
    // Lots of code comes here ...
    ...
} // while( true )

switch( type )
{
    case TYPE_1 -> ...
    ...
    case TYPE_n -> ...
    default -> throw new IllegalArgumentException()
} // switch( type )

if( isSpecialCase() )
    // Lots of code comes here ...
    ...
} // if( isSpecialCase() )
else
{
    // Not so much code here
    ...
}
```

²²This also means that you should be careful with removing or changing labels in the code.



```
}
```

7.3. Maintenance Comments

It is very likely that source code will be changed more than once during its lifetime. Bugs will be fixed, functionality is added or removed, refactorings will be applied, or the code will be migrated to other platforms or different versions of the programming language, the underlying libraries, the connected systems and/or the operating system.

There is a practice that all the changes made in the code will be commented in the code. These comments are usually referred to as “Maintenance Comments”.

This may look like this:²³

```
...
//<<BEGIN FSP-0815 -- applied by Mirabel Madrigal
<Additional Code>
...
//>>END FSP-0815
...
//<<BEGIN FSP-4711 -- applied by Micky Mouse
//<Old Code>
//...
//>><<
<New Code>
...
//>>END FSP-4711
...
//<<BEGIN FSP-4917 -- applied by Donald Duck
//<Old Code>
//...
//>>
<Unmodified Code>
...
//<<
<New Code>
...
```

²³The abbreviation “FSP” stands for “Field Service Patch” ...



7. Writing proper Comments

```
//>>END FSP-4917  
...
```

Also it looks good at a first glance, it has proved that this is not a good practice at all, and a really bad practice if dealing with code that is managed by an SCCS.

First, this practice causes problems for the compare tools coming with the SCCS – at least it will make it more difficult to read the comparison results from those tools.²⁴ And the main function of those comments – documenting the changes – is much better served by the SCCS tools themselves.

Second, this gets even worse when the fix needs a fix. Just think about overlapping changes, like FSP-4712 changes lines from the new code of FSP-4711, together with lines directly below, or changes that has to be partially reverted, and so on.

And finally, the comments does not help to identify if your are currently running the patched code, or the old one.

On the other hand, nothing can be said against adding a line to the file or class comment that lists the patches that were applied to the code.

Therefore I recommend to introduce an annotation that can be used to mark elements that are affected by a fix.

Such an annotation can provide the BUG number of the fix, together with a short description of the issue. This annotation replaces a comment about the applied patches, with the advantage that it can be retrieved also from the compiled classes, even at runtime.

Chapter “9.6.6 Patch Identification” on page 488 provides an example of such an annotation.

Old code will be removed and not commented out, or just replaced by the new code. One or the other short comment with a hint is not mandatory, but does not harm either.

²⁴Most comparison tools will recognise the out-commenting of the old code as a change and the new code as additional code instead of the replacement for the old code. This is at least confusing when a code revisions are made on the fix.



When using the suggested annotation from chapter 9.6.6, this could look like this:

```
...
@BUG( id = "BUG-0815", comment = "Superseded_by_BUG-4711" )
@BUG( id = "BUG-4711", comment = "No_end_criterion_for_loop" )
@BUG( id = "BUG-4712", comment = "Exception_was_swallowed" )
public final void myMethod()
{
    ForeverLoop: while( true )
    {
        try
        {
            ...

            if( !hasMore() ) break ForeverLoop; // BUG-4711
        }
        catch( final IllegalStateException e )
        {
            log( e );
            break ForeverLoop; // BUG-4712: Added exception handling
        }
    } // ForeverLoop:
} // myMethod()
...
```

The relevant details about the fix/change/enhancement can now be found in the SCCS and/or in your Bug Tracking system, referenced through the id of the annotation.

7.4. Special Comments

The “Code Conventions for the Java™ Programming Language”[109] suggests some special comments: XXX in a comment flags something that is bogus but works. FIXME flags something that is bogus and broken (and has to be fixed very soon).

Eclipse knows a setting that allows to externalise such comments into a task list:

Window|Preferences|General|Editors|Structured Text Editors|Task Tags.



It also adds TODO to that list, for something that still needs to be implemented.

7.5. Commenting out Code

Sometimes you want to ‘comment out’ code that should (no longer) be compiled and/or executed, instead of removing it completely.

First, your production code should not contain any code that is commented out. Nevertheless, it is helpful that you have this option.

Use the “//” delimiter to comment out a complete line, only a partial line, or a whole bunch of consecutive lines of code.

Example:

```
if( foo > 1 )
{
    //---* Do a double-flip *-----
    ...
}
else
{
    return false;           // Explain why here.
}
//if( bar > 1 )
//{
//    //---* Do a triple-flip *-----
//    ...
//}
//else
//{
//    return false;
//}
```

To use a block comment (“/*...*/”) for commenting out sections of code is not a good idea, although you do not have to type that much: you cannot have a block comment inside a block comment.



The easiest way to comment out a selected code block this way in Eclipse is to use “Source | Toggle Comment” from the menu or the short key “CTRL+/²⁵”.

In IntelliJ IDEA, it is the menu command “Code | Comment with Line Comment” or the short key “CTRL+/²⁶”.

7.6. Comments when?

To repeat what was already said at the beginning of this chapter “7 Writing proper Comments”: The frequency of comments reflects the quality of the source code, but it can be “under-commented” or “over-commented”. This means that there is a “frequency band” for comments that has to be hit for *good* quality code.

The advice “When you feel compelled to add a comment, consider rewriting the code to make it clearer” raises the question for whom the code has to be made clearer, because the experts and the rookies are completely distinct audiences, with different needs in regard of comments in the source code.

Same with that other advice (“Even if you think, a comment might not be necessary, add it nevertheless”): it is the other extrema, and equally bad.

That leads us (again) to the conclusion, that writing good comments to source code is a complex art.

This means that this document cannot give you a complete rule set here, just some guidance.

That documentation comments are required everywhere possible, and how to write them was already covered in the chapter “7.1 Documentation Comments” on page 146, thus we focus here on the implementation comments.

P1. When something looks like a mistake, write a comment.

This could be an empty code block, an exception that is not handled, a fall-through in `switch` branch, or an assignment where a comparison is

²⁵On a German keyboard, it is “7” instead of “/”.

²⁶On a German keyboard, it is “÷” on the numeric keypad instead of “/”.



7. Writing proper Comments

expected. Here the comment should make it clear that this “mistake” is also intentional.

P2. When you break a rule, write a comment.

The “Coding Guardrails for Java in a Nutshell” say in bullet points 9 and 10: *“Special cases aren’t special enough to break the rules although practicality beats purity”*. You will encounter lots of occasions in your daily work where this seems to fit – according to your opinion! Others may think differently about that, but a comment at least tells them why you wrote the codes as you did.

And by the way: “practicality” does not mean “laziness in typing”.

P3. Don’t let them guess, tell them the solution.

In bullet point 14 of the “Coding Guardrails for Java in a Nutshell” you are advised not to guess – that’s an advice also for the readers of the code. Add a comment when you think that something can be interpreted in different ways.

P4. Describe the short-cuts.

Assume the following case: you have to parse a string with some input data, and you know from the specification, that this input data has a determined content when the string a specified length, and that this length will never be possible for other contents.

In that case, you can skip the parse process after you determined the length of the string and just set the result. But you should add some lengthy comment to your code that explains that ‘magic’.

And you will have this kind of ‘short-cut’ also in less esoteric cases: in chapter “8.3 Checking Method Parameters and Return Values” on page 250, I request that the arguments to methods needs to be validated before they are used. But when the method is `private`, all the callers to that method are well known and under your control. Therefore you may have checked the arguments already on the caller and can omit the check in the method itself. Or you delegated the validation to another method – in both cases, add a comment.



7.6.1. Empty Code Blocks are forbidden

That means that the sequence “{}” may not occur anywhere in your source code. If there is really no code in a block, there have to be at least a comment, just to show that this is intentional. For a `catch` block, there should be a good (at least some) explanation, why it is empty (see also chapter “8.2.3 General Exception Handling”).

Some samples:

```
//---* Skip input stream until then end *-----
while( input.read() != EOF ) { /* empty */ }

// Better
//---* Skip input stream until then end *-----
while( input.read() != EOF );
```

```
//---* Spent CPU cycles on counting *-----
for( var i = 0; i < maxValue; ++i ) { /* empty */ }

// Better
//---* Spent CPU cycles on counting *-----
for( var i = 0; i < maxValue; ++i );
```

In this case it is likely that the compiler will optimise the loop, just be removing it, because the code does nothing useful.

```
InputStream input = null; // null if file cannot be opened
try
{
    input = new FileInputStream( "myFile" );
}
catch( final FileNotFoundException ignored ) { /* Deliberately ignored */ }

// Better
final InputStream input; // null if file cannot be opened
try
{
    input = new FileInputStream( "myFile" );
}
catch( final FileNotFoundException ignored )
{
    input = null;
}
```



7. Writing proper Comments

```
}
```

```
public MyClass() { /* Just exists */ }
```

```
public interface MyInterface { /* No methods */ }
```

```
@MountPoint  
public Data customAction( final Data data ) { /* Does nothing */ }
```

The usage of the annotation `@MountPoint`^[340] is described in the chapters “8.4.2 Non-final Classes” on page 266 and “8.4.3 Non-final Methods” on page 267, a sample implementation can be found in chapter 9.6.5 on page 487.

7.6.2. Description of the used Algorithm

A short description of the algorithm that was implemented by the current source code is always helpful; even just mentioning its name can make a difference when hunting a bug. But in case the implementation changes, such a comment needs to be adjusted as well.

The same is valid for the pattern that you implement with your code, although this belongs more often into the documentation comments.

7.6.3. Broken Rules

Whenever you hurt or ignore one of the rules, guidelines, or recommendations from this document, it is worth a comment; same when you use a short-cut.

Some examples:

```
value = 7.0 / a; // a != 0.0 was already checked above  
  
o.execute(); // o != null was checked in foo.bar( o )  
  
/*  
 * bar.getQ() will never return null, so an explicit check on  
 * null for q was omitted
```



```
*/  
q = bar.getQ();  
q.execute();
```

In all these samples, arguments or values are not checked for `null` or zero, *because this was already done elsewhere* – and not necessarily just one line above the current location. The comment tells a maintenance engineer that the check was made – at least in the initial version of the code – and that the root cause for the problem might be elsewhere (perhaps because someone eliminated that other location or at least the value check there).

Non-obvious class casts can be seen as a short-cut or a broken rule, but they should be always explained with a comment, in order to document that you knew what you did:

```
private final void valueProcessor( List<Object> values )  
{  
    ...  
    Value value;  
    for( Object o : values )  
    {  
        /*  
         * We know that values can only contain Value objects;  
         * otherwise this method would not have been called to  
         * process the list.  
         */  
        value = (Value) o;  
        ...  
    }  
}
```

This makes sense here because a check like

```
if( o instanceof Value ) value = (Value) o;
```

is relatively expensive – especially if the contract for this `private` method is that it is called only with lists containing `Value` objects so that the check would not be positive only in very, very, very rare cases. And finally: what else can be done in cases where the object is not of the right type than throwing a `ClassCastException`? That's the same that is done by the code above in such a case, too.

First, the version



```
if( o instanceof Value value )
{
    ...
}
```

might be clearer, but it is not less expensive.

Second, having a list of **Value** objects declared as a **List<Object>** is bad design anyway – or legacy code (and still bad design).

A fall-through in a **switch** is another broken rule and needs a comment, as said already in chapter “5.4.2 ‘switch’ Statements” on page 95.

7.6.4. Swallowed Exceptions

The “Coding Guardrails for Java in a Nutshell” says in bullet point 11 that errors may not be ignored and that exceptions may not be swallowed; this is most often done by an empty **catch** block.

But people forget that **catch** blocks like those below will also swallow the exception, although it does not look like that:

```
1  ...
2  catch{ final Exception e }
3  {
4      e.printStackTrace();
5  }
6
7  ...
8  catch{ final IOException e }
9  {
10     m_Logger.atError()
11         .withThrowable( e )
12         .log( "Opening_the_file_failed" );
13 }
14
15 ...
16 catch( final NumberFormatException e )
17 {
18     m_LastError = e;
19 }
```



All this might be intended, but at a first glance it looks like a mistake. This mean that a comment is mandatory each time an exception is swallowed, no matter if silently or not, meaning the exception is logged somewhere. If the program is continued after an exception was caught, write a comment (see also chapter “8.2.3 General Exception Handling”).

Also when you catch `java.lang.Error` or a child class of it, or `java.lang.Throwable`, you should provide a comment why you are doing that (and why you think it is allowed in your special case ...).

7.6.5. Suppressed Warnings

With the `@SuppressWarnings` annotation[11], you can switch off a compiler warning or error caused by code that is somehow “hurting the rules”. Basically, the `@SuppressWarnings` annotation is a replacement for a comment about that deviation from the rules, an additional comment is required only in cases it is not obvious why that annotation was applied. More details on this are provided in “8.24 Compiler Warnings and Errors” on page 403.

7.6.6. Comments vs. Annotations

Sometimes a comment can be replaced or enhanced by an annotation. Obvious examples are the `@Override`[10] and the `@Deprecated`[8] annotations, and to some extent, also the `@FunctionalInterface`[9] annotation.

Other annotations from the Java Runtime library that could (and should) be used this way are (the list is not exhaustive):

- `@Generated`[12]
- `@Native`[7]
- `@Serial`[6]

The `@API`[13] annotation can be seen as a comment replacement, too. Refer to chapter “8.20 The Annotation `@API`” on page 400 to learn more about this annotation and its usage.

My Foundation library will have some more samples:



7. Writing proper Comments

- `@BUG`[337]
- `@ClassVersion`[338]
- `@MountPoint`[340]
- `@PlaygroundClass`[341]
- `@ProgramClass`[342]
- `@UtilityClass`[343]

Similar annotations can be found in other libraries (search for the annotation `javax.annotation.OverridingMethodsMustInvokeSuper` or for the annotation `org.jetbrains.annotations.MustBeInvokedByOverriders`), and when you found yourself in the situation that you have to write the same comment to methods or classes over and over again, think about creating your own annotation that can replace that comment – or you look whether there is a library that comes with that annotation already.

To some extent is even the `@SuppressWarnings`[11] annotation a “replacement” for a comment: it tells the compiler to suppress a warning or an error, but it also tells the readers of your source code that you had been aware of what you were doing when implementing that ‘wrong’ piece of code.

Of course using an annotation instead of a comment can help to spare typing, but the greatest advantage of annotations over comments is that you can easily write tools that helps you with checking the source code in accordance with the annotations. Or someone else has already created those tools.

7.7. Updating Comments

When the source code is modified, no matter if due to regular maintenance, bug fixing or a migration, all related comments has to be updated accordingly. That is mandatory!

There is no value in keeping “historical comments”, although mentioning the old algorithm when the former implementation has been replaced completely might be useful in some cases.

But keeping the outdated code as a comment is definitely not useful, in particular not when there is an SCCS in place that is used to manage the code.



8. Coding Guidelines

These coding guidelines are a collection of coding standards and best practices. Obeying them should make your code better readable and less error prone. Some of them will even help to increase the program's overall performance. So perhaps you should see them not as optional, but more as obligatory rules.

But as myself, you may have issues to accept something as a “best practice” just because some guru told so. That's a good approach! First because what's a “best practice” today can be an “anti-pattern” tomorrow, and not just because some fashion trends have changed, but also because some new technology was introduced, or – happens all too often! – someone unmasked a false guru.

Another reason is that too often the explanations are missing why the “best practice” is better the alternatives. I have tried to give that explanations when I describe a best practice, but sometimes I just quote one of the gurus of our business and their explanations.

Coding standards are something different; a standard means to do the same thing always in the same way, and that is definitely an *undoubted best practice*. Nevertheless there could be better ways to do that thing – in that case, change the standard¹

But as always there may be good reasons to do it different from what is recommended or even requested by the guidelines². In such case a comment

¹And I would appreciate if you update me accordingly so that I can modify this document.

²You remember the bullet points 9 and 10 from the “Coding Guardrails for Java in a Nutshell”? If not, here are they again: “9 Special cases aren't special enough to break the rules ...” and “10 ... although practicality beats purity”.



is required that describes that reason.³

One basic recommendation is that you should not write the same code over and over again. This is also known as the “DRY Principle” (“Don’t Repeat Yourself”) and we will discuss this later again.

And please keep in mind that not always the shortest, most compact source code is the best. Also avoid what is known as “Premature Optimization”⁴. Modern optimising compilers and run-time optimisers do a very good job to create compact object code, so in most cases the programmer can concentrate fully on writing readable and comprehensible code. In this context I would like to remind you on the quotation from Martin Fowler’s book that I put in front of this document, and again to the “Coding Guardrails for Java in a Nutshell”, here the points “4. Simple is better than complex” and “5. Complex is better than complicated”.

But although optimisation still may have some limits, comments will have never any impact on the runtime performance of a program. So please refer to chapter “7.6 Comments when?” on page 185 (if not done already) and see the recommendations on when to apply comments to your code.

Types of Products

Some of the coding guidelines below are different for the type of product or project you are working on. Basically, we can distinguish the following types that will be explained in the following chapters⁵:

- Function Libraries
- Feature Libraries
- Tools

³Omitting this comment is also a deviance from the rule, requiring a comment to explain it. Also known as the Catch 22[32].

⁴Donald E. Knuth made the following statement on optimisation: “We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.”[237]

⁵If you miss the terms ‘Framework’ and ‘Server’ on the list above: a *Framework* is in this regard a *Feature Library*, and a *Server* is a *Standalone Application* (what else could a server be?).



- Standalone Applications
- Server-based Applications
- Extensions

The individual types cannot be clearly distinguished one from another, there are some overlappings and gray areas. So when applying a guideline, you still have to use your judgement which implementation really fits for *your project*.

Function Library A function library is a collection of functions (often organised in utility classes – refer to chapter 8.42 on page 423) and helper classes. A function library does not have a state or requires an initialisation or configuration.

Samples are my Foundation Util library[359], the Commons Lang library from the Apache Commons project[15], or Google Guava[132].

My JavaComposer library[357] is a sample for the beforementioned ‘gray area’: I decided to treat it as a function library, but it could have been a feature library, too. The various XML parsers and JSON parsers/generators will also belong to this gray area.

Feature Library A feature library adds a service or a complex functionality to your application. Usually, the library as a whole has an internal state, and it has its own configuration and initialisation. Quite often the library’s functionality is accessed like an external service, and the library runs its own threads.

Some feature libraries can even be started standalone, in a server mode. Samples for this are the H2 Database Engine[169], ActiveMQ[14], or Jetty[333].

Others are only providing access to external services that runs in external processes even on different machines. The examples here are the various JDBC drivers or Hibernate[171].

In the gray area here I would place JUnit[234] and Log4j[18].



Tool A tool in this context is a program that is started, performs a single task and terminates afterwards. Most probably it will be invoked from the command line, and it will not have a UI, instead it takes all input data somehow from the command line. Perhaps it may even work as a filter[125], reading from standard input and writing to standard output.

Programs like `ls` or `grep` belongs to this type, although these are usually not provided as Java programs, although there are implementations written in Java.

Gray area candidates are `awk` and also `sed` (again usually not provided as Java programs, but available in versions written in Java), `jAlbum`[218], but even `javac`, the Java compiler, or `javadoc`, the documentation tool for Java.

Standalone Application A standalone application will run indefinitely (meaning until deliberately terminated by the user) and takes input continuously. Samples are text editors like `jEdit`[232], an IDE, an application server like `WebSphere`[174] or a web container like `Tomcat`[22].

A gray area candidate is `jAlbum` that I already mentioned as a tool, above, others are the H2 Database Engine, `ActiveMQ` or `Jetty` in their standalone/server modes.

Server-based Application Server-based applications are applications that require a special environment to be executed; the best example are JEE applications that need an application server like `JBoss`[326], `WebLogic`[316] or `WebSphere`[174], and web applications, requiring `Tomcat`[22], `Jetty`[333] or any of the appservers mentioned before.

In this case, the program code has to follow several special rules, determined by the server environment. On the other side the environment provides several services that can be utilised by the application.

Other samples are `Maillets for James`[17] or the customisations (“mods”) for `Minecraft`[311] (although these could be regarded both as the gray area candidates here, because both could be seen also as extensions).



Extension An extension or a plugin requires also an environment to run in, but it is not an application as such. It just changes the behaviour of that environment. Annotation processors are samples for this (they change the behaviour of the Java compiler), as well as Maven[19] plugins.

Extensions written in Java are also possible for programs that are not written in Java themselves; examples for that are Apache OpenOffice[20] (refer to [21] for an overview) and LibreOffice[242]⁶.

8.1. Swap Logic Errors for Compiler Errors

From a book about Java programming:

“One fundamental principle of programming is that, generally, it is best to swap a logic error for a compiler error. Compiler errors tend to be found in seconds and are corrected just as fast. Syntax errors are a good example [...]

Logic errors, on the other hand, are the bane of all programmers. They hide and hate to reveal themselves. Logic errors seem to have minds of their own, constantly evading detection and dodging your efforts to pin down their cause. They can easily take a thousand times more effort to solve than the worst compiler errors. Worst of all, many logic errors are not found at all and occur only intermittently in sensitive places, which causes your customer to scream for a fix. Logic errors often require you to throw thousands of man-hours at them, only to finally discover that they are minor typos.”

Robert Simmons Jr.: *Hardcore Java*[328]

⁶Although *how* to create these extensions in Java is quite well hidden in the documentation for LibreOffice ...



Principles

- P1. Utilise the capabilities of your development tools to detect bugs as soon as possible.

Java is a strongly typed language, and this makes the compiler a powerful tool in the regard, given that you design your code in a manner that it can detect errors.

- P2. Take warnings serious!

The `javac` in its standard configuration emits already several warnings, build tools like Maven[19] or Gradle[312] can add to them, and finally you can configure additional warnings in your favourite IDE.

- P3. Avoid to outsmart the compiler!

Casting an object to another type is a sample for this, invoking methods through Reflection is another.⁷

A quite simple example for swapping a potential logic error for a compiler error is the recommendation to write comparisons always with the unchangeable value on the left side⁸. So if you forget the second equal sign for a comparison on equal, the compiler will complain immediately:

```
if( length() == len ) ...  
if( 5 == len ) ...
```

Another sample for this is the recommendation to name fields with the “`m_`” prefix (refer to chapter “6.10 Fields” on page 135) instead of using the `this.` prefix. The compiler will never complain if you omit that prefix when accessing a field, and it will not complain if you name a local variable in the same way as a field – but it will scream loudly about a non-existing reference if you forget the “`m_`” prefix when accessing a field named along the rules defined

⁷Thanks to the creators of Java that you cannot invoke a method through a pointer, as it is non uncommon in C and C++ ...

⁸This recommendation originates from C/C++ programming where `if` conditions are checking values of type integer – and in C, nearly everything can be an integer, or at least be interpreted like one. Java forces that the type of the expression in the `if` condition has to be `boolean`, therefore this approach is less useful for Java code.



here. Although you can configure both Eclipse and IntelliJ Idea to raise an error or a warning if you access a field without `this.`, or if you shadow a field through a local variable with the same name. Nevertheless, I prefer the “`m_`” prefix for fields because it is also easier to read.

Also defining and using value types and domain value types instead of simple types can prevent logic errors from remaining undetected: providing an article number to a method that expects a customer id will not work, when you defined the domain value types `ArticleId` and `CustomerId` and declared the respective method to take an argument of type `CustomerId` instead of a simple `String` or `int`. Refer to the chapter “8.10 Value Types” on page 336 for more details about this topic.

For the same reason you should always consider to use enums instead of defining magic numbers.

The best thing about compiler errors is, that they usually show up early during development – and not first after deployment at customer side. Under this aspect you should take compiler warnings seriously, no matter whether emitted by `javac` or your development environment. When CI/CD pipelines are used, additional code checking tools may have been integrated. Utilise them, and respond to their warnings, at least by thinking twice before adding the `@SuppressWarnings[11]` annotation.

See also chapter 8.24 on page 403 about compiler warnings and errors.

In this regard the use of `java.lang.Class.forName[256]` and the use of Reflection in general are another nasty source for bugs that might be difficult to identify.

Basically, `Class.forName()` takes a string as its argument that is the fully qualified name of a class. If there is at least one instance of `Class` on the current classpath, the method will return that object – otherwise it will throw a `ClassNotFoundException[46]`. This will not happen at compile time, but only during the execution of the program, depending on the execution path of your program – meaning the bug that you introduced by having a typo in the class or package name may remain undetected for a long time.

So if you want to access a class object that you expect to be there, always use “`<ClassName>.class`” where `<ClassName>` is the name of the class; you can



use the fully qualified name, or you can import it. Either way, the compiler will complain if the class is not there.

Only if a class is optional for your application (for example, it may come with a plugin), use “`Class.forName( <ClassName> )`”; of course you also have to add the proper error handling for the case that the class is not there.

For Reflection, the situation is similar: you refer to the class element that you want to get access to by its name, provided as a string. The compiler is unable to validate that name, so a potential error will surface earliest at runtime. Refer to the chapter “8.11 Accessing Fields or Methods using Reflection” on page 343 for more details about Reflection and its alternatives.

8.2. Error Handling

A proper error handling is crucial for any code, no matter in which type of program it lives – but the definition of *proper* depends significantly on the type of the final product, as it was defined above.

Principles

- P1. Errors should never pass silently and Exceptions may never be swallowed.

You may have read this already right at the beginning of this document; it is bullet point 11 of the “Coding Guardrails for Java in a Nutshell”.

The first part implies that you need to check for error conditions, the second part means that you may not throw away error notifications (in Java, these are mainly the Exception).

- P2. Error handling must not obscure the essentials.

When you write the code that checks for an error condition or handles an error condition, come to the point fast, otherwise the readability of the code suffers.



P3. Fail fast.

Place all your error checks to the beginning of a method, and do not hesitate to abort a function when an error condition is detected.

P4. Ensure that errors will be reported in an understandable manner.

The error message “An error occurred, the program was aborted” is not really helpful when investigating the root cause of a problem with your program. Confessed, showing a pop-up window with the error message “Exception java.lang.ArithmeticException: / by zero”, followed by a stack trace, is in a similar way not very useful for the average end user (and may even cause a security risk), but that’s where logging was invented for.

8.2.1. The Basics

Basically, in a Java program we have to deal with only a very limited set of possible error categories:

1. An exception is thrown by an operation triggered by our code.
2. An operation returns an error state.
3. The result of an operation is an unexpected or an unwanted value.
4. A method in our code is called with an invalid argument.
5. Input data is invalid or corrupted.
6. A command line argument is invalid.
7. An external resource is not or no longer available.

Of course, all those categories are possible triggers for an exception in Java. This means that we can quite simply generalise the response to an error: *in case of an error condition, throw an exception!*

Consequently, we can also simplify *error* handling to *exception* handling. And there are only a few options on how to respond to an exception:

- Throw an exception and make it an S.E.P.⁹ – particularly suitable (no irony!) for libraries.

⁹S.E.P = Someone Else’s Problem (refer to [4]).



- Perform an alternate operation or provide a default value.
- Retry/repeat the operation.
- Abort the current operation – either the current thread or the whole program.

As said, throwing an exception will always work, but at some point exceptions need to be handled in some way.

The other alternatives are feasible only in some special contexts:

- Repeating or retrying a failed operation depends on the kind of operation, and not that much on the type of the product. This also should not be done indefinitely; at some point, it gets useless to repeat or retry a constantly failing operation.
- Same for an alternate operation that will be executed in case of an error: if it fails, too, abort the operation.
- Providing a default value demands that this default value makes sense in the current context. It should be obvious that this is not always the case.
- To abort a Java program (in fact, to abort the JVM that executes that Java program), you call the method `java.lang.System::exit[272]` with a negative integer as the argument. But this is only acceptable for a tool and, with some limitations, for a standalone application. Libraries, extensions and especially server-based applications are not allowed to call `System.exit()`! Never!

A thread will be aborted automatically when an exception is thrown and not caught. If this thread is the last non-daemon thread, the JVM will go down, too. So usually the best thing to abort the current operation is to throw an **Error** – for the details, refer to chapter 8.2.3.

Only these four options do exist for handling an exception! This means that afterwards either the error condition is fixed, or it persists.

I bet that you now ask yourself why the actions below are missing from my list above:

- Log an error message.



- Write an error message to the console, a message window or status bar.
- Display an error dialog.

That is because ***none of these are handling the error!*** Of course you should log an error, of course you should inform the user about an error condition. But none of these actions will change the state of the system!¹⁰

Far too often you find code like this:

```
...
try
{
    writeStatusToBillingSystem( invoice );
}
catch( final TimeOutException e )
{
    m_Logger.error( e );
}
invoice.markAsPaid();
...
```

Looks good, right?

Really?

You mark the invoice as paid, despite the writing to the billing system failed?

You think that this cannot happen, because you have logged the exception?

I think that the accountants are no longer your friend when they find out that this piece of code was written by you ...

The chapters “8.2.2 Throwing Exceptions” on page 204 and “8.2.3 General Exception Handling” on page 207 discuss throwing and handling exception. More about reporting the various error conditions can be found in chapter “8.2.5 Reporting Errors” on page 220 and the chapter “8.2.6 Logging” on page 222 deals with logging in particular.

¹⁰The only exception would be an error handling window allowing the user to abort the operation or to ignore the error condition.

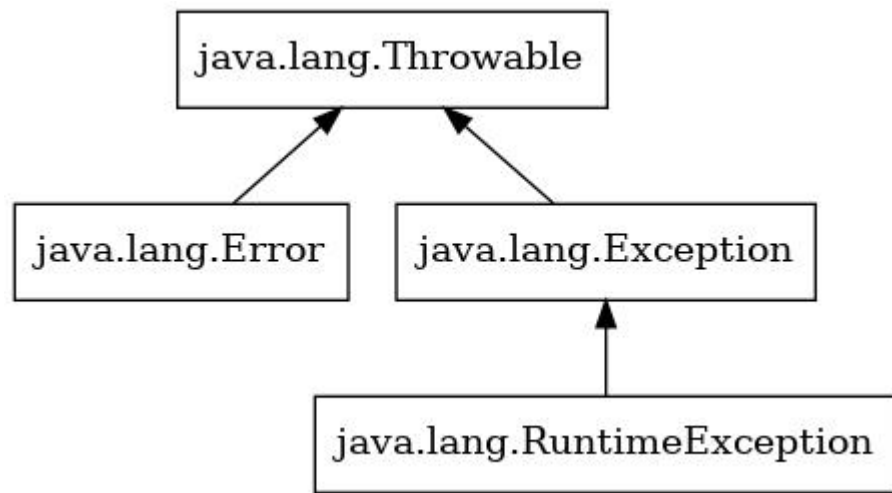


8.2.2. Throwing Exceptions

Your code will emit an exception when it encounters an error condition of some kind. Exceptions are discussed in detail in the Java Language Specification[134]; I just repeat the basics here.

For the Java programming language, an exception is an instance of a class that is derived from `java.lang.Throwable`[71], or – more precise – from one of the child classes of `Throwable`:

- `java.lang.Exception`[49]
- `java.lang.RuntimeException`[65]
- `java.lang.Error`[48]



The different kinds of exceptions are discussed in [135].

Exception classes that extend `java.lang.Exception` are so-called *checked* exceptions; if a method wants to throw one of these, it has to declare that with the `throws` clause. The class `java.lang.RuntimeException` is a direct subclass of `java.lang.Exception` and the superclass of all *runtime* or *unchecked* exceptions – declaring these is not mandatory.



While an ordinary program can *potentially* recover from an error condition signalled by an **Exception** instance, it is assumed that instances of the class **java.lang.Error** and its subclasses will indicate unrecoverable error conditions, causing the program to abort.

The class of the thrown exception should be chosen appropriate to the error condition it indicates. So throwing just an instance **java.lang.Exception** or **java.lang.RuntimeException** is usually not a good idea – same if throwing a **java.lang.NullPointerException**[58] in case a divisor is 0 (zero)¹¹.

When selecting the exception class, your first look should be to the already existing exception implementations if there is already one that fits your needs.¹² If none of the existing ones fit, implement your own exception class by extending one of **Exception**, **RuntimeException**, or **Error**, depending on your needs¹³. This is always useful in the case of a feature library to simplify its API¹⁴, and it can be also helpful for an application.

When implementing an extension or a server-based application (or when using a particular framework), you have to implement quite often methods that declare only one, sometimes two exceptions. Nevertheless, it may happen that a method call in your code throws an exception that cannot be handled by the current code, but it cannot be delegated also, because the interface for the method does not declare that particular exception. In such a case, you have to catch the exception and to wrap it into an instance of the declared exception that will be thrown afterwards. For a servlet implementation[33], this may look like this:

```
protected final void doGet( HttpServletRequest request,
                             HttpServletResponse response )
```

¹¹The proper exception class would be **java.lang.ArithmeticException**[43].

¹²Although sometimes the name of an exception looks good, you should also consider which package it lives in; throwing an exception from a JNDI related package like **javax.naming.OperationNotSupportedException** in a code segment that does nothing with the “Java Naming and Directory Interface” at all will cause more confusion than it helps. Not to mention that it may require an additional module.

¹³You can create your own exception class that extends **Throwable** directly, but so far I have not found a use case for that.

¹⁴All methods will only throw one exception especially created for that library, or exceptions derived from that. Samples for this pattern are the Servlet API (throwing only instances of **IOException** and **ServletException**) or JDBC (throwing only **SQLException** instances).



```
        throws ServletException, IOException
    {
        ...
        try
        {
            ...
        }
        catch( final TimeOutException e )
        {
            throw new ServletException( e );
        }

        ...
    } // doGet()
```

When instantiating an exception instance, you should always provide a textual message that describes the error condition and/or provides additional information, like the name of the file that cannot be found, or the SQL statement with the invalid syntax.¹⁵ These text have to be in English language, for the same reason why the names and the comments are in English.

Of course, messages that will be presented to the end user in a UI have to be localised, but the UI should not display the raw exception message anyway at all.

Some exception classes do not have a constructor that allows to provide a cause, or they have constructors that allow either a message or a cause, but not both. A sample for that is `java.lang.ExceptionInInitializerError`[50].

Some of these exception classes will allow a workaround that allows you to still provide both, cause and message, with them, like below:¹⁶:

```
static
{
    try
    {
        ...
    }
    catch( final Exception e )
```

¹⁵In this regard, the sample above is bad, because it does not provide such an explanation.

¹⁶Unfortunately, `ExceptionInInitializerError` does not belong to this group of exception classes.



```
{  
    throw new DumpException( "Message" )  
        .initCause( e );  
}  
}
```

For the method `initCause()` refer to [281].

Unfortunately, you have to test whether the exception that you have to deal with allows this workaround. And you should add this test to the unit tests for your code, to protect you from changes coming with new versions.

8.2.3. General Exception Handling

A method is only allowed to handle an exception if it is able to restore the program state, otherwise it has to delegate the exception. Delegation can be done either by ignoring the exception – in this case it will be propagated to the caller automatically (it “bubbles up”) – or by catching and wrapping it, as described in chapter “8.2.2 Throwing Exceptions”.

To handle an exception you first have to catch it.

Catching `java.lang.Exception` is discouraged in most cases; it *can* be acceptable when you need to wrap all checked exceptions into a runtime exception or into an error:

```
public final void run()  
{  
    try  
    {  
        ...  
    }  
    catch( final Exception e )  
    {  
        throw new WrapperException( e );  
    }  
    // run()  
}
```



8. Coding Guidelines

You should absolutely avoid to catch `java.lang.Error` and all of its subclasses, `java.lang.RuntimeException`, and of course, you should not never ever catch `java.lang.Throwable`!¹⁷

Provide a `catch` block for each and every exception that you want to handle:

```
public final void run()
{
    try
    {
        ...
    }
    catch( final FileNotFoundException e )
    {
        throw new ApplicationError( "Cannot_find_file", e );
    }
    catch( final IOException e )
    {
        throw new ApplicationError( "An_error_occurred_on_processing_the_input_file", e );
    }
    catch( final NumberFormatException e )
    {
        throw new ApplicationError( "Input_Data_is_corrupted", e );
    }
    catch( final PatternSyntaxException e )
    {
        throw new ApplicationError( "Pattern_invalid", e );
    }
} // run()
```

Sometimes when a large number of exceptions has to be handled with all the same code, you may be tempted to just catch `java.lang.Exception`. As already said: don't do it! This approach will definitely cause problems as it may handle runtime exceptions in the same `catch`-block as the checked exceptions, and usually this is not intended.

A simplification for the sample above could look like this:

```
public final void run()
{
    try
```

¹⁷But no rule without an exception: refer to chapter 8.2.4 on page 212.



```
{
    ...
}
catch( final IOException | NumberFormatException |
       PatternSyntaxException e )
{
    throw new ApplicationError( "Processing_failed", e );
}
// run()
```

Still only the declared exceptions will be handled in the `catch`-block, but unexpected exceptions will now bubble up¹⁸.

Sometimes an exception indicates a regular program state (although this would indicate poor design in most cases)¹⁹. Usually you will avoid this situation by e.g. checking the operands before an operation, but a very popular example where this is not practicable is the conversion of a string into a numerical value:

```
...
var amount = 0.0;
final var requestParam = request.getParameter( "amount" );
try
{
    if( nonNull( requestParam ) )
    {
        amount = Double.parseDouble( requestParam );
    }
}
catch( final NumberFormatException ignored )
{
    //---* If amount in request is not numeric, set it to 0.0 *-----
    amount = 0.0;
}
...
```

As shown in the example, a comment is required in the catch clause, because you do not further propagate the exception neither do you log it.

¹⁸The exception class `java.io.FileNotFoundException` is not listed because its superclass `java.io.IOException` is already on the list.

¹⁹Nevertheless, you can find this even in the Java Runtime Library ... but no one has ever said that the authors of the JDK had never made doubtful decisions.



It is also possible that you checked the preconditions, but you still have to deal with some checked exceptions:

```
...
final var file = new File( ... );
if( file.exists() )
{
    try( final var input = new FileInputStream( file ) )
    {
        ...
    }
    catch( final FileNotFoundException e )
    {
        throw new UnexpectedExceptionError( "File_was_there_when_
            checked!", e );
    }
}
...
```

A sample for the class **UnexpectedExceptionError** can be found in [350].

Some APIs in the JDK declares a checked exception, but that will never be thrown for the given arguments by any means. Most famous for this are the conversions from byte arrays to Strings and back:

```
...
final String s;
try
{
    s = new String( byteArray, "UTF-8" );
}
catch( final UnsupportedEncodingException e )
{
    throw new ImpossibleExceptionError( "UTF-8_must_exist!", e );
}
...
```

According to [332], the UTF-8 encoding is mandatory for each JDK/JVM implementation, therefore the code sequence above will *never* throw an **UnsupportedEncodingException**. Nevertheless, it is not allowed to swallow the exception; instead an error should be thrown in the **catch** clause, indicating the error condition. In the improbable case UTF-8 is not supported by the



current Java runtime environment, the problem will surface, and the program will not die silently without any hint about the reason.²⁰ It also protects you from malicious changes to the name of the character encoding: a none existing **Charset** will cause noise.

Of course, the problem could be solved differently, by using an instance of `java.nio.charset.Charset`[77] as the argument to the **String** constructor, instead of the name of the character encoding. For this case, the constructor does not declare an **UnsupportedEncodingException**.

Another example is the interface `java.lang.Appendable`[181]: it is implemented by `java.io.Writer` and `java.io.PrintStream`, but also by `java.lang.StringBuilder` (and a few more). The method `java.lang.Appendable::append` declares to throw an **IOException**, but the implementation of that method for **StringBuilder** will never throw it and does not even declare it.

```
...
final Appendable appendable = new StringBuilder();
try
{
    appendable.append( "Some_Text" );
}
catch( final IOException e )
{
    throw new ImpossibleExceptionError( "Append_to_StringBuilder", e );
}
...
```

Again, a sample for **ImpossibleExceptionError** can be found in [347].

Sometimes a method wants to act upon an error condition without handling the exception. In this case, it may catch the exception, does whatever necessary, then throw it again. An example:

```
public final void myMethod() throw IOException
{
    final Predicate<String> isNetworkError =
        m_NetworkErrorPattern.asMatchPredicate();
    ...
    try
    {
```

²⁰If, of course, there is no catch for **Throwable** somewhere on the path.



```
    ...
}
catch( final IOException e )
{
    final var message = e.getMessage();
    if( nonNull( message ) && isNetworkError.test( message ) )
        ++networkErrors;
    throw e;
}
...
} // myMethod()
```

It is usually not a good idea to log an exception and to rethrow or to wrap it afterwards, because this might cause the exception to be logged twice (or even more often). More details about logging can be found in chapter 8.2.6 on page 222.

Finally, you handle the exception somehow, by assigning a default value²¹, or by performing an alternative action. In any case, you need to provide a comment in the `catch` why you do not further delegate the handling of the exception.

If an exception was not caught in your code, it “bubbles up” until it reaches the main method for the current thread (that is `main()` for the main or program thread, and `run()`[267] for any other thread). If this method will not handle the exception, the thread will be aborted. If this thread is the last non-daemon thread, the JVM will be terminated, too.

8.2.4. Exceptions in Threads

When we execute the Java program below, the **Error** is printed to the console:

Listing 8.1: Test Abort main()

```
1 public final class TestAbortMain
2 {
3     public static final void main( final String... args )
4     {
```

²¹see the sample about the conversion from string to number, above.



```
5      throw new Error( "Aborted" );
6  }    // main()
7  }
8  // class TestAbortMain
```

```
$ java TestAbortMain.java
Exception in thread "main" java.lang.Error: Aborted
    at TestAbortMain.main(TestAbortMain.java:5)
$
```

In the sample below, we abort a secondary thread:

Listing 8.2: Test Abort run()

```
1 public final class TestAbortRun
2 {
3     public static final void run()
4     {
5         throw new Error( "run()_aborted!" );
6     }    // run()
7
8     public static final void main( final String... args )
9     {
10        try
11        {
12            final var thread = new Thread( TestAbortRun::run );
13            thread.start();
14            thread.join();
15        }
16        catch( final InterruptedException e )
17        {
18            e.printStackTrace();
19        }
20        throw new Error( "main()_aborted!" );
21    }    // main()
22 }
23 // class TestAbortRun
```

The output looks like this²²:

²²When using an older version of Java, no output from the thread would be shown at all.



8. Coding Guidelines

```
$ java TestAbortRun.java
Exception in thread "Thread-0" java.lang.Error: run() aborted!
    at TestAbortRun.run(TestAbortRun.java:5)
    at java.base/java.lang.Thread.run(Thread.java:833)
Exception in thread "main" java.lang.Error: main() aborted!
    at TestAbortRun.main(TestAbortRun.java:20)
$
```

This behaviour is sufficient for a tool in most cases, but not for a standalone application and not for a feature library. A function library usually relies on the a caller, while serverbase applications and extensions use the facilities provided by the host applications.

For both a standalone application and for a feature library, you want that error conditions will be logged, under all circumstances.

For the main thread of an application this can be achieved quite easily, by implementing the method `main()` like this:

```
/**
 * The program entry method.
 *
 * @param args The command line arguments.
 */
public static final void main( final String... args )
{
    try
    {
        final var application = new MyClass();
        applicaton.execute( ... );
    }
    catch( final Throwable t )
    {
        //---* Log the previously unhandled exceptions *-----
        m_Logger.error( "Unhandled_Exception/Error", e );
    }
    // main()
}
```

In this case, the `catch` statement catches `Throwable` and the `catch`-block does nothing else but logging the exception!



This is acceptable here only because the program ends immediately after the closing curly brace of the `catch`-block!²³

Theoretically, you can do the same thing for threads:

```
public final void run()
{
    try
    {
        //---* The thread's payload *-----
        ...
    }
    // NOT Recommended
    catch( final Throwable t )
    {
        //---* Log the previously unhandled exceptions *-----
        m_Logger.error( "Unhandled_Exception/Error", e );
    }
} // run()
```

One reason is that the method `Runnable::run`[267] can be called also in other contexts than just as the main method of a thread; swallowing the exception like shown in the example is a bad idea in that case.

Another reason is that when the method is implemented as part of a library, an extension or even a server-based application, different facilities may be used to report the error.

But the main reason is that there is a much better mechanism that handles those otherwise unhandled exceptions: the “Uncaught Exception Handler”[189].

The interface `java.lang.Thread.UncaughtExceptionHandler` is a functional interface[165, 164, 9] providing the method `uncaughtException()`[277].

There are basically two ways²⁴ to implement an uncaught exception handler; the classical way would be to create a new class:

²³Ok, technically only the main thread ends here because other non-deamon threads may still run at this point. But the goal should be that the main thread dies always at last.

²⁴In fact, three ways. But as you should avoid anonymous classes, this option is omitted here.



8. Coding Guidelines

Listing 8.3: Class `UncaughtExceptionHandlerImpl`

```
1 /**
2  * The implementation of the interface
3  * {@link java.lang.Thread.UncaughtExceptionHandler}
4  * that is used by this project.
5  */
6 public final class UncaughtExceptionHandlerImpl implements
7     Thread.UncaughtExceptionHandler
8 {
9     /**
10     * The logger that is used to report the uncaught exceptions.
11     */
12     private static final Logger m_Logger = getLogger(
13         UncaughtExceptionHandlerImpl.class );
14
15     /**
16     * {@inheritDoc}
17     */
18     @Override
19     public final void uncaughtException( final Thread t, final Throwable e
20     )
21     {
22         m_Logger.error( "Unhandled_Exception/Error_in_Thread_
23             '%s'".formatted( t.getName(), e );
24     } // uncaughtException()
25 }
26 // class UncaughtExceptionHandlerImpl
```

Otherwise you can benefit from `java.lang.Thread.UncaughtExceptionHandler` being a functional interface and implement the uncaught exception handler as a method in your class:

```
public final class MyClass
{
    /**
     * The logger that is used by this class.
     */
    private static final Logger m_Logger = getLogger( MyClass.class );

    /**
     * <p>{@summary This method will be invoked when the given
     * thread terminates due to the given uncaught exception.}</p>
     * <p>Any exception thrown by this method will be ignored by
     * the Java Virtual Machine.</p>
     */
}
```




```
*
* @param t    The thread that terminated.
* @param e    The exception that caused the termination.
*
* @see UncaughtExceptionHandler#uncaughtException(Thread,Throwable)
*/
private final void uncaughtException( final Thread t, final Throwable
    e )
{
    m_Logger.error( "Unhandled_Exception/Error_in_Thread_
        '%s'".formatted( t.getName(), e );
    } // uncaughtException()
}
// class MyClass
```

In the second case, the uncaught exception handler uses the resources provided by the class it lives in. This may be desired or not.

A thread can be configured with the uncaught exception handler in three different ways:

1. Each single thread can get its own uncaught exception handler by calling `setUncaughtExceptionHandler()`[275] on it, with an instance of an implementation of `Thread.UncaughtExceptionHandler` as the argument:

```
public final class MyClass
{
    /**
     * Does the program's work.
     *
     * @throws InterruptedException    The worker thread was
     *                                 interrupted.
     */
    private final void execute() throws InterruptedException
    {
        final var thread = new Thread( this::run, "MyWorkerThread" );
        thread.setUncaughtExceptionHandler( this::uncaughtException );
        thread.start();
        thread.join();
    } // execute()

    /**
     * The thread's main method.
     */
}
```



8. Coding Guidelines

```
private final void run() { ... }  
}  
// class MyClass
```

2. On creation of a thread, a **ThreadGroup**[70] instance will be assigned to it; if none is explicitly set, it will be the group of the current thread.

The instance has a method **uncaughtException()**[278] method is called in the case the thread itself does not have an uncaught exception handler assigned to it. To change the behaviour of the handler from the thread group it is necessary to derive a new class from **ThreadGroup** and to overwrite the **uncaughtException()**.

Chapter 9.6.8 on page 493 shows an implementation for **ThreadGroup** (named **ThreadGroupExt**) that allows to configure the uncaught exception handler; when using that class, the configuration can look like this:

```
public final class MyClass  
{  
    /**  
     * The thread group for all threads used by this program.  
     */  
    private final ThreadGroup m_ThreadGroup;  
  
    /**  
     * Creates a new instance of {@code MyClass}.  
     */  
    public MyClass()  
    {  
        m_ThreadGroup = new ThreadGroupExt( "MyWorkerThreads",  
            this::uncaughtException );  
    } // MyClass()  
  
    /**  
     * Does the program's work.  
     *  
     * @throws InterruptedException The worker thread was  
     * interrupted.  
     */  
    private final void execute() throws InterruptedException  
    {  
        final var thread = new Thread( m_ThreadGroup, this::run,  
            "MyWorkerThread" );
```



```
        thread.start();
        thread.join();
    } // execute()

    /**
     * The thread's main method.
     */
    private final void run() { ... }
}
// class MyClass
```

3. By calling `java.lang.Thread::setDefaultUncaughtExceptionHandler`^[274] it is possible to install an instance of `Thread.UncaughtExceptionHandler` for all threads that does neither have their own handler nor got one from their thread group. This uncaught exception will be called as a last resort.

```
public final class MyClass
{
    /**
     * Does the program's work.
     *
     * @throws InterruptedException The worker thread was
     *                             interrupted.
     */
    private final void execute() throws InterruptedException
    {
        //---* Install the default uncaught exception handler *-----
        Thread.setDefaultUncaughtExceptionHandler( new
            UncaughtExceptionHandlerImpl() );

        final var thread = new Thread( this::run, "MyWorkerThread" );
        thread.start();
        thread.join();
    } // execute()

    /**
     * The thread's main method.
     */
    private final void run() { ... }
}
// class MyClass
```

The last option can be used only from standalone applications and from tools.



The environments for server-based applications and extensions do quite often restrict the creation of additional threads. A feature library should accept an uncaught exception handler and/or a thread group as a configuration value.

If a function library creates new threads, the respective methods must accept at least an uncaught exception handler and optionally a thread group. Alternatively, a thread factory[198] can be injected.

8.2.5. Reporting Errors

As already said earlier, at some point we need to report an error condition to the user of our program; this can happen in various ways. The most common is to log the error somehow, but as logging has various additional aspects, it is covered by a chapter of its own: chapter “8.2.6 Logging” on page 222.

Other means to report the error condition are the following:

- Write an error message to the console, a message window or status bar.
- Display an error dialog.
- Send an email, a tweet or another kind of message.
- Trigger an external signalling device (an alarm bell, a horn, a flashlight, ...).
- ...

Before discussing this further, I want to repeat: *Reporting an error condition is not handling it!*

Obviously, not all ways of reporting an error condition are feasible for all types of products.

- First, a *function library* should not report an error condition in any other way than just throwing the appropriate exception. It should not even log them.



- *Extensions* and *server-based applications* should use the APIs that are provided to them by their hosts/containers for reporting the error conditions, although server-based applications may introduce additional ways to report an error.
- Code in a *function library*, in a *server-based application* and in an *extension* should never write to the console or display an error dialog. The code of a *feature library* can use a message window, status bar or error dialog, if it provides these resources itself (basically, when it provides the UI).

Application servers, frameworks and the hosts for extensions usually provides a facility for error reporting that can be used; this may result in updating a message window or status bar, but for your code, this is transparent.

- Sending messages around to report an error condition should be considered mainly for dark (headless) applications, either standalone or server-based. Of course, this something that can be provided by a feature library (as the new featured), but a feature library should not use it.

Whether it makes sense for a tool depends on the kind of the tool.

- Same for the use of external signalling devices.

In most cases, it is not sufficient to provide the user with the caught exception only – meaning just calling `java.lang.Throwable.printStackTrace` is not enough. In fact, for most users, this is even too much information. Also providing the call stack for an operation in an *error message* is sometimes considered as a security loophole, because it may give a potential attacker information about code internals that they can use to shape their attack against your system.

You should also consider to localise the error message to the locale of the user; that is mainly translating it to the user's language, but not only. You should always add the time zone to timestamps, or at least the offset to UTC, even when you adjust them to the user's time zone.



It is a good idea to have a unique error code with your error message. That allows you to identify it when the user reports it back to you, and you do not understand their language.

Obviously, you should avoid to have sensitive data in your error messages. But 'sensitive data' may depend from the context your program is running in.

8.2.6. Logging

Logging is a mean to report errors and to make the respective messages available even after the program is down, for a post mortem analysis. But you can use logging for other purposes, too.

Nevertheless, *logging* is mainly for the recording of technical issues. If you need to keep track of business transactions (if you have to provide an audit log or a journal), we talk about *journalling* instead. There may be some overlapping between those two, and the requirements look similar, but logging and journalling are two distinct topics and should be treated independently from each other.

Another only slightly related topic is performance logging or monitoring where you issue alert messages in case operations took longer than expected. Of course these can be written to the regular error logs, but in most cases, you should consider a dedicated infrastructure for this feature.

Logging Infrastructure

The simplest way to implement logging would be to open a file during startup and to write the log messages there. That works, but this is not very flexible, although it might be sufficient for a tool.

Because logging is an important feature, Java comes with a logging facility right out of the box, the so called 'JDK Logging'²⁵, provided by the package `java.util.logging`[320, 222].

²⁵Sometimes also referred to as 'JUL', by the name of the package.



Despite the fact that it has improved significantly since its introduction with Java 1.4, most people prefer still use the logging subsystem provided by the external Log4j[18] library.

Next, only tools and standalone applications should configure a logging framework. Function libraries should not log at all, while feature libraries rely on the logging facilities provided by the host environment, same as server-based applications and extensions.

For the latter two the host environment also defines the whole API for logging, but for feature libraries (and for function libraries that really require logging) it is recommended to integrate the SLF4J[329] library.

SLF4J is not a logging framework itself, the library provides only a facade for a logging framework (“SLF4J” stands for “Simple Logging Facade for Java”), its methods delegate to a *real* logging framework; refer to [27] on how this is done. Originally, the API was inspired from Log4j, but it has evolved since then. SLF4J supports Log4j, JDK Logging, Apache Commons Logging[16] (formerly known as JCL – Jakarta Commons Logging) and Logback[244] as well as ‘Simple’ logging. Support for other frameworks can be implemented easily if required.

Looking at the code to write, all three (JDK Logging, Log4j and SLF4J) work basically the same:

1. During the initialisation of your class, obtain an instance of a logger class; the instances are named, usually by the name of the the class that obtains it²⁶. These names are used for the configuration of a logger.
2. Keep the reference in a **private static final** variable with the name **m_Logger**.
3. Check whether the desired log level is active.
4. If the desired log level is active, log your message.

Even the APIs are quite similar, down to the names²⁷:

```
1 // For JDK Logging:
2 import static java.util.logging.Logger.getLogger;
3 import static java.util.logging.Level.*;
```

²⁶This is discussed in more detail in chapter 8.2.6 on page 237.

²⁷Of course the code sample will not compile.



```
4 import java.util.logging.Logger;
5
6 // For Log4j:
7 import static org.apache.logging.log4j.LogManager.getLogger;
8 import static org.apache.logging.log4j.Level.*;
9 import org.apache.logging.log4j.Logger;
10 import org.apache.logging.log4j.message.*;
11
12 // For SLF4J:
13 import static org.slf4j.LoggerFactory.getLogger;
14 import org.slf4j.Logger;
15
16 public final class MyClass
17 {
18     /*-----*\
19     ===** Static Initialisations **=====
20     \*-----*/
21     /**
22      * The logger that is used by this class.
23      */
24     private static final Logger m_Logger = getLogger(
25         MyClass.class.getName() );
26
27     /* For Log4j and SLF4J, this works, too: */
28     private static final Logger m_Logger = getLogger( MyClass.class );
29
30     /* For Log4j, even this will have the same result: */
31     private static final Logger m_Logger = getLogger();
32
33     /*-----*\
34     ===** Methods **=====
35     \*-----*/
36     public final void myMethod()
37     {
38         /*---* Logging an error message *-----
39         final var message = "An_error_occurred!";
40         final var error = new Error( "Something_happened!" );
41
42         /* With JDK Logging */
43         m_Logger.severe( message ); // Without the exception...
44         m_Logger.log( SEVERE, message, error );
45
46         /* With Log4j and SLF4J */
47         m_Logger.error( message, error );
```




```
48      /* With Log4j */
49      m_Logger.log( ERROR, message, error );
50      m_Logger.atError()
51          .withThrowable( error )
52          .log( message );
53
54      //---* Logging an informational message *-----
55      /* With JDK Logging */
56      if( m_Logger.isLoggable( INFO ) )
57      {
58          final var message = getCurrentStatus();
59          m_Logger.log( INFO, message );
60      }
61
62      /* With Log4j and SLF4J */
63      if( m_Logger.isInfoEnabled() )
64      {
65          final var message = getCurrentStatus();
66          m_Logger.info( message );
67      }
68
69      /* With Log4j */
70      m_Logger.atInfo()
71          .log( () -> new SimpleMessage( getCurrentStatus() ) );
72  } // myMethod()
73 }
74 // class MyClass
```

For the details, refer to the respective documentation:

- JDK Logging: `java.util.logging.Logger`[93]
- Log4j: `org.apache.logging.log4j.Logger`[210]
- SLF4J: `org.slf4j.Logger`[211]

The Log Levels

A logger can be configured that it logs only messages with at least a certain priority or *log level*. For JDK Logging, Log4j and SLF4J, the predefined log levels differ:



Table 8.1.: Log Levels

JDK Logging	Log4j	SLF4J	Description
	FATAL		A fatal event that will prevent the application from continuing.
SEVERE	ERROR	ERROR	An error in the application, possibly recoverable. A message level indicating a serious failure.
WARNING	WARN	WARN	An event that might possibly lead to an error.
INFO	INFO	INFO	An event for informational purposes.
CONFIG			A message level for static configuration messages.
FINE	DEBUG		A general debugging event. A message level providing tracing information.
FINER			A fairly detailed tracing message.
FINEST	TRACE	TRACE	A fine-grained debug message, typically capturing the flow through the application. A highly detailed tracing message.

In addition, they also know the ‘levels’ ALL (log everything) and OFF (log nothing), but you cannot check for these.

A lower level includes all the higher levels, too. So when a logger is configured for the log level INFO, `m_Logger.isLoggable()` will return `true` for INFO, WARNING (or WARN), SEVERE, ERROR and FATAL, but not for DEBUG or FINER.

JDK Logging and Log4j allow both to define additional log levels. Basically, this is also possible for SLF4J, but without knowledge about the implementation that is really used for logging, it is quite difficult to determine how these new log levels in SLF4J are mapped to those of the effective logging framework.

So sometimes you want to have a log level that allows to log informational messages under all circumstances. Of course you could use SEVERE or FATAL for that purpose, but that would pollute the logs with false error messages. A better idea would be to introduce a log level INFO_FORCED or VITAL.



The example below shows how to create additional log levels for the JDK Logging:

Listing 8.4: LogLevel.java for JDK Logging

```

1 package org.tquadrat.foundation.logging;
2
3 import static org.apiguardian.api.API.Status.STABLE;
4 import static org.tquadrat.foundation.lang.Objects.requireNonNullArgument;
5 import static org.tquadrat.foundation.lang.Objects.requireNotBlankArgument;
6
7 import java.util.logging.Level;
8
9 import org.apiguardian.api.API;
10 import org.tquadrat.foundation.annotation.ClassVersion;
11
12 /**
13  * <p>{@summary This class is a replacement for
14  * {@link Level}.} It provides alias names for the log levels</p>
15  * <ul>
16  *     <li>{@link Level#FINE}</li>
17  *     <li>{@link Level#FINEST}</li>
18  *     <li>{@link Level#SEVERE}</li>
19  * </ul>
20  * <p>so that the log levels for the JDK logging are better aligned
21  * with those from Log4j.</p>
22  * <p>In addition it provides an additional log level
23  * {@link #INFO_FORCED}</p>
24  *
25  * @author Thomas Thrien - thomas.thrien@tquadrat.org
26  */
27 public class LogLevel extends Level
28 {
29     /*-----*\
30     ===** Static Initialisations **=====
31     \*-----*/
32     /**
33      * {@code DEBUG} is the alias for
34      * {@link #FINE}.
35      */
36     public static final Level DEBUG = new LogLevel( "DEBUG", FINE );
37
38     /**
39      * {@code ERROR} is the alias for
40      * {@link #SEVERE}.
41      */

```



8. Coding Guidelines

```
42     public static final Level ERROR = new LogLevel( "ERROR", SEVERE );
43
44     /**
45      * <p>{@summary {@code INFO_FORCED} is the log level for
46      * informational messages that have to be logged independently
47      * from any logger configuration, under all circumstances.} It
48      * will be initialized to
49      * {@link Integer#MAX_VALUE}
50      * {@code - 1}. This means that it can be still switched off
51      * by setting a logger's log level to
52      * {@link #OFF}.
53      */
54     public static final Level INFO_FORCED = new LogLevel( "INFO_FORCED",
55         Integer.MAX_VALUE - 1 );
56
57     /**
58      * {@code TRACE} is the alias for
59      * {@link #FINEST}.
60      */
61     public static final Level TRACE = new LogLevel( "TRACE", FINEST );
62
63     /*-----*\
64     ===** Constructors **=====
65     \*-----*/
66     /**
67      * Creates a named LogLevel with a given integer value.
68      *
69      * @param name    The name of the LogLevel.
70      * @param value   An integer value for the LogLevel.
71      */
72     protected LogLevel( final String name, final int value )
73     {
74         super( requireNotBlankArgument( name, "name" ), value );
75         // LogLevel()
76
77     /**
78      * Creates a named LogLevel as an alias of the given level.
79      *
80      * @param name    The name of the LogLevel.
81      * @param level   The log level to rename.
82      */
83     protected LogLevel( final String name, final Level level )
84     {
85         super( requireNotBlankArgument( name, "name" ),
86             requireNonNullArgument( level, "level" ).intValue() );
87     }
88 }
```



```
85     } // LogLevel()
86 }
87 // class LogLevel
88
89 /*
90  * End of File
91 */
```

To create a log level `INFO_FORCED` for Log4j, create the respective constant somewhere in your code:

```
public final ApplicationMainClass
{
    ...

    /*-----*\
    ===** Static Initialisations **=====
    \*-----*/
    /**
     * The log level &quot;{@code INFO_FORCED}&quot;.
     */
    public static final Level INFO_FORCED = Level.forName( "INFO_FORCED",
        1 );

    ...
}
// class ApplicationMainClass
```

Note: For Log4j, the integer value for the higher log level is closer to 0 (zero), while for JDK Logging, it is closer to `Integer.MAX_VALUE`.

With Log4j, you can achieve a similar result by using `Logger::always`^[308] instead of creating a new log level:

```
...
m_Logger.always()
    .withThrowable( e )
    .withLocation()
    .log( "Vital_message!" );
...
```



How to log

This chapter discusses how to emit log messages, but also when you should log something, and what you should log.

Errors First, you should log error conditions at some point in the program flow. Usually this means that you will log an exception. And you will log an exception only when you do not further delegate the handling of that exception. For tools and standalone applications, this means that you will log an exception latest in the `main()` method or in the `UncaughtExceptionHandler` for threads other than the main thread (refer to chapter “8.2.4 Exceptions in Threads” on page 212).

We said already that a function library will not log at all. For a feature library, an extension and server-based application, you should log when the control flow is about to leave the realm of your component and you do not know how the host code will handle the exception (or if it will handle and/or them at all).

You should also log (uncaught) exceptions in threads that your code creates; this was already discussed in chapter “8.2.4 Exceptions in Threads”.

As a rule of thumb, you log an exception at the latest possible occasion.

The log message should contain all information that you have at this point; this is at least the exception itself with its message and stacktrace. But perhaps you know additional things, for example what your code attempted when the exception was thrown, the name of resources that should be tried to access, the user name, the access rights and many other things – as long as these are not sensible data! So you should never add a password to a log message. And even the user name is already in a gray area.

Obviously, the log levels `FATAL`, `SEVERE` and `ERROR` are meant for these messages. The other log levels are not meant for reporting *current* error conditions.



Warnings The description for the log levels `WARNING` and `WARN` says that these indicate “an event that might possibly lead to an error” somewhen in the future.

Such an event could be the repeatedly failed login attempt of a user, as that could be the indication for a hacker attack. Most probably, you should not log the first failed attempt, perhaps not even the second or third (at least not as a warning), but consecutive attempts from the same device and/or with the same user name should be logged. And obviously, the respective log message must contain all the necessary data that can be used to repel that attack. Whether the used passwords should be added to the log messages can be discussed.

Obviously, you have to implement the necessary infrastructure to collect the data for all failed (and successful) logins, and you do this only to be able to log the respective warning.²⁸ This means that you can switch it off when the log level is disabled.

In the same way, you should log attempts of unauthorised access to data or functionality by authenticated users.

Another reason to log a warning would be when you handle an error condition by repeating an operation or by performing an alternative operation. Although the current problem is solved for now, it is worth to keep track of it in the logs.

Then you should emit a warning to the logs when a resource gets short, but before it is exhausted; even when the application recovers from that, it should be tracked. If you run out of a particular resource, this would be an error condition, no longer a warning condition.

Obviously, this is not a complete list.

Info You *may* also log other important events that are clearly no error condition nor a warning. The log level for this kind of log entries is `INFO`. You

²⁸Although you can collect this data also for general monitoring if your application lives in an environment where this is provided. Or you make it available through JMX – see chapter 8.47 on page 425.



should assume that this is the default log level for each new logger, meaning that messages with the log levels INFO, WARNING/WARN and FATAL/ERROR/SEVERE are always written to the log file.

An INFO log message may be confirmation for the successfully established connection with an external system, for the start and the termination of a long running background process, for the creation of a new user and alike.

It is also a good idea to log the system configuration on startup with the log level INFO. The log level CONFIG from JDK logging is lower than INFO, therefore using that may cause unwanted behaviour.

You should not log (at least not with a log level of INFO) the successful termination of common operations.²⁹

In general, you should avoid to log the “happy path” in your code – at least, you should not do this with the log level INFO.

Debug and Trace A very basic technic for the debugging of a program is to add statements to the code that prints the current state to the console.

Logging with the log levels DEBUG, TRACE, FINE, FINER, and FINEST is basically the same, just with the difference that you do not emit to a console, but to the log file.

There are some recommendations to log the entry and exit for each method, together with the arguments and the return value. In JDK Logging, the class `java.util.logging.Logger` has a set of specialised methods to support this approach (`Logger::entering` and `Logger::exiting`) that emit messages with the log level FINER. That will look like this:

```
public final class myClass
{
    private static final String CLASS_NAME = myClass.class.getName();
    private static final Logger m_Logger = getLogger( CLASS_NAME );

    ...
}
```

²⁹The kind of application that you create may request an audit log of some kind, but this is not covered by *logging*. Instead we talk about *journalling* here.



```
public final String myMethod( final String value, final boolean flag )
{
    final var METHOD_NAME = "myMethod";
    m_Logger.entering( CLASS_NAME, METHOD_NAME, value,
        Boolean.valueOf( flag ) );

    //---* Does some work here ... *-----
    ...

    //---* Done *-----
    m_Logger.exiting( CLASS_NAME, METHOD_NAME, retValue );
    return retValue;
} // myMethod()
}
// class myClass
```

Log4j has the methods `Logger::traceEntry` and `Logger::traceExit` for the same purpose, that are easier to use:

```
public final class myClass
{
    private static final Logger m_Logger = getLogger();

    ...

    public final String myMethod( final String value, final boolean flag )
    {
        m_Logger.traceEntry( "value: %s - flag: %b", value,
            Boolean.valueOf( flag ) );

        //---* Does some work here ... *-----
        ...

        //---* Done *-----
        m_Logger.traceExit( retValue );
        return retValue;
    } // myMethod()
}
// class myClass
```

But you should be careful with applying these methods to your code. First, because the kind of information provided by this kind of logging is useful only in rare case. And next, if really each and every method is instrumented



8. Coding Guidelines

in that way, the log files will be flooded with messages when the general log level is set to FINER or FINEST for JDK Logging or TRACE for Log4j respectively, making it very difficult to find the information you really need.

In addition, for JDK Logging, it requires a real lot amount of unnecessary code to write, because neither the name of the class nor that of the method will be detected automatically; instead, these values have to be provided as a constant. That also makes some refactorings quite challenging ...

As a result, I do not recommend to use entry/exit logging. In fact, you should generally think twice before you add a log statement for DEBUG and TRACE proactively.

Of course, there are scenarios that are well known to cause issues during runtime quite often and therefore DEBUG logging should be added for them. One good sample is the processing of emails, others are REST and SOAP communications.

But in general, you should never add TRACE logging *proactively*, and DEBUG logging only in special cases.

On the other hand, you should never remove DEBUG and TRACE logging that you added during a bug hunt; it may get useful again if a regression tests fails. This also means that the DEBUG and TRACE log messages should be very verbose so that you can identify them even years later.

```
public final String myMethod( final String value, final boolean flag )
{
    // DISCOURAGED!!
    m_Logger.trace( value );

    // RECOMMENDED
    m_Logger.trace( "BUGHUNT_2022-12-14_{}_entering_myMethod()|value={},_{}_flag={}", value, flag );

    final var buffer = retrieveData( value, flag );

    // DISCOURAGED
    m_Logger.debug( buffer );

    // RECOMMENDED
    if( m_Logger.isDebugEnabled )
    {
```



```

m_Logger.debug( "BUGHUNT_2022-12-14_{}_myMethod()_called_{}_
    retrieveData()|value={},_flag={}", value, flag );
if( isNull( buffer )
{
    m_Logger.debug( "BUGHUNT_2022-12-14_{}_retrieveData()_returned_{}_
        null!" );
}
else
{
    m_Logger.debug( "BUGHUNT_2022-12-14_{}_retrieveData()_returned_{}_
        '{}'", buffer );
}
}

final var retValue = process( buffer );

//---* Done *-----
// DISCOURAGED
m_Logger.debug( retValue );

// RECOMMENDED
m_Logger.trace( "BUGHUNT_2022-12-14_{}_exting_myMethod()|retValue={},_
    '{}'", retValue );

return retValue;
} // myMethod()

```

In chapter 8.2.3 I said that it is usually not a good idea to log an exception and to rethrow or to wrap it afterwards. But there is one exception to this rule: sometimes you want to know about an exception that will be handled later and will be not logged then. Or you have some additional information at the current code location that you cannot add to the wrapper exception (perhaps because of its size). In this case, you can log that exception for debugging (the sample uses Log4j/SLF4J):

```

public final void myMethod() throw IOException
{
    ...
    try
    {
        ...
    }
    catch( final IOException e )
    {

```



```
//---* Log and rethrow *-----  
m_Logger.catching( DEBUG, e );  
throw e;  
}  
...  
try  
{  
    ...  
}  
catch( final IllegalStateException e )  
{  
    //---* Log and wrap *-----  
    if( m_Logger.isDebugEnabled() )  
    {  
        final var message = composeIllegalStateExceptionMessage();  
        m_Logger.debug( message, e );  
    }  
    throw new ApplicationError( "Already_in_use", e );  
}  
...  
} // myMethod()
```

Here you also accept that the same exception might be logged twice.

Configuration I have to confess that I never understood the purpose of the log level CONFIG for the JDK Logging. The description says “A message level for static configuration messages”. So you should use it to log the configuration parameters of your program – things like the current IP address, but also the system properties, or at least the command line parameters.

Of course, these could be relevant information for debugging issues, and it would make sense to log them with each restart of your program.

But CONFIG has a lower priority than INFO – this means that when you activate CONFIG, you have also activated INFO for that logger. This is probably not wanted.

Therefore most people uses a dedicated logger³⁰ to log the configuration settings – but this makes the log level CONFIG completely obsolete.

³⁰refer to chapter “8.2.6 Naming of Loggers” on page 237.



My recommendation would be to completely ignore the log level `CONFIG` and to use a dedicated logger with log level `INFO` instead. If you are using another logging framework than JDK Logging, you don't have another choice either.

Naming of Loggers

Each call to `Logger::getLogger` (for JDK Logging), `LogManager::getLogger` (for Log4j), or `LoggerFactory::getLogger` (for SLF4J) with the same argument will always return the *same* logger instance:

```
if( MyClass.class
    .getName().equals( "org.tquadrat.foundation.sample.logging.MyClass" ) )
{
    // Always true!!
    getLogger( MyClass.class.getName() )
        == getLogger( "org.tquadrat.foundation.sample.logging.MyClass" );
}
```

This means that a logger is created just once for a given name.

Both JDK Logging and Log4j organise their loggers in a parent-child hierarchy with a root logger at the top.³¹

Global configuration settings are applied only to the root logger and will be inherited by/propagated to the children and grand children. Of course you can apply a setting also to a logger that represents a branch in that tree, and this will affect all the loggers in that branch. Settings made specifically for a logger are not overwritten by changes to a parent logger.

For JDK Logging, you can set the parent for each logger explicitly, through `Logger::setParent`[296]. But usually, that relationship is determined by the name of the loggers that is like a path from the root logger to the specific 'leaf' logger, with the dot (".") as the path separator.

In all samples we have seen so far in this chapter, we used the fully qualified name of the owning class as the name for the logger for that class:

³¹This root logger may or may not have a name itself, and your application may have more than one logger hierarchy tree in parallel, each with a separate root logger.



```
package org.tquadrat.foundation.sample.logging;

...

public final class MyClass
{
    ...

    /*-----*\
    ==** Static Initialisations **=====
    \*-----*/
    /**
     * The logger that is used by this class.
     */
    private static final Logger m_Logger = getLogger(
        MyClass.class.getName() );

    /* For Log4j and SLF4J, this works, too: */
    private static final Logger m_Logger = getLogger( MyClass.class );

    /* And for Log4j, this will also return the same result: */
    private static final Logger m_Logger = getLogger();

    ...
}
// class MyClass
```

Obviously, the logger referenced by `m_Logger` will have the name “`org.tquadrat.foundation.sample.logging.MyClass`”. But the call to `getLogger()` will also create the missing parents:

```
function getLogger( name ):Logger
if exists( name )
then getLogger = retrieve( name )
else
parent = getLogger( parentName( name ) )
getLogger = create( name, parent )
```

In the sample above, after the call to `getLogger()`, you will end up with the loggers

- Logger “`org.tquadrat.foundation.sample.logging.MyClass`”
- Logger “`org.tquadrat.foundation.sample.logging`”
- Logger “`org.tquadrat.foundation.sample`”



- Logger “org.tquadrat.foundation”
- Logger “org.tquadrat”
- Logger “org”
- RootLogger

For the regular loggers, it makes a lot of sense to organise them this way. It allows you to switch them on and off per class or per package, as well as to configure them on this level, and usually this is sufficient.

But there are no special requirements for the name of a logger. This means that “logger”, “my.logger”, “1.2.3.4.5.6.7.8” or “Fritz” are all valid logger names. Using the fully qualified class name for the main loggers gives you a predictable naming scheme for the configuration (the log level is not the only configuration parameter to set, see “8.2.6 Logger Configuration” on page 241).

But when you are on a bug hunt and add additional log statements to emit DEBUG or TRACE messages, this usually affects more than one class, and more than one package. So if you use the loggers named after the classes for the debug or trace logging, you always have to configure multiple loggers. That’s not only inconvenient, but also error prone. Not to mention that you might get lots of entries to the log files that are unrelated to the problem that you are currently investigating.

Same when you want to provide a debugging facility for a logical transaction in your application; usually this will also span multiple classes from different packages.

So instead of using multiple class loggers, you should consider to introduce dedicated loggers that are referenced in multiple classes – all the classes that are involved in the respective operation:

```
/**
 * The logger for this class.
 */
private static final Logger m_Logger = getLogger( MyClass.class.getName()
);

/**
 * The loggers for the database transactions.
 */
```



8. Coding Guidelines

```
private static final Logger m_InsertLogger = getLogger( "CRUD.Insert" );
private static final Logger m_SelectLogger = getLogger( "CRUD.Select" );
private static final Logger m_UpdateLogger = getLogger( "CRUD.Update" );
private static final Logger m_DeleteLogger = getLogger( "CRUD.Delete" );

/**
 * The logger for the bug hunt started on 2022-12-14.
 */
private static final Logger m_BugHunt_2022-12-14_Logger = getLogger(
    "BugHunt.2022-12-14" );

...

public final void myMethod( final Operation operation, final Record data )
    throws SQLException
{
    m_BugHunt_2022-12-14_Logger.debug( "entered_MyClass.myMethod|operation_
    =_{ }'", operation );
    try( final var connection = obtainConnection() )
    {
        OperationSwitch: switch( requireNonNullArgument( operation,
            "operation" ) )
        {
            case INSERT:
                m_InsertLogger.get().debug( "Performing_an_Insert" );
                ...
                break OperationSwitch;

            case ...
        } // OperationSwitch:
        m_BugHunt_2022-12-14_Logger.debug( "MyClass.myMethod_-Operation_
        successfully_terminated|operation_={ }'", operation );
    }
    catch( final PoolExhaustedException e )
    {
        m_Logger.warn( "Connection_Pool_exhausted;-operation_postponed", e
        );
        m_BugHunt_2022-12-14_Logger.debug( "MyClass.myMethod_-Operation_
        postponed|operation_={ }'", operation );
        postponeOperation( operation, data );
    }
    catch( final SQLException e )
    {
        m_BugHunt_2022-12-14_Logger.debug( () -> format( "MyClass.myMethod_
        -Operation_aborted|operation_=%s'", operation ), e );
    }
}
```




```
        throw e;
    }
} // myMethod()
```

When using Log4j, instead of having dedicated loggers, you can use *Markers*; refer to [245] to learn more about that.

Logger Configuration

Both, JDK Logging and Log4j are best configured through configuration files.³² These configuration files should be provided with the source code for your product.

But both frameworks allow also a programmatical configuration, at least for the log level. This can be used to add a feature to your application, allowing to change the log levels during runtime and to switch debugging on and off without requiring a restart of the application. It makes definitely sense for server-based applications, to some extent for feature libraries, and of course for long running standalone applications.

JDK Logging and Log4j will also support JMX for changes to the log levels[202, 233]; you can use this with long running tools to enable or disable some debug output during runtime.

JDK Logging The basic information about the configuration of JDK Logging are given in [223, 224] and some important details can be found at [94].

Usually JDK Logging will be configured through a properties file; the default path for that file is `${JAVA_HOME}/conf/logging.properties`, and out-of-the-box, it looks like this:

³²SLF4J does not have a configuration of its own, as it is just a facade to a ‘real’ logging framework, like JDK Logging or Log4j ...



Listing 8.5: Default JDK Logging Configuration

```
1 #####
2 #       Default Logging Configuration File
3 #
4 # You can use a different file by specifying a filename
5 # with the java.util.logging.config.file system property.
6 # For example java -Djava.util.logging.config.file=myfile
7 #####
8
9 #####
10 #       Global properties
11 #####
12
13 # "handlers" specifies a comma separated list of log Handler
14 # classes. These handlers will be installed during VM startup.
15 # Note that these classes must be on the system classpath.
16 # By default we only configure a ConsoleHandler, which will only
17 # show messages at the INFO and above levels.
18 handlers= java.util.logging.ConsoleHandler
19
20 # To also add the FileHandler, use the following line instead.
21 #handlers= java.util.logging.FileHandler, java.util.logging.ConsoleHandler
22
23 # Default global logging level.
24 # This specifies which kinds of events are logged across
25 # all loggers. For any given facility this global level
26 # can be overridden by a facility specific level
27 # Note that the ConsoleHandler also has a separate level
28 # setting to limit messages printed to the console.
29 .level= INFO
30
31 #####
32 # Handler specific properties.
33 # Describes specific configuration info for Handlers.
34 #####
35
36 # default file output is in user's home directory.
37 java.util.logging.FileHandler.pattern = %h/java%u.log
38 java.util.logging.FileHandler.limit = 50000
39 java.util.logging.FileHandler.count = 1
40 # Default number of locks FileHandler can obtain synchronously.
41 # This specifies maximum number of attempts to obtain lock file by
42 # FileHandler implemented by incrementing the unique field %u as per
43 # FileHandler API documentation.
44 java.util.logging.FileHandler.maxLocks = 100
```



```

45 java.util.logging.FileHandler.formatter = java.util.logging.XMLFormatter
46
47 # Limit the message that are printed on the console to INFO and above.
48 java.util.logging.ConsoleHandler.level = INFO
49 java.util.logging.ConsoleHandler.formatter =
    java.util.logging.SimpleFormatter
50
51 # Example to customize the SimpleFormatter output format
52 # to print one-line log message like this:
53 #     <level>: <log message> [<date/time>]
54 #
55 # java.util.logging.SimpleFormatter.format=%4$s: %5$s [%1$tc]%n
56
57 #####
58 # Facility specific properties.
59 # Provides extra control for each logger.
60 #####
61
62 # For example, set the com.xyz.foo logger to only log SEVERE
63 # messages:
64 com.xyz.foo.level = SEVERE

```

To provide your own configuration, you can modify this file (not recommended) or you create your own configuration file that you announce through the system property `java.util.logging.config.file`.³³

A sample configuration file for the sample provided above may look like this:

Listing 8.6: Sample JDK Logging Configuration

```

1 handlers=java.util.logging.FileHandler
2 .level=SEVERE
3
4 java.util.logging.FileHandler.pattern=%h/target/Sample.log
5 java.util.logging.FileHandler.level=ALL
6 java.util.logging.FileHandler.formatter=java.util.logging.SimpleFormatter
7
8 BugHunt_2022-12-14_Logger.level=FINEST
9 CRUD.level=OFF

```

³³Chapter “9.6.4 Load JDK Logging Configuration” on page 485 describes two additional methods how to load the configuration.



It defines a configuration with two loggers, “BugHunt_2022-12-14_Logger” and “CRUD”. The default log level is set to SEVERE, while the logger CRUD is switched off, together with all it is child loggers, and the log level for BugHunt_2022-12-14_Logger is set to FINEST.

A handler determines the destination for the log messages, and here we selected the file handler. A handler has its own log level, and you define the formatter that determines how the log messages will look like when written by the respective handler. Finally, ‘pattern’ specifies the name of the logfile.

This is more or less the simplest configuration you can make. Refer to [222, 320, 94] and the JavaDoc for the various handlers and formatters for more details.

Log4j The configuration for Log4j is described in [116]. It allows four different formats for the configuration file (XML, JSON, YAML, or properties format), but the XML format is still the most common one.

Log4j has the ability to automatically configure itself during initialization.

1. When Log4j starts it will inspect the “log4j2.configurationFile” system property and, if set, will attempt to load the configuration using a **ConfigurationFactory** that matches the file extension.

Note that this is not restricted to a location on the local file system and may contain a URL.

2. If no system property is set, the “properties **ConfigurationFactory**” will look for log4j2-test.properties on the classpath.
3. If no such file is found, the “YAML **ConfigurationFactory**” will look for log4j2-test.yaml or log4j2-test.yml on the classpath.
4. If no such file is found, the “JSON **ConfigurationFactory**” will look for log4j2-test.json or log4j2-test.jsn on the classpath.
5. If no such file is found, the “XML **ConfigurationFactory**” will look for log4j2-test.xml on the classpath.



6. If a test file cannot be located, the “**properties ConfigurationFactory**” will look for `log4j2.properties` on the classpath.
7. If a properties file cannot be located, the “**YAML ConfigurationFactory**” will look for `log4j2.yaml` or `log4j2.yml` on the classpath.
8. If a YAML file cannot be located, the “**JSON ConfigurationFactory**” will look for `log4j2.json` or `log4j2.jsn` on the classpath.
9. If a JSON file cannot be located, the “**XML ConfigurationFactory**” will try to locate `log4j2.xml` on the classpath.
10. If no configuration file could be located, the **DefaultConfiguration**” will be used. This will cause logging output to go to the console.

A `log4j2.xml` file may look like that one below:

Listing 8.7: Sample Log4j Configuration

```
1 <?xml version="1.0"
2   encoding="UTF-8"?>
3 <Configuration name="SampleConfiguration"
4   status="debug"
5   strict="true">
6   <Properties>
7     <Property name="filename">target/Sample.log</Property>
8   </Properties>
9
10  <Appenders>
11    <Appender type="File"
12      name="File"
13      fileName="${filename}">
14      <Layout type="PatternLayout">
15        <Pattern>%d %p %C{1.} [%t] %m%n</Pattern>
16      </Layout>
17    </Appender>
18  </Appenders>
19
20  <Loggers>
21    <Logger name="BugHunt_2022-12-14_Logger"
22      level="trace"
23      additivity="false">
24      <AppenderRef ref="File"/>
25    </Logger>
26
```



```
27     <Logger name="CRUD"  
28         level="off"  
29         additivity="false">  
30         <AppenderRef ref="File"/>  
31     </Logger>  
32  
33     <Root level="error">  
34         <AppenderRef ref="File"/>  
35     </Root>  
36 </Loggers>  
37 </Configuration>
```

It defines a configuration with the name “Sample”, and that has one appender, named “File” and two named loggers, “CRUD” and that infamous “BugHunt_2022-12-14_Logger”, together with the default configuration (with the name “Root”).

An appender is basically the definition of the target for the log messages and how the messages will look like (the ‘layout’); you can define as many appenders as you want. The name of the defined loggers match with those we used in the program code, and you will set the level and the destination (the ‘appender reference’) for it.

But the configuration is much more powerful than that what I will show here. For more details, see the documentation in [116].

Side Effects of Logging

Logging may not have side-effects!

This means that your code have to behave always in the same way, no matter which log levels are active for any of the defined loggers.

Although this is literally impossible (a call to the `log()` method will consume more or less CPU cycles, depending on whether the respective log level is active or not), you should take care to reduce the impact as much as possible.

On the other side, an inactive log level also should not have any impact.



The good thing is that this problem usually shows up only for DEBUG and TRACE, rarely for INFO.

Some samples:

```
1 public final void myMethod()
2 {
3     // AVOID!
4     final var logMessage = composeLogMessage();
5     m_Logger.debug( logMessage );
6
7     // BETTER
8     if( m_Logger.isDebugEnabled() )
9     {
10         final var logMessage = composeLogMessage();
11         m_Logger.debug( logMessage );
12     }
13
14     // RECOMMENDED
15     m_Logger.debug( this::composeLogMessage );
16 }
```

The method `composeLogMessage()` is just a placeholder here, but to keep the code readable, it is sometimes really a good idea to place the generation of the logging output into a dedicated `private` method.

```
1 public final void myMethod()
2 {
3     // AVOID!
4     final var bufferForLogOutput = new StringBuilder();
5     while( input.hasMore() )
6     {
7         final var line = processInput( input.next() );
8         output.println( line );
9         bufferForLogOutput.append( line )
10             .append( '\n' );
11     }
12     m_Logger.debug( "Output:_{}", bufferForLogOutput.toString() );
13
14     // A LITTLE BIT BETTER
15     StringBuilder bufferForLogOutput = null;
16     if( m_Logger.isLoggable( DEBUG ) ) bufferForLogOutput = new
17         StringBuilder();
18     while( input.hasMore() )
19     {
```



8. Coding Guidelines

```
19     final var line = processInput( input.next() );
20     output.println( line );
21     if( nonNull( bufferForLogOutput ) )
22     {
23         bufferForLogOutput.append( line )
24         .append( '\n' );
25     }
26 }
27 m_Logger.debug( "Output:_{}", bufferForLogOutput.toString() );
28
29 // RECOMMENDED
30 while( input.hasMore() )
31 {
32     final var line = processInput( input.next() );
33     output.println( line );
34     m_Logger.debug( "Output:_{}", line );
35 }
36 } // myMethod()
```

The variants starting in line 3 and line 14 have both the risk that they may cause an **OutOfMemoryError** when the output is larger. And it is generally a bad idea to have a variable that exists only for debugging purposes.

On the other hand, the variant starting on line 29 may pollute the logs with a high number of messages. But for debugging purposes, this should be acceptable.

Something that I have seen in real life code, another variant of the pattern above:

```
1 public final void myMethod() throws IOException
2 {
3     // AVOID! BY ALL MEANS!!
4     PrintStream logOutput = null;
5     if( m_Logger.isLoggable( DEBUG ) )
6     {
7         final var logOutputName = generateFileName();
8         logOutput = new PrintStream( new FileOutputStream( logOutputName )
9         );
10        m_Logger.debug( "Writing_logOutput_File_{}", logOutputName );
11    };
12    while( input.hasMore() )
13    {
14        final var line = processInput( input.next() );
```




```
14     output.println( line );
15     if( nonNull( logOutput ) )
16     {
17         bufferForLogOutput.append( line )
18         .append( '\n' );
19     }
20 }
21 if( nonNull( logOutput ) ) logOutput.close();
22 } // myMethod()
```

In this case, enabling DEBUG for the logger can cause exceptions that will never occur in normal operation.

One more?

```
1 public final Data myMethod() throws IOException
2 {
3     // AVOID! BY ALL MEANS!!
4     final Data retValue;
5
6     if( m_Logger.isTraceEnabled() )
7     {
8         retValue = composeOutputDataWithTrace();
9     }
10    else
11    {
12        retValue = composeOutputData();
13    }
14
15    //---* Done *-----
16    return retValue;
17 } // myMethod()
```

Do I have to comment this?

As a conclusion, you should avoid side effects of (debug) logging as much as possible. The impact on time (CPU cycles) can be reduced if the operations done for logging are kept short. And instead of collecting large data chunks, log each small chunk separately.



8.3. Checking Method Parameters and Return Values

Principles

P1. A `NullPointerException` indicates a serious programming bug.

A thrown `NullPointerException`[58] reveals that either a method argument or a return value was not properly checked before it was further used.

Your code should never throw a `NullPointerException`!³⁴ Nevertheless, values can still be `null`, for various reasons, and if this is indicating an error condition, this has to be signalled.

P2. Each method or constructor argument needs to be validated.

The arguments must be checked for `null`, but also to ensure that they meet any further specifications (e.g. that they are in a given range of values).

P3. Consolidate the validation methods in one place.

To avoid that you repeat yourself on validating the arguments (refer to the “Coding Guardrails for Java in a Nutshell” bullet point 3) you should consider to provide a utility class as central location for validation methods; using (domain) value types would be an alternative (refer to chapter “8.10 Value Types” on page 336).

P4. Validate any return value.

If `null` is a valid return value of an invoked method, check whether the returned value is not `null` before further using that value. In particular if the invoked method is not part of the code written by you.

³⁴And, of course, your code should also never catch a `NullPointerException`. Instead it should be treated in the same way as an **Error**.



P5. Fail fast.

Arguments and return values should be checked as soon as possible, so that illegal values are detected fast – and before damage can be caused.

P6. Avoid `null` as a valid return value.

At least methods that are part of a public API (methods that are `public` or `protected`) should not return `null`.

Consequently, you need to check the arguments for each and every method and constructor. In case an argument value is invalid, you should throw an `java.lang.IllegalArgumentException`[52] or another exception derived from that; chapter 9.6.2 on page 471 provides some sample implementations for exceptions that can be used to signal an invalid argument.

Beside the arguments to a method or constructor, you also have to check the values that are returned by a method call, at least that they are not `null`. But don't be paranoid: if the specification of the called method clearly declares that the method will not return `null`, you should trust it³⁵.

8.3.1. How to implement the Validation

A simple implementation for validating an argument could look like this:

```
1 public final void myMethod( final String s )
2 {
3     if( s == null ) throw new IllegalArgumentException( "s_is_null" );
4     if( s.isEmpty() ) throw new IllegalArgumentException( "s_is_empty" );
5     if( s.isBlank() ) throw new IllegalArgumentException( "s_is_blank" );
6
7     ...
8 } // myMethod()
```

That works for methods, but not for a constructor that calls another constructor:

³⁵Although you can still add an `assert` statement to ensure that – see chapter “8.3.4 Assertions” on page 263.



8. Coding Guidelines

```
1 // DOES NOT COMPILE!
2 public final class MyClass
3 {
4     public final void MyClass( final String s )
5     {
6         if( s == null ) throw new IllegalArgumentException( "s_is_null" );
7         if( s.isEmpty() ) throw new IllegalArgumentException( "s_is_empty"
8         );
9         if( s.isBlank() ) throw new IllegalArgumentException( "s_is_blank"
10        );
11
12        this( s, true );
13    }
14    // class MyClass
```

This is because the call to `this()` or `super()` has to be the first statement in the constructor's body code block.

The class `java.util.Objects`[95] provides a method `requireNonNull()`[303] that allows you to write at least the `null` check like this:

```
1 import static java.util.Objects.requireNonNull;
2
3 public final class MyClass
4 {
5     public final void MyClass( final String s )
6     {
7         this( requireNonNull( s, "s_is_null" ), true );
8     }
9     // MyClass()
10 }
11 // class MyClass
```

Unfortunately, `java.util.Objects::requireNonNull` throws a `NullPointerException`[58] and that is not wanted. Therefore I invented a replacement and an extension for `java.util.Objects`; the class `org.tquadrat.foundation.lang.Objects`[354] is part of my foundation library[356].

With my `Objects` class, you can write:



```

1 import static org.tquadrat.foundation.lang.Objects.requireNotBlankArgument;
2
3 public final class MyClass
4 {
5     public final void MyClass( final String s )
6     {
7         this( requireNotBlankArgument( s, "s" ), true );
8         ...
9     } // MyClass()
10 }
11 // class MyClass

```

The method `Objects::requireNotBlankArgument` (see below for the implementation) throws a `NullArgumentException`[348] in case the argument is `null`, an `EmptyArgumentException`[346] if the argument is empty, and when the argument is string that contains only whitespace, a `BlankArgumentException`[345]. All three exception classes extend `ValidationException`[352], and that extends `java.lang.IllegalArgumentException`[52].

To check an argument for `null` only, you can use `Objects::requireNonNullArgument`, and for the empty check, the method `Objects::requireNotEmptyArgument` (see below) is available.

The method `requireNonNullArgument()` looks like this:

Listing 8.8: `requireNonNullArgument()`

```

1 /**
2  * Checks if the given argument {@code a} is {@code null} and throws a
3  * {@link NullArgumentException}
4  * if it is {@code null}.
5  *
6  * @param <T> The type of the argument to check.
7  * @param a   The argument to check.
8  * @param name The name of the argument; this is used for the
9  *            error message.
10 * @return The argument if it is not {@code null}.
11 * @throws NullArgumentException  {@code a} is {@code null}.
12 */
13 public static final <T> T requireNonNullArgument( final T a, final String
14     name )
15 {
16     if( isNull( name ) ) throw new NullArgumentException( "name" );

```



8. Coding Guidelines

```
16     if( name.isEmpty() ) throw new EmptyArgumentException( "name" );
17     if( isNull( a ) ) throw new NullArgumentException( name );
18
19     //--* Done *-----
20     return a;
21 } // requireNonNullArgument()
```

In general, it is a good idea to check the arguments for a method or a constructor as soon as possible, but it needs to be checked latest before it is used in some way.

We can check an argument for being empty, and Strings can be even tested for being blank, in the same way as for the null check:

Listing 8.9: requireNotEmptyArgument()

```
1  /**
2   * <p>{@summary Checks if the given argument {@code arg} is
3   * {@code null} or empty and throws a
4   * {@link NullArgumentException}
5   * if it is {@code null}, or an
6   * {@link EmptyArgumentException}
7   * if it is empty.</p>
8   * <p>Strings, arrays, instances of
9   * {@link java.util.Collection} and
10  * {@link java.util.Map}
11  * as well as instances of
12  * {@link java.lang.StringBuilder},
13  * {@link java.lang.StringBuffer},
14  * and
15  * {@link java.lang.CharSequence}
16  * will be checked on being empty.</p>
17  * <p>For an instance of
18  * {@link java.util.Optional},
19  * the presence of a value is checked in order to determine whether
20  * the
21  * {@link Optional} is empty or not.</p>
22  * <p>Because the interface
23  * {@link java.util.Enumeration}
24  * does not provide an API for the check on emptiness
25  * ({@link java.util.Enumeration#hasMoreElements() hasMoreElements()})
26  * will return {@code false} after all elements have been taken from
27  * the {@code Enumeration} instance), the result for arguments of
28  * this type has to be taken with caution.</p>
29  * <p>For instances of
```



```

30 * {@link java.util.stream.Stream},
31 * this method will only check for {@code null} (like
32 * {@link #requireNonNullArgument(Object,String)}).
33 * This is because any operation on the stream itself would render
34 * it unusable for later processing.</p>
35 * <p>In case the argument is of type
36 * {@link Optional},
37 * this method behaves different from
38 * {@link #requireNotEmptyArgument(Optional,String)};
39 * this one will return the {@code Optional} instance, while the
40 * other method will return the contents of the
41 * {@code Optional}.</p>
42 * <p>This method will not work properly for instances of
43 * {@link java.util.StringJoiner}, because its method
44 * {@link java.util.StringJoiner#length() length()}
45 * will not return 0 when a prefix, suffix, or an
46 * &quot;{@linkplain java.util.StringJoiner#setEmptyValue(CharSequence)
47 * empty value}&quot; was provided.</p>
48 *
49 * @param <T> The type of the argument to check.
50 * @param arg The argument to check; may be {@code null}.
51 * @param name The name of the argument; this is used for the
52 * error message.
53 * @return The argument if it is not {@code null} or empty.
54 * @throws NullPointerException {@code arg} is {@code null}.
55 * @throws EmptyArgumentException {@code arg} is empty.
56 */
57 public static final <T> T requireNotEmptyArgument( final T arg, final
    String name )
58 {
59     if( isNull( name ) ) throw new NullPointerException( "name" );
60     if( name.isEmpty() ) throw new EmptyArgumentException( "name" );
61
62     switch( arg )
63     {
64         /*
65          * When using guarding expressions, the code would not get
66          * better to read and to understand, as the positive cases
67          * will be handled all by the default case then.
68          */
69         case null -> throw new NullPointerException( name );
70         case CharSequence charSequence ->
71         {
72             if( charSequence.isEmpty() ) throw new EmptyArgumentException(
                name );

```



8. Coding Guidelines

```
73     }
74     case Collection<?> collection ->
75     {
76         if( collection.isEmpty() ) throw new EmptyArgumentException(
            name );
77     }
78     case Map<?,?> map ->
79     {
80         if( map.isEmpty() ) throw new EmptyArgumentException( name );
81     }
82     case Enumeration<?> enumeration ->
83     {
84         /*
85          * The funny thing with an Enumeration is that it could
86          * have been not empty in the beginning, but it may be
87          * empty (= having no more elements) now.
88          * The good thing is that Enumeration.hasMoreElements()
89          * will not change the state of the Enumeration - at
90          * least it should not do so.
91          */
92         if( !enumeration.hasMoreElements() ) throw new
            EmptyArgumentException( name );
93     }
94     case Optional<?> optional ->
95     {
96         if( optional.isEmpty() ) throw new EmptyArgumentException(
            name );
97     }
98     default ->
99     {
100         if( arg.getClass().isArray() )
101         {
102             if( Array.getLength( arg ) == 0 ) throw new
                EmptyArgumentException( name );
103         }
104         else
105         {
106             /*
107              * Other data types are not further processed; in
108              * particular, instances of Stream cannot be checked
109              * on being empty. This is because any operation on
110              * the Stream itself will change its state and may
111              * make the Stream unusable.
112              */
113         }
```




```

114     }
115 }
116
117 //---* Done *-----
118 return arg;
119 } // requireNotEmptyArgument()

```

Listing 8.10: requireNotBlankArgument()

```

1 /**
2  * <p>{@summary Checks if the given String argument {@code arg} is
3  * {@code null}, empty or blank and throws a
4  * {@link NullPointerException}
5  * if it is {@code null}, an
6  * {@link EmptyArgumentException}
7  * if it is empty, or a
8  * {@link BlankArgumentException}
9  * if it is blank.</p>
10 *
11 * @param <T> The type of the argument to check.
12 * @param arg The argument to check; may be {@code null}.
13 * @param name The name of the argument; this is used for the
14 * error message.
15 * @return The argument if it is not {@code null}, empty or blank.
16 * @throws NullPointerException {@code arg} is {@code null}.
17 * @throws EmptyArgumentException {@code arg} is empty.
18 * @throws BlankArgumentException {@code arg} is blank.
19 *
20 * @see String#isBlank()
21 */
22 public static final <T extends CharSequence> T requireNotBlankArgument(
23     final T arg, final String name )
24 {
25     if( isNull( name ) ) throw new NullPointerException( "name" );
26     if( name.isEmpty() ) throw new EmptyArgumentException( "name" );
27
28     switch( arg )
29     {
30         case null -> throw new NullPointerException( name );
31         case String string ->
32         {
33             if( string.isEmpty() ) throw new EmptyArgumentException( name );
34             if( string.isBlank() ) throw new BlankArgumentException( name );
35         }
36     }
37 }

```



```
34     }
35     case CharSequence charSequence ->
36     {
37         if( charSequence.isEmpty() ) throw new EmptyArgumentException(
38             name );
39         if( charSequence.toString().isBlank() ) throw new
40             BlankArgumentException( name );
41     }
42     //---* Done *-----
43     return arg;
44 } // requireNotBlankArgument()
```

As said, the methods above are implemented in the utility class **Objects**[354] from the package **org.tquadrat.foundation.lang** that also provides reimplementations of the methods from **java.util.Objects**; these methods throw now **NullArgumentException** instead of **NullPointerException**.

That utility class provides some more methods that support the validation of arguments; for the details, refer to [354].

- **checkFromIndexSize()**[361] Checks if a **fromIndex** value fits into a given range for a given size of a sub-range.

```
public final byte [] cutOut( final byte [] source, final int
    fromIndex, final int size )
{
    final var retValue = copyOfRange( requireNonNullArgument( source
        ), checkFromIndexSize( fromIndex, size, source.length ),
        fromIndex + size );

    //---* Done *-----
    return retValue;
} // cutOut()
```

The method **Arrays::copyOfRange**[286] will of course check the arguments by itself, but the main goal of the sample code above is to illustrate how to use the methods from **Objects**.

- **checkFromToIndex()**[362] Checks if a **fromIndex** and a **toIndex** value fits into a given range.



```
public final byte [] cutOut( final byte [] source, final int
    fromIndex, final int toIndex )
{
    final var retValue = copyOfRange( requireNonNullArgument( source
        ), checkFromToIndex( fromIndex, toIndex, source.length ),
        toIndex );

    //---* Done *-----
    return retValue;
} // cutOut()
```

- **checkIndex()**[363] Checks if the given value is within the bounds of the range from 0 (inclusive) to the given length (exclusive).

```
public final void setAge( final int age )
{
    m_Age = checkIndex( age, MAX_AGE + 1 );
} // setAge()
```

- **require()**[364] Applies a given validation on the given value, and if that validation fails, a **ValidationException** with a specified message is thrown. For the check on argument, prefer **requireValidArgument()**[366], this method is more feasible for the check on return values.

```
final var connection = require( obtainConnection(), "Connection_is_
    closed", Connection::isOpen );
```

- **requireNonNull()**[365] This message throws a **ValidationException** when the given argument is **null**. For the **null** check on arguments, you should prefer **requireNonNullArgument()**, this method is more feasible for the **null** check on the return value of a method.

```
final var configFileName = "config.file";
final var resourceStream = requireNonNull(
    getClass().getResourceAsStream( configFileName ), () -> format(
        "Cannot_open_resource_ '%s'", configFileName ) );
```

- **requireValidArgument()**[366] Applies a given validation on the given value, and if that validation fails, a **ValidationException** with a specified message is thrown.



```
private final boolean checkPassword( final String password )
{
    final var retValue = (requireNotBlankArgument( password,
        "password" ).length() > 8)
        && containsDigit( password )
        && containsPunctuation( password )
        && !password.equals( password.toLowerCase( ROOT ) )
        && !password.equals( password.toUpperCase( ROOT ) );

    //---* Done *-----
    return retValue;
} // checkPassword()

public final void setPassword( final String password )
{
    m_Password = requireValidArgument( password, "password",
        this::checkPassword );
} // setPassword()
```

Similar methods exist for the primitive types `double`, `int` and `long`.

You may have noticed that I always use `isNull()`[\[301\]](#) and `nonNull()`[\[302\]](#) from `java.util.Objects` instead of `... == null` and `... != null`. I think that the method calls are easier to read than the `==` and `!=` operators. Both methods are also defined in `org.tquadrat.foundation.lang.Objects`[\[354\]](#).

8.3.2. Delegating the Validation

It is possible to delegate the argument check to another method, like shown in lines 20 and 23:

```
1 public final class MyClass
2 {
3     public MyClass( final String value )
4     {
5         if( isNull( value ) ) throw new IllegalArgumentException( "value_
6             is_null" );
7         m_Value = value;
8     } // MyClass()
9 }
```



```

10 public final void myMethod( final String value )
11 {
12     if( isNull( value ) ) throw new IllegalArgumentException( "value_
        is_null" );
13
14     if( value.equals( m_Value ) ) ...
15 } // myMethod()
16
17 public static final MyClass myFactory( final String value1, final
    String value2 )
18 {
19     //---* Null check is done by constructor *-----
20     final var retValue = new MyClass( value1 );
21
22     //---* Null check is done by myMethod() *-----
23     retValue.myMethod( value2 );
24 } // myFactory()
25 }
26 // class MyClass

```

The comments as in lines 19 and 22 are mandatory.

8.3.3. Foreign Code

The methods and constructors from external libraries (but also from the Java Runtime library) may still throw a `NullPointerException` if called with a `null` argument, but we are not allowed to catch that exception ...

This means that we have to check the values of the arguments on `null` *before* we call a method or a constructor from that foreign code:

```

final var partialArray = copyOfRange( requireNonNull( source ), fromIndex,
    toIndex );

```

Or we wrap the call to that 3rd method into a method of our own:

```

public final static byte [] copyOfRange( final byte [] source, final int
    fromIndex, final int toIndex )
{
    final var retValue = Arrays.copyOfRange( requireNonNullArgument(
        source ), checkFromToIndex( fromIndex, toIndex, source.length ),
        toIndex );
}

```



8. Coding Guidelines

```
    //---* Done *-----  
    return retValue;  
}    // copyOfRange()
```

Of course we can also provide a facade class for the foreign stuff, either by extending the classes (if possible), or by wrapping it:

```
public final class ForeignFacade extends ForeignClass  
{  
    public ForeignFacade( final String value, final Data data )  
    {  
        super( requireNonNullArgument( value, "value" ),  
               requireNonNullArgument( data, "data" ) );  
    }    // ForeignFacade()  
  
    @Override  
    public final void foreignMethod( final String value, final Data data )  
    {  
        super.foreignMethod( requireNonNullArgument( value, "value" ),  
                              requireNonNullArgument( data, "data" ) );  
    }    // foreignMethod()  
}  
// class ForeignFacade  
  
public final class ForeignWrapper  
{  
    private final ForeignClass m_WrappedInstance;  
  
    public ForeignWrapper( final String value, final Data data )  
    {  
        m_WrappedInstance = new ForeignClass( requireNonNullArgument(  
            value, "value" ), requireNonNullArgument( data, "data" ) );  
    }    // ForeignWrapper()  
  
    @Override  
    public final void foreignMethod( final String value, final Data data )  
    {  
        m_WrappedInstance.foreignMethod( requireNonNullArgument( value,  
            "value" ), requireNonNullArgument( data, "data" ) );  
    }    // foreignMethod()  
}  
// class ForeignWrapper
```



Regarding other validations, you can decide that your code will either perform them, too, before calling the foreign code, or that your code catches the respective exceptions and wraps them into an `IllegalArgumentException` or a `ValidationException`, or you just go with the exceptions from that 3rd party code³⁶.

8.3.4. Assertions

Java 1.4 introduced the new keyword `assert`[137] to the language. At first glance, this looks like the appropriate argument-checking feature – but ...

“[...]Java’s `assert` keyword is unique in two very interesting ways:

1. The intended use of this keyword is for test environments, or for temporarily performing debugging routines on production systems. No other keyword is targeted toward testing and debugging in the same way as the Java `assert` statement.
2. The Java `assert` keyword is ignored when the Java Virtual Machine (JVM) is run using a default configuration. At startup, a special `enableassertions` runtime variable must be passed to the JVM as a command-line argument in order for code associated with with a Java `assert` to be executed. There is no other keyword in the Java language that is disabled by default at runtime.

[...]”

Cameron McKenzie, TechTarget[250]

The validation of arguments and return values has to be active also in production, not only during test, and it should not be possible to switch off that validation. Therefore `assert` is not suitable for general validation.

As said in the Java Language Specification[137]:

³⁶In the latter case, you should check whether the foreign code will be consistent in this regard; if not, you should prefer one of the other two options.



“[Because they can be disabled], assertions should not be used for argument checking in **public** methods. Argument checking is typically part of the contract of a method, and this contract must be upheld whether assertions are enabled or disabled.

A secondary problem with using assertions for argument checking is that erroneous arguments should result in an appropriate run-time exception [...]. An assertion failure will not throw an appropriate exception. Again, it is not illegal to use assertions for argument checking on public methods, but it is generally inappropriate. It is intended that **AssertionError** never be caught, but it is possible to do so, thus the rules for **try** statements should treat assertions appearing in a **try** block similarly to the current treatment of throw statements.”

Because it is acceptable to omit the validation for **private** methods and constructors, when the *caller* guarantees that the arguments are valid, you should add **assert** statements here that check the arguments[323]. These are active during the testing phase, verifying that the caller will really invoke the method with valid arguments only, but switched off in production.

This can look like this:

```
public final class MyClass
{
    private MyClass( final String value )
    {
        assert nonNull( value ) : "value_is_null";

        m_Value = value;
    } // MyClass()

    private final void myMethod( final String value )
    {
        assert nonNull( value ) : "value_is_null";

        if( value.equals( m_Value ) ) ...
    } // myMethod()

    public static final Optional<MyClass> myFactory( final String value1,
        final String value2 )
    {
```




```
final var retValue = Optional.ofNullable( nonNull( value1 ) ? new
    MyClass( value1 ) : null );
if( nonNull( value2 ) ) retValue.ifPresent( v -> v.myMethod(
    value2 ) );

//---* Done *-----
return retValue;
} // myFactory()
}
// class MyClass
```

In the same way you can check the values returned by method calls, in particular those returned by methods from foreign code. But you should not use **assert** to check results that depend from external conditions, like the existence of files or whether a connection attempt was successful.

8.4. Extending Classes, Overriding Methods

This chapter provides some hints for the design and implementation of classes and methods that should be/can be extended. This is basically relevant for libraries, but also for applications that should be customisable in some way and therefore providing an API.

Principles

P1. Avoid complex class hierachies.

Inheritance is a 'chainsaw tool': very powerful, but with the inherent capability to cut off your foot. And its dangerousness increases with the complexity of the inheritance hierarchy. Therefore keep your hierarchies simple and small.

P2. The extensibility of your code should not be based on inheritance.

Implementing an interface or extending a simple abstract class from an SPI is alright, but the requirement of plugging in new child classes into a multi-level class hierarchy can cause severe problems later.



P3. Extendable classes have to be build for being extended.

This means first that the mount or extension points have to be marked clearly, and that they need to be documented properly.

Next it means that all classes that are not meant to be extended have to be `final`.

8.4.1. '`final`' for Classes and Methods

Each non-`private` method, that is not explicitly meant to be overridden by an extending class has to be `final`. Each non-`final` class is implicitly meant to be extended – even if it contains only `final` methods.

Non-`final` methods require a comment about when and how they can be overridden, and if the super implementation has to be called, and when. On how to write the comments for classes and methods, refer to the respective chapters 7.1.1 on page 153 and 7.1.1 on page 157.

`private` and `static` methods are implicitly `final`. Nevertheless, in the source code they will be marked as `final`, too, although that is redundant. If your IDE issues a warning about that, you should switch that off.

8.4.2. Non-`final` Classes

As already said above, each non-`final` class is implicitly meant to be extended. This means, that it has to be designed in a way that supports that extension, and it has to provide proper documentation on how to implement the extension and when.

Only when the class is `sealed`[157, 160], this is obsolete³⁷: for a `sealed` class (and for a `sealed` interface), all implementations have to be provided by your code. This is discussed further in chapter “8.35.1 Encapsulation with Modules” on page 420.

³⁷Ok, ‘redundant’ may be the better term here, because it could also support maintenance when the documentation elaborates on the inheritance concept also for these classes.



Unfortunately it is difficult to provide a globally valid recipe for extensible classes. In general, such a class should provide some so called ‘mount points’ or ‘extension points’ where the behaviour could be changed or more features could be added. This could be an **abstract** method, an empty, non-**final** place holder method, or a non-**final** method providing some default behaviour. Such a method should be marked with the annotation **@Mountpoint**³⁸ (if not **abstract** or part of an interface), to indicate that you have not just forgotten to add the **final** keyword.

An extensible class can also provide useful and/or convenience functionality as **protected final** methods.

Although *it is not recommended*, an extensible class can even provide direct access to fields by declaring them **protected** instead of **private**.

8.4.3. Non-final Methods

This chapter deals with non-**final** and non-**abstract** methods. Such a method will be overridden to change the behaviour of the child class, compared with the base class. The method either has already a behaviour, or it is empty, doing nothing.

In the latter case, it is very likely that this method was added with the focus on designing an extendable class.

In the other case, the new method’s behaviour will completely replace that of the original one with new functionality. If this is not possible – meaning that some of the original method’s functionality has to be kept, this needs to be described in the documentation comment for that method. Or, even better, you should change the design of your class to use the Template Method Pattern[127].

As said already is it difficult to come up with a generic recipe for good design of an extendable class, so let me try it with an example here. Assume your class has to implement the following process:

1. Initialisation

³⁸Refer to chapter 9.6.5 on page 487 for an implementation of the Annotation.



2. Gathering data
3. Validate the input data
4. Filtering the input data
5. Processing the data
6. Formatting the output data
7. Cleanup and housekeeping
8. Writing the output

Your first, naïve implementation looks like this:

```
1 public class MyClass
2 {
3     public String process() throws IOException
4     {
5         //---* Initialise the process context *-----
6         final var context = ...
7
8         //---* Gather the input data *-----
9         final Data inputData;
10        try( final var inputFile = new FileInputStream( "inputFileName" ) )
11        {
12            inputData = readStream( inputFile );
13        }
14
15        //---* Validate and filter the data *-----
16        final var filteredData = ... // Do something with inputData
17
18        //---* Process the filtered data to the result data *-----
19        final var resultData = ... // Do something with filteredData
20
21        //---* Format the data *-----
22        final var retValue = ... // Do something with resultData
23
24        //---* Cleanup *-----
25        // Do whatever is necessary ...
26
27        //---* Done *-----
28        /*
29         * Let the caller write the data to wherever it should end up.
30        */
```



```

31     return retValue;
32 }    // process()
33 }
34 // class MyClass

```

Works! Job done! Until your project manager returns to you with the requirement that the input data should be possible to specify alternative sources for the input data than just a file, that different filter schemes are possible when the class is used in different contexts, and that the output format has to be customisable, depending on the part of the application that uses the class. But the initialisation step, the validation and the housekeeping are fine ...

Ok, the method `MyClass::process` is not final, you will override it in `CustomClass`:³⁹

```

1 public class CustomClass extends MyClass
2 {
3     @Override
4     public String process() throws IOException
5     {
6         //---* Initialise the process context *-----
7         final var context = ...
8
9         //---* Gather the input data *-----
10        final Data inputData;
11        try( final var inputFile = new URL( "192.168.0.1" ).openStream() )
12        {
13            inputData = readStream( inputFile );
14        }
15
16        //---* Validate and filter the data *-----
17        final var filteredData = ... // Do something with inputData
18
19        //---* Process the filtered data to the result data *-----
20        final var resultData = ... // Do something with filteredData
21
22        //---* Format the data *-----
23        final var retValue = ... // Do something with resultData that's
24        // different from what MyClass::process did here

```

³⁹Of course implementing inheritance is not the only option you have to fulfil the requirements of your project manager in this case; it could be done also through implementing the Command Pattern[127], and there are still more ways it could be done. But we are discussing inheritance, the extension of classes and overriding of methods here.



8. Coding Guidelines

```
25
26     //---* Cleanup *-----
27     // Do whatever is necessary...
28
29     //---* Done *-----
30     /*
31      * Let the caller write the data to wherever it should end up.
32      */
33     return retValue;
34 }    // process()
35 }
36 // class CustomClass
```

But obviously, this cannot be the solution ... so you modify the original class, **MyClass**:

```
1  public class MyClass // Second attempt
2  {
3      /**
4       * Validate and filters the input data. Usually, no filtering should
5       * be required.
6       * An implementation of this method in a subclass needs to call this
7       * implementation in order to validate the data.
8       *
9       * @param data    The raw input data.
10      * @param context The process context.
11      * @return The validated and filtered data.
12      * @throws IllegalArgumentException The input data is invalid.
13      */
14     @MountPoint
15     protected Data filterData( final Data data, final ProcessContext
16                               context )
17     {
18         if( /* data is not valid */ ) throw new IllegalArgumentException();
19
20         //---* Done *-----
21         return data;
22     }    // filterData()
23
24     /**
25      * Formats the output data.
26      *
27      * @param data    The data to prepare for the output.
28      * @param context The process context.
29      * @return The formatted data.
```



```

29  */
30  @MountPoint
31  protected String formatData( final Data data, final ProcessContext
    context )
32  {
33      final var retValue = ... // Do something with data
34
35      //---* Done *-----
36      return retValue;
37  } // formatData()
38
39  /**
40   * Obtains the input data from somewhere; the default
41   * implementation reads it from the file {@code inputFileName}.
42   *
43   * @return The input data.
44   * @throws IOException An I/O error occurred while gathering the
45   *         data.
46   */
47  @MountPoint
48  protected Data obtainInputData() throws IOException
49  {
50      final Data retValue;
51      try( final var inputFile = new FileInputStream( "inputFileName" ) )
52      {
53          retValue = readStream( inputFile );
54      }
55
56      //---* Done *-----
57      return retValue;
58  } // obtainInputData()
59
60  public final String process() throws IOException
61  {
62      //---* Initialise the process context *-----
63      final var context = ...
64
65      //---* Gather the input data *-----
66      final var inputData = obtainInputData();
67
68      //---* Validate and filter the data *-----
69      final var filteredData = filterData( inputData, context );
70
71      //---* Process the filtered data to the result data *-----
72      final var resultData = ... // Do something with filtererdData

```



8. Coding Guidelines

```
73
74      //---* Format the data *-----
75      final var retValue = formatData( resultData, context );
76
77      //---* Cleanup *-----
78      // Do whatever is necessary ...
79
80      //---* Done *-----
81      /*
82       * Let the caller write the data to wherever it should end up.
83       */
84      return retValue;
85  }    // process()
86  }
87  // class MyClass
```

Looks good now! Tests were successful! Job done!

A custom class that needs to get the data from the net can implement the method **obtainInputData()** as

```
/**
 * Obtains the input data from the server with the IP4 address
 * 192.168.0.1.
 *
 * @return {@inheritDoc}
 * @throws {@inheritDoc}
 */
@Override
protected final Data obtainInputData() throws IOException
{
    final Data retValue;
    try( final var inputFile = new URL( "192.168.0.1" ).openStream() )
    {
        inputData = readStream( inputFile );
    }

    //---* Done *-----
    return retValue;
} // obtainInputData()
```

Similarly, it can override **formatData()** and **filterData()**.



Your colleagues implemented some custom classes extending your new version of **MyClass**, and basically, all is fine. Just once in a blue moon, the programs using one of the custom classes crashes spectacularly.

When you looked into the source for these customised implementations, you found this implementation for **filterData()**

```
@Override
protected final Data filterData( final Data data, final ProcessContext
    context )
{
    final var validData = super.filterData( data, context );

    final var retValue = ... // Do some filtering on validData

    //---* Done *-----
    return retValue;
} // filterData()
```

This was not what you meant! It should be like shown below, to ensure that *after* filtering the data is valid (remember: the method **filterData()** of the base class **MyClass** is responsible for *filtering* and *validation* of the input data)⁴⁰:

```
@Override
protected final Data filterData( final Data data, final ProcessContext
    context )
{
    final var filteredData = ... // Do some filtering on data
    final var retValue = super.filterData( filteredData, context );

    //---* Done *-----
    return retValue;
} // filterData()
```

But your colleagues says that they have to check the input data – and it seems that they are right! But the data has to be checked after the filtering again ...

So you modified your base class once more:

⁴⁰Yes, this should not be the case, but if done correct, there would not be a story to tell, and no example to illustrate the issue



8. Coding Guidelines

```
1 public class MyClass // Third attempt
2 {
3     /**
4      * Filters the input data. This implementation does nothing.
5      *
6      * @param data    The raw input data.
7      * @param context The process context.
8      * @return The filtered data.
9      */
10    @MountPoint
11    protected Data filterData( final Data data, final ProcessContext
12                               context )
13    {
14        /* Does nothing */
15
16        //---* Done *-----
17        return data;
18    } // filterData()
19
20    /**
21     * Formats the output data.
22     *
23     * @param data    The data to prepare for the output.
24     * @param context The process context.
25     * @return The formatted data.
26     */
27    @MountPoint
28    protected String formatData( final Data data, final ProcessContext
29                                 context )
30    {
31        final var retValue = ... // Do something with data
32
33        //---* Done *-----
34        return retValue;
35    } // formatData()
36
37    /**
38     * Obtains the input data from somewhere; the default
39     * implementation reads it from the file {@code inputFileName}.
40     *
41     * @return The input data.
42     * @throws IOException An I/O error occurred while gathering the
43     *                    data.
44     */
45 }
```



```

43 @MountPoint
44 protected Data obtainInputData() throws IOException
45 {
46     final Data retValue;
47     try( final var inputFile = new FileInputStream( "inputFileName" ) )
48     {
49         retValue = readStream( inputFile );
50     }
51
52     //---* Done *-----
53     return retValue;
54 } // obtainInputData()
55
56 /**
57  * Validates the input data.
58  *
59  * @param data    The input data.
60  * @param context The process context.
61  * @throws IllegalArgumentException The input data is invalid.
62  */
63 private final void validateData( final Data data, final ProcessContext
64     context )
65 {
66     if( /* data is not valid */ ) throw new IllegalArgumentException();
67 } // validateData()
68
69 public final String process() throws IOException
70 {
71     //---* Initialise the process context *-----
72     final var context = ...
73
74     //---* Gather the input data *-----
75     final var inputData = obtainInputData();
76
77     //---* Validate and filter the data *-----
78     validateData( inputData, context );
79     final var filteredData = filterData( inputData, context );
80     if( !inputData.equals( filteredData ) ) validateData(
81         filteredData, context );
82
83     //---* Process the filtered data to the result data *-----
84     final var resultData = ... // Do something with filtererdData
85
86     //---* Format the data *-----
87     final var retValue = formatData( resultData, context );

```



8. Coding Guidelines

```
86
87     //---* Cleanup *-----
88     // Do whatever is necessary...
89
90     //---* Done *-----
91     /*
92      * Let the caller write the data to wherever it should end up.
93      */
94     return retValue;
95 }    // process()
96 }
97 // class MyClass
```

Now `filterData()` will only filter data, it is no longer responsible for the validation, too. And the default implementation of that method is now empty.

So you have basically the following options when designing an extendable class:

- The method (and the base class) is **abstract**; this forces that any instantiable implementation (any non-**abstract** subclass) has to implement that method.
- The method does nothing (the method body is empty or contains only a return statement) and can be implemented to add functionality if needed/desired.
- The method provides a default behaviour that can be *completely* replaced in a subclass.

You should avoid the *requirement* to call the super implementation of the overridden method, by all means! But if you cannot avoid it, provide a detailed comment on how this has to be done!

That does not mean that an overriding method is *not allowed* to call the super implementation! Assume that in the sample above, the implementation of the method `formatData()` from `MyClass` is more or less doing what you want, but there is just one caption that you want to have different. Then a completely legal implementation for `CustomClass::formatData` could look like this:

```
/**
 * {@inheritDoc}
 */
```



```
@Override
protected final String formatData( final String data, final ProcessContext
    context )
{
    final var formattedData = super.formatData( data, context );
    final var retValue = formattedData.replace( "Caption:", "Description:"
    );

    //---* Done *-----
    return retValue;
} // formatData()
```

And before it gets lost: each overriding method has to be annotated with the `@Override` annotation^[10], no matter if it was declared in a class or in an interface. Usually, you can use just `{@inheritDoc}` as the documentation for that method.

8.4.4. Adapter Classes

An adapter class implements an interface that declares mainly optional methods; all methods in the class are empty. In addition, an adapter class does not define any attributes⁴¹.

A sample for that pattern is the class `java.awt.event.KeyAdapter`^[34]; it implements the interface `java.awt.event.KeyListener`^[176].

This is used like this:

```
public final class MyClass
{
    public static final void main( final String... args )
    {
        final var frame = new JFrame( "Key_Listener" );
        final var contentPane = frame.getContentPane();
        final KeyListener listener = new KeyAdapter()
        {
            @Override
            public void keyTyped(KeyEvent event)
            {
```

⁴¹Otherwise, it would need at least accessor and probably also mutator methods for these attributes that could not be empty.



8. Coding Guidelines

```
        printEventInfo("Key_Typed", event);
    } //      keyTyped()

    private final void printEventInfo( final String str, final
        KeyEvent e )
    {
        System.out.println( str );
        int code = e.getKeyCode();
        System.out.println( "Code:_" + KeyEvent.getKeyText(
            code ) );
        System.out.println( "Char:_" + e.getKeyChar() );
        int mods = e.getModifiersEx();
        System.out.println( "Mods:_" +
            KeyEvent.getModifiersExText( mods ) );
        System.out.println( "Location:_" + keyboardLocation(
            e.getKeyLocation() ) );
        System.out.println( "Action?_" + e.isActionKey() );
    } // printEventInfo()

    private final String keyboardLocation( final int keyboard )
    {
        final var retValue = switch( keyboard )
        {
            case KeyEvent.KEY_LOCATION_RIGHT -> "Right";
            case KeyEvent.KEY_LOCATION_LEFT -> "Left";
            case KeyEvent.KEY_LOCATION_NUMPAD -> "NumPad";
            case KeyEvent.KEY_LOCATION_STANDARD -> "Standard";
            default -> "Unknown";
        };

        //---* Done *-----
        return retValue;
    } //      keyboardLocation()
};

final var textField = new JTextField();
textField.addKeyListener( listener );
contentPane.add( textField, BorderLayout.NORTH );
frame.pack();
frame.setVisible( true );
    } // main()
}
// class MyClass
```



The advantage from using **KeyAdapter** instead of implementing the interface **KeyListener** directly is that you do not have to implement those methods you are not interested in. Below the relevant part when using the interface:

```
public final class MyClass
{
    public static final void main( final String... args )
    {
        ...
        final KeyListener listener = new KeyListener()
        {
            @Override
            public void keyPressed( KeyEvent event ) { /* Does nothing */ }

            @Override
            public void keyReleased(KeyEvent event) { /* Does nothing */ }

            @Override
            public final void keyTyped( final KeyEvent event )
            {
                printEventInfo("Key_Typed", event);
            } // keyTyped()

            ...
        };
        ...
    } // main()
}
// class MyClass
```

For this interface and this adapter class, the advantage is marginal, but already the Java Runtime Library knows some interfaces with much more methods that do have an adapter class, and these would benefit much more from an adapter class.

But since Java 8, interfaces may have **default** methods[120, 161]. Therefore, today the concept of an adapter class has got obsolete. Instead, you would define the interface differently, using **default** methods.

If adjusting the interface **java.awt.event.KeyListener** accordingly, this would look like this:

```
package java.awt.event;
```



8. Coding Guidelines

```
import java.util.EventListener;

/**
 * The listener interface for receiving keyboard events (keystrokes).
 * [...]
 *
 * @author Carl Quinn
 *
 * @see KeyAdapter
 * @see KeyEvent
 */
public interface KeyListener extends EventListener
{
    /**
     * <p>{@summary Invoked when a key has been pressed.} See the
     * class description for
     * {@link KeyEvent}
     * for a definition of a key pressed event.</p>
     *
     * @param e The event to be processed.
     */
    public default void keyPressed( final KeyEvent e ) { /* Does nothing
        */ }

    /**
     * <p>{@summary Invoked when a key has been released.} See the
     * class description for
     * {@link KeyEvent}
     * for a definition of a key released event.</p>
     *
     * @param e The event to be processed.
     */
    public default void keyReleased( final KeyEvent e ) { /* Does nothing
        */ }

    /**
     * <p>{@summary Invoked when a key has been typed.} See the
     * class description for
     * {@link KeyEvent}
     * for a definition of a key typed event.</p>
     *
     * @param e The event to be processed.
     */
    public default void keyTyped( final KeyEvent e ) { /* Does nothing */ }
}
```




```
// interface KeyListener
```

This would allow to use the interface in the same way as previously the adapter class, and to implement only the method `KeyListener::keyTyped`.

8.4.5. Alternatives to Extending a Class

The goal for the design of `MyClass` in chapter 8.4.3 to allow modifications to the behaviour of the method `process()`; there we used inheritance and extended the class.

But it can be done also as in the implementation below:

```
1 public final class MyClass
2 {
3     /*-----*\
4     ===** Inner Classes **=====
5     \*-----*/
6     @FunctionalInterface
7     public interface DataSupplier
8     {
9         /*-----*\
10        ===** Methods **=====
11        \*-----*/
12        /**
13         * Returns an instance of
14         * {@link Data}.
15         *
16         * @return The data.
17         * @throws IOException Something went wrong.
18         */
19        public Data get() throws IOException;
20    }
21    // interface DataSupplier
22
23    /*-----*\
24    ===** Attributes **=====
25    \*-----*/
26    /**
27     * The filter method.
28     */
29    private final BiFunction<Data,ProcessContext,Data> m_FilterMethod;
```



8. Coding Guidelines

```
30
31  /**
32   * The formatter method.
33   */
34  private final BiFunction<Data,ProcessContext,String> m_FormatterMethod;
35
36  /**
37   * The data supplier method.
38   */
39  private final DataSupplier m_DataSupplier;
40
41  /*-----*\
42  ===** Constructors **=====
43  \*-----*/
44  /**
45   * Creates a new instance of {@code MyClass}.
46   */
47  public MyClass() { this( null, null, null ); }
48
49  /**
50   * Creates a new instance of {@code MyClass}.
51   *
52   * @param dataSupplier The data supplier method; can be
53   *   {@code null}.
54   * @param filter The filter method; can be {@code null}.
55   * @param formatter The formatter method; can be {@code null}.
56   */
57  public MyClass( final DataSupplier dataSupplier, final
58                  BiFunction<Data,ProcessContext,Data> filter, final
59                  BiFunction<Data,ProcessContext,String> formatter )
60  {
61      m_DataSupplier = isNull( dataSupplier )
62          ? this::obtainInputData
63          : dataSupplier;
64      m_FilterMethod = isNull( filter )
65          ? this::filterData
66          : filter;
67      m_FormatterMethod = isNull( formatter )
68          ? this::formatData
69          : formatter;
70  } // MyClass()
71
72  /*-----*\
73  ===** Methods **=====
74  \*-----*/
```



```

73  /**
74   * Filters the input data. This implementation does nothing.
75   *
76   * @param data    The raw input data.
77   * @param context The process context.
78   * @return The filtered data.
79   */
80  private final Data filterData( final Data data, final ProcessContext
      context )
81  {
82      /* Does nothing */
83
84      //---* Done *-----
85      return data;
86  } // filterData()
87
88  /**
89   * Formats the output data.
90   *
91   * @param data    The data to prepare for the output.
92   * @param context The process context.
93   * @return The formatted data.
94   */
95  private final String formatData( final Data data, final ProcessContext
      context )
96  {
97      final var retValue = ... // Do something with data
98
99      //---* Done *-----
100     return retValue;
101 } // formatData()
102
103 /**
104  * Obtains the input data from somewhere; the default
105  * implementation reads it from the file {@code inputFileNames}.
106  *
107  * @return The input data.
108  * @throws IOException An I/O error occurred while gathering the
109  *         data.
110  */
111 private final Data obtainInputData() throws IOException
112 {
113     final Data retValue;
114     try( final var inputFile = new FileInputStream( "inputFileName" ) )
115     {

```



8. Coding Guidelines

```
116         retValue = readStream( inputFile );
117     }
118
119     //---* Done *-----
120     return retValue;
121 }    // obtainInputData()
122
123 /**
124  * Validates the input data.
125  *
126  * @param data    The input data.
127  * @param context The process context.
128  * @throws IllegalArgumentException The input data is invalid.
129  */
130 private final void validateData( final Data data, final ProcessContext
    context )
131 {
132     if( /* data is not valid */ ) throw new IllegalArgumentException();
133 }    // validateData()
134
135 public final String process() throws IOException
136 {
137     //---* Initialise the process context *-----
138     final var context = ...
139
140     //---* Gather the input data *-----
141     final var inputData = m_DataSupplier.get();
142
143     //---* Validate and filter the data *-----
144     validateData( inputData, context );
145     final var filteredData = m_FilterMethod.apply( inputData, context
        );
146     if( !inputData.equals( filteredData ) ) validateData(
        filteredData, context );
147
148     //---* Process the filtered data to the result data *-----
149     final var resultData = ... // Do something with filtererdData
150
151     //---* Format the data *-----
152     final var retValue = m_FormatterMethod.apply( resultData, context
        );
153
154     //---* Cleanup *-----
155     // Do whatever is necessary ...
156 }
```



```
157      //---* Done *-----  
158      /*  
159       * Let the caller write the data to wherever it should end up.  
160       */  
161      return retValue;  
162  }    // process()  
163  }  
164  // class MyClass
```

Basically the functionality is provided as arguments to the constructor; if an argument is `null`, a default is used. The inner class (interface) **DataSupplier** is needed because the method `get()` of out-of-the-box functional interface `java.util.function.Supplier`^[200] does not declare an exception.

This is of course only one possible approach, and it does not work in all cases. So it does not allow to add additional attributes, on the other side it can be used to introduce some nasty side effects that are not immediately visible from the code of the class. When you have configured logging in the usual way, the injected methods will use different loggers than the other methods of the class.

But it is a valuable addition to our tool box that complements inheritance.

8.5. Method Signatures

The question how to define the signature of a method or constructor is important not only for a public API; doing it right helps with building better readable code in general.

At first you need to understand what the ‘method signature’ is in Java; it is what identifies a method and distinguishes it from another.

In Java, the method signature is the name of the method plus the sequence of the types of formal parameters for that method. Neither the names of the formal parameters nor the return type nor any declared exceptions are part of the signature. This reflects in the method `Class::getMethod`^[259, 258] that takes just the method name and a list of `Class` instances as arguments to identify a method inside a class.



- P1. The selected name for a method has to be meaningful.

For more details on this, refer to P1. in chapter “6 Naming Conventions”, and to chapter “6.7 Methods” that exhausts the topic regarding the naming of methods.

- P2. The type for an argument should be as general as possible.

That type from a class hierarchy, that provides exact all required functionality, has to be chosen as the type for the argument.

- P3. An argument should be as specific as possible.

At a first glance this seems to contradict P2., but as that refers to the general type only, does this principle refers also to the value. It means that you should prefer to provide the object itself instead of a key to it, or its string representation.

In this context, P4. is important, too.

- P4. Do not cast the provided argument value.

If you need to cast the given argument value, you have most probably chosen the wrong argument type, although there are exceptions to the rule – especially if you have to implement an interface or to override a method from a foreign class. `Object::equals` is a good sample for such an exception to this rule.

- P5. Keep the parameters list short.

Method calls with a long list of arguments are difficult to read and to understand, and they are error prone.

- P6. Avoid to have multiple arguments with the same type on the parameters list.

This is also error prone and may cause errors that are really nasty to debug.



8.5.1. Mutable Types for Arguments

A method (or constructor) *must* declare in its documentation comment that it modifies a given argument value. This is important because otherwise the method could be called with an immutable implementation of the type for the formal parameter, what would cause an exception⁴². It is also important for the caller as it cannot rely any longer that an object instance that was used as an argument in a method call has still the same state after the method call as it had before.

Conversely, it also means a method, that *does not specify* to modify one of its arguments, *will not change* any of the arguments, and that it would be safe to call that method with a mutable object – at least safe for the caller.

But when the called method or constructor stores the provided argument in the instance, this may call for trouble if it keeps just the given reference:

```
public final class MyClass
{
    private List<String> m_Attribute;

    public MyClass( final List<String> attribute )
    {
        // AVOID!!
        m_Attribute = attribute;
    } // MyClass()
}
// class MyClass
```

If the caller holds a reference to the **List** instance provided as the argument to the constructor in this sample, it can still modify that list – and change the internal state of the **MyClass** instance at the same time.⁴³

Therefore you must create a copy for all (potentially) mutable arguments⁴⁴:

⁴²Usually, it would be an `java.lang.UnsupportedOperationException`[72], sometimes an `java.lang.IllegalStateException`[53], but others are possible, too.

⁴³You may encounter a similar situation if the provided object instance is accessible from multiple threads than may attempt to modify it concurrently.

⁴⁴The examples will show a constructor, but a mutator method (a ‘setter’) needs to be implemented along the same lines.



8. Coding Guidelines

```
public final class MyClass
{
    private List<String> m_Attribute;

    public MyClass( final List<String> attribute )
    {
        // Better!
        m_Attribute = new ArrayList( attribute );
    } // MyClass()
}
// classMyClass
```

If the code of the class will not modify the attribute, you can of course use `List::copyOf`^[294], too.

Or you can do it like this⁴⁵:

```
public final class MyClass
{
    private final List<String> m_Attribute = new ArrayList<>();

    public MyClass( final List<String> attribute )
    {
        // PREFERRED!!
        m_Attribute.addAll( attribute );
    } // MyClass()
}
// classMyClass
```

In the samples above, I used lists of `java.lang.String`, and `String` is an immutable type – meaning that it would not be required to copy the contents of the list, too. But if it would be a mutable type, a deep copy is required:

```
public final class MyClass
{
    private final List<StringBuilder> m_Attribute = new ArrayList<>();

    public MyClass( final List<StringBuilder> attribute )
    {
        for( final var s : attribute )
```

⁴⁵I deliberately omitted any otherwise necessary `null` checks in these samples as they do not help to illustrate the issue.



```
{
    m_Attribute.add( new StringBuilder( s );
} // MyClass()
}
// classMyClass
```

This is easy to implement for `java.lang.StringBuilder`, but it can become arbitrarily complex for custom classes.

8.5.2. Abstract vs. concrete Argument Types

If you have the choice, you should prefer always an abstraction over a concrete implementation, given that the abstraction reveals all the functionality you need. This is often referred to as “Programming to the Interface” and discussed in more detail in chapter 8.28 on page 406.

You encounter this situation mostly when your method takes a collection type as an argument: should it be `Collection`[194] or `ArrayList`[83]?

Another class hierarchy that raises this question is built around `InputStream`[41] and `OutputStream`[42].

A similar situation is there about `String`[66] and `CharSequence`[183].

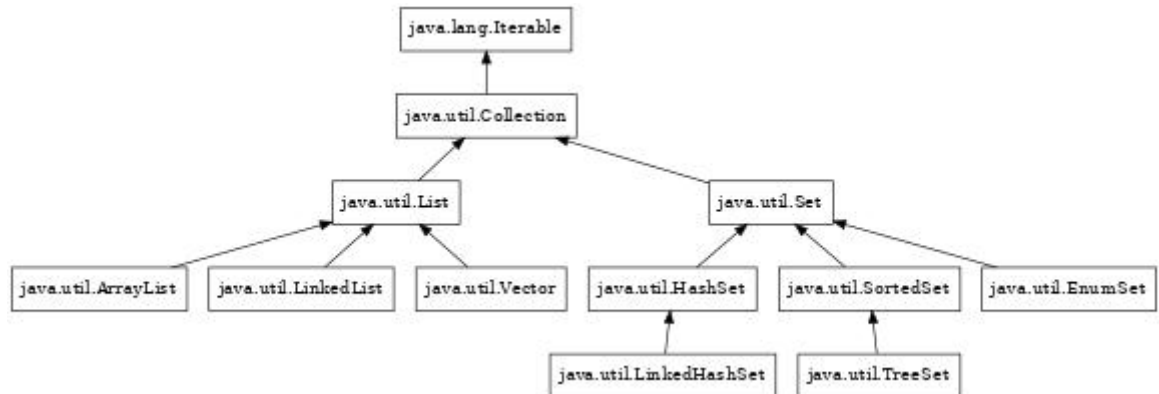
I will discuss this topic here using these three areas as samples, and hopefully, it gives you an idea how to handle this question in other contexts.

This approach allows a method to be called with a wider range of valid arguments, without causing issues with the type safety and without the need to cast the arguments before it can be processed. This also means that the extrema to use always `java.lang.Object`[59] for the formal parameters is definitely a bad idea.



Choose the proper Collection Type

The diagram below shows the relevant class hierarchy for the collection types (we will discuss the map types later).



If your method will only iterate over the contents of a collection, the argument type of choice would be `java.util.Collection`[194]. It even allows to add and to remove entries.

The interface `java.lang.Iterable`[186] is usually not the proper selection – although it looks like that – because it is also implemented by types like `java.nio.file.Path`[190] or `java.beans.beancontext.BeanContext`[177].⁴⁶

You use the interface `java.util.List`[201] as the argument type, if somehow the sequence of the entries is important and/or you want to have duplicate entries in the collection.

The implementations of the interface `java.util.Set`[205] does not guarantee any particular sequence, but therefore all entries are distinct.

Finally the interface `java.util.SortedSet`[207] allows you to force a particular order of the distinct entries.

⁴⁶To be precise, `Path` and `BeanContext` extend `java.lang.Iterable`, as these are interfaces, too.

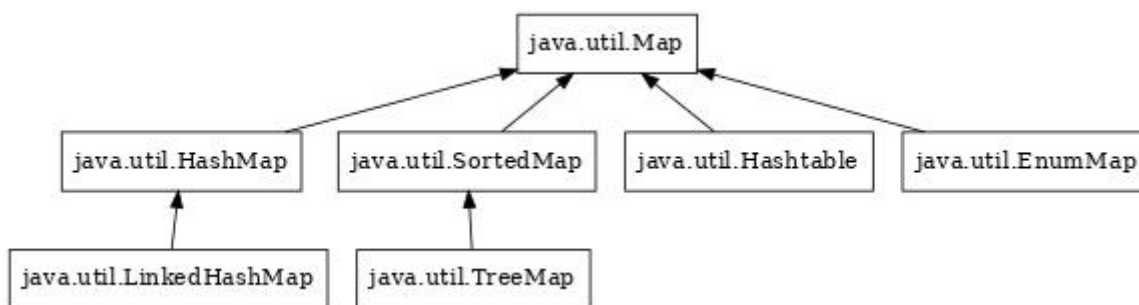


The concrete implementations

- `java.util.ArrayList`[83]
- `java.util.LinkedList`[92]
- `java.util.Vector`[105]
- `java.util.HashSet`[51]
- `java.util.LinkedHashSet`[56]
- `java.util.EnumSet`[88]
- `java.util.TreeSet`[103]

are rarely used as argument types, with the exception of `LinkedHashSet` in cases where you need distinct entries in their insertion order.

The corresponding class hierarchy for a map looks like this:



In most cases you use `java.util.Map`[203] for the type of a method's formal parameter, or `java.util.SortedMap`[206] if a defined sequence is relevant on iterating over the entries. The concrete implementation `java.util.LinkedHashMap`[55] can be used if the insertion order has to be maintained somehow, but the other implementations

- `java.util.HashMap`[90]
- `java.util.Hashtable`[91]
- `java.util.EnumMap`[87]
- `java.util.TreeMap`[102]



are usually not used as argument types.

When using the most general type (the interfaces in case of collection and map) for the arguments types, your methods get a wider range of usability with sacrificing type safety.

All the type discussed here are generic, and you should also consider to use a type variable with them instead of a concrete type if you are not specifically interested in the content type:

```
public final <T> void myMethod( final List<T> argumentList ) {}
```

instead of

```
// AVOID!  
public final void myMethod( final List<Object> argumentList ) {}  
  
// NEVER!  
public final void myMethod( final List argumentList ) {}
```

For more details refer to chapter “8.16 Generics”.

Choose the proper I/O Stream Type

While the collection types are a clear class hierarchy, this is different for the I/O streams. Basically, there are the (**abstract**) classes `java.io.InputStream`^[41] and `java.io.OutputStream`^[42], implementations like `java.io.FileInputStream`^[39] and `java.io.FileOutputStream`^[40], and classes like `java.io.BufferedInputStream`^[35] and `java.io.BufferedOutputStream`^[36] that are decorators for the implementations, easily to identify because the constructors for the decorators take a stream instance as their arguments.

In most cases you use just the abstract classes as the types for the formal parameters of your methods, in rare cases one of the decorator classes. There are nearly no use cases that would require to use an implementation class like `FileInputStream` as argument type.



String vs. CharSequence

For the type of method and constructor arguments you should prefer `java.lang.CharSequence`^[183] over `java.lang.String`^[66], even if you in fact need a `String`: calling `toString()` on an instance of `String` is a NOOP that will be optimised away by the `javac` compiler or by the JIT, but line 3 from the sample below is easier to read than line 5, and the latter does not provide any additional information to the reader.

```
1 final var buffer = new StringBuilder();
2 ...
3 final var parts = splitString( buffer );
4 ...
5 final var parts = splitString( buffer.toString() );
```

Of course, there will be always methods that will be called only with `String` instances as the arguments, and never with an instance of `CharBuffer`^[76] or `StringBuilder`^[68], just because of the context they are defined in. Then it does not make any sense at all to define the formal parameter as `CharSequence`.

8.5.3. boolean Arguments

You should avoid method or constructor signatures that have formal parameters of type `boolean` or `java.lang.Boolean`. A single flag for a setter is acceptable, but something like the constructor below will cause issues:

```
1 public final MyClass
2 {
3     /*-----*\
4     ===** Attributes **=====
5     \*-----*/
6     private final boolean m_Flag01;
7     private final boolean m_Flag02;
8     private final boolean m_Flag03;
9     private final boolean m_Flag04;
10    private final boolean m_Flag05;
11    private final boolean m_Flag06;
12    private final boolean m_Flag07;
13    private final boolean m_Flag08;
14    private final boolean m_Flag09;
```



8. Coding Guidelines

```
15     private final boolean m_Flag10;
16
17     /*-----*\
18     ===** Constructors **=====
19     \*-----*/
20     // BAD!!
21     public MyClass( final boolean flag01, final boolean flag02, final
22                     boolean flag03, final boolean flag04, final boolean flag05, final
23                     boolean flag06, final boolean flag07, final boolean flag08, final
24                     boolean flag09 final boolean flag10 )
25     {
26         m_Flag01 = flag01;
27         m_Flag02 = flag02;
28         m_Flag03 = flag03;
29         m_Flag04 = flag04;
30         m_Flag05 = flag05;
31         m_Flag06 = flag06;
32         m_Flag07 = flag07;
33         m_Flag08 = flag08;
34         m_Flag09 = flag09;
35         m_Flag10 = flag10;
36     } // MyClass()
37
38     /*-----*\
39     ===** Methods **=====
40     \*-----*/
41     public static final void main( final String... args )
42     {
43         final var instance = new MyClass( true, false, false, true, false,
44                                           true, false, true, false, false );
45     } // main()
46 }
47 // class MyClass
```

Can you say with a quick look how that **MyClass** object is instantiated in line 40? This approach produces code that is definitely bad to read!

An alternative is this approach:

```
1 public final MyClass
2 {
3     /*-----*\
4     ===** Inner Classes **=====
5     \*-----*/
6     public enum Flag
```



```

7      {
8          FLAG01,
9          FLAG02,
10         FLAG03,
11         FLAG04,
12         FLAG05,
13         FLAG06,
14         FLAG07,
15         FLAG08,
16         FLAG09,
17         FLAG10
18     }    // enum Flag
19
20     /*-----*\
21     ===** Attributes **=====
22     \*-----*/
23     private final boolean m_Flag01;
24     private final boolean m_Flag02;
25     private final boolean m_Flag03;
26     private final boolean m_Flag04;
27     private final boolean m_Flag05;
28     private final boolean m_Flag06;
29     private final boolean m_Flag07;
30     private final boolean m_Flag08;
31     private final boolean m_Flag09;
32     private final boolean m_Flag10;
33
34     /*-----*\
35     ===** Constructors **=====
36     \*-----*/
37     public MyClass( final Flag... flags )
38     {
39         final Set<Flag> flagSet = switch( requireNonNullArgument( flags,
40             "flags" ).length )
41         {
42             case 0 -> EnumSet.noneOf( Flag.class );
43             case 1 -> EnumSet.of( flags [0] );
44             default -> EnumSet.of( flags [0], flags );
45         }
46         m_Flag01 = flagSet.contains( FLAG01 );
47         m_Flag02 = flagSet.contains( FLAG02 );
48         m_Flag03 = flagSet.contains( FLAG03 );
49         m_Flag04 = flagSet.contains( FLAG04 );
50         m_Flag05 = flagSet.contains( FLAG05 );
51         m_Flag06 = flagSet.contains( FLAG06 );

```



8. Coding Guidelines

```
51     m_Flag07 = flagSet.contains( FLAG07 );
52     m_Flag08 = flagSet.contains( FLAG08 );
53     m_Flag09 = flagSet.contains( FLAG09 );
54     m_Flag10 = flagSet.contains( FLAG10 );
55 } // MyClass()
56
57 /*-----*\
58 ===** Methods **=====
59 \*-----*/
60 public static final void main( final String... args )
61 {
62     final var instance = new MyClass( FLAG01, FLAG04, FLAG06, FLAG08 );
63 } // main()
64 }
65 // class MyClass
```

That is definitely easier to read than the original version. You should even consider whether you really want to have all the `boolean` attributes: you can replace them by an `EnumSet`^[88] that holds values of type `MyClass.Flag`. A getter is now implemented like in this sample:

```
public final MyClass
{
    /*-----*\
    ===** Inner Classes **=====
    \*-----*/
    public enum Flag
    {
        ...
    } // enum Flag

    /*-----*\
    ===** Attributes **=====
    \*-----*/
    private final Set<Flag> m_Flags = EnumSet.noneOf( Flag.class );

    /*-----*\
    ===** Constructors **=====
    \*-----*/
    public MyClass( final Flag... flags )
    {
        for( final flag : requireNonNullArgument( flags, "flags" ) )
        {
            m_Flags.add( flag );
        }
    }
}
```




```
    }  
    // MyClass()  
  
    /*-----*\  
    ===** Methods **=====   
    \*-----*/  
    public final boolean isFlag01() { return m_Flags.contains( FLAG01 ) };  
  
    public static final void main( final String... args )  
    {  
        final var instance = new MyClass( FLAG01, FLAG04, FLAG06, FLAG08 );  
    } // main()  
}  
// class MyClass
```

8.5.4. Multiple Parameters with same Type

Chapter 8.5.3 describes the problem that multiple `boolean` parameters can cause in a method or constructor signature.

It should be obvious that long parameter lists with only `int`, `String` or `MyClass` arguments (or whatever else ...) will cause the same kind of problems. Therefore you should avoid that kind of signature declarations.

Unfortunately, the solution is not that simple as for `boolean` arguments.

For a constructor, you can think about the implementation of the Builder pattern[127], but that does not work for methods.

Another alternative is described in chapter 8.5.5.

8.5.5. Generic Parameter Types

The topic of this chapter is closely related to that of chapter “8.5.4 Multiple Parameters with same Type”, and it touches also the topic of parameter validation that is discussed in chapter 8.3 on page 250 as well as chapter “8.10 Value Types” on page 336.



‘Generic parameter type’ means here a type whose values can have multiple meanings: an `int` could be a x coordinate of a point in a graphic, a color component, the price for something or a day in a month. In the same way, a `String` could be text message, an email address or a customer id.

The problem with these that their value does not have a meaning outside a defined context: `10` means nothing, but as argument to `Thread.sleep[276]` is stands for the ten milliseconds that the current thread will be asleep:

```
try
{
    sleep( 10 );
}
catch( final InterruptedException ignored )
{
    /* Deliberately ignored */
}
```

But in another context, it could be a length, a port number or a magic number.

First, avoid the use of magic numbers! Use enums instead! At least use constants if the magic numbers are defined by an external library.

A sample for that is the class `java.sql.Types`[79]: the various SQL types are represented by an integer number. The enum class `java.sql.JDBCType`[78] that was introduced with JDBC 4.2 shows how this could be handled: when calling `JDBCType.getVendorTypeNumber`[284] on a SQL type, it returns the corresponding value from `Types`, while `JDBCType.valueOf`[285] returns the enum constant for the corresponding integer value from `Types`.

When using an enum, you also benefit from the strict typing of Java:

```
public void assignDataType( final int type );
```

can be called with any integer value; of course you should use one of the constants from `Types`, but this is not enforced in any way. And the validation whether the given number is allowed is a nightmare.

But when declaring

```
public void assignDataType( final JDBCType type );
```



the valid arguments are limited to the values in the enum class, and the argument validation can be reduced to a simple `null` check. The compiler will reliably protect you from using an invalid data type ...

But it also it helps to protect your code from other bugs; assume a method with the following method signature:

```
public void assignDataTypeToColumn( final int type, final int columnIndex )
{
    ...
} // assignDataTypeToColumn()
```

The compiler will not complain about this method invocation:

```
final var columnIndex = 7;
assignDataTypeToColumn( columnIndex, Types.BOOLEAN );
```

But this will not compile until you get the arguments on the method call right:

```
public void assignDataTypeToColumn( final JDBCType type, final int
    columnIndex )
{
    ...
} // assignDataTypeToColumn()

...

final var columnIndex = 7;
// DOES NOT COMPILE!!
assignDataTypeToColumn( columnIndex, JDBCType.BOOLEAN );
```

Refer also to chapter “8.1 Swap Logic Errors for Compiler Errors” on page 197.

The same kind of bug is this one:

```
public final Order createOrder( final String customerId, final String
    articleNumber )
{
    ...
} // createOrder()

...
```



8. Coding Guidelines

```
final var customerId = "C-1234";  
final var articleNumber = "BigBook 4711";  
final var order = createOrder( articleNumber, customerId );
```

You can avoid this easily by defining the domain value types **CustomerId** and **ArticleNumber** and using them for the formal parameters of **createOrder()**; then the compiler will already complain.

A related problem class is this:

```
final var minutes = 10;  
try  
{  
    Thread.sleep( minutes );  
}  
catch( final InterruptedException ignored )  
{  
    /* Deliberately ignored */  
}
```

Do you remember? **Thread::sleep** takes the waiting time in milliseconds, not in minutes ...

This class of bugs can be addressed by the definition of ‘value types’ or ‘dimensioned values’⁴⁷.

Value types and domain value types are covered in more detail in chapter 8.10 on page 336.

8.5.6. Arrays and **varargs** as Arguments

The signatures **myMethod(final String... args)** (using **vararg**) and **myMethod(final String [] args)** are basically the same. There is only a difference on how to call a method with one or the other signature.

The only option for the call to the variant with the array argument signature looks like this:

⁴⁷Although this will not necessary help to prevent providing values with the wrong dimension to already existing methods ...



```
public final void myMethod( final String [] args ) { ... }  
  
...  
  
final var args = new String [] { "value1", "value2", ... };  
myMethod( args );
```

The vararg variant additionally allows to be invoked like this:

```
public final void myMethod( final String... args ) { ... }  
  
...  
  
final var args = new String [] { "value1", "value2", ... };  
myMethod( args );  
  
myMethod( "value1", "value2", ... );
```

The only other difference is that a vararg has to be the last formal parameter in a method or constructor signature, while a mere array argument can be anywhere on the parameters list.

Internally, the arguments are represented as arrays in both cases; this means, the body would look identically irrespective the kind of the formal parameter.

And arrays are not immutable: despite the `final` qualifier, I can always assign a new value to an (existing) array element. As a consequence you cannot store just the reference for a provided array to an attribute of your class, you have to copy the array (and if the array elements are also not immutable, it has to be a deep copy). In this regard, arrays are like collections.

For a shallow copy, there are multiple options:

```
1 public final void myMethod( final String... args )  
2 {  
3     final var c1 = args.clone();  
4  
5     final var c2 = new String [args.length];  
6     System.arraycopy( args, 0, c2, 0, args.length );  
7  
8     final var c3 = Arrays.copyOf( args, args.length );
```



```
9  
10     final var c4 = Arrays.stream( args ).toArray( String []::new );  
11 }    // myMethod()
```

The Java Runtime library does not provide any pre-build solutions for a deep copy of arrays; if you do not want to implement it yourself, you can look for 3rd libraries providing the respective functionality.

8.6. Types for Return Values

This topic is relevant for all types of code, not only for code in a public API. Basically, it can be reduced to two questions that needs to be answered:

1. Interface vs. concrete Class
2. Mutable vs. Immutable

In general, the answers are “Interface” and “Immutable”, but this is not always the best option.

Principles

- P1. A return value may not provide access to the internal status of an object.

There are exceptions to this rule, in particular for internal code, but in general it must be taken care that a modification to a returned value does not have an unwanted impact to the status of the object that owned the accessor method.

- P2. Chose the return type based on what the caller may want to do with the return value.

Together with P1. this can result in return types for accessor methods that are completely unrelated to the types of the associated attributes.



P3. Avoid to return magic values to indicate errors.

This also means that you should avoid to return `null`. We discussed that already in chapter “8.3 Checking Method Parameters and Return Values”.

8.6.1. Abstract vs. concrete Return Types

Same as for the types of the formal parameters (see chapter “8.5.2 Abstract vs. concrete Argument Types” on page 289), the type for a return value of a method should always be the most abstract type (interface or class) possible, based on the features that the caller might require or expect from the return type. Consequently, `java.lang.Object` is rarely the correct type for the return value as is the most abstract type, but does provide nearly no functionality.

This means, you will usually use `Collection` or `List` instead of `LinkedList`, `OutputStream` instead of `ByteArrayOutputStream`^[37], and so on.

Only when a concrete class or a more specialised interface provides features that are relevant for the caller, the method should be defined to return that type.

One difference is in regard of methods returning strings: here you should *not* use `CharSequence` as the return type! In nearly all cases where a method returns a string this value will really be an instance of `java.lang.String`^[66] anyway – not to mention that `String` implements `CharSequence`.

Another reason is that the interface `CharSequence` is implemented by immutable types (see `java.lang.String`) and mutable types (e.g. `java.lang.StringBuilder`) as well. Depending on how the caller wants to use the return value, it may have to check for the concrete type first⁴⁸.

In general, the idiom to prefer an abstraction over a concrete implementation is usually referred to as “Programming to the Interface”; it will be discussed in more detail in chapter 8.28 on page 406.

⁴⁸... just to find out that it is a `String` in most cases.



8.6.2. Mutable Return Types

In general, it is not evil if a method returns a mutable type, but it can get nasty under some circumstances – for example when you expose the object's internal state this way:

```
public final class MyClass
{
    private final List<String> m_Attribute = new ArrayList<>();

    // PROBLEMATIC!!
    public final List<String> getAttribute() { return m_Attribute; }
}
// class MyClass
```

This means that code like that below would modify `myObject` – and not only the lines 3 to 5, but also the lines 9 to 11:

```
1 final var myObject = new MyClass();
2
3 myObject.getAttribute().add( "Some_String" );
4 myObject.getAttribute().sort( String.CASE_INSENSITIVE_ORDER );
5 myObject.getAttribute().clear();
6
7 final var list = myObject.getAttribute();
8
9 list.add( "Some_String" );
10 list.sort( String.CASE_INSENSITIVE_ORDER );
11 list.clear();
```

Usually, this is not desired! Otherwise, it should be clearly documented:

```
...
/**
 * Returns a reference to the attribute.
 *
 * @note Any modifications of the returned value will be reflected
 *       to the attribute itself and changes the state of the object.
 *
 * @return The reference to the attribute.
 */
public final List<String> getAttribute() { return m_Attribute; }
...
```




But keep in mind that changes to the state of the object may be reflected also to the return value if your method returns a reference to a mutable attribute:

```

23 public final class MyClass
24 {
25     private final List<String> m_Attribute = new ArrayList<>();
26
27     public final void addAttributeEntry( final String s )
28     {
29         m_Attribute.add( s );
30     } // addAttributeEntry()
31
32     // PROBLEMATIC!!
33     public final List<String> getAttribute() { return m_Attribute; }
34 }
35 // class MyClass
36
37 ...
38
39 final var myObject = new MyClass();
40 final var list = myObject.getAttribute();
41 final var size1 = list.size();
42 myObject.addAttributeEntry( "Some_String" );
43 final var size2 = list.size();
44
45 final var status = size1 == size2;

```

The value of **status** in line 23 will always be **false**!

Some better implementations of **getAttribute()** would be:

```

1 /**
2  * Returns the attribute.
3  *
4  * @return The attribute.
5  */
6 public final List<String> getAttribute()
7 {
8     final var retValue = List.copyOf( m_Attribute );
9
10    final List<String> retValue = unmodifiableList( m_Attribute );
11
12    final List<> retValue = new ArrayList<>( m_Attribute );
13

```



```
14     final List<String> retValue = unmodifiableList( new ArrayList<>(
15         m_Attribute ) );
16     final List<String> retValue = m_Attribute.clone();
17
18     //---* Done *-----
19     return retValue;
20 } // getAttribute()
```

Line 8 creates an unmodifiable copy of `m_Attribute`. The method `copyOf()` exists for `java.util.List`[294], `java.util.Set`[305] and `java.util.Map`[298].

`copyOf()` has some limitations: first is that `null` is not valid as a value, and – for `Map` – as a key.

Next it is important to know, that `Set::copyOf` returns an implementation of `Set` that resembles a `java.util.HashSet`[51], and for `Map::copyOf` the return value is resembling a `java.util.HashMap`[90] – this means that any sort order gets lost for the copy.

Line 10 wraps `m_Attribute` into an unmodifiable instance of `List`. Drawback is that the returned value changes when the attribute itself is modified somehow.

The utility class `java.util.Collections`[84] provides not only the shown method `unmodifiableList()`[291] but also corresponding methods for `Collection`[290], `Set`[293] and `Map`[292].

Line 12 creates an instance of `java.util.ArrayList`[83] for the return value and initialises that with `m_Attribute`. The returned value will be mutable, but changes will not affect the attribute and therefore will not affect the state of the object instance, too. And `null` values are no problem, other than for the solution with `copyOf()`.

Line 14 basically combines lines 10 and 12, eliminating the shortcoming of the solutions from lines 8 and 10.



Line 16 ‘clones’ `m_Attribute`; obviously this works only for instances that implement `java.lang.Cloneable`[184], but although collections (and arrays) are cloneable themselves, they may contain elements that are not cloneable, causing a `CloneNotSupportedException`[47].

Remember that arrays are not immutable; you can always assign a new value to an existing array element. Therefore you have to take the same precautions when your method should return an array attribute as for returning a collection.

```
1 public final class MyClass
2 {
3     private String [] m_Attribute;
4
5     /**
6      * Returns the attribute.
7      *
8      * @return The attribute.
9      */
10    public final String [] getAttribute()
11    {
12        final var retValue = m_Attribute.clone();
13
14        final var retValue = new String [m_Attribute.length];
15        System.arraycopy( m_Attribute, 0, retValue, 0, m_Attribute.length
16        );
17
18        final var retValue = Arrays.copyOf( m_Attribute,
19        m_Attribute.length );
20
21        final var retValue = Arrays.stream( m_Attribute )
22        .toArray( String []::new );
23
24        //---* Done *-----
25        return retValue;
26    } // getAttribute()
27 }
28 // class MyClass
```

In the examples, the element type for both, the collections and the arrays,



is immutable⁴⁹; if the elements would be mutable, you need to take the corresponding precautions for them, too. Basically that would mean a deep copy of the collection or the array.

In general you have to take care that you will not return a reference to an attribute with a mutable type that would allow to modify the internal state of the owning object. This clearly does not affect just collections – although this is the most common case – but for example also instances of `java.util.Date`^[86]⁵⁰. Obviously, for other types the details on how to create a copy or to make them immutable will differ.

If a method creates (generates, obtains, reads, loads, ...) the value it returns, it is part of the methods' contract whether that value is mutable or not.

8.6.3. Special Return Values

'Special return values' here does not mean those 'magic numbers' mentioned in chapter 8.5.5 on page 297 and elsewhere, although you should avoid them as return values in the same way as you should not use them as arguments to a method call.

This term indicates otherwise impossible return values indicating a special condition, like in the samples below:

```
1 final var input = "Some_Text_follows_.";
2 final var pos = input.indexOf( "value" );
3 if( index > 0 ) process( input.substring( 0, pos ) );
4
5 final Map<Key,Value> map = loadData();
6 final var key = getKey();
7 final var value = map.get( key );
8 if( nonNull( value ) ) process( value );
```

In line 2, the method `String::indexOf`^[270] return -1 in case the given text is not part of the `String`.

⁴⁹`java.lang.String` is immutable.

⁵⁰Another good reason to abandon `java.util.Date` and to use the classes from the `java.time`^[318] package instead.



In the same way, the method `Map::get`^[299] in line 7 returns `null` if the given key is not in the map. Unfortunately, `null` could also be a valid value, so in this case it does not necessarily indicate that key is not in the map.

You should avoid this kind of return values.

A better declaration for the two methods would be⁵¹:

```
public interface Map<K,V>
{
    public Optional<V> get( final K key );
} // interface Map

public final class String
{
    public OptionalInt indexOf( final String str ) { ... }
} // class String
```

With this declarations, you could now write the examples from above as:

```
1 final var input = "Some_Text_follows_.";
2 final OptionalInt pos = input.indexOf( "value" );
3 pos.ifPresent( p -> process( input.substring( 0, p ) ) );
4
5 final Map<Key,Value> map = loadData();
6 final var key = getKey();
7 final Optional<Value> value = map.get( key );
8 value.ifPresent( this::process );
```

The classes `java.util.OptionalInt`^[98], `java.util.OptionalLong`^[99] and `java.util.OptionalDouble`^[97] can help you to avoid “magic numbers” as return values, indicating special results.

In cases where `null` seemed the proper response, choose `java.util.Optional`^[96] instead.

So you can implement the methods in your code they *never* return `null` nor a magic number of some kind. Nevertheless, when you are implementing a (3rd party) interface that requests a return value of `null` or another special value under some conditions, you have to obey that requirements.

⁵¹Both methods had been part of the Java Runtime library since version 1.0, and no one seriously suggests to change them now; this is just to illustrate the topic.



8. Coding Guidelines

In Java, you usually do not return an error code if an operation fails – that `Map::get` returns `null` if the key is not mapped is not indicating an error: that a key is not mapped is one expected result. So instead of returning an error code, a method will throw an exception.

But what if you do not want to throw an exception, for whatever reason?

Try this:

Listing 8.11: Status Record

```
1 public record Status( ResultCode resultCode, Optional<Data> data )
2 {
3     /*-----*\
4     ===** Inner Classes **=====
5     \*-----*/
6     /** */
7     public enum ResultCode
8     {
9         SUCCESS, NO_DATA, FAILURE, TIMEOUT, INCOMPLETE
10    }
11    // enum ResultCode
12
13    /*-----*\
14    ===** Constructors **=====
15    \*-----*/
16    public Status( final ResultCode resultCode, final Data data )
17    {
18        this( resultCode, Optional.ofNullable( data ) );
19    } // Status()
20
21    public Status
22    {
23        requireNonNullArgument( resultCode, "resultCode" );
24        requireNonNullArgument( data, "data" );
25    } // Status()
26 }
27 // record Status
```

This would allow you to write:

```
public final Status retrieveData() { ... }
...
var proceed = true;
while( proceed )
```



```
{
    final var status = retrieveData();
    switch( status.resultCode )
    {
        case SUCCESS -> process( status.data().get() );
        case INCOMPLETE -> status.data().ifPresent(
            this::processIncomplete );
        case NO_DATA -> proceed = false;
        case TIMEOUT -> reconnect();
        case FAILURE -> throw new ProcessStatusException();
        default -> throw new UnsupportedOperationException( status.resultCode() );
    }
}
```

If you need this kind of return values more often for your project, you can enhance that **Status** class in various ways. So you can use a type variable instead of the concrete class **Data**, to make the class generic. Or you add a method like this:

```
...
public final void onSuccess( final Consumer<Data> processor )
{
    if( resultCode == SUCCESS ) data.ifPresent( processor::accept );
}
// onSuccess()
...
```

8.7. Implementing Immutable Types

In Java⁵², an immutable type does not allow that the internal state of one of its instances can be modified after creation. But to achieve that is more difficult than it sounds.

Principles

P1. To be immutable, all attributes of a class have to be **private** and **final**.

⁵²And most probable, in any other programming language, too



Even without a proper mutator method, non-**final** attributes could be changed, at least via Reflection.

P2. For an immutable type, all attribute *values* have to be immutable, too.

Otherwise, they could be modified externally, again by using (abusing?) Reflection. But this does not necessarily mean that the type of the attributes must be an immutable type.

P3. The class for the immutable type has to be **final**.

This is to prevent the creation of a mutable subclass whose instances can be used instead of the (apparently) immutable base class.

8.7.1. The **final** Keyword in Java

In Java, variables are mutable by default, meaning you can change the value they hold.

By using the **final** keyword when declaring a variable, the Java compiler won't let you change the value of that variable; Instead, a compile-time error will be issued:

```
final String name = "Java";  
name = "Borneo"; // does not compile
```

In order to understand this it is necessary to remember that **name** is *technically not a variable of type 'String'*, but a variable of type 'Reference to String'⁵³.

So **final** only forbids you from changing the reference a variable *holds*, it doesn't protect you from changing the internal state of the object it *refers* to by using its public API:

```
final List<String> strings = new ArrayList<>();  
assertEquals( 0, strings.size() );  
strings.add( "Java" );  
assertEquals( 0, strings.size() );
```

⁵³Although the general wording corresponds to the first variant: when Java programmers talk about a 'String variable **name**', they mean of course **String name**.



The second call to `assertEquals()` fails because adding an element to the list changes its size, therefore, it isn't an immutable object. The `final` keyword *does not prevent* you from calling `add()` on the list.

This is different from the behaviour of the `const` keyword in C++; a declaration like

```
const MyClass object;
```

would mean that you cannot call any methods on `object` that would change the internal state of the instance. 'Safe' methods are marked as `const`⁵⁴:

```
class MyClass
{
    int safeMethod() const;
    int unsafeMethod();
}
```

Therefore the code below does not compile because of the call to `unsafeMethod()` in line 2:

```
1 const MyClass object;
2 object.unsafeMethod(); // does not compile
```

But this code variant will work:

```
MyClass object;
object.unsafeMethod();

const MyClass constObject;
constObject.safeMethod();
```

Unfortunately, it is not that simple in Java; we all know that

```
1 final List<String> list = new ArrayList<>();
2 list.add( "Message" );
```

works fine – in fact, that is a general pattern.

Nevertheless, you should use `final` wherever you do not intend to alter a (reference) variable, not only for properties.

⁵⁴Unfortunately the C++ compiler does not detect if a `const` method will change the internal state; at least it will not abort the compilation.



8.7.2. Property Types, Constructor Arguments and Return Values for Accessors

Basically, this was already discussed in the chapters “8.5.1 Mutable Types for Arguments” on page 287 and “8.6.2 Mutable Return Types” on page 304:

- A constructor argument value of a mutable type needs to be copied before stored to the property; and it should be converted to an immutable type.
- The return value of a getter should be a copy of the property value if that is mutable.

In particular the latter sounds simple: the attributes have to be immutable, so we can just return them ...

Let's see this code:

```
1 public final class MyClass
2 {
3     private final Date m_Date;
4     private final String [] m_Names;
5
6     public MyClass( final String [] names, final Date date )
7     {
8         m_Date = requireNonNullArgument( date, "date" );
9         m_Names = copyOfRange( requireNonNullArgument( names, "names" ),
10                               0, names.length );
11     } // MyClass()
12
13     public final Date getDate() { return m_Date; }
14
15     public final String [] getNames() { return copyOfRange( m_Names, 0,
16                                                             m_Names.length ); }
17 }
18 // class MyClass
```

Unfortunately, the class `java.util.Date`^[86] is *not* immutable. And the although we copied the array, it can still be modified through Reflection.



A better implementation would look like this, assuming we cannot get rid of `java.util.Date`⁵⁵:

```
1 public final class MyClass
2 {
3     private final long m_Date;
4     private final List<String> m_Names;
5
6     public MyClass( final String [] names, final Date date )
7     {
8         m_Date = requireNonNullArgument( date, "date" ).getTime();
9         m_Names = List.of( requireNonNullArgument( names, "names" ) );
10    } // MyClass()
11
12    public final Date getDate() { return new Date( m_Date ); }
13
14    public final String [] getNames() { return m_Names.toArray( String
15        []::new ); }
16 } // class MyClass
```

The implementation of `java.util.List`[201] that is returned by `List::of`[295] does not provide any API that would allow to modify it. Nevertheless, it is not safe from being modified through Reflection, if someone would spent enough time and effort to research how to do it. This means that it is difficult to get a collection really secured against this kind of abuse, but on the other side, you should not make it *too* easy!

8.7.3. Mutators for Immutable Types

`java.lang.String`[66] and `java.math.BigInteger`[73] are two prominent immutable types in Java. Both classes provide methods that *appear* to allow the modification of the current instance – for example:

- `String::replace`[271]
- `BigInteger::clearBit`[283]

⁵⁵Otherwise, we would replace it by a type from the `java.time` package[318]; these are immutable.



On a second look you will find that both methods will have a return value other than `void` as one might expect for a mutator method. And the documentation tells you that these methods will return a new instance of the respective class with the modified status:

```
1 final var s1 = "Ping";
2 final var s2 = s1.replace( "P", "Str" );
3 var isSame = s1 == s2; // Will be false
4
5 final var b1 = new BigInteger( "1234" );
6 final var b2 = b1.clearBit( 2 );
7 isSame = b1 == b2; // Will be false;
```

8.7.4. Apparently immutable Types

That a type is immutable does not necessarily imply that this type is also quite simple. Such a type can be still fairly complex, and when its instances are used as keys in maps, or their string representations are used quite often, you may think about an implementation like this:

```
1 public final MyClass
2 {
3     private final T1 m_Attribute1;
4     private final T2 m_Attribute2;
5     ...
6     private final Tn m_AttributeN;
7
8     private transient Integer m_HashValue = null;
9     private transient String m_StringRepresentation = null;
10
11     public MyClass( final T1 attribute1, final T2 attribute2, ..., Tn
12         attributeN )
13     {
14         m_Attribute1 = attribute1;
15         m_Attribute2 = attribute2;
16         ...
17         m_AttributeN = attributeN;
18     } // MyClass()
19
20     @Override
21     public final int hashCode() // DISCOURAGED!
22     {
```



```

22     if( isNull( m_HashValue ) )
23     {
24         m_HashValue = Integer.valueOf( Objects.hash( m_Attribute1,
25                                                     m_Attribute2, ..., m_AttributeN ) );
26     }
27     //---* Done *-----
28     return m_HashValue;
29 } // hashCode()
30
31 @Override
32 public final String toString() // DISCOURAGED!
33 {
34     if( isNull( m_StringRepresentation ) )
35     {
36         m_StringRepresentation = ...
37     }
38
39     //---* Done *-----
40     return m_StringRepresentation;
41 } // toString()
42 }
43 // MyClass

```

As all methods will return always the same values (we assume that there are only `equals()` and the getters for the attributes ...), objects of this type *appears* to be immutable.

For most cases, it does not matter that they are *technically* not immutable, but if it matters, this may cause some errors that are really difficult to track down.

But caching the values for the hash and for the string representation is still a good idea⁵⁶ for an immutable type, you just should implement it differently⁵⁷:

```

1 public final MyClass
2 {
3     private final T1 m_Attribute1;
4     private final T2 m_Attribute2;

```

⁵⁶... although it is in a grey area in regards of 'premature optimisations' – refer to [237].

⁵⁷If **MyClass** is serialisable, **m_HashValue** and **m_StringRepresentation** can be **transient**; you just need to implement **readObject()** that it will reconstruct them properly.



```
5      ...
6      private final Tn m_AttributeN;
7
8      private final Integer m_HashValue;
9      private final String m_StringRepresentation;
10
11     public MyClass( final T1 attribute1, final T2 attribute2, ..., Tn
12                    attributeN )
13     {
14         m_Attribute1 = attribute1;
15         m_Attribute2 = attribute2;
16         ...
17         m_AttributeN = attributeN;
18
19         m_HashValue = Integer.valueOf( Objects.hash( m_Attribute1,
20                                                     m_Attribute2, ..., m_AttributeN );
21         m_StringRepresentation = ...
22     } // MyClass()
23
24     @Override
25     public final int hashCode() { return m_HashValue; }
26
27     @Override
28     public final String toString() { return m_StringRepresentation; }
29 }
30 // MyClass
```

The methods `hashCode()` and `toString()` are just samples here; your class can have lots of other methods that will follow that same or a similar pattern, allowing to pregenerate their results: if you will do this, it should happen in the constructor!

8.7.5. Changes through Reflection

Confessed, protecting your code against data modification through Reflection seems paranoid at a first glance: in order to exploit such a weakness, someone needs to have access to the system in a way that would allow them to do much worse things ...

But you do not need to think about malicious code introduced by hackers; too often your own colleagues are similarly dangerous!



Assume the following implementation for a class that is *intended* to be immutable:

```
1 public final class MyClass
2 {
3     private final Date m_ContractEnd;
4
5     public MyClass( final Date contractEnd )
6     {
7         m_ContractEnd = requireNonNullArgument( contractEnd, "contractEnd"
8             ).clone();
9     } // myClass()
10
11     @Override
12     public final boolean equals( final Object o )
13     {
14         var retValue = this == o;
15         if( !retValue && o instanceof MyClass other )
16         {
17             retValue = m_ContractEnd.equals( o.m_ContractEnd );
18         }
19         //---* Done *-----
20         return retValue;
21     } // equals()
22
23     public final Date getContractEnd() { return m_ContractEnd; }
24
25     @Override
26     public final int hashCode() { return Objects.hash( m_ContractDate ); }
27 }
28 // class MyClass()
```

Of course that class had some other data than just the date for the end of the contract, but this is not relevant for the scenario here. But you can assume that the design of this class was according the business specification.

Instances of this class had been given from the backend layer to the frontend layer (in fact, the backend returned them on request from the frontend) as attributes of another object that was not immutable. And when the frontend returned that container object, this had been persisted completely, including the incorporated **MyClass** instance⁵⁸, although this had not been necessary.

⁵⁸Of course the real names had been different, but that does not matter here.



8. Coding Guidelines

As it was not possible to modify those **MyClass** instance, it was required to create a new instance in case of a contract extension – and this was desired that way by the backend, for some business reasons.

But the guys from the frontend team had a different idea: they just want to change the end date as this was much, much easier for them to implement. So they implemented the following code⁵⁹:

```
1 public final void extendContract( final MyClass value, final Date
   newContractEnd )
2 {
3     final var valueClass = value.getClass();
4     final var contractField = valueClass.getDeclaredField(
       "m_ContractEnd" );
5     contractField.setAccessible( true );
6     final var currentContractEnd = (Date) contractField.get( value );
7     currentContractEnd.setTime( newContractEnd.getTime() );
8 } // extendContract()
```

Everyone was happy with this solution: the frontend team because of their simple, straightforward implementation of the contract extension, and the backend team because they did not know about it ...

Sometime later, the procedure for new contracts was changed; additional data was required, and amongst that the acceptance of the GDPR. It was decided that no special action should be taken for the existing customers because when their contracts ends, a new contract will be created that automatically requires that new data. Only that the hack above avoid the creation of these new contracts ...

It took some time before someone detected that nearly none of the old clients accepted the GDPR, and some more time before the reason why was found. Of course no one was responsible for the problem, the developers that created the code had left the company already some time ago ...

That the objects of the ‘immutable’ class **MyClass** could be manipulated through Reflection was not the only fault in this scenario, but that one that made the whole thing possible.

⁵⁹I omitted all the required error handling stuff, as it does not add more clarity.



Refer to the chapter “8.11 Accessing Fields or Methods using Reflection” for some more details about Reflection.

8.8. Local Variable Type Inference

Java 10 introduced type inference[166] for local variables[144, 248, 369]. Previously, all local variable declarations required an explicit (manifest) type on the left-hand side. With type inference for local variable, the explicit type can be replaced by the reserved type name `var`⁶⁰ for local variable declarations that have initialisers. The type of the variable is inferred from the type of the initialiser:

```
final var s = "String";           // s is java.lang.String
final var i = 9;                  // i is int
final var d = 3.1415;             // d is double
final var list = new ArrayList<String>(); // list is
    java.util.ArrayList<String>
```

There is a certain amount of controversy over this feature. Some welcome the concision it enables; others fear that it deprives readers of important type information, impairing readability. And both groups are right, to some extent. It can make code more readable by eliminating redundant information, and it can also make code less readable by eliding useful information. Another group worries that it will be overused, resulting in more bad Java code being written. This is also true, but it's also likely to result in more good Java code being written. Like all features, it must be used with judgement. There's no blanket rule for when it should and shouldn't be used.

Local variable declarations do not exist in isolation; the surrounding code can affect or even overwhelm the effects of using `var`. The goal of this chapter is to examine the impact that surrounding code has on `var` declarations, to

⁶⁰That `var` was not introduced as a new *keyword* for the language means it can be still used as name for local variables, fields and methods – although it is definitely discouraged to do so!



explain some of the tradeoffs, and to provide guidelines for the effective use of `var`.⁶¹

Principles

P1. Reading code is more important than writing code.

Code is read much more often than it is written. Further, when writing code, you usually have the whole context in your head, and take your time; when reading code, you are often context-switching, and may be in more of a hurry. Whether and how particular language features are used ought to be determined by their impact on future readers of the program, not its original author. Shorter programs can be preferable to longer ones, but shortening a program too much can omit information that is useful for understanding the program. The central issue here is to find the right size for the program such that understandability is maximized.

You should be specifically unconcerned here with the amount of keyboarding that is necessary to input or to edit a program. While concision may be a nice bonus for the author, focusing on it misses the main goal, which is to improve the understandability of the resulting program.

P2. Code should be clear from local reasoning.

The reader should be able to look at a `var` declaration, along with uses of the declared variable, and understand almost immediately what is going on. Ideally, the code should be readily understandable using only the context from a snippet or a patch. If understanding a `var` declaration requires the reader to look at several locations around the code, it might not be a good situation in which to use `var`. Then again, it might indicate a problem with the code itself.

⁶¹This chapter is basically a summary of the article “Local Variable Type Inference - Style Guidelines”, published by Stuart W. Marks in March 2018[248].



P3. Code readability shouldn't depend on IDEs.

Code is often written and read within an IDE, so it's tempting to rely heavily on the code analysis features of IDEs. For type declarations, why not just use `var` everywhere, since one can always point at a variable to determine its type?

The main reason is that code is often read outside an IDE. Code appears in many places where IDE facilities are not available, such as snippets within a document, browsing a repository on the internet, or in a patch file. It is counterproductive to have to import code into an IDE simply to understand what the code does.

Code should be self-revealing. It should be understandable on its face, without the need for assistance from tools.

P4. Explicit types are a tradeoff.

Java has historically required local variable declarations to include the type explicitly. While explicit types can be very helpful, they are sometimes not very important, and are sometimes just in the way. Requiring an explicit type can add clutter that crowds out useful information.

Omitting an explicit type can reduce clutter, but only if its omission does not impair understandability. The type is not the only way to convey information to the reader. Other means include the variable's name and the initialiser expression. You should take all the available channels into account when determining whether it is appropriate to mute one of these channels.

8.8.1. Naming

I discussed the naming of local variable already in chapter 6.8 on page 128 and I want elaborate on this again here.

First you have to choose a variable name that provides useful information to the reader of your code. This is good practice in general, but it is much more important in the context of `var`. In a declaration using `var`, information about the meaning and use of the variable can be conveyed mainly using



8. Coding Guidelines

the variable's name. Replacing an explicit type with `var` should often be accompanied by improving the variable name.

For example:

```
// BAD
final List<Customer> x = dbConnection.executeQuery( query );

// BETTER
final var customerList = dbConnection.executeQuery( query );
```

In this case, a useless variable name has been replaced with a name that is evocative of the type of the variable, which is now implicit in the `var` declaration.

Encoding the variable's type in its name, taken to its logical conclusion, may result in "Hungarian Notation"[173, 213]. Just as with explicit types, this is sometimes helpful, but in most times just clutter. In this example the name `customerList` implies that a `List` with customer data is being returned. It might not be significant that it is in fact a `List`. Instead of the exact type, it's sometimes better for a variable's name to express the role or the nature of the variable, such as `customers`:

```
// BAD
try( final Stream<Customer> result = dbConnection.executeQuery( query ) )
{
    return result.map( ... )
                .filter( ... )
                .findAny();
}

// BETTER
final Optional<Customer> retValue;
try( final var customers = dbConnection.executeQuery( query ) )
{
    retValue = customers.map( ... )
                    .filter( ... )
                    .findAny();
}

//---* Done *-----
return retValue;
```



8.8.2. Scope

You should also try to minimize the scope of your local variables. This practice is described in “Effective Java (3rd Edition)”[28], Item 57.

Limiting the scope of a local variable is good practice in general, but it applies with extra force if **var** is in use.

In the following example, the **add()** method clearly adds the special item as the last list element, so it’s processed last, as expected.

```
final var items = new ArrayList<Item>( ... );
items.add( MUST_BE_PROCESSED_LAST );
for( final var item : items ) { ... }
```

Now suppose that in order to remove duplicate items, you modify this code to use a **HashSet**[51] instead of an **ArrayList**[83], like below:

```
final var items = new HashSet<Item>( ... );
items.add( MUST_BE_PROCESSED_LAST );
for( final var item : items ) { ... }
```

Your code now has a bug, since a **Set** does not have a defined iteration order. However, it is likely that you will fix this bug immediately, as the uses of the **items** variable are adjacent to its declaration.

Now suppose that this code is part of a large method, with a correspondingly large scope for the **items** variable:

```
final var items = new HashSet<Item>( ... );

// ... 100 lines of code ...

items.add( MUST_BE_PROCESSED_LAST );
for( final var item : items ) { ... }
```

The impact of changing from an **ArrayList** to a **HashSet** is no longer readily apparent, since **items** is used so far away from its declaration. It seems likely that this bug could survive for much longer.

If **items** had been declared explicitly as **List<String>**, changing the initialiser would also require changing the type to **Set<String>**. This *might* prompt you



to inspect the rest of the method for code that would be impacted by such a change (then again, it might not). Use of `var` would remove this prompting, possibly increasing the risk of a bug being introduced in code like this.

This might seem like an argument against using `var`, but it really is not. The initial example that uses `var` is perfectly fine. The problem only occurs when the variable's scope is large. Instead of simply avoiding `var` in these cases, you should change the code to reduce the scope of the local variables, and only then declare them with `var`.

8.8.3. Initialisers

First, you cannot use `var` if you do not have a proper initialiser:

```
// DOES NOT WORK!
final var x;

// DOES NOT WORK EITHER!
final var x = null;
```

Local variables are often initialised with constructors. The name of the class being constructed is often repeated as the explicit type on the left-hand side. If the type name is long, use of `var` provides concision without loss of information:

```
// ACCEPTABLE
final ByteArrayOutputStream outputStream = new ByteArrayOutputStream();

// BETTER
final var outputStream = new ByteArrayOutputStream();
```

It is also reasonable to use `var` in cases where the initialiser is a method call, such as a `static` factory method, instead of a constructor, and when its name contains enough type information:

```
// ACCEPTABLE
final BufferedReader reader = Files.newBufferedReader( ... );
final List<String> stringList = List.of("a", "b", "c");

// BETTER
```



```
final var reader = Files.newBufferedReader(...);  
final var stringList = List.of("a", "b", "c");
```

In these cases, the method names strongly imply a particular return type, which is then used for inferring the type of the variable.

So the rule of thumb is to consider the use of `var` when the initialiser provides sufficient information to the reader.

8.8.4. `var` and “Programming to the Interface”

A common idiom in Java programming is to construct an instance of a concrete type but to assign it to a variable of an interface type. This binds the code to the abstraction instead of the implementation, which preserves flexibility during future maintenance of the code. For example:

```
// Original  
final List<String> list = new ArrayList<>();
```

If `var` is used, however, the concrete type is inferred instead of the interface:

```
// Inferred type of list is ArrayList<String>  
final var list = new ArrayList<String>();
```

Usually this idiom is referred to as “Programming to the Interface”, and it will be discussed in more detail in chapter 8.28 on page 406.

In the context that we discuss here, the recommendation is just: don’t worry too much about “programming to the interface” with local variables. As we saw above, the compiler will always infer the current type returned by the initialiser for the variable that is declared with `var`.

It must be reiterated here that `var` can only be used for local variables. It cannot be used to infer field types, method parameter types, and method return types. The principle of “programming to the interface” is still as important as ever in those contexts.



The main issue is that code that uses the variable can form dependencies on the concrete implementation. If the variable's initialiser were to change in the future, this might cause its inferred type to change, causing errors or bugs to occur in subsequent code that uses the variable.

If, as recommended in chapter 8.8.2, the scope of the local variable is small, the risks from “leakage” of the concrete implementation that can impact the subsequent code are limited. If the variable is used only in code that's a few lines away, it should be easy to avoid problems or to mitigate them if they arise.

In this particular case, `ArrayList` only contains a couple of methods that aren't on `List`, namely `ensureCapacity()` and `trimToSize()`. These methods don't affect the contents of the list, so calls to them don't affect the correctness of the program. This further reduces the impact of the inferred type being a concrete implementation rather than an interface.

8.8.5. `var` with Literals

Primitive literals can be used as initialisers for `var` declarations. It's unlikely that using `var` in these cases will provide much advantage, as the type names are generally short. However, `var` is sometimes useful, for example, to align variable names. And it usually does no harm either.

But you should take some care when using `var` with literals.

There is no issue with `boolean`, `character`, `long`, and string literals. The type inferred from these literals is precise, and so the meaning of `var` is unambiguous:

```
// Original
boolean ready = true;
char ch = '\ufffd';
long sum = 0L;
String label = "wombat";

// GOOD
var ready = true;
var ch = '\ufffd';
var sum = 0L;
```




```
var label = "wombat";
```

Particular care should be taken when the initialiser is a numeric value, especially an integer literal. With an explicit type on the left-hand side, the numeric value may be silently widened or narrowed to types other than `int`. With `var`, the value will be inferred as an `int`, which may be unintended.

```
// Original
byte flags = 0;
short mask = 0x7fff;
long base = 17;

// DANGEROUS: all infer as int
var flags = 0;
var mask = 0x7fff;
var base = 17;
```

Floating point literals are mostly unambiguous:

```
// Original
float f = 1.0f;
double d = 2.0;

// GOOD
var f = 1.0f;
var d = 2.0;
```

Note that `float` literals can be widened silently to `double`. It is somewhat obtuse to initialise a `double` variable using an explicit `float` literal such as `3.0f`, however, cases may arise where a `double` variable is initialised from a `float` field. Caution with `var` is advised here⁶²:

```
// Original
static final float INITIAL = 3.0f;
...
double temp = INITIAL;

// DANGEROUS: now infers as float
var temp = INITIAL;
```

⁶²Indeed, this example violates the guideline from chapter 8.8.3 on page 326, because there isn't enough information in the initialiser for a reader to see the inferred type.



8.8.6. Using `var` with the ‘Diamond Operator’ or Generic Methods

Both `var` and the “diamond” feature⁶³ allow you to omit explicit type information when it can be derived from information already present. Can you use both in the same declaration?

Consider the following:

```
final PriorityQueue<Item> itemQueue = new PriorityQueue<Item>();
```

This can be rewritten using either diamond or `var`, without losing type information:

```
// OK: both declare variables of type PriorityQueue<Item>
final PriorityQueue<Item> itemQueue = new PriorityQueue<>();
final var itemQueue = new PriorityQueue<Item>();
```

It is syntactically legal to use both `var` and diamond, but the inferred type will change:

```
// DANGEROUS: infers as PriorityQueue<Object>
final var itemQueue = new PriorityQueue<>();
```

For its inference, diamond can use the target type (typically, the left-hand side of a declaration) or the types of constructor arguments. If neither is present, it falls back to the broadest applicable type, which is often `java.lang.Object`. This is usually not what was intended.

Generic methods have employed type inference so successfully that it’s quite rare for programmers to provide explicit type arguments. Inference for generic methods relies on the target type if there are no actual method arguments that provide sufficient type information. In a `var` declaration, there is no target type, so a similar issue can occur as with diamond. For example,

⁶³The name ‘diamond’ stems from the appearance of the empty angle brackets, and it is not an operator at all. Nevertheless, it is referred to as the “Diamond operator” in quite a lot of sources.



```
// DANGEROUS: infers as List<Object>
final var list = List.of();
```

With both diamond and generic methods, additional type information can be provided by actual arguments to the constructor or method, allowing the intended type to be inferred. Thus like below:⁶⁴

```
// OK: itemQueue infers as PriorityQueue<String>
final Comparator<String> comp = ... ;
final var itemQueue = new PriorityQueue<>( comp );

// OK: infers as List<BigInteger>
final var list = List.of( BigInteger.ZERO );
```

If you decide to use `var` with diamond or a generic method, you should ensure that method or constructor arguments provide enough type information so that the inferred type matches your intent. Otherwise, avoid using both `var` with diamond or a generic method in the same declaration.

8.8.7. Examples

Some examples of where `var` can be used to greatest benefit.

- The following code removes at most max matching entries from a `Map`[203]. Wildcarded type bounds are used for improving the flexibility of the method, resulting in considerable verbosity. Unfortunately, this requires the type of the `Iterator`[187] to be a nested wildcard, making its declaration more verbose. This declaration is so long that the header of the for-loop no longer fits on a single line, making the code even harder to read.

```
// Original
public final void removeMatches( final Map<? extends String,? extends
    Number> map, final int max )
{
```

⁶⁴But notice that the instances are slightly different: the `PriorityQueue` is now using an explicit comparator, and the `List` is not empty ...



```
for( Iterator<? extends Map.Entry<? extends String,? extends
    Number>> iterator = map.entrySet().iterator();
    iterator.hasNext(); )
{
    Map.Entry<? extends String,? extends Number> entry =
        iterator.next();
    if( max > 0 && matches( entry ) )
    {
        iterator.remove();
        --max;
    }
} // removeMatches()
```

Use of **var** here removes the noisy type declarations for the local variables. Having explicit types for the **Iterator** and **Map.Entry**[204] locals in this kind of loop is largely unnecessary. This also allows the **for**-loop control to fit on a single line⁶⁵, further improving readability.

```
// GOOD
public final void removeMatches( final Map<? extends String,? extends
    Number> map, final int max )
{
    for( var iterator = map.entrySet().iterator();
        iterator.hasNext(); )
    {
        var entry = iterator.next();
        if( max > 0 && matches( entry ) )
        {
            iterator.remove();
            --max;
        }
    }
} // removeMatches()
```

- Consider code that reads a single line of text from a socket using the **try-with-resources** statement. The networking and I/O APIs use an object wrapping idiom. Each intermediate object must be declared as a resource variable so that it will be closed properly if an error occurs while opening a subsequent wrapper. The conventional code for this

⁶⁵Confessed, not with the line length enforced in this document ...



requires the class name to be repeated on the left and right sides of the variable declaration, resulting in a lot of clutter:

```
// Original
try( final InputStream is = socket.getInputStream();
    final InputStreamReader isr = new InputStreamReader( is, charset
    );
    final BufferedReader buf = new BufferedReader( isr ) )
{
    return buf.readLine();
}
```

Using `var` reduces the noise considerably:

```
// GOOD
try( final var inputStream = socket.getInputStream();
    final var reader = new InputStreamReader( inputStream, charset );
    final var bufReader = new BufferedReader( reader ) )
{
    return bufReader.readLine();
}
```

- Consider code that takes a collection of strings and finds the string that occurs most often. This might look like the following:

```
final String retValue = strings.stream()
    .collect( groupingBy( s -> s, counting() ) )
    .entrySet()
    .stream()
    .max( Map.Entry.comparingByValue() )
    .map( Map.Entry::getKey );
```

This code is correct, but it is potentially confusing, as it looks like a single stream pipeline. In fact, it's a short stream, followed by a second stream over the result of the first stream, followed by a mapping of the `Optional`[96] result of the second stream. The most readable way to express this code would have been as two or three statements; first group entries into a map, then reduce over that map, then extract the key from the result (if present), as shown below:

```
// BETTER READABLE
final Map<String, Long> freqMap = strings.stream()
    .collect(groupingBy( s -> s, counting() ) );
```



```
final Optional<Map.Entry<String,Long>> maxEntryOpt =  
    freqMap.entrySet()  
        .stream()  
        .max( Map.Entry.comparingByValue() );  
final String retVal = maxEntryOpt.map( Map.Entry::getKey );
```

But you probably resisted doing that because writing the types of the intermediate variables seemed too burdensome, so instead they distorted the control flow. Using `var` here allows you to express the code more naturally without paying the high price of explicitly declaring the types of the intermediate variables:

```
// EVEN BETTER READABLE  
final var freqMap = strings.stream()  
    .collect( groupingBy( s -> s, counting() ) );  
final var maxEntryOpt = freqMap.entrySet()  
    .stream()  
    .max( Map.Entry.comparingByValue() );  
final var retVal = maxEntryOpt.map( Map.Entry::getKey );
```

You might legitimately prefer the first snippet with its single long chain of method calls. However, in some cases it is better to break up long method chains. Using `var` for these cases is a viable approach, whereas using full declarations of the intermediate variables as in the second snippet makes it an unpalatable alternative. As with many other situations, the correct use of `var` might involve both taking something out (explicit types) and adding something back (better variable names, better structuring of code).

8.8.8. Conclusion

Using `var` for declarations can improve code by reducing clutter, thereby letting more important information stand out. On the other hand, applying `var` indiscriminately can make things worse. Used properly, `var` can help improve good code, making it shorter and clearer without compromising understandability.



8.9. Access to Properties

Principles

P1. Encapsulation is an important design principle for classes.

It means that the internal state of an object can be manipulated only in a well defined manner, through the declared `public` methods.

When the object state may be changed only through public method, this means it may not be possible to modify the attributes directly, by a direct assignment.

To achieve that all non-`final` mutable instance or class variables – also known as properties, attributes or fields – have to be `private`.

If it is necessary to set or retrieve an instance variable (a property) directly, you have to provide the appropriate methods for this (setter and/or getter, or mutator and/or accessor methods). But often attributes are set or retrieved as a side effect of method calls that modify the internal state of the object instance, or rely on it.

Programmers are sometimes inclined to use `public` fields when they need a data structure like a `struct` in C/C++, and not a full-fledged class. But as Java does not know that `struct` data structure, it seems to be a quick solution to have a `class` with only `public` fields and no methods, just to spare typing effort⁶⁶.

And if you want to avoid the typing, use a `record`[219, 158, 29] instead. Sometimes also an instance of `java.util.Map` or another collection implementation could be an alternative to a specialised class.

⁶⁶And sometimes also with the idea, that the direct access to the `public` field is much faster than accessing it through a method. But modern optimising compilers will inline the code of a simple getter method, so that there is no difference in the end.



Sometimes fields from base classes are defined as **protected**, to make them directly accessible by methods from the derived implementations, but this is also discouraged.⁶⁷

It does not matter if we talk about properties (instance variables) or **static** fields (class variables): both should always be **private**.

Constants are the only exception, obviously, because a constant is explicitly defined as a **public static final** field.

To read more about encapsulation in general, refer to chapter 8.35.1 on page 420.

8.10. Value Types

Basically, we distinguish two types of 'Value Types': first there are the 'Domain Value Types', and then we have 'Dimensioned Values'.

Principles

P1. Value types are immutable.

Once created, a value type instance cannot be modified, like a number.

P2. A value type instance has to be validated on creation.

Once instantiated, the value type instance remains valid.

Assume the following specification: "A person is identified by an alphanumeric id with seven places; the first place has to be a letter, all other places can be letters and digits; only capital letters are allowed, but not 'I', 'J', 'O' and 'Q'."

⁶⁷The idea behind that is the same false assumption that the direct access to the **protected** field is much faster than accessing it through a method.



Next, you have to provide the definition for the class **Person** that holds the id, the last name, the first name and a list of contacts, represented by the ids of those persons.

The constructor for that class may look like this:

```
public Person( final String id, final String lastName, final String
    firstName, final String... contactIds )
{
    m_Id = requireValidArgument( id, "id", this::validatePersonId );
    m_LastName = requireNotBlankArgument( lastName, "lastName" );
    m_FirstName = requireNotBlankArgument( firstName, "firstName" );
    var counter = 0;
    for( final var contactId : requireNonNullArgument( contactIds,
        "contactIds" ) )
    {
        m_ContactIds.add( requiredValidArgument( contactId, format(
            "contactIds_[%d]", counter++ ), this::validatePersonId ) );
    }
} // Person()
```

When you call that constructor, there are a lot of strings on the arguments list, and when you miss a name, suddenly the new person got the person id of a contact as its first name. This problem was already discussed in the chapters “8.1 Swap Logic Errors for Compiler Errors” on page 197, “8.5.4 Multiple Parameters with same Type” and “8.5.5 Generic Parameter Types” on page 297, and using value types were suggested as a solution.

Another issue is that you have to validate the provided person ids before storing them. Most probably this happened already elsewhere, perhaps already several times, but the constructor is public and you have to do it here again. Of course you can provide (and you should do!) the validation method through a utility class, but it still costs time and CPU cycles.

Using a value type – in this case, a ‘domain value type’ as it is defined for a specific problem domain – could solve this:

Listing 8.12: Value Type ‘PersonId’

```
1 public record PersonId( String id ) implements Cloneable,
   Comparable<PersonId>
2 {
3     /*-----*\
```



```
4      ===** Constructors **=====
5      \*-----*/
6      public PersonId { requireValidArgument( id, "id", this::validate ); }
7
8      /*-----*\
9      ===** Methods **=====
10     \*-----*/
11     @Override
12     public final PersonId clone()
13     {
14         final PersonId retValue;
15         try
16         {
17             retValue = (PersonId) super.clone();
18         }
19         catch( final CloneNotSupportedException e )
20         {
21             throw new UnexpectedExceptionError( e );
22         }
23
24         //---* Done *-----
25         return retValue;
26     } // clone()
27
28     @Override
29     public final int compareTo( final PersonId o )
30     {
31         return id.compareTo( o.id );
32     } // compareTo()
33
34     @Override
35     public final String toString() { return id; }
36
37     private final boolean validate( final String id )
38     {
39         ...
40     } // validate()
41 }
42 // record PersonId
```

Implementing the **PersonId** as a **record** has some advantages[29]: it provides working implementations for the methods **equals()**[262] and **hashCode()**[264] from **java.lang.Object**[59] as well as an accessor method for the raw string representing the id. You need to implement only the methods **clone()**[261],



`toString()`[265]⁶⁸ and `compareTo()`[260]⁶⁹ – the latter allows to sort instances of `PersonId` according to their natural order.

With this new type, we can declare the constructor `Person` like this:

```
public Person( final PersonId id, final String lastName, final String
    firstName, final PersonId... contactIds )
{
    m_Id = requireNonNullArgument( id, "id" );
    m_LastName = requireNotBlankArgument( lastName, "lastName" );
    m_FirstName = requireNotBlankArgument( firstName, "firstName" );
    stream( requireNonNullArgument( contactIds, "contactIds" ) )
        .filter( Objects::nonNull )
        .forEach( m_ContactIds::add );
} // Person()
```

Most domain value types follow the same a pattern as `PersonId` here: they are not much more than a wrapper around a simple value; is not always a `String`, you can also wrap a `UUID`[104] or a numerical value.

A ‘dimensioned value’ is similar to a domain value type, but the purpose is slightly different. Consider the following class:

```
public final class CardboardBox
{
    final double m_Height;
    final double m_Length;
    final double m_Width;

    public CardboardBox( final double height, final double length, final
        double width )
    {
        final var values = new double []
        {
            requireValidDoubleArgument( height, "height", h -> h > 0.0 ),
            requireValidDoubleArgument( length, "length", l -> l > 0.0 ),
            requireValidDoubleArgument( width, "width", w -> w > 0.0 )
        }
        sort( values );
        m_Height = values [2];
        m_Length = values [0];
    }
}
```

⁶⁸A record provides also an implementation of `toString()` but that is not feasible for our purpose here.

⁶⁹from the interface `java.lang.Comparable`[185]



8. Coding Guidelines

```
        m_Width = values [1];
    }    // CardboardBox()
}
// class CardboardBox
```

Again, the signature of the constructor contains multiple formal parameters of the same type, but we solved the problem here by rearranging them in the body of the constructor.

The real issue with this constructor is different: from the code, we cannot see how large a cardboard box is ... The instance that you create with

```
final var box = new CardboardBox( 1.0, 2.5, 4.0 );
```

can be as tiny as a matchbox or big enough for an aircraft carrier – because we do not know the dimensions for the given arguments. Of course, they could be written to the specification, perhaps you have added this information to the documentation comment for the class, or even to that for the constructor, but it is still error prone.

A solution may look like this:

Listing 8.13: Dimensioned Value Length

```
1 public record Length( Dimension dimension, double value ) implements
   Cloneable, Comparable<Length>
2 {
3     /*-----*\
4     ===** Inner Classes **=====
5     \*-----*/
6     public enum Dimension
7     {
8         /*-----*\
9         ===** Enum Declaration **=====
10        \*-----*/
11        MILLIMETER(      1.0, "mm" ),
12        CENTIMETER(     10.0, "cm" ),
13        METER           ( 1_000.0, "m" );
14
15        /*-----*\
16        ===** Attributes **=====
17        \*-----*/
18        private final double m_Factor;
```



```

19     private final String m_Unit;
20
21     /*-----*\
22     ===** Constructors **=====
23     \*-----*/
24     private Dimension( final double factor, final String unit )
25     {
26         m_Factor = factor;
27         m_Unit = unit;
28     } // Dimension()
29
30     /*-----*\
31     ===** Methods **=====
32     \*-----*/
33     public final double factor() { return m_Factor; }
34     public final String unit() { return m_Unit; }
35 }
36 // enum Dimension
37
38 /*-----*\
39 ===** Constructors **=====
40 \*-----*/
41 public Length
42 {
43     requireNonNullArgument( dimension, "dimension" );
44     value = requireValidDoubleArgument( value, "value", v -> v > 0.0 )
45         * dimension.factor();
46 } // Length()
47
48 /*-----*\
49 ===** Methods **=====
50 \*-----*/
51 @Override
52 public final Length clone()
53 {
54     final Length retValue;
55     try
56     {
57         retValue = (Length) super.clone();
58     }
59     catch( final CloneNotSupportedException e )
60     {
61         throw new UnexpectedExceptionError( e );
62     }

```



8. Coding Guidelines

```
63         //---* Done *-----
64         return retValue;
65     } // clone
66
67     @Override
68     public final int compareTo( final Length o ) { return Double.compare(
        value, o.value ); }
69
70     /**
71     * Factory method for your convenience ...
72     */
73     public static final Length( final double value, final Dimension
        dimension )
74     {
75         return new Length( value, dimension );
76     } // length()
77
78     @Override
79     public final String toString() { return format( "%2$f_ %1$s",
        dimension.unit(), value( dimension() ) ); }
80
81     public final double value( final Dimension otherDimension )
82     {
83         final var retValue = value / requireNonNullArgument(
            otherDimension, "otherDimension" ).factor();
84
85         //---* Done *-----
86         return retValue;
87     } // value()
88 }
89 // record Length
```

Now **CardboardBox** would look like this:

```
public final class CardboardBox
{
    final Length m_Height;
    final Length m_Length;
    final Length m_Width;

    public CardboardBox( final Length height, final Length length, final
        Length width )
    {
        final var values = new Length []
        {
```



```
        requireNonNullArgument( height, "height" ),
        requireNonNullArgument( length, "length" ),
        requireNonNullArgument( width, "width" )
    }
    sort( values );
    m_Height = values [2];
    m_Length = values [0];
    m_Width = values [1];
} // CardboardBox()
// class CardboardBox
```

and we can write for a new instance:

```
final var box = new CardboardBox( length( 1.0, MILLIMETER ), length( 2.5,
    MILLIMETER ), length( 4.0, MILLIMETER ) );
```

Obviously, the resulting box is even smaller than a regular matchbox ...

Another advantage is that you have to change your code only at one single location if you have to support also inch and foot as valid units: you just add the appropriate enum values to `Length.Dimension`.

8.11. Accessing Fields or Methods using Reflection

“Reflection” is a feature in the Java programming language that allows an executing Java program to examine or “*introspect*” upon itself, and manipulate internal properties of the program. For example, it’s possible for a Java class to obtain the names of all its members and display them.

The ability to examine and manipulate a Java class from within itself may not sound like very much, but in other programming languages this feature simply doesn’t exist. For example, there is no way in a PASCAL, C, or C++ program to obtain information about the functions defined within that program.

Reflection is mainly implemented by the interfaces/classes

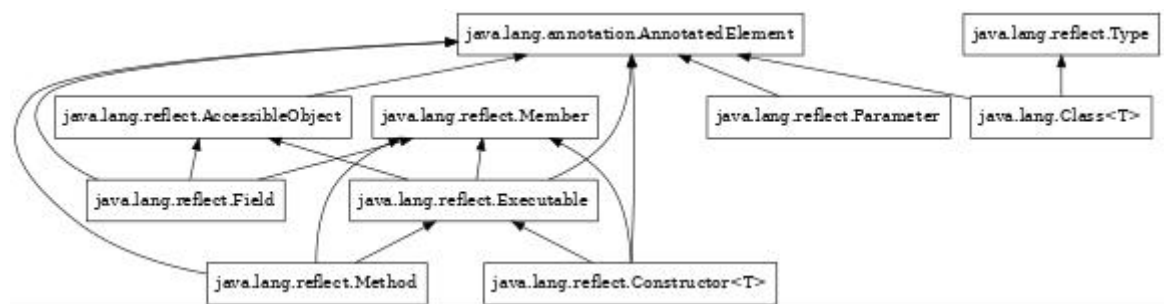
- `java.lang.Class`[45]



- `java.lang.Package`[60]
- `java.lang.Module`[57]
- `java.lang.annotation.Annotation`[179]
- `java.lang.reflect.Constructor`[62]
- `java.lang.reflect.Method`[64]
- `java.lang.reflect.Field`[63]
- `java.lang.reflect.Array`[61]

and their superclasses/superinterfaces.

The class diagram shows only an excerpt of the whole API:



Principles

- P1. First of all, in production code you should avoid to use Reflection whenever possible. Just do not use it!

One reason is that accessing a field or a method via Reflection causes some overhead that decreases a program's performance.

Next, such code is usually not easy to read or to understand, not only because of all the necessary error handling code around it.

But as always, there are a few exceptions: if your project is to write a programming tool – a code generator, an IDE, a code checker – or an



annotation processor, you cannot avoid Reflection. It is also useful for the implementation of some unit tests⁷⁰.

Next, some factory methods for arrays and/or generic types also require the use of Reflection.

- P2. The Reflection types may cause errors that are difficult to debug when used in public APIs.

Most problems with instances of these types will reveal only at runtime; also refer “8.1 Swap Logic Errors for Compiler Errors” on page 197), therefore they should not be used as parameter or return types. Of course this restriction does not apply for programming tools or test code.

- P3. Do not use Reflection to circumvent access restrictions.

The designer of a class made a method `private` for some reason, so it is usually a bad idea to use Reflection to invoke this method from the outside. Again, unit tests could be an exception. Refer to chapter 8.7.5 on page 318 that discusses a scenario where this went wrong.

- P4. When using modules[324], the results of calls to the Reflection APIs get unpredictable.

As long as these calls refer to classes within the same module, they behave as expected (mainly as before Java 9), but for classes from foreign modules, some results can be fairly surprising. Of course these results are not random, they still follow some strict rules. Nevertheless, they depend on the module definitions and on some command line arguments that can override those definitions.

8.11.1. Callback Methods

A callback is used by a server to report an event to a client; in Java, you will usually implement this through dedicated interfaces. A common pattern for

⁷⁰Although one could argue that tests are not production code.



8. Coding Guidelines

this is the listener pattern that can be implemented in Java like this⁷¹:

```
public interface AnEventListener extends EventListener
{
    /**
     * The callback method.
     *
     * @param event The event.
     */
    public void receiveEvent( final MyEvent event );
}
// interface AnEventListener
```

```
public interface AnEventSource
{
    public void addListener( final AnEventListener listener );
    public void removeListener( final AnEventListener listener );
}
// interface AnEventSource
```

```
public final class MyEvent extends EventObject
{
    public MyEvent( final AnEventSource source )
    {
        super( source );
    }
    MyEvent()
}
// class MyEvent
```

A real event object would have some payload in addition to the source, but for the purpose to explain the basic pattern, I omitted that, too.

```
public final class MyEventSource implements AnEventSource
{
    private final List<AnEventListener> m_Listeners = new ArrayList<>();

    @Override
    public final void addListener( final AnEventListener listener )
    {
        m_Listeners.add( listener );
    }
    // addListener()
```

⁷¹I omitted all error handling code, and some synchronisation would be required, too, but both would not help to understand the pattern



```
@Override
public final void removeListener( final AnEventListener listener )
{
    m_Listeners.remove( listener );
} // removeListener()

private final void sendEvent()
{
    final var event = new MyEvent( this );
    for( final var listener : m_Listeners )
    {
        listener.receiveEvent( event );
    }
} // sendEvent()
}
// class MyEventSource
```

```
public final class MyEventListener implements AnEventListener
{
    public final void registerToSource( final AnEventSource source )
    {
        source.addListener( this );
    } // registerToSource()

    @Override
    public final void receiveEvent( final MyEvent event )
    {
        // Respond to the event ...
    } // receiveEvent()
}
// class MyEventListener
```

A lot of code ... but the good thing is, that problems with the definition will be detected by the compiler.

But what if instances of already existing classes should be used as the event listener? Classes that do not share a common superclass and that does not expose the callback method in a convenient interface⁷². In C and C++, you could solve this with a method pointer, and in Java, Reflection seems to be the solution.

⁷²Another scenario would be that your are too lazy to write all that code ...



8. Coding Guidelines

Such an already existing “listener” class may look like this:

```
public final class AListener
{
    /**
     * The method that we want to use as the callback method.
     */
    public final void update() { /* Do something */ }
}
// class AListener
```

And you could implement the “event source” class like this:

```
public final class MySource
{
    private final Map<Object,Method> m_Listeners = new IdentityHashMap<>();

    public final void addListener( final Object listener )
    {
        try
        {
            final var method = listener.getClass().getMethod( "update" );
            m_Listeners.put( object, method );
        }
        catch( final NoSuchMethodException e )
        {
            throw new IllegalArgumentException( "Not_a_valid_listener!" );
        }
    } // addListener()

    private final void sendEvent()
    {
        for( final var entry : m_Listeners.entrySet() )
        {
            try
            {
                entry.getValue().invoke( entry.getKey() );
            }
            catch( final IllegalAccessException e )
            {
                /*
                 * update() is public.
                 */
                throw new UnexpectedExceptionError( e );
            }
        }
    }
}
```



```
        catch( final InvocationTargetException e )
        {
            switch( e.getCause() )
            {
                case null -> throw new UnexpectedExceptionError( e );
                case RuntimeException re -> throw re;
                case Error error -> throw error;
                default -> throw new UnexpectedExceptionError(
                    e.getCause() );
            }
        }
    } // sendEvent()
}
// class MySource
```

Unfortunately, the compiler cannot check anything here, if something is wrong, you will get to know it during runtime – if ever.

But since Java 8, there is an alternative; you can write the class **MySource** like this:

```
public final class MySource
{
    private final List<Runnable> m_Listeners = new ArrayList<>();

    public final void addListener( final Object listener )
    {
        switch( listener )
        {
            case null -> throw new NullPointerException( "listener" );
            case AListener aListener -> m_Listeners.add( aListener::update );
            default -> throw new IllegalArgumentException( "Not_a_valid_listener!" );
        }
    } // addListener()

    private final void sendEvent()
    {
        // No boiler plate error handling required!!
        for( final var listener : m_Listeners )
        {
            listener.run();
        }
    }
}
```



8. Coding Guidelines

```
    } // sendEvent()  
}  
// class MySource
```

This implementation uses method references[145] that can be checked at compile time.

But although this works, for the listener pattern you should prefer the first approach, because it shows your intent in a much better way. For other use cases, the last one is a good alternative to be used instead of Reflection.

8.11.2. Factory Methods with Reflection

Assume you have a class `RingBuffer<T>` that has a method `RingBuffer::toArray`.

A straight forward implementation of that method would look like this – and it would not compile:

```
1 public final class RingBuffer<T>  
2 {  
3     public final Iterator<T> iterator() { ... }  
4     public final int size() { ... }  
5  
6     public final T [] toArray()  
7     {  
8         // DOES NOT COMPILE!  
9         final T [] retValue = new T [size()];  
10        var index = 0;  
11        for( var iterator = iterator(); iterator.hasNext; ++index )  
12        {  
13            retValue [index] = iterator.next();  
14        }  
15  
16        //---* Done *-----  
17        return retValue;  
18    } // toArray()  
19 }  
20 // class RingBuffer
```



The compiler will complain about line 9 because you cannot instantiate a type variable directly. You can now change the return value of `toArray()` to `Object []`, or you implement it like this:

```

1 public final class RingBuffer<T>
2 {
3     public final Iterator<T> iterator() { ... }
4     public final int size() { ... }
5
6     public final T [] toArray( final T [] destination )
7     {
8         final var retValue = destination.length >= size()
9             ? destination
10            : (T []) Array.newInstance(
11                destination.getClass().getComponentType(), size() );
12         var index = 0;
13         for( var iterator = iterator(); iterator.hasNext; ++index )
14         {
15             retValue [index] = iterator.next();
16         }
17
18         //---* Done *-----
19         return retValue;
20     } // toArray()
21 }
22 // class RingBuffer

```

In line 10, we use Reflection to ...

- ... get the component type of the `destination` argument by calling the method `Class::getComponentType`[257]
- ... create the array of that type through a call to `Array::newInstance`[266]

We know that `destination` is an array, because we declared it that way. So we know also that `Class::getComponentType` will not return `null`. And we know that this new array will be of the right type, for the same reason, therefore the cast will be successful. This implementation of `toArray()` will not work without the use of Reflection.

But why do we need the `destination` parameter? Why do not implement it this way:

```

1 public final class RingBuffer<T>
2 {

```



```
3 public final Iterator<T> iterator() { ... }
4 public final int size() { ... }
5
6 // Will compile, but sometimes, it does not work ...
7 public final T [] toArray()
8 {
9     final T [] retValue;
10    var index = 0;
11    for( var iterator = iterator(); iterator.hasNext; ++index )
12    {
13        final var entry = iterator.next();
14        if( index == 0 )
15        {
16            retValue = (T []) Array.newInstance( entry.getClass(),
17                                                size() );
18        }
19        retValue [index] = entry;
20    }
21    //---* Done *-----
22    return retValue;
23 } // toArray()
24 }
25 // class RingBuffer
```

It would work fine for `RingBuffer<String>`, but for `RingBuffer<CharSequence>` it will crash – because the call to `entry.getClass()` in line 16 will never return `java.lang.CharSequence`^[183], therefore `Array.newInstance()` will not return an array of type `T` (equals to `java.lang.CharSequence` in this case) and the cast will fail – at runtime! This sample shows again how dangerous the use of Reflection can be.

Refer also to the implementations of `Collection::toArray` here: ^[289, 288, 287].

8.11.3. Using Reflection to write Unit Tests

Reflection allows you to access non-`public` members of a class that are usually not accessible by your code. This is considered a “dirty hack” and must



not be used in production code. In particular as it may not work properly with modules.

But there are useful applications for this, too: you can and should use it when writing unit tests for **private** or **protected** methods that are not made **public** for good reasons, and if these methods cannot be tested indirectly.⁷³

I recommend to use the following pattern if you want to access a **private** method in your unit tests; first the class under test (**MyClass**):

```
public final class MyClass
{
    /**
     * Does something.
     *
     * @param value The value.
     * @return The result.
     * @throws IOException Something went wrong.
     */
    private final String myMethod( final CharSequence value ) throws
        IOException
    {
        final var retValue = ...

        //---* Done *-----
        return retValue;
    } // myMethod()
}
// class MyClass
```

The test class:

```
1 /**
2  * This class provides some unit tests for
3  * {@link MyClass}.
4  */
5 public final class TestMyClass
6 {
7     /*-----*\
8     ===** Static Initialisations **=====
9     \*-----*/
```

⁷³The alternative would be to make the method at least **protected**, but this would make it (more) visible to the consumers of the API, and perhaps even accessible – but as said, there was a reason why that method was originally **private**.



8. Coding Guidelines

```
10  /**
11   * The reference to {@code myMethod()}.
12   */
13  private static final Method METHOD_myMethod;
14
15  static
16  {
17      final var targetClass = MyClass.class;
18      String methodName;
19      try
20      {
21          methodName = "myMethod";
22          METHOD_myMethod = targetClass.getDeclaredMethod( methodName,
23                                                         CharSequence.class );
24          METHOD_myMethod.setAccessible( true );
25      }
26      catch( final NoSuchMethodException ignored )
27      {
28          throw new ExceptionInInitializerError( "Cannot find method
29                                                  '%s()' in class '%s'".formatted( methodName,
30                                                  targetClass.getName() ) );
31      }
32  }
33
34  /**
35   * -----*|
36   * ==** Methods **=====
37   * |*-----*/
38   /**
39   * Calls
40   * {@link MyClass#myMethod(CharSequence)}
41   *
42   * @param instance The candidate.
43   * @param value The value.
44   * @return The result.
45   * @throws IOException Something went wrong.
46   */
47  protected static final String myMethod( final MyClass instance, final
48                                           CharSequence value )
49  {
50      final String retValue;
51      try
52      {
53          retValue = (String) METHOD_myMethod.invoke(
54              requireNonNullArgument( instance, "instance" ), value );
55      }
56  }
```



```

50     catch( final IllegalAccessException | ClassCastException e )
51     {
52         throw new AssertionError( e );
53     }
54     catch( final InvocationTargetException e )
55     {
56         switch( e.getCause() )
57         {
58             case null -> throw new AssertionError( e );
59             case IOException ioe -> throw ioe;
60             default -> throw new AssertionError( e.getCause() );
61         }
62     }
63 } // myMethod()
64
65 @Test
66 final void testMyMethod() throws Exception
67 {
68     final var candidate = new MyClass();
69     final var result = myMethod( candidate, "" );
70     assertTrue( result instanceof String );
71 } // testMyMethod()
72 }
73 // class TestMyClass

```

This allows you to call the method `MyClass::myMethod` nearly directly; the method `TestMyClass::myMethod` behaves in the same way as the original method.

`java.lang.AssertionError`[44] is the base class for the errors thrown by JUnit[234].

8.11.4. Annotations and Reflection

Annotations with their retention policy[123, 180] set to `RUNTIME` “[...] are to be recorded in the class file by the compiler and retained by the VM at run time, so they may be read reflectively.”

To check whether the class of the given object was annotated with a particular annotation, you will use code like that below:



```
public final <T extends Annotation> Optional<T> retrieveAnnotation( final
    Object object, final Class<T> annotationClass )
{
    final AnnotatedElement annotatedElement = requireNonNullArgument(
        object, "object" ).getClass();
    final var retValue = retrieveAnnotation( annotatedElement,
        annotationClass );

    //---* Done *-----
    return retValue;
} // retrieveAnnotation()

public final <T extends Annotation> Optional<T> retrieveAnnotation( final
    AnnotatedElement annotatedElement, final Class<T> annotationClass )
{
    final Optional<T> retValue = Optional.ofNullable(
        requireNonNullArgument( annotatedElement, "annotatedElement"
        ).getAnnotation( requireNonNullArgument( annotationClass,
        "annotationClass" ) ) );

    //---* Done *-----
    return retValue;
} // retrieveAnnotation()
```

On the returned object instance you can call the ‘methods’ that had been defined for the given annotation, to get the values that were set with the annotation.

This is used by various popular libraries and frameworks:

- For XML parsing: JAXB and its predecessors[317, 216]
- For JSON parsing: GSON[168] and Jackson[214]
- For WebServices: JAX-WS[231, 217]
- For ORM: JPA[226, 215] and Hibernate[171]
- For command line handling: ARGS4J[236, 235]

In fact, these frameworks/libraries are only possible because annotations had been introduced with Java 5: the annotations allow to apply meta information to classes above the information provided by the class definition itself.



All the above listed projects will somehow map external data to a Java class (*unmarshalling*), or from a Java class into an external format (*marshalling*), and annotations provide column names or field names.

When your project does not focus on another tool in this category, plain attributes are a better choice in most cases.

And if you have to provide a mapper from an internal data exchange format (that is not XML or JSON ...), you should consider to write an annotation processor (AP) – in particular if you are in control of the full build process for the project that is intended to use that mapper.

8.12. Implementing the Methods from 'java.lang.Object'

In Java, all *classes*⁷⁴ are somehow extending the class `java.lang.Object`[59], and therefore, they inherit several methods from it. Four of these methods⁷⁵ can be overridden to adjust the behaviour of your class to your needs:

- `clone()`[261]
- `equals()`[262]
- `hashCode()`[264]
- `toString()`[265]

The chapters below will provide some guidelines on how to code new implementations for these methods.

8.12.1. `equals()` and `hashCode()`

Overriding the method `java.lang.Object::equals`[262] always requires to override the method `java.lang.Object::hashCode`[264], too – and vice versa.

⁷⁴This includes *records* and *enums*, but excludes *interfaces* and *annotations*.

⁷⁵In fact, it is five methods, but the method `finalize()`[263] is deprecated and should not be used anymore. Refer to chapter “8.49 Finalisation” on page 426 for more details on this topic.



This is not optional!

The method `equals()` returns `true` if the given reference refers to an object that is equal to the current one, according to *your criteria* what “being equal” means in this context, and – obviously – `false` otherwise.

So two objects can be considered to be equal when they both have the same unique id, or you require for equality that all attributes do have the same values (are also equal), or something in between. The default implementation of `java.lang.Object.equals()` returns `true` only if the two objects are identical⁷⁶.

If two objects are equal, the result of `hashCode()` has to be the same for both objects, but that the hash values for two objects are the same does not necessarily imply that these two objects are equal.⁷⁷

An implementation for the two methods should look like this:

Listing 8.14: Methods `equals()` and `hashCode()`

```
1 public class MyClass
2 {
3     /**
4      * {@inheritDoc}
5      */
6     public boolean equals( final Object o )
7     {
8         var retValue = o == this;
9         if( !retValue && o instanceof MyClass other
10            && getClass().equals( other.getClass() ) )
11         {
12             retValue = Objects.equals( <attribute>, other.<attribute> )
13                && Objects.equals( <...>, other.<...> );
14         }
15
16         //---* Done *-----
17         return retValue;
18     } // equals()
19 }
```

⁷⁶This means that both objects are the *same*; the given reference points to the current object itself.

⁷⁷Mainly this is the reason why you always have to provide implementations for both methods.



```

20  /**
21   * {@inheritDoc}
22   */
23  public int hashCode()
24  {
25      final var retValue = Objects.hash( <attribute>, <...> );
26
27      //---* Done *-----
28      return retValue;
29  } // hashCode()
30 }
31 // class MyClass

```

The check in line 10 can be omitted if **MyClass** is **final**. If that check is omitted for a non-**final** class, it means that instances of derived classes can be equal to an instance of the superclass – something that is rarely wanted, especially because it would break the rule that any implementation of **equals()** has to guarantee that

```
a.equals( b ) == b.equals( a )
```

is always valid.

The attributes that are compared in the lines 12 and following have all to be used in **hashCode()** to calculate the hash value.

If **java.lang.Object::hashCode** is implemented by both a superclass and its derived classes, the implementation of the derived class *may* call the superclass implementation of **hashCode()**:

```

1  public class OtherClass extends MyClass
2  {
3      /**
4       * {@inheritDoc}
5       */
6      public int hashCode()
7      {
8          final var retValue = Objects.hash( Integer.valueOf( super.hashCode
9              ), <attribute>, <...> );
10
11          //---* Done *-----
12          return retValue;
13      } // hashCode()

```



```
13 }  
14 // class OtherClass
```

Obviously, `OtherClass.hashCode()` considers only the attributes that comes with the definition of the derived class; the attributes of the superclass are already covered.

This does not work that well with `equals()`.

8.12.2. `toString()`

According to [265], the method `toString()`

“Returns a string representation of the object.

In general, the `toString()` method returns a string that ‘textually represents’ this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method. [...]”

What does “textually represents” mean?

The implementation for `java.lang.Object::toString` ...

“[...] returns a string consisting of the name of the class of which the object is an instance, the at-sign character ‘@’, and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of: `getClass().getName() + '@' + Integer.toHexString( hashCode() )`”

But for an instance of `java.lang.Integer`[54], that ‘string representation’ is just a string containing the digits for the numerical value of that object, and for an instance of `java.lang.StringBuilder`[68], it is the current contents of the buffer.

For the class `java.util.StringJoiner`[101], `toString()` is even the method that provides the result.

Originally, the textual representation of an object as provided by the `toString()` method was meant only for debugging purposes, but soon it was also used



for the conversion of the object's value to a string, as you can see for classes like `java.lang.Integer`, `java.util.UUID` or `java.time.Instant`.

So how to implement the method `toString()` for your class?

If your class represents objects that can be easily written as a string, you should implement `toString()` accordingly:

```
public final class PhoneNumber
{
    private final int m_AreaCode;
    private final int m_CountryCode;
    private final int m_SubscriberNumber;

    ...

    /**
     * {@inheritDoc}
     */
    @Override
    public final String toString()
    {
        final var retValue = "+%d_%d_%d".formatted( m_CountryCode,
            m_AreaCode, m_SubscriberNumber );

        //---* Done *-----
        return retValue;
    } // toString()
}
// class PhoneNumber
```

Some samples for this from the Java Runtime Library are the classes below:

- `java.lang.StringBuilder`[68]
- `java.lang.StringBuffer`[67]
- `java.util.UUID`[104]
- `java.time.Instant`[82] and the other classes representing time/date values from the `java.time` package[318]
- `java.io.File`[38]



- `java.lang.Integer`[54] and the other wrapper classes for the primitives

In all these cases, you can use a call to `toString()` to embed the value of the instance into a regular text:

```
final var phoneNumber = new PhoneNumber( ... );
out.printf( "The_customer's_phonenumber_is_%s.", phoneNumber.toString() );
```

If your class is more complex and/or an output makes only sense for debugging purposes or requires additional formatting instructions, you should consider a different implementation of `toString()`:

```
public final class EmailMessage
{
    private final String m_Body;
    private final Map<RecipientType,List<EmailAddress>> m_Recipients;
    private final EmailAddress m_Sender;
    private final ZonedDateTime m_SentWhen;
    private final String m_Subject;

    ...

    /**
     * {@inheritDoc}
     */
    @Override
    public final String toString()
    {
        final var buffer = new StringJoiner( ",", "%s[" .formatted(
            getClass().getName(), "]" )
            .add( "Body='%s'".formatted( m_Body ) )
            .add( "Recipients=%s".formatted( Objects.toString(
                m_Recipients ) ) )
            .add( "Sender='%s'".formatted( Objects.toString(
                m_EmailAddress ) ) )
            .add( "Sent_when=%s".formatted( Objects.toString( m_SentWhen )
                ) )
            .add( "Subject='%s'".formatted( m_Subject ) ) );

        /*---* Compose the return value *-----
        final var retValue = buffer.toString();

        /*---* Done *-----
        return retValue;
    } // toString()
```



```
}  
// class EmailMessage
```

This is basically how the IDEs will generate the `toString()` method. The output may look like this⁷⁸:

```
org.tquadrat.sample.EmailMessage[Body='This is the body of the\  
email', Recipients=[a.b@c.de], Sender='thomas.thrien@tquadrat.\  
org', Sent_when=2022-11-26T20:31:17.884636950+01:00[Europe/Ber\  
lin], Subject='Ping!']
```

This will work fine for a debug log, but to get it 'pretty printed', you may have to provide another method. Refer to chapter 8.38 on page 422 on how this could look like.

If your class is not `final`, the method `toString()` should not be `final` as well.

8.12.3. clone()

The JavaDoc for `java.lang.Object::clone`[261] says:

"Creates and returns a copy of this object. The precise meaning of 'copy' may depend on the class of the object. The general intent is that, for any object `x`, the expression:

```
x.clone() != x
```

will be `true`, and that the expression:

```
x.clone().getClass() == x.getClass()
```

will be `true`, but these are not absolute requirements. While it is typically the case that:

```
x.clone().equals(x)
```

⁷⁸The backslash indicates where I inserted a linebreak so that it looks fine in this document; otherwise it would be just one long line.



will be `true`, this is not an absolute requirement.

By convention, the returned object should be obtained by calling `super::clone`. If a class and all of its superclasses (except `java.lang.Object`) obey this convention, it will be the case that

```
x.clone().getClass() == x.getClass()
```

is `true`.

By convention, the object returned by this method should be independent of this object (which is being cloned). To achieve this independence, it may be necessary to modify one or more fields of the object returned by `super::clone` before returning it. Typically, this means copying any mutable objects that comprise the internal ‘deep structure’ of the object being cloned and replacing the references to these objects with references to the copies. If a class contains only primitive fields or references to immutable objects, then it is usually the case that no fields in the object returned by `super::clone` need to be modified.”

When instances of your class should be cloneable, it first has to implement the interface `java.lang.Cloneable`[184], and then you need to override the method `clone()`; usually, this method is `protected` and it declares to throw a `java.lang.CloneNotSupportedException`[47]. The default implementation does nothing else than throwing that exception when called.

A simple implementation may look like this:

Listing 8.15: A simple `clone()` Method

```
1 public final MyClass implements Cloneable
2 {
3     /**
4      * {@inheritDoc}
5      */
6     @Override
7     public final MyClass clone()
8     {
9         final MyClass retValue;
10        try
11        {
```



```
12         retVal = (MyClass) super.clone();
13     }
14     catch( final CloneNotSupportedException e )
15     {
16         throw new UnexpectedExceptionError( e );
17     }
18
19     //---* Done *-----
20     return retVal;
21 } // clone()
22 }
23 // class MyClass
```

This works despite the fact that the class `java.lang.Object` itself *does not* implement `java.lang.Cloneable`!

But it may fail in case your class extends a class other than `java.lang.Object` that does not implement `java.lang.Cloneable`.

And because we know that `super.clone()` will work in this case, our implementation does not declare to throw `java.lang.CloneNotSupportedException`; instead it will swallow it⁷⁹.

The implementation of `java.lang.Object::clone` (that one we have called with `super.clone()`) uses native code to make a shallow copy of the current object. If all attributes of the class are either primitives or immutable, the implementation shown above is sufficient, also for properly implemented immutable types (refer to chapter “8.7 Implementing Immutable Types” on page 311 that elaborates on this topic).

But if any of the attributes are collections, arrays or generally mutable types, there is some more work to do; in this case, your implementation of `clone()` should declare the `java.lang.CloneNotSupportedException`.

Such an extended implementation of `clone()` may look like below; sometimes this pattern is referred to as ‘Deep Cloning’. It is assumed that the class `T` is *not* immutable but implements `java.lang.Cloneable`. If `T` will not implement

⁷⁹But to protect us from problems that may come our way when someone decides to extend another class than `java.lang.Object`, we wrap that `CloneNotSupportedException` into an `UnexpectedExceptionError`[350].



8. Coding Guidelines

Cloneable, we could reduce the implementation of `clone()` for this class to just

```
@Override
public final MyClass clone() throws CloneNotSupportedException
{
    throw new CloneNotSupportedException();
} // clone()
```

But even if `T` implements `java.lang.Cloneable`, the method `T::clone` can still throw an `CloneNotSupportedException`, so our implementation has to declare to throw that exception.⁸⁰

```
1 public final MyClass implements Cloneable
2 {
3     // NONE OF THE ATTRIBUTES MAY BE FINAL!!
4     private T [] m_Array;
5     private List<T> m_List;
6     private T m_Mutable;
7
8     /**
9      * {@inheritDoc}
10    */
11    @Override
12    public final MyClass clone() throws CloneNotSupportedException
13    {
14        final var retValue = (MyClass) super.clone();
15
16        if( nonNull( m_Array ) )
17        {
18            retValue.m_Array = m_Array.clone();
19            for( var i = 0; i < m_Array.size; ++i )
20            {
21                retValue.m_Array [i] = nonNull( m_Array [i] )
22                    ? m_Array [i].clone()
23                    : null;
24            }
25        }
26
27        if( nonNull( m_List ) )
28        {
29            retValue.m_List = new ArrayList<>();
```

⁸⁰You may have noticed that I did not declare `MyClass` as generic (like “`MyClass<T extends Cloneable>`”). I will come back to that later in this chapter.



```

30         for( final var t : m_List )
31         {
32             retValue.m_List.add( nonNull( t ) ? t.clone() : null );
33         }
34     }
35
36     retValue.m_Mutable = nonNull( m_Mutable )
37         ? m_Mutable.clone()
38         : null;
39
40     //---* Done *-----
41     return retValue;
42 } // clone()
43 }
44 // class MyClass

```

If one of the attributes that are requiring 'special treatment' is **final**, this implementation does not work any longer! In that case, you need to implement a "copy constructor":

```

public final MyClass implements Cloneable
{
    private final T [] m_Array;
    private final List<T> m_List;
    private final Map<String,T> m_Map = new HashMap();
    private final T m_Mutable;

    /**
     * Creates a new instance of {@code MyClass}.
     */
    public MyClass( ... )
    {
        // The regular constructor.
        ...
    } // MyClass()

    /**
     * Creates a new instance of {@code MyClass} from the provided
     * other instance.
     *
     * @note Used by the implementation of {@code clone()}.
     *
     * @param other The other instance.
     * @throws CloneNotSupportedException One of the attributes do

```



8. Coding Guidelines

```

    *      not allow to be cloned.
    */
private MyClass( final MyClass other ) throws
    CloneNotSupportedException
{
    if( nonNull( other.m_Array ) )
    {
        m_Array = other.m_Array.clone();
        for( var i = 0; i < other.m_Array.size; ++i )
        {
            m_Array [i] = nonNull( other.m_Array [i] )
                ? m_Array [i].clone()
                : null;
        }
    }

    if( nonNull( other.m_List ) )
    {
        m_List = new ArrayList<>();
        for( final var t : other.m_List )
        {
            m_List.add( nonNull( t ) ? t.clone() : null );
        }
    }

    for( final var entry : other.m_Map.entrySet() )
    {
        m_Map.put( entry.getKey(), entry.getValue.clone() );
    }

    m_Mutable = nonNull( other.m_Mutable )
        ? other.m_Mutable.clone()
        : null;
} // MyClass()

/**
 * {@inheritDoc}
 */
@Override
public final MyClass clone() throws CloneNotSupportedException
{
    final var retValue = new MyClass( this );

    //---* Done *-----

```




```
        return retValue;
    } // clone()
}
// class MyClass
```

Such a copy constructor may be required for all super classes of your **MyClass**, too.

Unfortunately, the interface **java.lang.Cloneable** does not declare the method **clone()** and **java.lang.Object::clone** is **protected**, therefore the code below will not work – it will not even compile:

```
// WILL NOT COMPILE!!
public static final <T extends Cloneable> List<T> deepCloneList( final
    List<T> list ) throws CloneNotSupportedException
{
    final var retValue = new ArrayList<T>();
    for( final var t : requireNonNullArgument( list, "list" )
    {
        if( isNull( t ) )
        {
            retValue.add( null );
        }
        else
        {
            // Cloneable does not declare clone()!
            retValue.add( t.clone() );
        }
    }

    //---* Done *-----
    return retValue;
} // deepCloneList()
```

A method that would allow to clone an arbitrary instance of a class that supports cloning must look like this:⁸¹

```
public static final <T extends Cloneable> T cloneArbitrary( final T
    instance ) throws CloneNotSupportedException, NoSuchMethodException
{
    final var cloneMethod = requireNonNullArgument( instance, "instance" )
        .getMethod( "clone" );
```

⁸¹This is one of the rare cases where you cannot avoid the use of Reflection.



```
final T retValue;
try
{
    retValue = (T) cloneMethod.invoke( instance );
}
catch( final IllegalAccessException | IllegalArgumentException e )
{
    throw new UnexpectedExceptionError( e );
}
catch( final InvocationTargetException e )
{
    final var cause = e.getCause();
    switch( cause )
    {
        case null ->
        {
            final var cnse = new CloneNotSupportedException( "clone()_
                caused_Exception" );
            cnse.initCause( e );
            throw cnse;
        }

        case CloneNotSupportedException cnse -> throw cnse;

        case RuntimeException rte -> throw rte;

        case Error error -> throw error;

        default -> throw new UnexpectedExceptionError( cause );
    }
}

//---* Done *-----
return retValue;
} // cloneArbitrary()
```

There are several ongoing discussions whether the API that was defined through the `java.lang.Object::clone` is generally usefull or more a pain in the ass, and as far as I am aware, these discussions will last for some more time. But I assume that after the explanations above, you got an idea why this discussions arose.

I am not a friend of `clone()` and I try to avoid its implementation whenever



possible. Instead I provide a `public` copy constructor or a parameterless `copy()` method if I need to 'clone' an object.

Therefore my recommendation is to ignore the method `java.lang.Object::clone` unless there is a strict requirement to use it.

8.13. String Concatenation

How to concatenate strings has been a topic of discussion since the very beginning of Java. And the truth has changed with nearly each version of the language, not making it easier to decide how it is done correctly. This chapter provides some recommendations and best practices for the current versions of Java (Java 17 and later).

Principles

P1. The code that builds the resulting string should be readable.

Usually, a string concatenation is quite simple and straight forward. If you need to read it twice to understand it, it is most probably too complex.

P2. Spend *some* thoughts on the efficiency of your implementation.

In most cases, you will immediately come to the conclusion that the string concatenation will be fast enough. If you have doubts, think again – but the saying about “premature optimisations”^[237] should not be forgotten.

8.13.1. The Basics

The implementation of the concatenation of two (or more) strings – or other data types to create their representation as a text – is a trade off between readability and performance. This is due to one important characteristic of the class `java.lang.String`^[66] it is immutable. This means that



8. Coding Guidelines

```
String a = "part1";
String b = "part2";
a += b;
```

will not modify **a** but returns a new **String** object with the concatenated contents of **a** and **b** and assigns a reference to that new object to **a**.

Older sources described the internal implementation of the **+** operator for **java.lang.String** like this: `indexjava.lang!String`

```
// NOT THE REAL IMPLEMENTATION!!
private final String operatorPlus( String a, String b )
{
    final var buffer = new StringBuffer( a )
        .append( b );
    final var retValue = buffer.toString();

    //---* Done *-----
    return retValue;
} // operatorPlus()
```

This means that an intermediate object of type **java.lang.StringBuffer**^[67]`indexjava.lang!` would be created for each concatenation. This gets even worse if you want to append a numerical value to the **String**, like this:

```
String a = "part1";
a += 42;
```

The implementation for this was described as `indexjava.lang!String`

```
// NOT THE REAL IMPLEMENTATION!!
private final String operatorPlus( String a, int b )
{
    StringBuffer buffer = new StringBuffer( a );
    String bString = Integer.toString( b );
    buffer.append( bString );
    final var retValue = buffer.toString();

    //---* Done *-----
    return retValue;
} // operatorPlus()
```



meaning that two intermediate objects are created.

Knowing this, the recommendation was always to write

```
String a = new StringBuffer( b )  
    .append( c )  
    .toString();
```

instead of

```
String a = b + c;
```

and

```
String a = new StringBuffer( a )  
    .append( b )  
    .toString();
```

instead of

```
a += b;
```

in order to increase performance.⁸²

But with each Java version the *real* implementation of + and += for **String** changed, so that today there is no longer just only one recommendation.

8.13.2. Concatenating String Constants

In your code, string literals will be always concatenated with the plus operator:

```
String a = "StringOne" + "StringTwo";
```

⁸²These recommendations origin from a time when the class **java.lang.StringBuilder**^[68] did not yet exist. Java 5 introduced **StringBuilder** as the successor/replacement for **StringBuffer**; it is more performant than **StringBuffer** because its operations are not synchronised and therefore have less overhead than that of **StringBuffer**.



because this way, they will already be concatenated *during compile time*; using `StringBuilder` here would cause negative effects on both performance and readability. This is also true when `static final String` variables, initialised with a literal, are concatenated with each other or with another string literal:

```
public static final String CONSTANT_A = "StringOne";
public static final String CONSTANT_B = "StringTwo";
...
final var a = CONSTANT_A + CONSTANT_B;
String b = CONSTANT_A + "StringThree";
```

The compiler replaces each reference to the `static final String` variables by either the literal itself or a reference to the literal and concatenates them if required.

8.13.3. Concatenating String Variables

Benchmark tests showed that beginning with one of the later versions of Java 1.4 the variant

```
String a = "part1";
String b = "part2";
String s = a + b;
```

is significantly faster than

```
String a = "part1";
String b = "part2";
String s = new StringBuffer( a )
    .append( b );
```

Even using `StringBuilder` in Java 5 instead of `StringBuffer` is slower than the `+` operator.

Appending non-string values to a `String` can be done as

```
String a = "part1";
int b = 42;
String s = a + Integer.toString( b );
```



and that is still being faster than the `StringBu*er` versions.

8.13.4. Concatenating Strings in Loops

The picture changes if strings are extended permanently in a loop:

```
// AVOID!!!
public final String buildSentence( String... words )
{
    var retValue = "";
    for( final var s : words )
    {
        if( !retValue.isEmpty() ) retValue += "_";
        retValue += s;
    }
    retValue += ".";

    //---* Done *-----
    return retValue;
} // buildSentence()
```

Here it is the better option to use `StringBuilder` or even `java.util.StringJoiner`^[101]:

```
// BETTER
public final String buildSentence( String... words )
{
    final var buffer = new StringBuilder()
    for( final var s : words )
    {
        if( buffer.length() > 0 ) buffer.append( "_" );
        buffer.append( s );
    }
    buffer.append( "." );
    final var retValue = buffer.toString();

    //---* Done *-----
    return retValue;
} // buildSentence()

// RECOMMENDED
public final String buildSentence( String... words )
{
```



8. Coding Guidelines

```
final var buffer = new StringJoiner( "_", "", "." );
for( final var s : words )
{
    buffer.add( s );
}
final var retValue = buffer.toString();

//---* Done *-----
return retValue;
} // buildSentence()
```

Although the concatenation with `+` is usually faster than using `StringBuilder`, this is faster than the first version, because the shown version will definitely create much less objects that have to be garbage collected later – what will have a negative impact on performance. And the implementation with `StringJoiner` is even more concise than both others.

8.13.5. Composing Structured Strings

Quite often you see code like this

```
final var message = "The_requested_seat_" + number + "_is_unavailable_"
    + "because_" + reason;
```

An even more ugly version is this

```
final var xmlMessage = "<note>\n" +
    "    <script/>\n" +
    "    <to>" + recipient + "</to>\n" +
    "    <from>" + sender + "</from>\n" +
    "    <heading>" + subject + "</heading>\n" +
    "    <body>" + body + "</body>\n" +
    "</note>";

// It could be even worse:
final var xmlMessage = "<note>\n" +
    "    <script/>\n" +
    "    <to>" + recipient + "</to>\n" +
    "    <from>" + sender + "</from>\n" +
    "    <heading>" + subject + "</heading>\n" +
    "    <body>" + body + "</body>\n" +
    "</note>";
```




Obviously performance is not the issue here, but readability. And, for the first case, that the message cannot be identified for a translation into another language, if required (see the chapter 8.18 on page 399 about internationalisation).

For these samples, you should consider to utilize the capabilities of the class `java.util.Formatter`[89]. Usually, you do not need to instantiate an instance of `Formatter` yourself, instead you can use the methods `String::format`[268], `String::formatted`[269], `PrintWriter::printf`[254] or `PrintStream::printf`[252]⁸³.

To improve the readability of the second sample, you should also consider to use the new ‘text block’ feature[149, 239] that was introduced with Java 15.

So the sample code from the beginning of this chapter should look like this:

```
final var message = String.format( "The_requested_seat_%d_is_unavailable_
    because_%s", number, reason );

final var xmlMessage =
    """
    <note>
    <<<<<script/>
    <to>%1$s</to>
    <from>%2$s</from>
    <heading>%3$s</heading>
    <body>%4$s</body>
    </note>"""
    .formatted( recipient, sender, subject, body );
```

That’s perhaps not faster, but definitely much better readable.

8.13.6. Conclusion

The recommendation is to use the `+` operator for strings where to combine literals, `String` constants and/or `String` variables in a single, standalone

⁸³`PrintWriter::format`[253] and `PrintStream::format`[251] do exist, too, but are less often used; basically, they to the same as `printf()`.



statement, but to consider **StringBuilder** or even **StringJoiner** if strings have to be concatenated in loops or a (large) number of consecutive statements.⁸⁴

When “adding” primitives to a string, these should be translated to a **String** first by calling the **static toString()** method of the appropriate wrapper class. This is not necessary if calling **StringBuilder::append** as this exists as specialised versions for each primitive type.

Also when “adding” an instance of another type to a string, you should consider to first call **toString()** on that object. This is not mandatory, as it is done implicitly anyway, but it clearly shows your intent.

java.lang.StringBuilder should always be preferred over **java.lang.StringBuffer**. I have not yet found any use case where I could not use **StringBuilder**, but was forced to use **StringBuffer** instead.

For multi-line text in strings, use a text block instead of concatenation, and when composing a structured string, use **format()**.

8.14. try-with-resources

The feature **try-with-resources**^[141] was introduced with Java 7; it can help to make programs more stable and less error prone.

Principles

P1. (External) resources should be released where they had been acquired.

Java and the JVM allows you to be lazy in this regard, as there are mechanisms in place that do the housekeeping once an object representing an external resource (a file, a database connection, a network socket, ...) gets garbage collected. The resource will be freed in the end - latest when the JVM goes down. To this topic, refer also to the chapter “8.49 Finalisation” on page 426.

⁸⁴But keep [237] in mind, where Donald E. Knuth said something about “premature optimisation”.



But releasing a resource immediately after it is no longer used will increase the liveliness of a program dramatically, and in case of a lock, it is even crucial.

P2. Code should be made stable against slips.

To often closing of files or releasing locks will be forgotten, the respective code will be removed erroneously or – at worst case – new code will be added that prevents the returning of the external resource.

try-with-resources can help with this.

8.14.1. Basics

Basically, try-with-resources is an extension of the previously existing try-catch-finally^[140] feature.

Instead of writing

```
1  InputStream input = null;
2  try
3  {
4      input = new FileInputStream( file );
5      ...
6  }
7  catch( final IOException e )
8  {
9      // Handle the error
10 }
11 finally
12 {
13     try
14     {
15         if( input != null ) input.close();
16     }
17     catch( final IOException e )
18     {
19         // Handle the error
20     }
21 }
```

the new feature allows you to write



```
1 try( final var input = new FileInputStream( file ); )
2 {
3     ...
4 }
5 catch( final IOException e )
6 {
7     // Handle the error
8 }
```

It works because the interface `java.lang.AutoCloseable` is implemented by the class `java.io.InputStream`^[41]⁸⁵. For more details refer to [182].

The interface `java.lang.AutoCloseable` defines just one method, `close()`, that declares to throw an exception of type `java.lang.Exception`.

`close()` is called automatically on all instances of `AutoCloseable` that were declared and defined in the 'arguments list' of the new `try` when the scope of the `try` block is left. If there is more than one resource defined, the sequence is reversed to that of the definition: the last assigned resource will be closed first.

So a code snippet to copy data from an input stream to an output stream may look like below; this is obviously not a very good implementation, but it illustrates quite well how to use try-with-resources:

```
1 try
2 (
3     final var input = new FileInputStream( infile );
4     final var output = new FileOutputStream( outfile )
5 )
6 {
7     var value = EOF;
8
9     //---* Read the input, write to the output *-----
10    while( (value = input.read()) != EOF )
11    {
12        output.write( value );
13    }
14 }
15 catch( final IOException e )
```

⁸⁵In fact, `InputStream` still implements just `java.io.Closeable`^[178], as already before Java 7, but this interface will now extend the new interface `java.lang.AutoCloseable`.



```
16 {  
17     throw new Error( "Copy_failed!", e );  
18 }
```

Both streams will be closed properly in case of a problem or the work is done.

8.14.2. Error Handling

What will happen if the code in the `try` block throws an exception and closing the resource will throw one, too? Remember, `AutoCloseable::close`[255] declares to throw **Exception**!

For the ‘traditional’ pattern (using `try-catch-finally` as shown in the first example of the previous chapter) this could mean that the first exception would be ‘supplanted’ by the exception issued by `close()`, called in the `finally` block. For sure, in a `catch` block the original cause could be logged before the code in the `finally` block will be executed, but usually only checked exceptions (and “expected” ones) are covered this way.

Together with `try-with-resources`, a new feature was introduced to the language: the *suppressed* exception. This deals with the problem described above.

So if the `try` block throws an exception (for our example, it would be most probably an `IOException`) and the `AutoCloseable.close()` will fail with an exception, too, the latter one will be added to the first one as a “suppressed exception” by the JVM.

For this purpose, the API of the class `java.lang.Throwable` was extended by the methods `addSuppressed()`[279] and `getSuppressed()`[280].⁸⁶

When using `Throwable.printStackTrace()`[282], an output like that below will be produced:

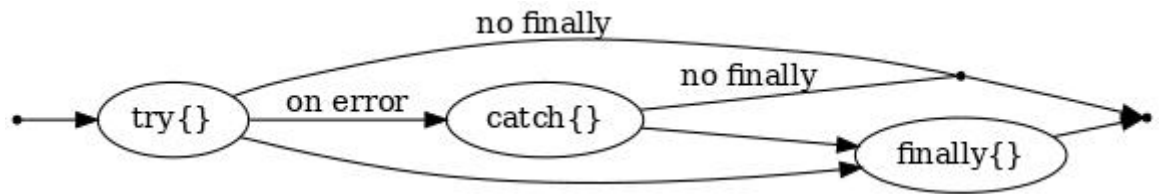
⁸⁶see [71] for more details.



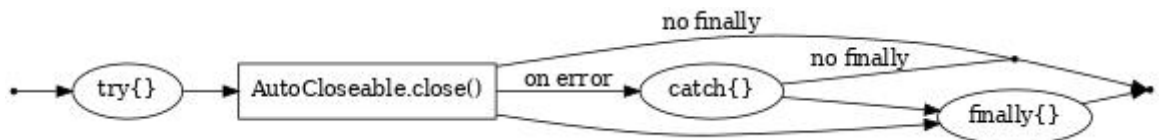
```
java.lang.Error
  at TryWithResources.main(TryWithResources.java:175)
  Suppressed: java.lang.Exception
    at TryWithResources$Resource2.close(TryWithResources.java:103)
    at TryWithResources.main(TryWithResources.java:176)
```

8.14.3. Execution Sequence

It is important to know how the execution sequence looks like when using try-with-resources. For the traditional pattern the execution sequence is



For try-with-resources the execution sequence looks like



This means that the method `close()` on the `AutoCloseable` objects is called *before* any code in an optional `catch` and/or `finally` block that would be attached to the `try` block. For the sample we used above this means that the `InputStream input` is already closed when the code in the `catch` block that handles the `IOException` will be executed.

Therefore the code below will not work as expected in case an exception is thrown in the `try` block:



```
// WILL NOT WORK!
final var logStream = new FileOutputStream( logfile )
try
(
    final var input = new FileInputStream( infile );
    final var output = new FileOutputStream( outfile );
    final var log = logStream
)
{
    var value = EOF;

    /* Read the input, write to the output */
    while( (value = input.read()) != EOF )
    {
        output.write( value );
    }
}
catch( final IOException e )
{
    logStream.write( "Copy_failed!\n".getBytes( UTF8 ) );
    /* Fails because the log file is already closed! */
}
```

Refer also to [142].

8.14.4. When to use?

try-with-resources is a very powerful feature that should be used whenever possible. Definitely it should be used with all the Java classes that already implement `java.lang.AutoCloseable` or `java.io.Closeable`:

- The `java.io` streams
- Sockets (`java.net.Socket`[75] and `java.net.ServerSocket`[74])
- `java.sql.Connection`[191], `java.sql.Statement`[193], `java.sql.ResultSet`[192]
- `jakarta.jms.Connection`[175]



IDEs like Eclipse and IntelliJ IDEA can be configured to issue a warning when an instance of a class that implements **AutoCloseable** or **Closeable** without try-with-resources.

The two following chapters (“Lifecycle” and “Post-Processing”) show some additional ways to make use of try-with-resources.

Lifecycle

In C++, it is a very common pattern to “wrap” the lifecycle of a resource into the lifecycle of an object:

```
class Resource
{
    //---* Attributes *-----
private:
    RTYPE m_Res;

    //---* Constructors *-----
public:
    Resource( RTYPE &r )
        : m_Res( r )
        { m_Res.open(); }

    //---* Destructor *-----
public:
    ~Resource() { m_Res.close(); }

    //---* Methods *-----
    // Some methods to access the resource
    ...
}
```

A use of that class might look like this:

```
...
{
    Resource resource( r );

    // Do something
    ...
}
```




```
...
```

The instance of **Resource** will be constructed and **open()** is called on **r** on the declaration of the variable **resource**. On leaving the scope the destructor of **Resource** is called implicitly and **close()** will be called on **r**.

The C++ STL is using a very similar pattern for smart pointers.

Unfortunately, Java does not know destructors⁸⁷, so this pattern could not be used in Java programs.

A workaround is to use a try-finally block with the cleanup (usually a call to a method **close()**) in the **finally** block. But too often we have seen that in the run of modifications and/or corrections (refactorings) suddenly the **finally** block and/or its contents had been removed (“optimised away”). Another problem were exceptions that are triggered from within a finally block.

Now, with the try-with-resources feature, we can have “Lifecycle” classes; they are still not that easy to use than a C++ class with real destructors, but we can come close.

A sample would be the class **AutoLock**; for the full code, see the chapter “9.6.1 AutoLock” on page 468; **org.tquadrat.foundation.lang.AutoLock** is a real life implementation that can be found at [360].

In programs that use **java.util.concurrent.locks.Lock**[197] or one of its implementations for thread synchronisation, you will find quite often code like this:

```
m_Lock.lock();
try
{
    // Do something
    ...
}
finally
{
```

⁸⁷The deprecated method **java.lang.Object.finalize()**[263] that is part of each Java class is not and was never a replacement for or an alternative to a destructor as it could never be predicted when it is called (just “sometime before the JVM terminates” – if ever).



```
m_Lock.unlock();  
}
```

This calls for a lifecycle class. Unfortunately the code below will not work, due to several reasons:

```
// Does not work!!  
try( final var unused = new Lock() )  
{  
    // Do something  
    ...  
}
```

First, **Lock** will not implement **java.lang.AutoCloseable**, and second – much more important – we cannot create a new instance of **Lock** each time we enter the critical section.⁸⁸

Fortunately, the try-with-resources feature will not call **close()** on the newly created object, but on the local reference to it (that is the reason why try-with-resources will not work with anonymous instances like in **try(new Lock())**). If we would now wrap the **Lock** instance into a class that implements **AutoCloseable**, we can write something like this:

```
1 ...  
2 AutoLock m_AutoLock = AutoLock.of( new ReentrantLock() );  
3 ...  
4 try( final var unused = m_AutoLock.lock() )  
5 {  
6     // Do something  
7     ...  
8 }
```

Post-Processing

Together with lambdas, try-with-resources can be (ab)used also to enforce a unconditional post-processing when a code block is left. This may look like this:

⁸⁸Not to mention that **java.util.concurrent.locks.Lock** is an interface so that **new Lock()** cannot work at all.



```
1 ...
2 Runnable doAfter = ...;
3 ...
4 try( final var p = new PostProcessor( doAfter ) )
5 {
6     // Do whatever necessary
7     ...
8 }
```

The **PostProcessor** instance will call **Runnable::run**[267] in its **close()** method when the **try** block is left. Chapter “9.6.7 PostProcessor” in the Appendices provides the source for the class.

‘Unconditional’ means here that the post-processing will be executed if the block terminates regularly or by a thrown exception. But conditions can be injected into the **Runnable**[188] implementation.

The difference between this approach and simply calling **doAfter.run()** in a **finally** block is that the **close()** method of **PostProcessor** is invoked before any code in a **catch** block (refer to chapter “8.14.3 Execution Sequence”).

The following code snippet could be a real-life example for where this is useful:

```
1 final var builder = new StringBuilder();
2
3 final Runnable addTrailer = () -> builder.append( "}\n" );
4
5 ...
6
7 try( final var p = new PostProcessor( addTrailer ) )
8 {
9     ...
10 }
```

This ensures that the string in **builder** terminates *always* with a closing curly brace followed by a linefeed.

Another sample is this code snippet:

```
1 final List<String> list = new LinkedList();
2
3 final Runnable forceSorting = () -> list.sort();
```



```
4
5 ...
6
7 try( final var p = new PostProcessor( forceSorting ) )
8 {
9     for( final var s : loadStrings() )
10    {
11        list.add( s );
12    }
13 }
```

Here the **PostProcessor** forces that the given list is always sorted after the values had been added.

8.15. Terminating a Method and returning Values

A consistent structure for methods does not only support the readability of the code, it also protects from unwanted side effects of modifications of that code.

Principles

P1. A method has just one single *regular* return point⁸⁹.

This is either *after* the last statement for a **void** method (in PASCAL, such a method was called a 'procedure'), or the very last statement of the method is the **return** statement with the return value (a 'function' in PASCAL).

P2. Additional *irregular* exit points have to be thrown exceptions.

That's the reason why exceptions have been introduced.

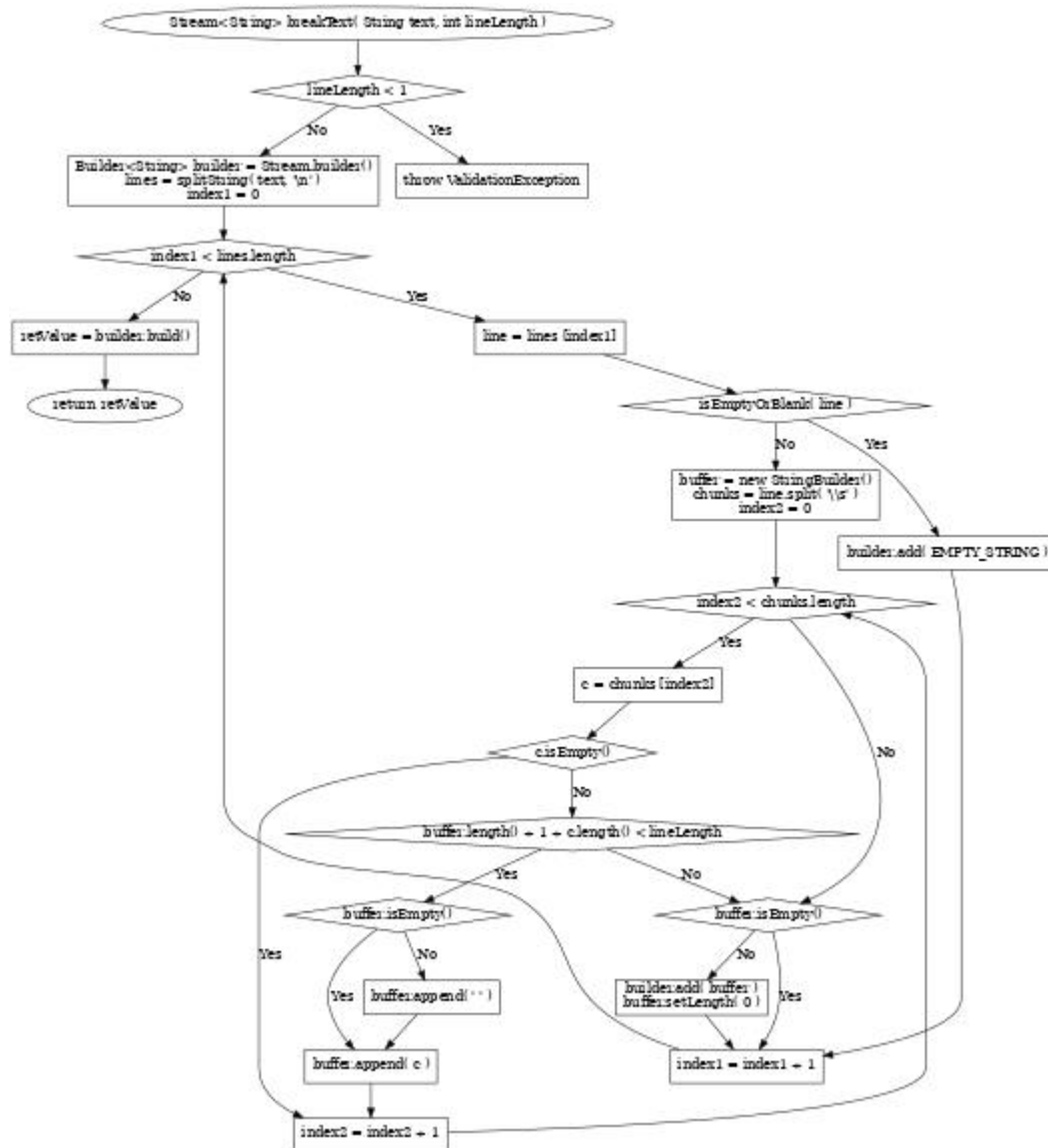
P3. The name of the variable that holds the return value is always **retValue**.

Only when a method (usually a getter or an accesor method) returns just a property value, that property can be used directly.

⁸⁹Sometimes also referred to as "exit point"



In those ancient times when I learned programming, we were forced by our teachers to provide a flow-chart of the code before we really started with coding it. This could look like this:

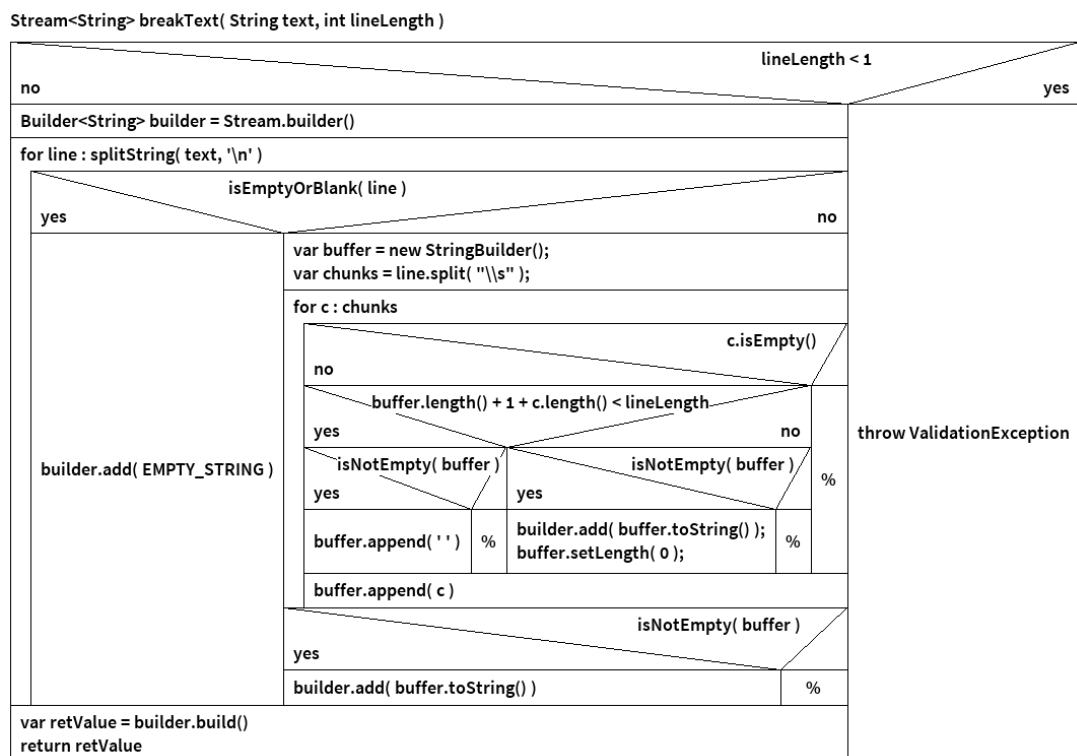




8. Coding Guidelines

This is a relatively simple function (method) that has only one regular exit point, but it is nevertheless quite confusing when presented as a standard flow-chart. In particular it is not easy to follow the program flow, and you cannot see immediately that there is really only one return point in it.

So we soon started to use 'structograms' or Nassi-Shneiderman diagrams[315]. As such a structogram, the same code looks like this:



The biggest advantage of such a 'structogram' is that you can take the code directly out of the diagram, because you have to read it strictly from top to bottom, and hence they have just one exit point⁹⁰ at the bottom. The resulting Java code looks like this:

⁹⁰Exceptions did not yet exist when these diagrams had been invented, but as shown, they can be easily implemented.



Listing 8.16: StringUtils::breakText

```

1 public static final Stream<String> breakText( final CharSequence text,
2       final int lineLength )
3 {
4     if( lineLength < 1 )
5         throw new ValidationException( format( "Line_length_size_must_not_
6             be_zero_or_a_negative_number:%d", lineLength ) );
7
8     final Builder<String> builder = Stream.builder();
9
10    for( final var line : splitString( requireNonNullArgument( text,
11        "text" ), '\n' ) )
12    {
13        if( isEmptyOrBlank( line ) )
14        {
15            builder.add( EMPTY_STRING );
16        }
17        else
18        {
19            final var buffer = new StringBuilder();
20            final var chunks = line.split( "\\s" );
21            SplitLoop: for( final var c : chunks )
22            {
23                if( c.isEmpty() ) continue SplitLoop;
24                if( (buffer.length() + 1 + c.length()) < lineLength )
25                {
26                    if( isEmpty( buffer ) ) buffer.append( ' ' );
27                }
28                else
29                {
30                    if( isEmpty( buffer ) )
31                    {
32                        builder.add( buffer.toString() );
33                        buffer.setLength( 0 );
34                    }
35                    buffer.append( c );
36                }
37            } // SplitLoop:
38            if( isEmpty( buffer ) ) builder.add( buffer.toString() );
39        }
40    }
41
42    final var retValue = builder.build();
43
44    //---* Done *-----

```



8. Coding Guidelines

```
42     return retValue;
43 }    // breakText()
```

It is easy to recognise where the elements of structogram ends up in the code. Therefore it is also quite easy to see in the structogram which impact a modification of the code would have, while the usual flow-chart would not help here.

So thinking how your code would look like as a Nassi-Shneiderman diagram helps to protect it from bugs that are introduced by code changes. One requirement for code that can be represented as a structogram is, that there is just one single regular exit point.⁹¹

A bad sample are the `equals()`[262] methods that are generated by the IDEs:

```
1  \\ DISCOURAGED!!
2  @Override
3  public boolean equals( final Object o )
4  {
5      if( this == o ) return true;
6      if( o == null || getClass() != o.getClass() ) return false;
7
8      final var other = (MyClass) o;
9      return m_Boolean == other.m_Boolean && Objects.equals( m_String,
10         other.m_String );
11 }
```

Obviously you cannot create a structogram for the code above.

The chapter 8.12.1 on page 357 describes how to implement `Object::equals`; according to that description, the code above would look like this:

```
1  @Override
2  public final boolean equals( final Object o )
3  {
4      var retValue = this == o;
5      if( !retValue && o instanceof MyClass other )
6      {
7          retValue = getClass() == other.getClass() )
8      }
```

⁹¹IntelliJ IDEA can be configured to emit an error for methods with more than a configured number (the default is 1) of return points.

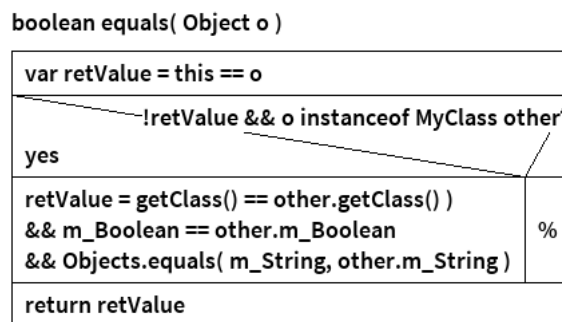


```

8      && m_Boolean == other.m_Boolean
9      && Objects.equals( m_String, other.m_String );
10     }
11
12     //---* Done *-----
13     return retValue;
14 }     // equals()

```

The structogram for this code looks like this:



As already said in chapter “6.8 Local Variables” on page 128, the name of the local variable that holds the return value is always **retValue**; only when the method returns just an attribute, the field name could be used instead. And it is also not necessary to assign **this** to **retValue** for method chaining. Hence these are all valid samples:

```

1 // A Getter method
2 public final String getAttribute() { return m_Attribute; }
3
4 // An accessor method
5 public final String attribute() { return m_Attribute; }
6
7 // Returning 'this' for method chaining
8 public final MyClass attribute( final String value )
9 {
10     m_Attribute = value;
11
12     //---* Done *-----
13     return this;

```



```
14 } // attribute()
15
16 // Any other methods
17 public final String compose( final String pattern )
18 {
19     final var retValue = pattern.formatted( m_Attribute );
20
21     //---* Done *-----
22     return retValue;
23 } // compose()
24
25 public final void print( final PrintStream destination )
26 {
27     destination.print( m_Attribute );
28 } // print()
```

8.15.1. Lambda Results

Most lambdas[146] are just one liners, and if they return something, it will be the return value of that line:

```
n -> (n + 1) * 4; // returns 20 for n=4
```

If the code for the lambda will be more complex, you should consider to place it to a method and refer to it with a method reference[145]:

```
public final class MyClass
{
    /**
     * Formatter function; implements
     * {@link java.util.function.UnaryOperator}.
     *
     * @param name The name to format.
     * @return The formatted name.
     */
    private final String format( final String name )
    {
        final var buffer = new StringJoiner( "},_{", "{", "}" );
        if( nonNull( name ) )
        {
            for( final var s: name.split( "_" ) )
            {
```



```

        buffer.add( s );
    }
}
else
{
    buffer.setEmptyValue( "No_Name" );
}
final var retValue = buffer.toString();

//---* Done *-----
return retValue;
} // format()

public final String myMethod()
{
    final var retValue = retrieveNames().stream()
        .map( this::format )
        .filter( n -> !n.equals( "No_Name" ) )
        .collect( joining( "},_Name={", "Names={Name={", "}" )

    //---* Done *-----
    return retValue;
} // myMethod()
}
// class MyClass

```

Only if a complex lambda needs to refer to local variables, it has to be defined locally:

```

public final class MyClass
{
    public final String myMethod( final String longTemplate, final String
        shortTemplate, final String nullText )
    {
        final var retValue = retrieveNames().stream()
            .map( n ->
            {
                final var result = nullText;
                if( n != null )
                {
                    result = n.length() > 15
                        ? longTemplate.formatted( n )
                        : shortTemplate.formatted( n );
                }
            }
        )
        return retValue;
    }
}

```



```
    } )  
    .filter( n -> !n.equals( "No_Name" ) )  
    .collect( joining( "},_Name={", "Names={Name={", "}}" ) )  
  
    //---* Done *-----  
    return retVal;    
  } // myMethod()  
}  
// class MyClass
```

The lambda cannot use the name **retValue** for its return value, because that is already defined in the surrounding method⁹². Instead, lambdas use the name **result** for their return values.

And you should write those complex lambdas also with just a single return point!

8.15.2. 'case' Results

With Java 12, **switch** expressions[221] have been introduced into the language. Additional details can be found in chapter 8.30.

Basically, a **switch** expression looks like this⁹³:

```
1  <result> = switch( <selector> )  
2  {  
3      case null -> <nullExpression>;  
4  
5      case <switchlabel1> -> <expression1>;  
6  
7      case <switchlabel2>, <switchlabel3> -> <expression2>;  
8  
9      case <switchlabel4> ->  
10     {  
11         <expressions>;  
12         yield <value1>;  
13     }  
14 }
```

⁹²And even if it is not (yet) defined, the lambda should not use it anyway, as it may collide with future code modifications.

⁹³Also refer to chapter 5.4.2 on page 97.



```
15 case <switchlabel4>, <switchlabel5> ->
16 {
17     <expressions>;
18     yield <value2>;
19 }
20
21 default -> <defaultExpression>;
22 }
```

Of course an expression can also throw an exception.

A concrete example may look like this:

```
1 public final <T> boolean isEmpty( final T arg )
2 {
3     final var retValue = switch( arg )
4     {
5         case null -> true;
6         case CharSequence charSequence -> charSequence.isEmpty();
7         case Collection<?> collection -> collection.isEmpty();
8         case Map<?,?> map -> map.isEmpty();
9         case Enumeration<?> enumeration -> !enumeration.hasMoreElements();
10        case Optional<?> optional -> optional.isEmpty();
11        case Iterator<?> iterator -> !iterator.hasNext();
12        case Throwable throwable -> isNull( throwable.getCause() );
13
14        case File file ->
15        {
16            var result = !file.exists();
17            if( !result )
18            {
19                if( file.isDirectory() )
20                {
21                    result = isEmpty( file.list() );
22                }
23                else if( file.isFile() )
24                {
25                    result = file.length() == 0;
26                }
27                else
28                {
29                    throw new IllegalArgumentException();
30                }
31            }
32        }
33    }
34    return retValue;
35 }
```



```
33         yield result;
34     }
35
36     default ->
37     {
38         if( !arg.getClass().isArray() ) throw new
39             IllegalArgumentException();
40
41         final var result = Array.getLength( arg ) == 0;
42
43         yield result;
44     }
45
46     //---* Done *-----
47     return arg;
48 } // isEmpty()
```

Same as for a lambda, you use **result** as the name of the local variable that holds the return value. This means that a collision could occur when using a complex lambda within a **case** block ... - if you really encounter this problem, you need to think about the structure of your code anyways - refer to “8.30 The “switch” Statement” on page 407 for more details.

And of course, there is only one **yield** statement per **case** block, and it has to be the last statement in that block.

8.16. Generics



8.17. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.18. Internationalisation



8.19. ----- Proceed from here!

8.19.1. ----- Proceed from here!

----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.20. The Annotation @API



8.21. ----- Proceed from here!

8.21.1. ----- Proceed from here!

----- Proceed from here!

[2] [13] Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.22. 'Convention over Configuration'

The phrase "Convention over Configuration" (or "Coding by Convention") got popular with the introduction of Ruby on Rails, but it is related to earlier ideas like the concept of "Sensible Defaults" and the "Principle of Least Astonishment" in user interface design.



Basically it means that an object instance can be created and used properly with only minimal configuration because all not absolutely mandatory settings will have meaningful – and useful! – default values.

On the other hand, there is that number 2 from the “Zen of Python”, saying “Explicit is better than implicit” ...

These are obviously contradictory statements – so whose right?

Both, to some extent!

Your design should support “Convention over Configuration”, but your code should rely on defaults only when a change of these defaults is unlikely, or such a change will not have an effect to your code.

So assume that you are using a 3rd party library that creates reports in HTML format; the default format was HTML3 with the previous version, but in the current version – that one used by you – it is HTML5. The generated reports are consumed by a tool that converts HTML5 input into PDF.

According to “Convention over Configuration”, you are fine: the convention is HTML5, you do not need to set the HTML version for the output format explicitly.

But what happens, if in a few years the next version of that report creator library will support HTML7 as the default, but your PDF generator sticks still with HTML5 for its input? Nothing happens until your successor as the maintainer of your software decides to use that new library ... afterwards you may see funny things in the generated PDF documents.

So the recommendation is: do not always rely on conventions! Whenever possible, provide an explicit configuration! At least leave a comment when you rely on the defaults, and that comment should describe what the anticipated defaults are.



8.23. ----- Proceed from here!

8.23.1. ----- Proceed from here!

----- Proceed from here!

[2] [13] Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.24. Compiler Warnings and Errors



8.25. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

It is not allowed to commit any code that emits warnings or even errors on a compiler run to the SCCS. It is also not allowed to switch off any compiler warning globally.

In fact, warnings should even not show up in the development environment, also with the most aggressive settings.

It is allowed to use the annotation `@SuppressWarnings[11]` to locally deactivate a warning. This is often necessary when dealing with legacy APIs that does not use Generics. So this sample would emit an “unchecked” warning for line 3:



```
1 public final Map<K,V> clone()  
2 {  
3     HashMap<K,V> retValue = (HashMap<K,V>) this.clone();  
4     return retValue;  
5 }
```

To avoid this, the annotation `@SuppressWarnings` with the value “unchecked” can be applied – preferably not to the method as a whole, but only to the problematic assignment, even if this means that an additional temporary variable is required (but not in this sample):

```
// RECOMMENDED  
public final Map<K,V> clone()  
{  
    @SuppressWarnings( "unchecked" )  
    HashMap<K,V> retValue = (HashMap<K,V>) this.clone();  
    return retValue;  
}  
  
// AVOID!!!  
@SuppressWarnings( "unchecked" )  
public final Map<K,V> clone()  
{  
    HashMap<K,V> retValue = (HashMap<K,V>) this.clone();  
    return retValue;  
}
```

As there is no rule without exception, here is one: I recommend to use labels to mark long code blocks (refer to the chapters “5.4.8 Labels and ‘break’ Statements”, “7.2.4 Trailing or End-Of-Line Comments”, and “7.6 Comments when?”), but if those code blocks do not reference these labels, they may cause an “Unused Label” warning in your IDE. The recommendation is here to deactivate that warning – globally.



8.26. The Ternary Operator ‘?’

8.27. ----- Proceed from here!

8.27.1. ----- Proceed from here!

----- Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.28. Programming to the Interface

A common idiom in Java programming is to construct an instance of a concrete type but to assign it to a variable of an interface type. This binds the code to the abstraction instead of the implementation, which preserves flexibility during future maintenance of the code. For example:

```
// Original
List<String> list = new ArrayList<>();
If var is used, however, the concrete type is inferred instead of the
interface:

// Inferred type of list is ArrayList<String>
var list = new ArrayList<String>();
```



8.29. ----- Proceed from here!

8.29.1. ----- Proceed from here!

----- Proceed from here!

[309][124][321][<empty citation>][<empty citation>][<empty citation>]
 [<empty citation>][<empty citation>][<empty citation>][<empty citation>]
 Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a
 faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl.
 Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis
 lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in
 sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu
 lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo
 lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula
 sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla
 egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat
 lacus vel est. Curabitur consectetur.

8.30. The "switch" Statement

In this document, I mentioned the `switch` statement[138] a first time in chapter "5.4.2 'switch' Statements" on page 95, where the formatting of the source code is discussed, and a second time in chapter 8.15.2 on page 396 that deals with return values.

Java inherited the problematic syntax of the `switch` statement from C; whether, when and how it should or could be used was always discussed controversially. There are projects that even banned the use of `switch` completely in their coding guidelines, because it is error prone.

Finally, a series of JEPs were launched to address the issues with the `switch` statement (JEP 325[130], JEP 354[25] and JEP 361[26]). Java 12 introduced



8. Coding Guidelines

the ‘new’ `switch` as a preview[330], since Java 14 it is an official part of the language.

Some sources (e.g. [221]) refer to the different syntax variants by the form of the `switch` labels as “case L:” (‘old’) and “case L ->” (‘new’).

“case L:” or ‘old’ switch	“case L ->” or ‘new’ switch
<pre>1 switch(<selector>) 2 { 3 case <label1>: 4 <expression1>; 5 break; 6 7 case <label2a>: 8 case <label2b>: 9 <expression2>; 10 break; 11 12 default: <expression>; break; 13 }</pre>	<pre>switch(<selector>) { case <label1> -> <expression1>; case <label2a>, <label2b> -> <expression2>; default -> <expression> }</pre>

As we have two different forms of the `switch` statement now, it is worth to define some coding guidelines for it.

Principles

P1. Prefer the ‘new’ `switch` over the ‘old’ one.

One motivation for the JEPs had been the error prone syntax of the original “case L:” `switch` statement; this is fixed by the new “case L ->” form – but only when it is used.

P2. Using `switch expressions` can make your code much more concise.

P3. `case` blocks should be short.

No matter if you are using the old or the new form, keep the code in the `case` blocks short and simple. Move the code to a `private` method, if it gets too long or too complex or both.



P4. Avoid 'fall-through'!

The new form does not allow 'fall-through', hence this is only relevant for the "case L:" form; 'fall-through' is the main reason why the old `switch` is considered to be error prone.

P5. Although it looks like one, a `switch` rule is no lambda.

That means that local variables that are referenced in a `switch` rule expression do not have to be effectively `final`.

P6. Never use the "case L:" label for a `switch` expression!

Although it works, the old label form for a `switch` expression looks just confusing – and is in a similar way error prone as for a regular `switch` statement.

8.30.1. Advantages of the new Syntax

The biggest advantage of the new syntax with the "case L ->" label is that it allows to avoid the problems of the original syntax with the "case L:" labels; previously, you wrote

```
1 switch( selector )
2 {
3     case 1:
4         executeOne();
5         break;
6
7     case 2:
8         executeTwo();
9         break;
10
11     default:
12         executeDefault();
13 }
```

Paranoid people (like me) would have added a `break` after `executeDefault()`, but the syntax rules does not require that.



8. Coding Guidelines

If – for whatever reason – the `break` in line 5 will get lost, it means that for `selector == 1`, both `executeOne()` and `executeTwo()` are invoked. Same if you add

```
13
14     case 3:
15         executeThree();
16         break;
17 }
```

for all values that are not 1, 2, or 3 (without being paranoid, of course): after `executeDefault()`, `executeThree()` is called.

With the new syntax, you do not need the `break` statement any longer:

```
1 switch( selector )
2 {
3     case 1 -> executeOne();
4     case 2 -> executeTwo();
5     default -> executeDefault();
6 }
7
8 }
```

Another advantage of the new syntax is that it looks much neater ...

The `switch` expressions[147, 148, 221] came with the new syntax, too, but it is not dependent on it; instead of writing

```
// RECOMMENDED!!
final var result = switch( selector )
{
    case 1 -> retrieveOne();
    case 2 -> retrieveTwo();
    default -> retrieveDefault();
}
```

you can write as well

```
// AVOID!!
final var result = switch( selector )
{
```



```
case 1: yield retrieveOne();  
case 2: yield retrieveTwo();  
default: yield retrieveDefault();  
}
```

replacing the `break` that usually terminates a branch by `yield` with the result value.

For `switch` expressions, my recommendation is to use always the "case L ->" syntax.

8.30.2. 'switch' Expressions

As already mentioned before, you can now have a `switch` expressions[147, 148, 221]; that means that each `case` block returns a result value. Based on the recommendation from the previous chapter, that looks like this:

```
final var result = switch( selector )  
{  
    case 1 -> retrieveOne();  
    case 2 -> retrieveTwo();  
    default -> retrieveDefault();  
}
```

Obviously the result type of each branch has to be compatible to those of the other branches (if it is not even always the same). If that is not really visible from your code, you should consider to use a concrete type for the receiving variable instead of `var`.



8.31. ----- Proceed from here!

5.4.2 8.15.2 [220]

[147, 148, 221]

[<empty citation>]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three



8.32. ----- Proceed from here!

5.4.2 8.15.2 [220] [221]

[147] [148]

[<empty citation>]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.32.1. More Enhancements



8.33. ----- Proceed from here!

5.4.2 8.15.2 [220] [221]

[147] [148]

[<empty citation>]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three

8.33.1. 'switch' Statements

This chapter describes how to format a `switch` statement together with the related `case` and `default` statements.

Beginning with the first preview in Java 12, an alternative syntax for `switch` was introduced, so that we have now two (in fact, three) different forms



of that construct. When to use which form and how is covered by chapter “8.30 The “switch” Statement” on page 407.

The traditional Form of ‘switch’

Originally, a `switch` statement in Java looked like this:

```
1 switch( <selector> )
2 {
3     case <switchlabel1>:
4         <statements1>;
5         // Falls through!
6
7     case <switchlabel2>:
8         <statements2>;
9         break;
10
11    case <switchlabel3>: <singleStatement>; break;
12
13    case <switchlabel4>:
14    {
15        <statements>;
16        break;
17    }
18
19    case <switchlabel5>: // Also a fall-through
20    case <switchlabel6>:
21    {
22        <statements>;
23        break;
24    }
25
26    default:
27        <statements>;
28        break;
29 }
```

First, these coding conventions make the blank line above each `case` (except the very first one) and above `default` mandatory!

If a `case` falls through, like in lines 3 to 9, a comment “*//Falls through!*” as in line 5 of the sample is required! But basically, such constructs should be



avoided when possible, because it also forces that the sequence of the `case` clauses cannot be changed, and that fact requires an additional comment to the `switch` itself. In fact here you should consider to ignore the “DRY Principle” – we will see this later – and repeat the code for the second `case` on the first one:

```
// BETTER
switch( <selector> )
{
    case <switchlabel1>:
        <statements1>;
        <statements2>;
        break;

    case <switchlabel2>:
        <statements2>;
        break;

    default: ...
}
```

Or you put the common code into a `private` method and call that from both branches.

Different to the case shown in lines 3 to 9, no comment is required in the case shown in lines 19 and 20: here we have two `case` clauses that triggers *exactly the same* action, and we can even omit the blank line between them (and also the comment from line 19).

For longer `switch` statements it is highly recommended to use a label with the `break` statement⁹⁴; this can look like this:

```
final Color color;
ColorSelector: switch( colorIndex )
{
    case 1: color = RED;
            break ColorSelector;

    case 2: color = BLUE;
            break ColorSelector;
}
```

⁹⁴In fact, I recommend to always use a label with `break`; see chapter “5.4.8 Labels and ‘break’ Statements” on this topic.



```
case 3: color = YELLOW;
      break ColorSelector;

case 4: color = GREEN;
      break ColorSelector;

default: color = NONE;
        break ColorSelector;
} // ColorSelector:
```

The **break** in the **default** branch is redundant, but it prevents a fall-through error if later another **case** branch is added – although it should be assured that the **default** branch is always the last branch in the **switch** statement. Of course no **break** is required when the branch is left by throwing an exception.⁹⁵

The new Form of ‘switch’

The alternative syntax for **switch** comes in two flavours (as *statement* and *expression*) and looks like this:

```
1 // switch statement
2 switch( <selector> )
3 {
4     case <switchlabel1> -> <singleStatement>;
5
6     case <switchlabel2>, <switchlabel3> -> <singleStatement>;
7
8     case <switchlabel4> ->
9     {
10         <statements>;
11     }
12
13     case <switchlabel4>, <switchlabel5> ->
14     {
15         <statements>;
16     }
```

⁹⁵A **break** would be also obsolete when the branch is left through a **return** statement, but in that case, the **return** would not be the last statement in the method, and this is another requirement!



```
17     default -> <singleStatement>;
18 }
19
20
21 // switch expression
22 var result = switch( <selector> )
23 {
24     case <switchlabel1> -> <expression>;
25
26     case <switchlabel2>, <switchlabel3> -> <expression>;
27
28     case <switchlabel4> ->
29     {
30         <expressions>;
31         yield <value>;
32     }
33
34     case <switchlabel4>, <switchlabel5> ->
35     {
36         <expressions>;
37         yield <value>;
38     }
39
40     default -> <expression>;
41 }
```

Of course the **default** in line 18 can be followed by a statement block as the **case** in line 8, and the **default** in line 40 can have a complex expression like the **case** in line 28.

As this format does not allow a fall-through in cases where the branches for two or more labels are doing the same stuff, it is possible here to have more than one switch label per **case**, as shown in lines 6, 13, 26 and 34.

If used with pattern matching (refer to [220]), a **switch** looks like this:

```
1 var selector = <anObject>
2 // switch statement
3 switch( selector )
4 {
5     case null -> <singleStatement>;
6
7     case <class1> <name1> -> <singleStatement>;
8 }
```



```

9      case <class2> <name2> ->
10     {
11         <statements>;
12     }
13
14     default -> <singleStatement>;
15 }
16
17 // switch statement
18 var result = switch( selector )
19 {
20     case null -> <expression>;
21
22     case <class1> <name1> -> <expression>;
23
24     case <class2> <name2> ->
25     {
26         <expressions>;
27         yield <value>;
28     }
29
30     default -> <expression>;
31 }

```

Same as for the `default` branch, the `case null` branch can be a statement block or a complex expression.

And an expression can also be always a `throw`.

General formatting Rules for 'switch'

- Every `switch` statement should include a `default` branch, even when the `case` labels are exhaustive.
- If really all possible values for the `case` labels are covered, the `default` branch should throw an exception about an unknown/unexpected value.⁹⁶ Refer to chapter “8.30 The “switch” Statement” on page 407 for additional details.
- The `default` branch should be always the last branch.

⁹⁶Refer to chapter “9.6.9 UnsupportedOperationException”[351] for a sample.



- A `switch` with pattern matching should always have a `case null` branch as its first branch.
- Syntactically, the use of curly braces in the branches is optional, even for multi-line statements, but here it is set as mandatory. One reason is that it allows to declare additional variables for a given branch that are local to the given branch.
- As already mentioned before, the blank line separating the branches is mandatory.

8.34. Encapsulation

8.35. ----- Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.35.1. Encapsulation with Modules

8.35.2. ----- Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in



sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.36. Lambdas

8.37. ————— Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.37.1. Functional Interfaces

8.37.2. ————— Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula



sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.38. The Interface `java.util.Formatter`

8.39. ----- Proceed from here!

[66] [67] [68] [101] [89] [199]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

8.40. The Interface `java.lang.Comparable`

8.41. ----- Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu



lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.42. Utility Classes

8.43. ----- Proceed from here!

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8.44. Date and Time Values

8.45. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est,



iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

8.46. Unique Ids

Data records stored to a database require a unique identifier in most cases. Sometimes, this identifier is provided externally – a social security number, an account number, or a tax id – but usually not.

When the database will be accessed (and updated) from different locations, this identifier has to be *universally* unique; alternatively, you can introduce a global service that provides such identifiers. But usually, this does make sense only when these identifiers are not only just technical – like the already mentioned social security numbers, account numbers, or tax ids.

How to get universally unique ids (UUID) is defined in RFC 4122[240], among others. Java provides an implementation for these UUIDs with the class `java.util.UUID`[104]. Unfortunately, the class provides only one method that creates new arbitrary UUIDs (`java.util.UUID::randomUUID()`[307]), and these are version 3 pseudo-random UUIDs.

These are large enough that collisions are unlikely, but when used as primary keys on a database table, the database indexing can get quite inefficient (lack of locality in the indexes). This is discussed in more detail in [310].

Partially, this is addressed by timebased UUIDs (version 1), and new formats will be discussed (see [119, 241]), but another issue is that these UUIDs occupy at least 128 bit – when you use the binary format. But usually, the textual representation is used (mainly because the Java class `UUID` does not provide a method to get the binary format), and that has even 288 bit (36 bytes or characters). I provide a utility class `UniqueIdUtils`[355] that provides some tools to address these issues. So it allows you to create timebased UUIDs or to get the binary representation from a UUID.



The already mentioned article in [310] suggests to use shorter unique ids that should be timebased; the article refers to a the `TSID_CREATOR` library[243], but alternatively, my `UniqueIdUtils` do also provide a `TSID` type with 64 bit length.

Another issue with UUIDs is that it should not be *too* easy to guess a valid instance. This means that a database sequence[371] that just increments a counter in order to get a new identifier is the worst choice in this regard.

8.47. Utilising JMX

8.48. ----- Proceed from here!

See 8.2.6, and there the part about warnings! Keeping track about failed logins.!!! Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



8.49. Finalisation

8.50. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

8.51. Deprecation of Elements

8.52. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



8.53. Multi-Threading and Synchronisation

8.54. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



8.55. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three



8.56. Inner Classes

Inner classes are ...

P1. one

P2. two

P3. three

- one
- two



- three

8.57. Chaining Constructors

The DRY (“Don’t repeat yourself”) principle should be also used for constructors. Basically there are two different scenarios. In the first, you have constructors with a different number of arguments, reflecting to amount of default settings for the new object. We call this the “Primary Constructor” pattern. In the other case you have constructors with different arguments, what we want to call the “Common Constructor” pattern. Both patterns are discussed below. Obviously there are situations where you have hybrids of these patterns, and sometimes you have to decide which one is better for the current design.

8.57.1. The “Primary Constructor” Pattern

Refer to the sample class below:⁹⁷

```
// AVOID!!
public class UnchainedConstructors extends JButton
{
    public UnchainedConstructors( String text )
    {
        setText( text );
        String tooltip = "A_Button_to_show_" + text;
        setToolTipText( tooltip );
    }

    public UnchainedConstructors( String text, String tooltip )
    {
        setText( text );
        setToolTipText( tooltip );
    }
}
```

⁹⁷I took the example from [328] and adopted it to my needs. As usual, I omitted the comments that are otherwise required for productive code.



8. Coding Guidelines

```
public UnchainedConstructors( String text, String tooltip,
                             ActionListener listener )
{
    setText( text );
    setToolTipText( tooltip );
    addActionListener( listener );
}
```

Although the code works, it is wasteful and prone to errors, especially when modified in later stages of the product. Making changes to one of the constructors will not reflect to the others, making the class working correctly under some circumstances and causing issues in others.

This can be done by declaring the constructor with the most arguments to the “primary constructor” and having all others calling it:

```
public class ChainedConstructors extends JButton
{
    public ChainedConstructors( String text )
    {
        this( text, "A_Button_to_show_" + text, null );
    }

    public ChainedConstructors( String text, String tooltip )
    {
        this( text, tooltip, null );
    }

    public ChainedConstructors( String text, String tooltip,
                               ActionListener listener )
    {
        setText( text );
        setToolTipText( tooltip );
        addActionListener( listener );
    }
}
```

In this case it would be possible also that the first constructor calls the second one:

```
...
public ChainedConstructors( String text )
{
    this( text, "A_Button_to_show_" + text );
}
```



...

that in turn will call the primary constructor. Now the code for the other constructors is limited to what they do differently.

8.57.2. The “Common Constructor” Pattern

Again, we want to look at a bad sample first:

```
// AVOID!!
public class UnchainedConstructors2
{
    private String m_Text;
    private int m_CurrentPos;
    private PropertyChangeSupport m_PropertyChangeSupport;

    public UnchainedConstructors2( String text )
    {
        m_Text = text;
        m_CurrentPos = 0;
        m_PropertyChangeSupport = new PropertyChangeSupport( this );
    }

    public UnchainedConstructors2( String [] text )
    {
        StringBuilder buffer = new StringBuilder();
        for( String s : text ) buffer.append( s ).append( '\n' );
        m_Text = buffer.toString();
        m_CurrentPos = 0;
        m_PropertyChangeSupport = new PropertyChangeSupport( this );
    }

    public UnchainedConstructors2( Collection<String> text )
    {
        StringBuilder buffer = new StringBuilder();
        for( String s : text ) buffer.append( s ).append( '\n' );
        m_Text = buffer.toString();
        m_CurrentPos = 0;
        m_PropertyChangeSupport = new PropertyChangeSupport( this );
    }

    public UnchainedConstructors2( BufferedReader reader )
```



8. Coding Guidelines

```
{
    StringBuilder buffer = new StringBuilder();
    String line = null;
    do
    {
        line = reader.readLine();
        if( line != null ) buffer.append( s ).append( '\n' );
    } while line != null;
    m_Text = buffer.toString();
    m_CurrentPos = 0;
    m_PropertyChangeSupport = new PropertyChangeSupport( this );
}
```

Obviously, in this case we do not have a constructor that can be designated as the primary constructor as each of the existing ones have just one argument, but each of a different type.

Saying that one taking the single String argument is the primary constructor and calling it like below does not work.

```
// DOES NOT WORK!!!
...
public UnchainedConstructors2( String [] text )
{
    StringBuilder buffer = new StringBuilder();
    for( String s : text ) buffer.append( s ).append( '\n' );
    this( buffer.toString() );
}
...
```

According to the language specification, the call to this() has to be the very first statement of the constructor. This is the same as for the call to the super constructor with super().

The issue is solved like shown below. Instead of repeating the initialisation steps that are necessary for each constructor, they are placed in an additional constructor with a different (usually shorter) signature, named the “common constructor” (and in this example, it is also the default constructor, but this is not always the case).

```
1 public class ChainedConstructors2
2 {
3     private String m_Text;
```




```
4 private int m_CurrentPos;
5 private PropertyChangeSupport m_PropertyChangeSupport;
6
7 private ChainedConstructors2()
8 {
9     m_CurrentPos = 0;
10    m_PropertyChangeSupport = new PropertyChangeSupport( this );
11 }
12
13 public ChainedConstructors2( String text )
14 {
15     this()
16     m_Text = text;
17 }
18
19 public ChainedConstructors2( String [] text )
20 {
21     this()
22     StringBuilder buffer = new StringBuilder();
23     for( String s : text ) buffer.append( s ).append( '\n' );
24     m_Text = buffer.toString();
25 }
26
27 public ChainedConstructors2( Collection<String> text )
28 {
29     this()
30     StringBuilder buffer = new StringBuilder();
31     for( String s : text ) buffer.append( s ).append( '\n' );
32     m_Text = buffer.toString();
33 }
34
35 public ChainedConstructors2( BufferedReader reader )
36 {
37     this()
38     StringBuilder buffer = new StringBuilder();
39     String line = null;
40     do
41     {
42         line = reader.readLine();
43         if( line != null ) buffer.append( s ).append( '\n' );
44     } while line != null;
45     m_Text = buffer.toString();
46 }
```



It is not necessary that the common constructor is private like in this example. It is even possible to have a cascade of constructor calls.

If required you can also introduce a private constructor with an artificial formal parameter that is never used as the “Common Constructor”:

```
private ChainedConstructor( final boolean ignored ) { ... }
```

8.57.3. Constructor Helpers

In the sample for the ChainedConstructors class in chapter 5.19.1 above we have a constructor like this:

```
1 public ChainedConstructors( String text )
2 {
3     this( text, "A_Button_to_show_" + text, null );
4 }
```

Sometimes the expressions are more complex like a simple string concatenation as in this case.

It is possible to call a helper method from the arguments list of this() (or super(), what is equivalent in this case):

```
1 public ChainedConstructors( String text )
2 {
3     this( text, composeToolTip( text ), null );
4 }
5 ...
6 private static String composeToolTip( String text )
7 {
8     String template = getText( TOOLTIP_DEFAULT );
9     if( template == null ) template = "A_Button_to_show_%1$s";
10    String retValue = format( template, text );
11    return retValue;
12 }
```

Java requires that this helper is static, and it should be private, or in case the class allows to be extended, protected and final.



Such a helper could also be used to check the arguments before conveying them to `this()` or `super()`, as shown in chapter “8.3 Checking Method Parameters and Return Values” on page 250.



8.58. ----- Proceed from here!

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

P1. one

P2. two

P3. three

- one
- two
- three



8.59. Miscellaneous

In this chapter I collected some dos and don'ts that do not fit into one of the other chapters, but are not relevant enough for a chapter on their own.

- Avoid octal numerical literals in your source code! Although this feature exists since the first versions of Java, it is not very well known.

If you do not know what I'm talking about: start `jshell`, type in `021 + 021` and be surprised that the result is not the ultimate answer to life, the universe, and everything.[4]



Or to summarise it: Do not prefix integer literals with 0!

- Avoid using an object to access a class (static) variable or method. If you cannot use a static import, refer the element through the class name instead. For example:

```
public final class AClass
{
    public static final void classMethod() { ... }
}
// class AClass

...

import static some.package.AClass.classMethod;

public final class MyClass
{
    public final void myMethod()
    {
        // RECOMMENDED!
        classMethod(); // Using the static import

        // OK
        AClass.classMethod();

        // AVOID!!
        final var anObject = new AClass();
        ...
        anObject.classMethod();
    } // myMethod()
}
// class MyClass
```

You can configure both Eclipse and IntelliJ IDEA to report the access to a static member of a class through an instance as a warning or even as an error.

- In general, static imports are preferred over using the class name as prefix for references to static class members, either to class methods or to constants.
- Try to initialise local variables where they are declared. The only reason not to initialise a variable where it's declared is if the initial value



depends on some computation occurring first.

- Avoid the multi-line initialisation for fields; use a constructor instead:

```
public final class MyClass
{
    private final String m_Field;
    // AVOID!!!!
    {
        final var propertyName = readConfig( "user.property" );
        m_Field = System.getProperty( propertyName );
    }

    /**
     * Creates a new instance for {@code MyClass}.
     */
    public MyClass()
    {
        // INSTEAD DO IT IN THE CONSTRUCTOR!!
        final var propertyName = readConfig( "user.property" );
        m_Field = System.getProperty( propertyName );
    } // MyClass()
}
// class MyClass
```

- Try to call a method only when its result is needed. This is even more true if the method does not return a value but has other effects.

Although this may seem to be self-evident, you may find often code like this:

```
// AVOID!!!
final var logMessage = composeMessage( params );
if( logEnabled )
{
    //---* Log the parameters *-----
    writeLog( logMessage );
}
...
```

or

```
// AVOID!!!
{
    final var param1 = retrieveData( data1 );
```



```
final var param2 = retrieveData( data2 );
if( option )
{
    process( param1 );
}
else
{
    process( param2 );
}
}
```

Instead, the samples should look like below:

```
// RECOMMENDED
if( logEnabled )
{
    //---* Log the parameters *-----
    final var logMessage = composeMessage( params );
    writeLog( logMessage );
}
...

// RECOMMENDED
{
    final Data param;
    if( option )
    {
        param = retrieveData( data1 );
    }
    else
    {
        param = retrieveData( data2 );
    }
    process( param );
}

// RECOMMENDED/Using the trinary operator
{
    final var param = option
        ? retrieveData( data1 )
        : retrieveData( data2 );
    process( param );
}
```



- Avoid anonymous classes! Although it (sometimes) reduces the code to write and the number of source files, it makes the resulting code very hard to read in most cases. And the number of generated class files remains exactly the same, no matter if anonymous classes, inner classes, non-`public` or `public` classes are used.

In most cases, you can use a lambda instead of an anonymous class.

- If you are using collections or maps, declare and define them with generics.

If a legacy interface returns a collection or map that is not declared with generics, map it. The resulting warning can be suppressed using the `@SuppressWarnings[11]` annotation.

Sometimes this may require the introduction of a temporary helper variable, as in the last sample, below.

```
// AVOID!!
List x = new LinkedList();

// RECOMMENDED!!
List<String> x = new LinkedList<String>();

public abstract Vector method1();
public abstract Vector method2() throws IOException;
...
public static final void main( String... args )
{
    @SuppressWarnings( "unchecked" )
    final List<String> method1Result = method1();
    List<String> method2Result = null;
    try
    {
        @SuppressWarnings( "unchecked" )
        final List<String> temporary = method2();
        method2Result = temporary;
    }
    catch( final IOException e ) { ... }
} // main()
```

- Using `java.lang.Object` as the type parameter for a generic data type is useless in most cases and should be avoided:



```
// AVOID!!!
private final List<Object> m_List = new LinkedList<Object>();
```

If you want to declare a variable of a generic type that should work for any parameter class, you have to use the question mark:

```
Class<?> dataClass = data.getClass();
```

- The classes `java.util.Vector`[105] or `java.util.Hashtable`[91] should not be used! Although their implementation had been modernised already with Java 1.2, their performance is still inferior to the alternative implementations, because `Vector` and `Hashtable` are still synchronised.

For method arguments and return values, you should use the interfaces `java.util.List`[201] instead of `Vector`, and `java.util.Map`[203] instead of `Hashtable`. Refer also to chapter ?? on page ?? that elaborates further on this topic.

If you need an implementation of the `List` interface, you should prefer `java.util.ArrayList`[83] over `java.util.LinkedList`[92]. The latter is only more performant in some very rare cases, and it also has a bigger memory footprint⁹⁸.

Instead of `Hashtable`, you should use `java.util.HashMap`[90] as the implementation for the `Map` interface.

If you really need a synchronised list, you can still consider to use `Vector` as your implementation of `List`, but for a synchronised implementation of `Map`, you should take `java.util.concurrent.ConcurrentHashMap`[85].

- Use the enhanced `for-loop` when iterating over collections⁹⁹. It is also preferred when iterating over arrays.

Alternatively, you can use an iterator, the `java.lang.Iterable::forEach` method, or the Stream API:

⁹⁸Internally, an `ArrayList` uses an array for the entries, and when this gets too small, a new array with twice the size will be allocated. This means that for some time two large array will exist. For really, really large lists, this may cause an issue. In the opposite, a `LinkedList` will grow entry by entry.

⁹⁹Ok, only implementations of `java.util.List`[201] provide methods for random access.



8. Coding Guidelines

```
// AVOID!  
for( var i = 0; i < list.length(); ++i )  
{  
    process( list.get( i ) );  
}  
  
// RECOMMENDED  
for( final var element : collection ) process( element );  
  
collection.forEach( this::process );  
  
// OK  
for( final var i = collection.iterator(); i.hasNext(); ) process(  
    i.next() );  
  
collection.stream()  
    .forEach( this::process );
```

The Stream API allows you to filter the elements in the collection.

- Consider to use the varargs feature when a method takes only one single argument, but can be called repeatedly with different arguments, or when it takes an array as argument:

```
public final void addListener( final Listener... listeners ) { ... }
```

instead of

```
public final void addListener( final Listener listener ) { ... }
```

Obviously, the implementation now has to deal with multiple entries, but it allows to write

```
addListener( listener1, listener2, listener3 );
```

instead of

```
addListener( listener1 );  
addListener( listener2 );  
addListener( listener3 );
```

making the code easier to read.



- “Forever” loops should be coded as

```
ForeverLoop: while( true )
{
    ...

    //---* Terminate the loop *-----
    if( <condition> ) break ForeverLoop;
} // ForeverLoop:
```

- Do not use `new String()` with a string constant or a string expression as the argument. There are only very few situations where this is useful or necessary, and most of them are related to JNI. Usually `new String()` is a waste of memory and computing time, except you use it with a `byte`, `char` or `int` array as the argument. Refer to [66] for the details.
- Some programmers do not like the Autoboxing feature that was introduced with Java 5. It is up to you to use it, or to transform the primitive types explicitly into their corresponding object types and vice versa.¹⁰⁰
- If a constructor calls a method of its own class or a superclass, this method has to be `static`, `private`, or `final`¹⁰¹.

In addition, it may call another constructor of the same or the super class, using `this()` or `super()`.

- Ensure that the main thread (that one that executes `main()`) always dies as the last non-deamon thread.

This means that you should keep a reference of all the threads that your code starts so that you can kill them explicitly before the program terminates.

- Ensure that assertions[137, 323] are enabled during testing and disabled in production.

To enable assertions, add `-ea` or `-enableassertions` to the JVM command line.

¹⁰⁰Nevertheless, you should consider “2. Explicit is better than implicit, and verbosity is your friend” – see chapter 1.11 on page 31.

¹⁰¹The important setting is `final`, because `private` and `static` methods are implicitly `final`



8. Coding Guidelines

- Although releasing resources when they are no longer needed is a good idea in most cases, it can cause trouble in rare occasions.

I found a code sequence like this in some real life code:

```
...
}
catch( Exception e )
{
    PrintStream ps = new PrintStream( System.out );
    ps.println( "Error_occured" );
    e.printStackTrace( ps );
    ps.close()
}
```

This will not only close the **PrintStream** **ps**, but also the wrapped **System.out** stream – with the consequence, that this exception was the last that was displayed on the console (or written to the log output).

So make sure that your code will only cleans up objects that your code is responsible for, either because it created them or the responsibility was clearly delegated to it. In case of a wrapper (like most implementations of **java.io.InputStream**, **java.io.OutputStream** **java.io.Reader** and **java.io.Writer**), your code should not call methods like **close()** if it is not responsible for the wrapped object, as in most cases the wrapper would delegate them to the wrapped object.

If your code has to provide references to resources to 3rd party code, you should consider to protect those resources from being freed (if required). For an instance of **PrintStream**, this could look like this:

```
...
final var tempStream = new PrintStream( myStream )
{
    /**
     * {@inheritDoc}
     */
    @Override
    public void close() { /* Does nothing! */ }
};
externalMethodWritingToPrintStream( tempStream );
...
```



- This is about the usage of “++” and “--”, because they have both a prefix and postfix notation.

The prefix notation will change the value first and returns the new value, while the postfix version will return the current value and change it afterwards. So the usage of prefix or postfix notation depends from the context.

In case where the return value is not used immediately, it is strongly recommended to use the prefix notation always. This is especially true in `for` loops.

Examples:

```
// DISCOURAGED
for( int i = 0; i < max; i++ )
{
    ...
}

// RECOMMENDED
for( int i = 0; i < max; ++i )
{
    ...
}
```

Confessed, this is an *early* optimisation (to avoid “premature”), and most implementations of the Java compiler will translate the postfix version into the same binary code as for the prefix version if the return value is not used¹⁰². But developing the habit to prefer the prefix notation have no costs, and it may spare some cycles when using other languages than Java, or when the compiler does not optimise.

- Avoid setting several variables to the same value in a single statement. It is hard to read.

Example:

```
// AVOID!
fooBar.fChar = barFoo.lChar = 'c';
```

¹⁰²If not the javac, probably the JIT will do.



8. Coding Guidelines

Avoid to use the assignment operator in a place where it can be easily confused with the equality operator. If you really do not see another option, make sure that it cannot be confused.

Example:

```
// AVOID!  
boolean flag;  
while( flag = isEmpty() ) { ... }
```

should be written as

```
boolean flag;  
while( (flag = isEmpty()) == true ) { ... }
```

if it is really necessary to have the assignment at this place.

As Java do not allow other than boolean expression for conditions, something like

```
// DOES NOT WORK!  
while( c++ = --d ) { ... }
```

is not possible. But a common pattern is

```
BufferedReader reader = ...  
String line = "";  
while( (line = reader.readLine()) != null ) { ... }
```

Do not use embedded assignments in an attempt to improve run-time performance. This is the job of the compiler.

Example:

```
// AVOID!  
d = (a = b + c) + r;
```

should be written as

```
a = b + c;  
d = a + r;}
```



- It is generally a good idea to use parentheses liberally in expressions involving mixed operators to avoid operator precedence problems. Even if the operator precedence seems clear to you, it might not be to others – you should not assume that other programmers know the precedence rules as good as you do.

```
// AVOID!  
if( a == b && c == d ) { ... }  
  
// BETTER  
if( (a == b) && (c == d) ) { ... }
```

If an expression containing a binary operator appears before the “?” in the ternary “?:” operator, it should be parenthesized.

Example:

```
(x >= 0) ? x : -x;
```

-
-
-
-
-
-



8.60. ----- Proceed from here!

- If there is a getter for a field, it should be used even by the methods of the same class. Usually the same yields for setters, but there are exceptions from that rule³, in case a field can be set internally to values that are invalid if set from the outside. In such cases the direct access to the field has to be commented accordingly.
- A method or constructor should not have more than 7 (in words: seven) arguments. If you feel inclined to provide more arguments to construct an instance, consider to use an instance of Map instead.
- Do not use BitFields, at least do not expose them to the API. Although the Java API is using them itself, they are still problematic.
- Whenever you have to convert a date, time or date-time value to a String that is not primarily meant to be display, used an internal format that is independent from any locale. Our recommendation is to use the ISO 8601 format “YYYY-MM-dd'T'hh:mm:ss.SSS”, normalised to UTC, and to use the class SimpleDateFormat to format the date.⁴
- ...



9. Appendices

9.1. The Naming Dictionary

The names of program elements provide an implicit contract (or at least a kind of commitment) between the original author of the program and its users/maintainers. But because people understand words differently, I have added a dictionary of common verbs and their implicit contracts here, together with a list of suffixes for class names.

9.1.1. Verbs

This chapter provides a list of verbs¹ to be used with method names and a description of their implicit contract. These verbs are usually prefixes to a method name, although some of them could be used as standalone names, too. The form that used more often is mentioned first.

¹Ok, some names or prefixes are not verbs, like 'main', 'from' or 'to' ...



Table 9.1.: Verbs

Verb	Contract
acquire (<i><arg></i>) acquire ...() acquire ... (<i><arg></i>) acquire ()	This name or prefix for the name of a method is used for methods that requests an exclusive access to something, either the owning object (then usually there is no argument given) or the object that is identified by the given argument. Usually, the return value is some kind of handle that has to be released later, by a call to a ‘release’ method.
add ... (<i><arg></i>) add (<i><arg></i>)	A method with this name will add the given argument value to an internal list in the object, keeping the argument as it is, so that it could be retrieved again later. Usually, an ‘add’-method would not return something.
append (<i><arg></i>) append ... (<i><arg></i>)	The name ‘append’ for a method indicates that it will append the given argument to an extensible data structure, usually with interpreting the argument in some way. In most cases, it is not possible to retrieve the argument from the data structure afterwards. This a typical method for a builder, so usually the method returns the owning object itself (this). A sample for this is java.lang.StringBuffer.append() .
build ()	The terminal method for a builder has this name. It returns the built object. Usually it can be called only once on the owning object.
call ()	This is the name of the main method of a thread that returns a result (an implementation of the Callable interface) and should usually only used in this or a similar context. For details on this, refer to [319] and [196].

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
check() check(<arg>) check...()	Usually ‘to check’ is understood as testing for a given condition, but sometimes it is also used if an object should be marked (as used in “CheckBox”). I do not recommend the latter usage (use ‘mark’ instead), and because of the chance to generally misinterpret the verb, I do not recommend to use it at all. If used anyway, the respective method may not modify the object and it should return a boolean.
clear()	A method with this name will empty or reset the owning object.
clone()	The method clone() creates an identical copy of the owning object. Refer to chapter 8.12.3 on page 363 and to [261] and [184].
close() close...()	This name will be assigned to methods to close a connection to an external resource, like a file or device. Refer to [182] and [178] for details.
compose... (<arg>) compose...()	A ‘compose’ method will compose a new object or data structure from the internal state and the given arguments – if any. It will not modify the owning object itself, and each call with identical arguments (and on an unchanged object) will return the same value. Different from a ‘build’ method, a ‘compose’ method can be called more than once on the same object instance.
connect() connect(<arg>) connect...() connect... (<arg>)	A method with this name initialises the connection of the owning object to an external resource, like a device or a remote system.

Continued on next page



Table 9.1 – *Continued from previous page*

Verb	Contract
copy(<arg>)	A ‘copy’ method makes a copy of the given argument and returns it. If this is an identical copy (like through the invocation of clone()) or not depends from the implementation.
create... (<arg>) create...()	This name indicates a method that creates something based on the given arguments, but different from the ‘compose’ method, each call can have a different result, even for identical arguments. A ‘create’ method usually returns created entity as the result.
delete() delete(<arg>)	A method with the name ‘delete’ deletes something, either identified by the state of the owning object or by the given arguments. A sample would be java.io.File::delete that removes the file that is associated with the respective instance of java.io.File .

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
execute() execute(<arg>)	This is the name for a method that does the program's or application's work. Usually the arguments are the command line arguments for the program, although this is not mandatory. This is used in pattern similar this: <pre> 1 public final class MyProgram 2 { 3 public final void execute(final String [] 4 args) { ... } 5 public final boolean initialize(final 6 String [] args) { ... } 7 public static final void main(final 8 String... args) 9 { 10 final var application = new 11 MyProgram(); 12 if(application.initialize(args)) 13 application.execute(); 14 } // main() 15 } 16 // class MyProgram </pre> Obviously, the method initialize() is optional, but if available, only this may take the command line arguments instead of execute()—, too. This allows to use an application/program class also in the context of another program/application.
exec...() exec... (<arg>)	Use the prefix 'exec' to name methods the will execute an operation, identified by the main part of the name. The arguments are parameters for this operation. Nothing is said about the state of the owning object, any return values or the state of the arguments.

Continued on next page



Table 9.1 – *Continued from previous page*

Verb	Contract
find(<arg>) find...(<arg>)	A ‘find’ method searches something that is expected to be there (different from a ‘search’ method described below). The method returns the result or throws an appropriate exception when it fails, but it would be also possible to return an instance of java.lang.Optional [96]. The arguments are parameters for the search. The method may or may not change the state of the owning object.
from(<arg>) from...(<arg>)	Methods with the name from() or a name prefixed with ‘from’ had been introduced with the java.time API and Java 8. These are static factory methods that create an instance of the owning class based on the given argument (usually a String).
generate...() generate...(<arg>) generate() generate(<arg>)	A ‘generate’ method generates something based on the object state, with or without changing it. Each call to generate may have a different result, even for the same arguments – this distinguishes it from ‘compose’. Different from a ‘create’ method, a ‘generate’ method may work on entities outside the program (a file, an image, a report, ...); this means it does not have a return value, or the return value is just the status of the generation process.

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
get...()	The prefix get indicates a <i>getter</i> method; it does not take any arguments, and it does not change the state of the owning object. The remaining part of the method name is known as the ‘property name’: the getter returns the value for that property (attribute). For more details, refer to [228]². A special variant is the method public static <Type> getInstance() method: it returns an instance of the class <Type> – each invocation returns the same instance. It does not matter whether the first invocation creates that instance, or if it will exist previously. For optional properties, a getter may return an instance of java.util.Optional[96], but it should never fail (meaning it should be avoided that a getter throws an exception).
has...() has(<arg>) has...(<arg>)	A method with a name starting with has checks whether the owning object ‘contains’ something, either identified by the rest of the name, or by the given argument. Samples could be hasChildren() for a tree node object, or hasOrders() for a customer record instance. Calling the method may not change the state of the owning object.
initialize() initialize(<arg>)	The ‘initialize’ method does what its name says: it initialises the owning object. Obviously, this changes the state of that object. If the method cannot be called multiple times, it should throw an IllegalStateException on any invocation after the first.

Continued on next page

²Although the whole document [227] could be relevant here.



Table 9.1 – *Continued from previous page*

Verb	Contract
is...()	This is the prefix for a <i>getter</i> that returns a boolean . Refer to get... , above.
load() load(<arg>)	The method with the name load() ‘loads’ the data for the owning object from somewhere (usually an external source) and initialises it with this data. The argument is usually the reference to the source. The return type is either void , or the method returns some kind of status for the load operation.
load...() load...(<arg>)	If this is used as the prefix for a method name, that method also loads data from an external source, but it initialised and returns a data structure that is described by the remainder of the name. Depending on the context, a name like loadData() could be valid, but some more meaningful is preferred, like loadClientHistory() or loadArticleList() .
log(<arg>) log()	A method with this name will write something to the logging subsystem. The method is either a member of the class representing the logger instance, or it is a member of an arbitrary class. In the first case, the arguments determine what is logged. In the latter case, it logs the internal state of the respective object instance – without modifying it!

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
main() main(<arg>)	Although ‘main’ is not a reserved word in the Java language, it should be treated as one. Therefore a method should never be named as main() unless it is the program entry point. Not to mention that ‘main’ is not even a verb at all. The usual signature for the main method is <pre>public static void main(String [] args)</pre> but I prefer it as <pre>public static final void main(final String... args)</pre> Refer to [136].
make...() make...(<arg>)	The prefix ‘make’ is similar to ‘create’, ‘compose’, or ‘generate’, but usually a method with a name starting with ‘make’ produces something outside the program – like a file or a database (table). In this regard it is like ‘generate’, but the process of ‘making’ is less complex as that of ‘generation’.
mark() mark(<arg>) mark...() mark...(<arg>)	A ‘mark’ method changes the state of the owning object by setting (when taking a boolean argument) or toggling (when taking no argument) an internal flag (see also ‘check’). Usually those methods will not return something, but for chaining, they can return this, or they return the previous state of the flag.
obtain...() obtain...(<arg>)	A method with a name prefixed with ‘obtain’ is similar to a getter, but with the difference that an ‘obtain’ method can change the state of the owning object, and that it does not necessarily return just a property. An ‘obtain’ method should never fail (should not throw an exception), but it may return an instance of java.util.Optional[96].

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
of(<arg>)	The method name of() was more or less introduced with Java 9 and the static factory methods for java.util.List , java.util.Set and java.util.Map . The methods with the that name creates an instance of owning class containing the given arguments. If there is no argument, an empty instance will be created.
open()	An ‘open’ method opens a connection to an external resource that is represented by the owning object. Obviously, that object changes its state. Usually, open() will not return something; or when it returns something, in will be that status of the open operation. It will also not take any arguments as all necessary information are given by the state of the owning object.
open... (<arg>) open(<arg>)	A method with a name that is prefixed with ‘open’ will also open a connection to an external resource, but it will return an object representing this connection or the external resource. The required information can either be taken from the owning object or given as the arguments or both. Such a method may or may not change the state of the owning object. And the owning object may or may not provide a corresponding ‘close’ method. A sample could be a method <pre>public final File openLogFile(final File logFolder) { . . }</pre> that can be even used with try-with-resources .

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
peek()	A method with this name is meant to spy into a data structure without modifying the state of the owning object. The method <code>java.util.Queue::peek</code> [304] is a sample for this. The method <code>java.util.stream.Stream::peek</code> [306] does not examine a data structure, but it allows to spy on the intermediate results of the stream processing.
perform() perform(<arg>) perform() perform(<arg>)	A ‘perform’ method performs an action, either implicitly like for an implementation of the command pattern, or explicitly named by the remaining part of the method name; it is even an option that the action is given as the argument of the method. Usually such a method will not change the state if the owning object.
pull() push(<arg>)	This pair of methods is usually associated with data structures like stacks or queues: push() adds an entry to the data structure, pull() retrieves an entry from it.
put(<arg>) put... (<arg>)	‘put’ is somehow a synonym for ‘add’, put different from that it has some position identification: with ‘add’, the new part is just thrown on the big pile, while it will be placed to a special position with ‘put’. Refer to [300].
read()	The read() method loads the state of the owning object from an external resource. Usually, it does not return something, or it returns that status of the ‘read’ operation. I prefer to use load() instead.

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
read(<arg>) read... (<arg>)	A ‘read’ method read something from an external source and returns. This may or may not change the state of the owning object. Usually a reference to that external source is provided as a parameter, but it is also possible that this reference is implicit (as for streams).
release... (<arg>) release(<arg>)	A ‘release’ method provides the converse operation to an ‘acquire’ method: it releases the exclusive access to the object that is somehow identified by the given argument. It does not return something, or it returns the status of the operation.
remove... (<arg>) remove(<arg>)	A method with the name remove() or a name that is prefixed with ‘remove’ will remove something from an internal data structure of the owning method. The given argument is some kind of reference to that thing that should be removed. It does not return something, or it returns the removed object or the status of the operation. A ‘remove’ method is somehow the reverse operation to an ‘add’ method.
retrieve...() retrieve... (<arg>)	A ‘retrieve’ method is similar to a ‘get’ method, with the differences that it may change the state of the owning object, that it is not limited in returning a property, and – that makes it also different to an ‘obtain’ method – that it can fail.
run()	run() is the name of the main method of all implementations of the interface java.lang.Runnable [188], and as such it is also the main method of all threads except the main thread (there it is main()). The name should not be used in a different context. Refer also to [267, 273].

Continued on next page



Table 9.1 – Continued from previous page

Verb	Contract
search(<arg>) search... (<arg>)	A ‘search’ method searches something, but it is a valid result that it does not find something (it will not throw an exception if nothing was found). That distinguishes it from a ‘find’ method as described above. The method returns an instance of java.lang.Optional [96] with the result. The arguments are parameters for the search. The method may or may not change the state of the owning object.
save() save(<arg>)	A method save() provides the converse operation to that of a method load() : it persists the current state of the owning object, usually to an external destination. The argument usually determines that destination. If the method has another return type than void , the return value is the status of the save operation. The method may or may not change the state of the owning object.
save... (<arg>)	A ‘save’ method provides the converse operation to that of a ‘load’ method: it persists the object that is identified by the given argument to an external destination. If it has a return type other than void , this will be the status of the save operation. The method may or may not change the state of the owning object.

That a method name is built using one of the verbs above does not free you from providing a proper JavaDoc comment that describes the purpose of the method in detail, together with the arguments, return values and exceptions.



9.1.2. Suffixes for Class Names

This chapter lists defined suffixes for class names and their function.

Table 9.2.: Suffixes for Class Names

Suffix	Description
...Adapter	A class that simplifies the implementation of an interface; refer to chapter “8.4.4 Adapter Classes” on page 277 for some details.
...Base	A base class of an interface implementation, usually abstract. Usually, it is either published as part of the SPI for a module, or it is only visible inside that module.
...Command	An implementation of a command for the Command pattern; use either this or ‘Operation’ as the suffix.
...DAO	...
...Entity	...
...Error	An implementation of <code>java.lang.Error</code> ; basically an unchecked exception that is thrown in case of an unrecoverable error that should never be caught nor handled internally.
...Exception	An implementation of either <code>java.lang.Exception</code> (for a checked exception) or <code>java.lang.RuntimeException</code> (for an unchecked exception).
...Handler	...
...Impl	The (default) implementation for an interface or an abstract base class, usually not visible outside of the current module.
...Listener	An implementation of a listener for the Observer ³ pattern.
...Operation	An implementation of a command for the Command pattern; use either this or ‘Command’ as the suffix.
...Service	...
...Visitor	The implementation of a visitor for the Visitor pattern.

Continued on next page

³sometimes also referred to as the “Listener” pattern



Table 9.2 – Continued from previous page

Suffix	Description
	t.b.c.

9.2. Configurable Errors and Warnings

A very convenient feature of most IDE's is the capability to configure additional warnings and even errors for the compilation.

9.2.1. Eclipse

tdb

9.2.2. JetBrains IntelliJ IDEA

tdb

9.3. IDE Configuration

This chapter provides samples of configuration files for some IDEs. See also the chapter 9.2 on page 463 about the errors and warnings that can be configured in Eclipse and IntelliJ IDEA.

9.3.1. Eclipse

tbd



Snippets

This chapter provides the XML code for Eclipse snippets.

Structuring Comments The snippets for the structuring comments as defined in chapter “7.2.1 Structuring Comments” on page 170.

```

<?xml version="1.0"
      encoding="UTF-16"
      standalone="no"?>
<snippets>
  <category filters="*"
            id="category_1145179107125"
            initial_state="0"
            label="Structuring_Comments"
            largeicon=""
            smallicon="">
    <description><![CDATA[Structuring Comments as defined by the Code
                          Conventions]]></description>
    <item category="category_1145179107125"
          class=""
          editorclass=""
          id="item_1145232938375"
          label="Enum_Declaration"
          largeicon=""
          smallicon="">
      <description><![CDATA[The header comment for the enum
                          definition part]]></description>
      <content><![CDATA[
=====** Enum Definitions
**=====
\*-----*/
]]></content>
    </item>
    <item category="category_1145179107125"
          class=""
          editorclass=""
          id="item_1145179869843"
          label="Inner_Classes"
          largeicon=""
          smallicon="">
      <description><![CDATA[The header comment for the inner classes
                          part]]></description>

```




```

        <content><![CDATA[                /*-----*\
====** Inner Classes
**=====
\*-----*/
]]></content>
    </item>
    <item category="category_1145179107125"
        class=""
        editorclass=""
        id="item_1251889697104"
        label="Constants"
        largeicon=""
        smallicon="">
        <description><![CDATA[The part comment for
            constants.]]></description>
        <content><![CDATA[                /*-----*\
====** Constants
**=====
\*-----*/
]]></content>
    </item>
    <item category="category_1145179107125"
        class=""
        editorclass=""
        id="item_1251888677777"
        label="Attributes"
        largeicon=""
        smallicon="">
        <description><![CDATA[The part comment for
            attributes.]]></description>
        <content><![CDATA[                /*-----*\
====** Attributes
**=====
\*-----*/
]]></content>
    </item>
    <item category="category_1145179107125"
        class=""
        editorclass=""
        id="item_1145179436656"
        label="Static Initialisations"
        largeicon="" smallicon="">
        <description><![CDATA[The header comment for the static
            initialisations part]]></description>
        <content><![CDATA[                /*-----*\

```



```
====** Static Initialisations
**=====
/*-----*/
]]></content>
</item>
<item category="category_1145179107125"
      class=""
      editorclass=""
      id="item_1145180117906"
      label="Constructors"
      largeicon=""
      smallicon="">
  <description><![CDATA[The header comment for the constructors
    part]]></description>
  <content><![CDATA[          /*-----*\
====** Constructors
**=====
/*-----*/
]]></content>
</item>
<item category="category_1145179107125"
      class=""
      editorclass=""
      id="item_1145180168796"
      label="Methods"
      largeicon=""
      smallicon="">
  <description><![CDATA[The header comment for the methods
    part]]></description>
  <content><![CDATA[          /*-----*\
====** Methods
**=====
/*-----*/
]]></content>
</item>
</category>
</snippets>
```

9.3.2. JetBrains IntelliJ IDEA

tbd



9.4. Embedded Code

Sometimes, it is necessary to embed code inside the Java source code. Most often, these are SQL statements, but sometimes it could be also fragments of HTML or XML documents.

9.4.1. Formatting SQL inside Java

9.4.2. Formatting XML inside Java

You should embed only small fragments of an XML document into the Java source; larger fragments and full documents can be handled better when provided as resources.

9.4.3. Formatting HTML inside Java

Same as for XML, also only small HTML fragments should be embedded into the Java source code. Anything else should go into a resource file.

9.5. The Reason why Prefix Notation should be preferred over Postfix Notation

At chapter 8.59 on page 445 I recommend to use always the prefix version of an unary operator if the result of the operation is not further used. This has its reason in the implementation for the operators ++ and --. As a C++ method, the prefix implementation may look like this:

```
int prefixIncrement( int& n )
{
    n = n + 1;
    return n;
}
```



and the postfix version would look like this:

```
int postfixIncrement( int& n )
{
    int oldValue = n;
    n = n + 1;
    return oldValue;
}
```

So the postfix version has an additional stack operation that is not necessary if the return value is discarded anyway. This saves only nanoseconds for a single operation, but used in a loop, and in an application server environment, these nanos will sum up to minutes over time.

I confess that there are other possible locations where one can save more CPU cycles, and I also know the advice not to begin too early with optimisations (to “avoid premature optimisation”). My opinion is that this is not an optimisation but a best practice, and that every cycle counts – especially if it is that easy to achieve.

As said also already, most Java compilers (and not only these) will optimise the particular code, but different Java compilers optimises this pattern differently (meaning some even do not touch it – for example that one for the LEGO® Mindstorms Controller ...), and even the runtime optimisation will treat it differently, so just from this point of view I would recommend this simple change of one’s personal habits.

9.6. Examples

9.6.1. AutoLock

This class is a sample implementation of the idea described in chapter “8.14.4 Lifecycle” on page 384, like a PoC; a real life implementation can be found at [360].



The Code

Listing 9.1: AutoLock.java

```

1 package org.tquadrat.util.concurrent;
2
3 import static java.util.Objects.requireNonNull;
4 import java.util.concurrent.locks.Lock;
5
6 /**
7  * A wrapper for locks that supports the {@code try-with-resources}
8  * feature of Java 7.
9  * The creation of the local reference to the wrapper object means
10 * some overhead but in very most scenarios this is negligible.
11 *
12 * {@code AutoLock} will only expose the methods
13 * {@link #lock()}
14 * and
15 * {@link #lockInterruptibly()}
16 * of the interface
17 * {@link java.util.concurrent.locks.Lock Lock},
18 * but with a return value. Exposing other methods is not
19 * reasonable.
20 * Calling
21 * {@link #close()}
22 * on the {@code AutoLock} instance or
23 * {@link Lock#unlock()}
24 * on the wrapped {@code Lock} object inside the {@code try} block
25 * may cause unpredictable effects.
26 *
27 * @author Thomas Thrien - thomas.thrien@tquadrat.org
28 *
29 * @see java.util.concurrent.locks.Lock
30 */
31 public class AutoLock implements AutoCloseable
32 {
33     /*-----*\
34     ===** Attributes **=====
35     \*-----*/
36     /**
37      * The wrapped lock.
38      */
39     private final Lock m_Lock;
40
41     /*-----*\
42     ===** Constructors **=====

```



9. Appendices

```
43      /*-----*/
44      /**
45       * Creates a new {@code AutoLock} object.
46       *
47       * @param lock    The wrapped lock.
48       */
49      public AutoLock( final Lock lock )
50      {
51          m_Lock = requireNonNull( lock );
52      } // AutoLock()
53
54      /*-----*\
55      ===** Methods **=====
56      /*-----*/
57      /**
58       * {@inheritDoc}
59       */
60      @Override
61      public final void close() { m_Lock.unlock(); }
62
63      /**
64       * Calls
65       * {@link java.util.concurrent.locks.Lock#lock() lock()}
66       * on the wrapped
67       * {@link java.util.concurrent.locks.Lock}
68       * instance.
69       *
70       * @return The reference to this {@code AutoLock} instance.
71       */
72      public final AutoLock lock()
73      {
74          m_Lock.lock();
75
76          //---* Done *-----
77          return this;
78      } // lock()
79  }
80  // class AutoLock
```



9.6.2. Illegal Argument Exceptions

As said in chapter “8.3 Checking Method Parameters and Return Values” on page 250, a `NullPointerException` that is thrown from your code has to be seen as a coding bug: a value was not properly checked before it was used. Nevertheless, values can be `null`, for various reasons, and this still can be an error that needs to be signalled.

For that, I suggested a bunch of custom exceptions that are shown here; they are also part of my Foundation Library: for `ValidationException` see [352], the `NullArgumentException` can be found at [348], the `EmptyArgumentException` is documented at [346], and finally the documentation for `BlankArgumentException` is at [345].

The Code

Listing 9.2: `ValidationException.java`

```
1 package org.tquadrat.foundation.exception;
2
3 import static org.tquadrat.foundation.lang.Objects.isNull;
4
5 import java.io.Serial;
6
7 /**
8  * This is a specialized implementation for the
9  * {@link IllegalArgumentException}
10  * that is meant as the root for a hierarchy of exceptions caused by
11  * validation errors.
12  *
13  * @author Thomas Thrien - thomas.thrien@tquadrat.org
14  */
15 public class ValidationException extends IllegalArgumentException
16 {
17     /*-----*\
18     ===** Constants **=====
19     \*-----*/
20     /**
21      * The message text for a validation error.
22      */
23     public static final String MSG_ValidationFailed = "Validation_failed";
24 }
```



```
25      /*-----*\
26      ===** Static Initialisations **=====
27      \*-----*/
28      /**
29       * The serial version UID for objects of this class: {@value}.
30       *
31       * @hidden
32       */
33      @Serial
34      private static final long serialVersionUID = 1174360235354917591L;
35
36      /*-----*\
37      ===** Constructors **=====
38      \*-----*/
39      /**
40       * Creates a new {@code ValidationException} instance.
41       */
42      public ValidationException()
43      {
44          super( MSG_ValidationFailed );
45      } // ValidationException()
46
47      /**
48       * Creates a new {@code ValidationException} instance.
49       *
50       * @param message The message that provides details on the
51       *                failed validation.
52       */
53      public ValidationException( final String message )
54      {
55          super( isNull( message ) || message.isEmpty() ?
56                MSG_ValidationFailed : message );
57      } // ValidationException()
58
59      /**
60       * Creates a new {@code ValidationException} instance.
61       *
62       * @param message The message that provides details on the failed
63       *                validation.
64       * @param cause The exception that is related to the validation
65       *                error.
66       */
67      public ValidationException( final String message, final Throwable
68                                cause )
69      {
```




```

68     super( isNull( message ) || message.isEmpty() ?
        MSG_ValidationFailed : message, cause );
69 } // ValidationException()
70
71 /**
72  * Creates a new {@code ValidationException} instance.
73  *
74  * @param cause The exception that is related to the validation
75  * error.
76  */
77 public ValidationException( final Throwable cause )
78 {
79     super( MSG_ValidationFailed, cause );
80 } // ValidationException()
81 }
82 // class ValidationException

```

Listing 9.3: NullArgumentException.java

```

1 package org.tquadrat.foundation.exception;
2
3 import static org.tquadrat.foundation.lang.CommonConstants.EMPTY_STRING;
4 import static org.tquadrat.foundation.lang.Objects.nonNull;
5
6 import java.io.Serial;
7
8 /**
9  * This is a specialized implementation for the
10  * {@link IllegalArgumentException}
11  * that should be used instead of the latter in cases where {@code null}
12  * is
13  * provided as an illegal argument value.
14  *
15  * @author Thomas Thrien - thomas.thrien@tquadrat.org
16  */
17 public sealed class NullArgumentException extends ValidationException
18     permits BlankArgumentException, EmptyArgumentException
19 {
20     /*-----*\
21     ===** Static Initialisations **=====
22     \*-----*/
23
24     /**
25      * The serial version UID for objects of this class: {@value}.
26      *
27      * @hidden
28      */
29 }

```



```
26      */
27      @Serial
28      private static final long serialVersionUID = 1174360235354917591L;
29
30      /*-----*\
31      ===** Constructors **=====
32      \*-----*/
33      /**
34       * Creates a new instance of {@code NullArgumentException}.
35       */
36      public NullArgumentException() { this( null ); }
37
38      /**
39       * Creates a new instance of {@code NullArgumentException}.
40       *
41       * @param argName The name of the argument whose value was
42       * provided as {@code null}; if {@code null} or the empty
43       * String, a default message is used that does not use the
44       * name of the argument.
45       */
46      public NullArgumentException( final String argName )
47      {
48          this( argName, "Argument_,'%1$s'_must_not_be_null", "Argument_must_
49              not_be_null" );
50      } // NullArgumentException()
51
52      /**
53       * <p>{@summary Creates a new instance of
54       * {@code NullArgumentException}.}</p>
55       * <p>This constructor was introduced for the
56       * {@link EmptyArgumentException}.</p>
57       *
58       * @param argName The name of the argument whose value was
59       * provided as {@code null}; if {@code null} or the empty
60       * String, a default message is used that does not use the
61       * name of the argument.
62       * @param msgName The regular message.
63       * @param msgNone The default message.
64       */
65      protected NullArgumentException( final String argName, final String
66          msgName, final String msgNone )
67      {
68          super( nonNull( argName ) && !argName.isEmpty() ?
69              msgName.formatted( argName ) : msgNone );
70      } // NullArgumentException()
```



```

68 }
69 // class NullArgumentException

```

Listing 9.4: EmptyArgumentException.java

```

1  package org.tquadrat.foundation.exception;
2
3  import java.io.Serial;
4
5  /**
6   * This is a specialized implementation for the
7   * {@link IllegalArgumentException}
8   * that should be used instead of the latter in cases where an empty
9   * String, array or
10  * {@link java.util.Collection}
11  * argument is provided as an illegal argument value.
12  *
13  * @author Thomas Thrien - thomas.thrien@tquadrat.org
14  */
15  public final class EmptyArgumentException extends NullArgumentException
16  {
17      /*-----*\
18      ===** Static Initialisations **=====
19      \*-----*/
20      /**
21       * The serial version UID for objects of this class: {@value}.
22       *
23       * @hidden
24       */
25      @Serial
26      private static final long serialVersionUID = 1174360235354917591L;
27
28      /*-----*\
29      ===** Constructors **=====
30      \*-----*/
31      /**
32       * Creates a new instance of {@code EmptyArgumentException}.
33       */
34      public EmptyArgumentException() { this( null ); }
35
36      /**
37       * Creates a new instance of {@code EmptyArgumentException}.
38       *
39       * @param argName The name of the argument whose value was
40       * provided as empty; if {@code null} or the empty String,

```



9. Appendices

```
41      *      a default message is used that does not use the name of
42      *      the argument.
43      */
44      public EmptyArgumentException( final String argName )
45      {
46          super( argName, "Argument_ '%1$s' must not be empty", "Argument_
47              must not be empty" );
48      } // EmptyArgumentException()
49      // class EmptyArgumentException
```

Listing 9.5: BlankArgumentException.java

```
1  package org.tquadrat.foundation.exception;
2
3  import java.io.Serial;
4
5  /**
6   * This is a specialized implementation for the
7   * {@link IllegalArgumentException}
8   * that should be used instead of the latter in cases where a blank
9   * String is provided as an illegal argument value.
10  *
11  * @author Thomas Thrien - thomas.thrien@tquadrat.org
12  */
13  public final class BlankArgumentException extends NullArgumentException
14  {
15      /*-----*\
16      ===** Static Initialisations **=====
17      \*-----*/
18      /**
19       * The serial version UID for objects of this class: {@value}.
20       *
21       * @hidden
22       */
23      @Serial
24      private static final long serialVersionUID = 1174360235354917591L;
25
26      /*-----*\
27      ===** Constructors **=====
28      \*-----*/
29      /**
30       * Creates a new instance of {@code BlankArgumentException}.
31       */
32      public BlankArgumentException() { this( null ); }
```



```

33
34  /**
35   *  Creates a new instance of {@code BlankArgumentException}.
36   *
37   *  @param  argName The name of the argument whose value was
38   *               provided as blank; if {@code null} or the empty String,
39   *               a default message is used that does not use the name of
40   *               the argument.
41   */
42  public BlankArgumentException( final String argName )
43  {
44      super( argName, "Argument_ '%1$s' must_not_be_blank", "Argument_
45             must_not_be_blank" );
46  } // BlankArgumentException()
47 // class BlankArgumentException

```

9.6.3. Lazy

The interface **Lazy** and the associated implementation **LazyImpl** provide a holder for a lazy initialised object instance. The initialisation happens on the first call to the method **Lazy::get** through a call to the **Supplier** instance the **Lazy** instance was created with.

```

public final class MyClass
{
    /**
     *  An attribute of type
     *  {@link AClass}.
     */
    private final Lazy<AClass> m_Attribute;

    public MyClass()
    {
        m_Attribute = Lazy.use( ()-> new AClass( this ) );
    }

    public final void myMethod()
    {
        m_Attribute.get().aMethod();
    } // myMethod()
}

```



```
// class MyClass
```

Lazy is part of my Foundation Library and can be found at [353].

The Code

Listing 9.6: Lazy.java

```
1 package org.tquadrat.foundation.lang;
2
3 import static org.tquadrat.foundation.lang.Objects.requireNonNull;
4
5 import java.util.Optional;
6 import java.util.function.Consumer;
7 import java.util.function.Function;
8 import java.util.function.Supplier;
9
10 import org.apiguardian.api.API;
11 import org.tquadrat.foundation.annotation.ClassVersion;
12 import org.tquadrat.foundation.lang.internal.LazyImpl;
13
14 /**
15  * <p>{@summary A holder for a lazy initialised object
16  * instance.}</p>
17  * <p>An instance of an implementation of this interface holds a
18  * value that will be initialised by a call to the supplier
19  * (provided with the method
20  * {@link #use(Supplier)})
21  * on a first call to
22  * {@link #get()}.</p>
23  * <p>Use
24  * {@link #isPresent()}
25  * to avoid unnecessary initialisation.</p>
26  * <p>As a lazy initialisation makes the value unpredictable, it is
27  * necessary that the implementations of
28  * {@link #equals(Object)}
29  * and
30  * {@link #hashCode()}
31  * do force the initialisation.</p>
32  * <p>{@link #toString()}
33  * will not force the initialisation.</p>
34  * <p>This interface is not feasible for the lazy initialisation of
35  * connections to resources of any kind, as these will usually
36  * throw (checked) exceptions when problems arise, and usually these
```



```

37 * should be handled on one central location.</p>
38 *
39 * @author Thomas Thrien - thomas.thrien@tquadrat.org
40 *
41 * @param <T> The type of the value for this instance of
42 * {@code Lazy}.
43 */
44 public sealed interface Lazy<T>
45     permits org.tquadrat.foundation.lang.internal.LazyImpl
46 {
47     /*-----*\
48     ===** Methods **=====
49     \*-----*/
50     /**
51      * {@inheritDoc}
52      */
53     @Override
54     public boolean equals( final Object obj );
55
56     /**
57      * Returns the value for this instance of {@code Lazy}.
58      *
59      * @return The value.
60      */
61     public T get();
62
63     /**
64      * {@inheritDoc}
65      */
66     @Override
67     public int hashCode();
68
69     /**
70      * If this {@code Lazy} instance has been initialised already,
71      * the provided
72      * {@link Consumer}
73      * will be executed; otherwise nothing happens.
74      *
75      * @param consumer The consumer.
76      */
77     public default void ifPresent( final Consumer<? super T> consumer )
78     {
79         if( isPresent() ) requireNonNullArgument( consumer, "consumer"
80         ).accept( get() );
81     } // ifPresent()

```



9. Appendices

```
81
82  /**
83   * Checks whether this {@code Lazy} instance has been
84   * initialised already. But even it was initialised,
85   * {@link #get()}
86   * may still return {@code null}.
87   *
88   * @return {@code true} if the instance was initialised,
89   *         {@code false} otherwise.
90   */
91  public boolean isPresent();
92
93  /**
94   * If this instance of {@code Lazy} is initialised, the provided
95   * mapper function will be executed on the value.
96   *
97   * @param <R> The result type for the mapper function.
98   * @param mapper The mapper function.
99   * @return An instance of
100   *         {@link Optional}
101   *         that holds the result for the mapping.
102   */
103  public default <R> Optional<R> map( final Function<? super T,? extends
104                                     R> mapper )
105  {
106      final Optional<R> retValue = isPresent() ? Optional.ofNullable(
107          mapper.apply( get() ) ) : Optional.empty();
108
109      //---* Done *-----
110      return retValue;
111  } // map()
112
113  /**
114   * Returns the value or throws the exception that is created by
115   * the given
116   * {@link Supplier}
117   * when not yet initialised.
118   *
119   * @param <X> The type of the implementation of
120   *             {@link Throwable}.
121   * @param exceptionSupplier The supplier for the exception
122   *                           to throw when the instance was not yet initialised.
123   * @return The value.
124   * @throws X When not initialised, the exception created by
125   *           the given supplier will be thrown.
```




```

124  */
125  public <X extends Throwable> T orElseThrow( Supplier<? extends X>
      exceptionSupplier ) throws X;
126
127  /**
128   * {@inheritDoc}
129   */
130  @Override
131  public String toString();
132
133  /**
134   * Creates a new {@code Lazy} instance that uses the given
135   * supplier to initialise.
136   *
137   * @param <T> The type of the value for the new instance of
138   *           {@code Lazy}.
139   * @param supplier The supplier that initialises the value
140   *               for this instance on the first call to
141   *               {@link #get()}.
142   * @return The new instance.
143   */
144  public static <T> Lazy<T> use( final Supplier<T> supplier )
145  {
146      return new LazyImpl<>( supplier );
147  } // use()
148 }
149 // interface Lazy

```

Listing 9.7: LazyImpl.java

```

1  package org.tquadrat.foundation.lang.internal;
2
3  import static org.tquadrat.foundation.lang.CommonConstants.NULL_STRING;
4  import static org.tquadrat.foundation.lang.Objects.isNull;
5  import static org.tquadrat.foundation.lang.Objects.nonNull;
6  import static org.tquadrat.foundation.lang.Objects.requireNonNullArgument;
7
8  import java.util.Optional;
9  import java.util.function.Supplier;
10
11 import org.tquadrat.foundation.lang.AutoLock;
12 import org.tquadrat.foundation.lang.Lazy;
13 import org.tquadrat.foundation.lang.Objects;
14
15 /**

```



9. Appendices

```
16  * <p>{@summary The implementation of the interface
17  * {@link Lazy}.}</p>
18  *
19  * @author Thomas Thrien - thomas.thrien@tquadrat.org
20  *
21  * @param <T> The type of the value for this instance of {@code Lazy}.
22  */
23  public final class LazyImpl<T> implements Lazy<T>
24  {
25      /*-----*\
26      ===** Attributes **=====
27      \*-----*/
28      /**
29       * The lock that protects the value creation. It will be set to
30       * {@linkplain Optional#empty() empty}
31       * after
32       * {@link #m_Value} is initialised.
33       */
34      @SuppressWarnings( "OptionalUsedAsFieldOrParameterType" )
35      private Optional<AutoLock> m_Lock;
36
37      /**
38       * The supplier for the value of this {@code Lazy} instance. It
39       * will be set to {@code null} after
40       * {@link #m_Value}
41       * is initialised.
42       */
43      private Supplier<T> m_Supplier;
44
45      /**
46       * The value of this {@code Lazy} instance; it is {@code null}
47       * if it was not yet initialised.
48       */
49      private T m_Value;
50
51      /*-----*\
52      ===** Constructors **=====
53      \*-----*/
54      /**
55       * Creates a new {@code Lazy} instance.
56       *
57       * @param supplier The supplier that initialises the value
58       * for this instance on the first call to
59       * {@link #get()}.
60       */
```



```

61 public LazyImpl( final Supplier<T> supplier )
62 {
63     m_Supplier = requireNonNullArgument( supplier, "supplier" );
64     m_Lock = Optional.of( AutoLock.of() );
65     m_Value = null;
66 } // LazyImpl()
67
68 /*-----*\
69 ==** Methods **=====
70 \*-----*/
71 /**
72  * {@inheritDoc}
73  */
74 @Override
75 public final boolean equals( final Object obj )
76 {
77     var retValue = this == obj;
78     if( !retValue && nonNull( obj ) )
79     {
80         if( obj instanceof Lazy other )
81         {
82             retValue = get().equals( other.get() );
83         }
84         else
85         {
86             retValue = get().equals( obj );
87         }
88     }
89
90     //---* Done *-----
91     return retValue;
92 } // equals()
93
94 /**
95  * {@inheritDoc}
96  */
97 @Override
98 public final T get()
99 {
100     m_Lock.ifPresent( l ->
101     {
102         try( @SuppressWarnings( "unused" ) final var lock = l.lock() )
103         {
104             /*
105              * When the lock is present, the supplier must have

```



9. Appendices

```
106         * been not null; this is enforced by the respective
107         * constructor.
108         * So when the supplier is NOW null, another thread
109         * has performed the initialisation while this one
110         * was waiting for the lock.
111         */
112         if( nonNull( m_Supplier ) )
113         {
114             m_Value = m_Supplier.get();
115             m_Lock = Optional.empty();
116             m_Supplier = null;
117         }
118     }
119 } );
120
121     //---* Done *-----
122     return m_Value;
123 } // get()
124
125 /**
126  * {@inheritDoc}
127  */
128 @Override
129 public final int hashCode() { return get().hashCode(); }
130
131 /**
132  * {@inheritDoc}
133  */
134 @Override
135 public final boolean isPresent() { return isNull( m_Supplier ); }
136
137 /**
138  * {@inheritDoc}
139  */
140 @Override
141 public <X extends Throwable> T orElseThrow( final Supplier<? extends
142     X> exceptionSupplier ) throws X
143 {
144     if( !isPresent() ) throw exceptionSupplier.get();
145
146     //---* Done *-----
147     return m_Value;
148 } // orElseThrow()
149
150 /**
```



```

150     * {@inheritDoc}
151     */
152     @Override
153     public final String toString()
154     {
155         final var retValue = Objects.toString( m_Value,
156             isPresent()
157             ? NULL_STRING
158             : "[Not_initialized]" );
159
160         //---* Done *-----
161         return retValue;
162     } // toString()
163 }
164 // class LazyImpl

```

9.6.4. Load JDK Logging Configuration

The default configuration for the JDK Logging can be found in the configuration file `${JAVA_HOME}/conf/logging.properties`; if you want to use a different file, you can announce that by providing the JVM command line argument `-Djava.util.logging.config.file=<filename>`. This also allows you to easily replace one logging configuration by another.

But in most cases, you want that a fixed logging configuration is used. You can enforce that by providing the configuration file as a resource and load that in the class with the method `main()` like this:

Listing 9.8: Load Logging Configuration from Resource

```

1  ...
2
3      /*-----*\
4  ===** Static Initialisations **=====
5      \*-----*/
6  /**
7   * The logger for this class.
8   */
9  private static final Logger m_Logger;
10
11  static

```



9. Appendices

```
12 {
13     //---* Load the logger configuration and create the logger *-----
14     final var c = Main.class;
15     Logger logger;
16     final var resourceURL = c.getResource( "logging.properties" );
17     if( nonNull( resourceURL )
18     {
19         try( final var logConfiguration = resourceURL.openStream() )
20         {
21             getLogManager().readConfiguration( logConfiguration );
22             logger = getLogger( c.getName() );
23         }
24         catch( final IOException e )
25         {
26             m_Logger = getAnonymousLogger();
27             throw new ExceptionInInitializerError( e );
28         }
29     }
30     else
31     {
32         m_Logger = getAnonymousLogger();
33         throw new ExceptionInInitializerError( "Cannot_find_resource_
34             'logging.properties'" );
35     }
36     m_Logger = logger;
37 }
38 ...
```

Or if you want to read it still from an external file, it can be done like this:

Listing 9.9: Load Logging Configuration from File

```
1 ...
2
3     /*-----*|
4 ====** Static Initialisations **=====
5 |*-----*/
6 /**
7  * The logger for this class.
8  */
9 private static final Logger m_Logger;
10
11 static
12 {
```



```
13  //---* Load the logger configuration and create the logger *-----
14  Logger logger;
15  try( final var logConfiguration = newInputStream( Path.of( ".",
16      "logging.properties" ), READ ) )
17  {
18      getLogManager().readConfiguration( logConfiguration );
19      logger = getLogger( Main.class.getName() );
20  }
21  catch( final IOException e )
22  {
23      m_Logger = getAnonymousLogger();
24      throw new ExceptionInInitializerError( e );
25  }
26  m_Logger = logger;
27  }
28  ...
```

The method `LogManager::readConfiguration`[\[297\]](#) should be invoked before the first logger is configured, so the code above should be placed in the static initialisation part of that class of your application that is loaded first - usually this is the class that contains the method `main()`.

9.6.5. MountPoint

The annotation `@MountPoint` and how to use it is described in the chapters “8.4.2 Non-final Classes” on page 266 and “8.4.3 Non-final Methods” on page 267.

The annotation is part of my foundation library, refer to [\[340\]](#).

The Code

Listing 9.10: MountPoint.java

```
1  package org.tquadrat.foundation.annotation;
2
3  import static java.lang.annotation.ElementType.METHOD;
4  import static java.lang.annotation.RetentionPolicy.SOURCE;
5
```



```
6 import java.lang.annotation.Documented;
7 import java.lang.annotation.Retention;
8 import java.lang.annotation.Target;
9
10 /**
11  * The marker annotation for methods that are meant to be
12  * overwritten in child classes.
13  *
14  * @author Thomas Thrien - thomas.thrien@tquadrat.org
15  */
16 @Documented
17 @Retention( SOURCE )
18 @Target( METHOD )
19 public @interface MountPoint
20 {
21     /**
22      * Optionally provides a short description on how to use this
23      * mount-point.
24      *
25      * @return The description of the mount point.
26      */
27     String value() default "";
28 }
29 // @interface MountPoint
```

9.6.6. Patch Identification

The chapter “7.3 Maintenance Comments” on page 181 discussed how to mark areas in the code that has been changed to fix a bug. There I suggested to use annotations for this task.

This can look like this:

```
1 @BUG( id = "BUG-123456", comment = "Introduced_base_class_MyClassBase" )
2 public final class MyClass extends MyClassBase
3 {
4     @BUG( id = "BUG-100000", comment = "Made_generic;_added_Typ_'String'" )
5     @BUG( id = "BUG-100003", comment = "Changed_type_to_'CharSequence'" )
6     private final List<CharSequence> m_Texts = new LinkedList<>();
7
8     @BUG( id = "BUG-123456", comment = "Introduced_base_class_
      MyClassBase;_calling_super()" )
```




```

9      public MyClass()
10     {
11         super();
12     } // MyClass()
13
14     @BUG( id = "BUG-100003", comment = "Changed_type_to_'CharSequence'" )
15     @BUG( id = "BUG-100022", comment = "Made_generic;_changed_to_'extends_'_CharSequence'" )
16     @BUG( id = "BUG-123456", comment = "Introduced_base_class_MyClassBase;_added_@Override" )
17     @Override
18     public final <T extends CharSequence> void addText( final T text ) { ...
19     }
20 // class MyClass

```

That the same annotation can be applied multiple times to the same element requires a “container annotation” for that annotation[163], but the use of this annotation is implicit.

The annotations `@BUG` and `@FixList` (the “container annotation”) are part of the Foundation Base project[356, 337, 339].

The Code

Listing 9.11: FixList.java

```

1 package org.tquadrat.foundation.annotation;
2
3 import static java.lang.annotation.ElementType.ANNOTATION_TYPE;
4 import static java.lang.annotation.ElementType.CONSTRUCTOR;
5 import static java.lang.annotation.ElementType.FIELD;
6 import static java.lang.annotation.ElementType.LOCAL_VARIABLE;
7 import static java.lang.annotation.ElementType.METHOD;
8 import static java.lang.annotation.ElementType.MODULE;
9 import static java.lang.annotation.ElementType.PACKAGE;
10 import static java.lang.annotation.ElementType.PARAMETER;
11 import static java.lang.annotation.ElementType.RECORD_COMPONENT;
12 import static java.lang.annotation.ElementType.TYPE;
13 import static java.lang.annotation.ElementType.TYPE_PARAMETER;
14 import static java.lang.annotation.ElementType.TYPE_USE;
15 import static java.lang.annotation.RetentionPolicy.RUNTIME;
16

```



```
17 import java.lang.annotation.Documented;
18 import java.lang.annotation.Retention;
19 import java.lang.annotation.Target;
20
21 /**
22  * The annotation container for
23  * {@link BUG &#64;BUG}
24  * annotations.
25  *
26  * @author Thomas Thrien - thomas.thrien@tquadrat.org
27  */
28 @Documented
29 @Retention( RUNTIME )
30 @Target( {ANNOTATION_TYPE, CONSTRUCTOR, FIELD, LOCAL_VARIABLE, METHOD,
31           MODULE, PACKAGE, PARAMETER, RECORD_COMPONENT, TYPE, TYPE_PARAMETER,
32           TYPE_USE} )
33 public @interface FixList
34 {
35     /*-----*\
36     ===** Attributes **=====
37     \*-----*/
38     /**
39      * Provides the list of
40      * {@link BUG &#64;BUG}
41      * annotations.
42      *
43      * @return The annotations.
44      */
45     public BUG [] value();
46 }
47 // @interface FixList
```

Listing 9.12: BUG.java

```
1 package org.tquadrat.foundation.annotation;
2
3 import static java.lang.annotation.ElementType.ANNOTATION_TYPE;
4 import static java.lang.annotation.ElementType.CONSTRUCTOR;
5 import static java.lang.annotation.ElementType.FIELD;
6 import static java.lang.annotation.ElementType.LOCAL_VARIABLE;
7 import static java.lang.annotation.ElementType.METHOD;
8 import static java.lang.annotation.ElementType.MODULE;
9 import static java.lang.annotation.ElementType.PACKAGE;
10 import static java.lang.annotation.ElementType.PARAMETER;
11 import static java.lang.annotation.ElementType.RECORD_COMPONENT;
```



```

12 import static java.lang.annotation.ElementType.TYPE;
13 import static java.lang.annotation.ElementType.TYPE_PARAMETER;
14 import static java.lang.annotation.ElementType.TYPE_USE;
15 import static java.lang.annotation.RetentionPolicy.RUNTIME;
16
17 import java.lang.annotation.Documented;
18 import java.lang.annotation.Repeatable;
19 import java.lang.annotation.Retention;
20 import java.lang.annotation.Target;
21
22 /**
23  * This annotation allows to add information about applied fixes to
24  * a program element.
25  *
26  * @author Thomas Thrien - thomas.thrien@tquadrat.org
27  */
28 @Documented
29 @Retention( RUNTIME )
30 @Target( {ANNOTATION_TYPE, CONSTRUCTOR, FIELD, LOCAL_VARIABLE, METHOD,
31          MODULE, PACKAGE, PARAMETER, RECORD_COMPONENT, TYPE, TYPE_PARAMETER,
32          TYPE_USE} )
31 @Repeatable( FixList.class )
32 public @interface BUG
33 {
34     /*-----*\
35     ===** Attributes **=====
36     \*-----*/
37     /**
38      * An optional comment regarding the bug fix.
39      *
40      * @return The comment.
41      */
42     String comment() default "";
43
44     /**
45      * The BUG id as provided by the bug tracking system.
46      *
47      * @return The BUG id.
48      */
49     String id();
50 }
51 // @interface BUG

```



9.6.7. PostProcessor

This implementation is basically a PoC; currently it is not part of any library.

The Code

Listing 9.13: PostProcessor.java

```
1 package util;
2
3 import static java.util.Objects.requireNonNull;
4
5 /**
6  * Use this class to implement an unconditional post-processing
7  * feature utilising try-with-resources.
8  *
9  * @author Thomas Thrien - thomas.thrien@tquadrat.org
10 */
11 public class PostProcessor implements AutoCloseable
12 {
13     /*-----*\
14     ===** Attributes **=====
15     \*-----*/
16     /**
17      * The action.
18      */
19     private final Runnable m_Action;
20
21     /*-----*\
22     ===** Constructors **=====
23     \*-----*/
24     /**
25      * Creates a new {@code PostProcessor} object.
26      *
27      * @param action The action that has to executed.
28      */
29     public PostProcessor( final Runnable action )
30     {
31         m_Action = requireNonNull( action );
32     } // PostProcessor()
33
34     /*-----*\
```



```

35  =====** Methods **=====
36  \*-----*/
37  /**
38   *  Calls the
39   *  {@link Runnable#run() run()}
40   *  method of the
41   *  {@linkplain #m_Action action}.
42   *
43   *  @see java.lang.AutoCloseable#close()
44   */
45  @Override
46  public void close() throws Exception { m_Action.run(); }
47  }
48  // class PostProcessor

```

9.6.8. ThreadGroup

The class `java.lang.ThreadGroup`^[70] provides a method `uncaughtException()` that has the same signature as the `UncaughtExceptionHandler::uncaughtException`^[277] method.

The method `java.lang.ThreadGroup:uncaughtException` is called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and no specific `Thread.UncaughtExceptionHandler`^[189] instance was installed to that thread.

The default implementation of that method does the following:

1. If this thread group has a parent thread group, the `uncaughtException()` method of that parent is called with the same two arguments.
2. Otherwise, this method checks to see if there is a default uncaught exception handler installed, and if so, its `uncaughtException()` method is called with the same two arguments.
3. Otherwise, this method determines if the `Throwable` argument is an instance of `java.lang.ThreadDeath`. If so, nothing special is done.

Otherwise, a message containing the thread's name, as returned from the thread's `getName()` method, and a stack backtrace, using the `Throwable`'s `printStackTrace()` method, is printed to the standard error stream.



Unfortunately, the class `java.lang.ThreadGroup` itself does not provide an API to change the behaviour of its `uncaughtException()` method. To do that, you have to override that method in a subclass.

Below you find an implementation of `ThreadGroup` that fixes that. A similar class is also part of my Foundation library – see [356, 344].

The Code

Listing 9.14: ThreadGroupExt.java

```
1 package org.tquadrat.foundation.lang;
2
3 import static org.tquadrat.foundation.lang.Objects.isNull;
4 import static org.tquadrat.foundation.lang.Objects.requireNonNullArgument;
5 import static org.tquadrat.foundation.lang.Objects.requireNotBlankArgument;
6
7 import java.lang.Thread.UncaughtExceptionHandler;
8
9 /**
10  * <p>{@summary An implementation of
11  *   {@link ThreadGroup}
12  *   that allows to configure the behaviour of
13  *   {@link #uncaughtException(Thread, Throwable)}}.</p>
14  *   <p>This class is not final, to allow further modifications.</p>
15  *
16  *   @author Thomas Thrien - thomas.thrien@tquadrat.org
17  */
18 public class ThreadGroupExt extends ThreadGroup
19 {
20     /*-----*\
21     ===** Attributes **=====
22     \*-----*/
23     /**
24      * The
25      *   {@link java.lang.Thread.UncaughtExceptionHandler}
26      *   for this thread group. It can be {@code null}.
27      */
28     private final UncaughtExceptionHandler m_UncaughtExceptionHandler;
29
30     /*-----*\
31     ===** Constructors **=====
32     \*-----*/
33     /**
```



```

34  * <p>{@summary Constructs a new thread group.}</p>
35  * <p>The parent of this new group is the thread group of the
36  * currently running thread.</p>
37  * <p>The behaviour of
38  * {@link #uncaughtException(Thread, Throwable)}
39  * is that of the superclass.</p>
40  *
41  * @param name    The name of the new thread group.
42  * @throws IllegalArgumentException    The {@code name} argument
43  *     is either {@code null}, empty or blank.
44  * @throws SecurityException    The current thread cannot create
45  *     a thread in this thread group.
46  */
47  public ThreadGroupExt( final String name ) throws
    IllegalArgumentException
48  {
49      super( requireNotBlankArgument( name, "name" ) );
50      m_UncaughtExceptionHandler = null;
51  } // ThreadGroupExt()
52
53  /**
54  * <p>{@summary Constructs a new thread group.}</p>
55  * <p>The parent of this new group is the specified thread
56  * group.</p>
57  * <p>The behaviour of
58  * {@link #uncaughtException(Thread, Throwable)}
59  * is that of the superclass.</p>
60  *
61  * @param parent    The parent thread group.
62  * @param name    The name of the new thread group.
63  * @throws IllegalArgumentException    The {@code name} argument
64  *     is either {@code null}, empty or blank, or the
65  *     {@code parent} thread group argument is {@code null}.
66  * @throws SecurityException    The current thread cannot create
67  *     a thread in this thread group.
68  */
69  public ThreadGroupExt( final ThreadGroup parent, final String name )
    throws IllegalArgumentException
70  {
71      super( requireNonNullArgument( parent, "parent" ),
72            requireNotBlankArgument( name, "name" ) );
73      m_UncaughtExceptionHandler = null;
74  } // ThreadGroupExt()
75  /**

```



9. Appendices

```
76      * <p>{@summary Constructs a new thread group.}</p>
77      * <p>The parent of this new group is the specified thread
78      * group.</p>
79      * <p>The behaviour of
80      * {@link #uncaughtException(Thread, Throwable)}
81      * is determined by the provided handler.</p>
82      *
83      * @param name    The name of the new thread group.
84      * @param handler The handler for the uncaught exceptions.
85      * @throws IllegalArgumentException The {@code name} argument
86      *         is either {@code null}, empty or blank, or the
87      *         {@code handler} argument is {@code null}.
88      * @throws SecurityException The current thread cannot create
89      *         a thread in this thread group.
90      */
91      public ThreadGroupExt( final String name, final
92                          UncaughtExceptionHandler handler ) throws IllegalArgumentException
93      {
94          super( requireNotBlankArgument( name, "name" ) );
95          m_UncaughtExceptionHandler = requireNonNullArgument( handler,
96                          "handler" );
97      } // ThreadGroupExt()
98
99      /**
100     * <p>{@summary Constructs a new thread group.}</p>
101     * <p>The parent of this new group is the specified thread
102     * group.</p>
103     * <p>The behaviour of
104     * {@link #uncaughtException(Thread, Throwable)}
105     * is determined by the provided handler.</p>
106     *
107     * @param parent The parent thread group.
108     * @param name    The name of the new thread group.
109     * @param handler The handler for the uncaught exceptions.
110     * @throws IllegalArgumentException The {@code name} argument
111     *         is either {@code null}, empty or blank, or one of the
112     *         {@code parent} thread group or the {@code handler}
113     *         arguments is {@code null}.
114     * @throws SecurityException The current thread cannot create
115     *         a thread in this thread group.
116     */
117     public ThreadGroupExt( final ThreadGroup parent, final String name,
118                         final UncaughtExceptionHandler handler ) throws
119                         IllegalArgumentException
120     {
```




```

117     super( requireNonNullArgument( parent, "parent" ),
118           requireNotBlankArgument( name, "name" ) );
119     m_UncaughtExceptionHandler = requireNonNullArgument( handler,
120                                                         "handler" );
121 } // ThreadGroupExt()
122
123 /*-----*\
124 ==** Methods **=====
125 \*-----*/
126 /**
127  * {@inheritDoc}
128  */
129 @Override
130 public final void uncaughtException( final Thread t, final Throwable e
131 )
132 {
133     if( isNull( m_UncaughtExceptionHandler ) )
134     {
135         super.uncaughtException( t, e );
136     }
137     else
138     {
139         m_UncaughtExceptionHandler.uncaughtException( t, e );
140     }
141 } // uncaughtException()
142 }
143 // class ThreadGroupExt

```

9.6.9. UnsupportedEnumError

This implementation of `java.lang.Error` is meant to be used in the `default` branch of a `switch` statement (refer to “5.4.2 ‘switch’ Statements” on page 95), in cases where the selector is an enum.

It will be used like this:

```

1 enum Color
2 {
3     RED, BLUE, GREEN, YELLOW
4 }
5
6 Color color = ...

```



```
7
8 // Traditional switch statement
9 switch( color )
10 {
11     case RED: ...; break;
12     case BLUE: ...; break;
13     case GREEN: ...; break;
14     case YELLOW: ...; break;
15
16     default: throw new UnsupportedOperationException( color );
17 }
18
19 // New switch statement
20 switch( color )
21 {
22     case RED -> ...;
23     case BLUE -> ...;
24     case GREEN -> ...;
25     case YELLOW -> ...;
26
27     default: throw new UnsupportedOperationException( color );
28 }
29
30 // switch expression
31 var result = switch( color )
32 {
33     case RED -> "Rot";
34     case BLUE -> "Blau";
35     case GREEN -> "Grün";
36     case YELLOW -> "Gelb";
37
38     default: throw new UnsupportedOperationException( color );
39 }
```

Also refer to [351].

The Code

Listing 9.15: UnsupportedOperationException.java

```
1 package org.tquadrat.foundation.exception;
2
3 import static org.tquadrat.foundation.lang.Objects.requireNonNullArgument;
4 import static org.tquadrat.foundation.lang.Objects.requireNotEmptyArgument;
```



```

5 import static org.tquadrat.foundation.lang.internal.SharedFormatter.format;
6
7 import java.io.Serial;
8
9 /**
10  * This is a specialized implementation for
11  * {@link Error}
12  * that is to be thrown especially from the {@code default} branch
13  * of a {@code switch} statement that uses an {@code enum} type as
14  * selector.
15  *
16  * @author Thomas Thrien - thomas.thrien@tquadrat.org
17  */
18 public final class UnsupportedEnumError extends Error
19 {
20     /*-----*\
21     ===** Constants **=====
22     \*-----*/
23     /**
24      * The message text.
25      */
26     private static final String MSG_UnsupportedEnum = "The_value_'%2$s'_of_
enum_class_'%1$s'_is_not_supported";
27
28     /*-----*\
29     ===** Static Initialisations **=====
30     \*-----*/
31     /**
32      * The serial version UID for objects of this class: {@value}.
33      *
34      * @hidden
35      */
36     @Serial
37     private static final long serialVersionUID = 1174360235354917591L;
38
39     /*-----*\
40     ===** Constructors **=====
41     \*-----*/
42     /**
43      * Creates a new instance of this class.
44      *
45      * @param <T> The type of the enum.
46      * @param value The unsupported value.
47      */
48     public <T extends Enum<T>> UnsupportedEnumError( final T value )

```



```
49     {
50         super( format( MSG_UnsupportedEnum, requireNonNullArgument( value,
51             "value" ).getClass().getName(), value.name() ) );
52     } // UnsupportedEnumError()
53
54     /**
55     * Creates a new instance of this class.
56     *
57     * @param type    The class of the enum.
58     * @param value   The unsupported value.
59     */
60     public UnsupportedEnumError( final Class<? extends Enum<?>> type,
61         final String value )
62     {
63         super( format( MSG_UnsupportedEnum, requireNonNullArgument( type,
64             "type" ).getName(), requireNotEmptyArgument( value, "value" )
65             ) );
66     } // UnsupportedEnumError()
67 }
68 // class UnsupportedEnumError
```



List of Tables

- 3.1. Parts of a **class** declaration 64
- 3.2. Parts of an **interface** declaration 66
- 3.3. Parts of an **enum** declaration 67
- 3.4. Parts of a **record** declaration 70
- 3.5. Parts of an annotation declaration 71

- 8.1. Log Levels 226

- 9.1. Verbs 450
- 9.2. Suffixes for Class Names 462





Listings

3.1. Beginning Comment for a Java file	58
3.2. Beginning Comment for a Resource or a Script file	58
3.3. Beginning Comment for XML and HTML files	59
3.4. Beginning Comment for a $\text{\TeX}$ / $\text{\LaTeX}$ file	59
3.5. Class Skeleton	63
3.6. Interface Skeleton	65
3.7. enum Skeleton	66
3.8. Record Skeleton	69
3.9. Annotation Skeleton	71
3.10 Closing Comment for a Java file	71
3.11 Closing Comment for a resource or a script file	73
3.12 Closing Comment for a HTML or an XML file	73
5.1. module-info.java	111
5.2. package-info.java	113
6.1. JavaBean	126
6.2. POJO	127
7.1. A tuple class	155
7.2. Gauss' Easter algorithm[118, 128]	176
8.1. Test Abort main()	212
8.2. Test Abort run()	213
8.3. Class UncaughtExceptionHandlerImpl	216
8.4. LogLevel.java for JDK Logging	227
8.5. Default JDK Logging Configuration	242
8.6. Sample JDK Logging Configuration	243
8.7. Sample Log4j Configuration	245
8.8. requireNonNullArgument()	253



8.9. requireNotEmptyArgument()	254
8.10 requireNotBlankArgument()	257
8.11 Status Record	310
8.12 Value Type 'PersonId'	337
8.13 Dimensioned Value Length	340
8.14 Methods equals() and hashCode()	358
8.15 A simple clone() Method	364
8.16 StringUtils::breakText	391
9.1. AutoLock.java	469
9.2. ValidationException.java	471
9.3. NullArgumentException.java	473
9.4. EmptyArgumentException.java	475
9.5. BlankArgumentException.java	476
9.6. Lazy.java	478
9.7. LazyImpl.java	481
9.8. Load Logging Configuration from Resource	485
9.9. Load Logging Configuration from File	486
9.10 MountPoint.java	487
9.11 FixList.java	489
9.12 BUG.java	490
9.13 PostProcessor.java	492
9.14 ThreadGroupExt.java	494
9.15 UnsupportedEnumError.java	498



Bibliography

- [1] ?
- [2] @API Guardian. Version 1.1.2. apiguardian-team. 2021-06-27. URL: <https://github.com/apiguardian-team/apiguardian#api-guardian> (visited on 2023-02-08).
- [3] Hal Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. With a forew. by Alan J. Perlis. 2nd ed. Cambridge, MA: MIT Press, 1996. Chap. Preface to the First Edition, p. xxii. ISBN: 0262510871. URL: <https://web.mit.edu/6.001/6.037/sicp.pdf> (visited on 2026-06-13).
- [4] Douglas Adams. *The Hitchhiker's Guide to the Galaxy*. 1979.
- [5] Konstantin Taranov et. al. *C# Coding Standards and Naming Conventions*. 2023-12-19. URL: <https://github.com/ktaranov/naming-convention/blob/master/C%23%20Coding%20Standards%20and%20Naming%20Conventions.md> (visited on 2026-06-14).
- [6] *Annotation Interface java.io.Serial*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Serial.html> (visited on 2023-02-08).
- [7] *Annotation Interface java.lang.annotation.Native*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Native.html> (visited on 2023-02-08).
- [8] *Annotation Interface java.lang.Deprecated*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Deprecated.html> (visited on 2022-11-12).



- [9] *Annotation Interface java.lang.FunctionalInterface*. Version Java 125. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/FunctionalInterface.html> (visited on 2022-06-22).
- [10] *Annotation Interface java.lang.Override*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Override.html> (visited on 2022-12-20).
- [11] *Annotation Interface java.lang.SuppressWarnings*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/SuppressWarnings.html> (visited on 2022-11-23).
- [12] *Annotation Interface javax.annotation.processing.Generated*. Version Java 25. Oracle. 2025. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.compiler/javax/annotation/processing/Generated.html> (visited on 2026-06-14).
- [13] *Annotation org.apiguardian.api.API*. Version 1.1.2. apiguardian-team. 2021-06-27. URL: <https://apiguardian-team.github.io/apiguardian/docs/current/api/org/apiguardian/api/API.html> (visited on 2023-02-08).
- [14] *Apache ActiveMQ. Flexible & Powerful Open Source Multi-Protocol Messaging*. The Apache Software Foundation. URL: <https://activemq.apache.org/> (visited on 2022-12-01).
- [15] *Apache Commons Lang*. Version 3.12.0. The Apache Software Foundation. 2021-03-01. URL: <https://commons.apache.org/proper/commons-lang/> (visited on 2022-12-01).
- [16] *Apache Commons Logging*. Version 1.2. The Apache Software Foundation. 2014-07-04. URL: <http://commons.apache.org/logging/> (visited on 2022-12-08).
- [17] *Apache James. Enterprise Mail Server*. Version 3.7.2. The Apache Software Foundation. URL: <https://james.apache.org/> (visited on 2022-12-01).
- [18] *Apache Log4j*. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/> (visited on 2022-12-01).



-
- [19] *Apache Maven*. Version 3.8.6. The Apache Software Foundation. 2022-11-30. URL: <https://maven.apache.org/> (visited on 2022-12-02).
 - [20] *Apache OpenOffice*. Version 4.1.13. The Apache Software Foundation. 2022-07-22. URL: <https://www.openoffice.org/> (visited on 2023-02-12).
 - [21] *Apache OpenOffice - Extensions development java*. The Apache Software Foundation. 2008-12-02. URL: https://wiki.openoffice.org/wiki/Extensions_development_java (visited on 2023-02-12).
 - [22] *Apache Tomcat*. Version 10.1.2. The Apache Software Foundation. 2022-11-14. URL: <https://tomcat.apache.org/> (visited on 2022-12-01).
 - [23] *ASCII*. en. 2026-06-05. URL: <https://en.wikipedia.org/wiki/ASCII> (visited on 2026-06-22).
 - [24] *Average typing speed infographic*. en. Ratatype. 2025-12-04. URL: <https://www.ratatype.com/learn/average-typing-speed/> (visited on 2026-06-13).
 - [25] Gavin Bierman. *JEP 354: Switch Expressions (Second Preview)*. OpenJDK. 2019-04-09. URL: <https://openjdk.org/jeps/354> (visited on 2023-04-02). last modified 2021-08-28. closed/delivered.
 - [26] Gavin Bierman. *JEP 361: Switch Expressions*. OpenJDK. 2019-09-04. URL: <https://openjdk.org/jeps/361> (visited on 2023-04-02). last modified 2022-03-11. closed/delivered.
 - [27] *Binding with a logging framework at deployment time. Simple Logging Facade for Java*. Version 2.0.5. QOS. URL: <https://www.slf4j.org/manual.html#swapping> (visited on 2022-12-08).
 - [28] Joshua Bloch. *Effective Java*. 3rd ed. Addison-Wesley Professional, 2017-12. ISBN: 9780134686097. URL: <https://www.oreilly.com/library/view/effective-java-3rd/9780134686097/> (visited on 2022-12-26).
 - [29] Keturah Carol Bruce Eckel. *Bruce Eckel on Java records*. Oracle. 2022-04-22. URL: <https://oraclejavacertified.blogspot.com/2022/04/bruce-eckel-on-java-records.html> (visited on 2026-06-22).



- [30] *C# Documentation*. Microsoft. 2025-10-10. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names> (visited on 2026-06-14).
- [31] *C++ Coding Standards*. ISO. URL: <https://isocpp.org/wiki/faq/coding-standards> (visited on 2026-06-14).
- [32] *Catch-22 (logic)*. en. 2022-12-15. URL: [https://en.wikipedia.org/wiki/Catch-22_\(logic\)](https://en.wikipedia.org/wiki/Catch-22_(logic)) (visited on 2022-12-21).
- [33] *Class jakarta.servlet.http.HttpServlet*. Version Jakarta EE Platform 9. Eclipse Foundation. URL: <https://jakarta.ee/specifications/platform/9/apidocs/jakarta/servlet/http/HttpServlet.html> (visited on 2022-12-03).
- [34] *Class java.awt.event.KeyAdapter*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/awt/event/KeyAdapter.html> (visited on 2023-02-18).
- [35] *Class java.io.BufferedInputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/BufferedInputStream.html> (visited on 2023-02-21).
- [36] *Class java.io.BufferedOutputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/BufferedOutputStream.html> (visited on 2023-02-21).
- [37] *Class java.io.ByteArrayOutputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/ByteArrayOutputStream.html> (visited on 2023-02-21).
- [38] *Class java.io.File*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/File.html> (visited on 2022-11-26).
- [39] *Class java.io.FileInputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/FileInputStream.html> (visited on 2022-12-21).



- [40] *Class java.io.FileOutputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/FileOutputStream.html> (visited on 2023-02-21).
- [41] *Class java.io.InputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/InputStream.html> (visited on 2022-12-21).
- [42] *Class java.io.OutputStream*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/OutputStream.html> (visited on 2023-02-19).
- [43] *Class java.lang.ArithmeticException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ArithmeticException.html> (visited on 2023-02-12).
- [44] *Class java.lang.AssertionError*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/AssertionError.html> (visited on 2022-11-24).
- [45] *Class java.lang.Class*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html> (visited on 2023-02-19).
- [46] *Class java.lang.ClassNotFoundException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ClassNotFoundException.html> (visited on 2023-02-27).
- [47] *Class java.lang.CloneNotSupportedException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/CloneNotSupportedException.html> (visited on 2022-11-26).
- [48] *Class java.lang.Error*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Error.html> (visited on 2022-12-03).
- [49] *Class java.lang.Exception*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Exception.html> (visited on 2022-12-03).



- [50] *Class java.lang.ExceptionInInitializerError*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ExceptionInInitializerError.html> (visited on 2022-12-03).
- [51] *Class java.lang.HashSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html> (visited on 2023-02-20).
- [52] *Class java.lang.IllegalArgumentException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/IllegalArgumentException.html> (visited on 2022-12-01).
- [53] *Class java.lang.IllegalStateException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/IllegalStateException.html> (visited on 2023-02-19).
- [54] *Class java.lang.Integer*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Integer.html> (visited on 2022-11-26).
- [55] *Class java.lang.LinkedHashMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedHashMap.html> (visited on 2023-02-20).
- [56] *Class java.lang.LinkedHashSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedHashSet.html> (visited on 2023-02-20).
- [57] *Class java.lang.Module*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Module.html> (visited on 2022-03-25).
- [58] *Class java.lang.NullPointerException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/NullPointerException.html> (visited on 2023-02-12).



- [59] *Class java.lang.Object*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html> (visited on 2022-11-24).
- [60] *Class java.lang.Package*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Package.html> (visited on 2022-03-25).
- [61] *Class java.lang.reflect.Array*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Array.html> (visited on 2023-03-25).
- [62] *Class java.lang.reflect.Constructor*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Constructor.html> (visited on 2022-03-25).
- [63] *Class java.lang.reflect.Field*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Field.html> (visited on 2022-03-25).
- [64] *Class java.lang.reflect.Method*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Method.html> (visited on 2022-03-25).
- [65] *Class java.lang.RuntimeException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/RuntimeException.html> (visited on 2022-12-03).
- [66] *Class java.lang.String*. Version Java 25. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html> (visited on 2026-06-23).
- [67] *Class java.lang.StringBuffer*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuffer.html> (visited on 2022-11-24).
- [68] *Class java.lang.StringBuilder*. Version Java 25. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html> (visited on 2026-06-28).
- [69] *Class java.lang.System*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/System.html> (visited on 2022-12-03).



- [70] *Class java.lang.ThreadGroup*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ThreadGroup.html> (visited on 2022-12-06).
- [71] *Class java.lang.Throwable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html> (visited on 2022-11-03).
- [72] *Class java.lang.UnsupportedOperationException*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/UnsupportedOperationException.html> (visited on 2023-02-19).
- [73] *Class java.math.BigInteger*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigInteger.html> (visited on 2023-03-21).
- [74] *Class java.net.ServerSocket*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/ServerSocket.html> (visited on 2023-03-27).
- [75] *Class java.net.Socket*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/Socket.html> (visited on 2023-03-27).
- [76] *Class java.nio.CharBuffer*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/CharBuffer.html> (visited on 2023-02-19).
- [77] *Class java.nio.charset.Charset*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/charset/Charset.html> (visited on 2022-12-04).
- [78] *Class java.sql.JDBCType*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/JDBCType.html> (visited on 2023-02-23).
- [79] *Class java.sql.Types*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/Types.html> (visited on 2023-02-23).
- [80] *Class java.text.SimpleDateFormat*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/text/SimpleDateFormat.html> (visited on 2022-11-13).



- [81] *Class java.time.format.DateTimeFormatterBuilder*. Version Java 25. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/format/DateTimeFormatterBuilder.html> (visited on 2026-06-28).
- [82] *Class java.time.Instant*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/Instant.html> (visited on 2022-11-26).
- [83] *Class java.util.ArrayList*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html> (visited on 2022-12-03).
- [84] *Class java.util.Collections*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html> (visited on 2022-12-21).
- [85] *Class java.util.concurrent.ConcurrentHashMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html> (visited on 2022-12-03).
- [86] *Class java.util.Date*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Date.html> (visited on 2022-11-13).
- [87] *Class java.util.EnumMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/EnumMap.html> (visited on 2023-02-20).
- [88] *Class java.util.EnumSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/EnumSet.html> (visited on 2023-02-20).
- [89] *Class java.util.Formatter*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Formatter.html> (visited on 2022-11-24).
- [90] *Class java.util.HashMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html> (visited on 2022-12-03).



- [91] *Class java.util.Hashtable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Hashtable.html> (visited on 2022-12-03).
- [92] *Class java.util.LinkedList*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedList.html> (visited on 2022-12-03).
- [93] *Class java.util.logging.Logger*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/Logger.html> (visited on 2023-02-16).
- [94] *Class java.util.logging.LogManager*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/LogManager.html> (visited on 2022-12-17).
- [95] *Class java.util.Objects*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html> (visited on 2022-12-18).
- [96] *Class java.util.Optional*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Optional.html> (visited on 2022-12-06).
- [97] *Class java.util.OptionalDouble*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/OptionalDouble.html> (visited on 2022-12-19).
- [98] *Class java.util.OptionalInt*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/OptionalInt.html> (visited on 2022-12-19).
- [99] *Class java.util.OptionalLong*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/OptionalLong.html> (visited on 2022-12-19).
- [100] *Class java.util.ResourceBundle*. Version Java 25. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/ResourceBundle.html> (visited on 2026-06-22).



- [101] *Class java.util.StringJoiner*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/StringJoiner.html> (visited on 2022-11-24).
- [102] *Class java.util.TreeMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/TreeMap.html> (visited on 2022-12-21).
- [103] *Class java.util.TreeSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/TreeSet.html> (visited on 2023-02-20).
- [104] *Class java.util.UUID*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/UUID.html> (visited on 2022-11-26).
- [105] *Class java.util.Vector*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Vector.html> (visited on 2022-12-03).
- [106] *Class jdk.javadoc.doclet.StandardDoclet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.javadoc/jdk/javadoc/doclet/StandardDoclet.html> (visited on 2022-11-19).
- [107] *Class PropertyResourceBundle*. Version Java 25. Oracle. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/PropertyResourceBundle.html> (visited on 2026-06-22).
- [108] *Code Conventions for the Java™ Programming Language*. Oracle. 1999-04-20. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-contents.html> (visited on 2026-06-13).
- [109] *Code Conventions for the Java™ Programming Language, chapter “10.5.4 Special Comments”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-programmingpractices.html#395> (visited on 2022-11-20).
- [110] *Code Conventions for the Java™ Programming Language, chapter “3.1.1 Beginning Comments”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-fileorganization.html#3441> (visited on 2022-11-13).



- [111] *Code Conventions for the Java™ Programming Language, chapter “4 - Indentation”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-indentation.html> (visited on 2026-06-14).
- [112] *Code Conventions for the Java™ Programming Language, chapter “6.3 Placement”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-declarations.html#16817> (visited on 2023-01-02).
- [113] *Code Conventions for the Java™ Programming Language, chapter “8.1 Blank Lines”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-whitespace.html#487> (visited on 2026-06-15).
- [114] *Code Conventions for the Java™ Programming Language, chapter “8.2 Blank Spaces”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-whitespace.html#682> (visited on 2023-01-02).
- [115] *Code Conventions for the Java™ Programming Language, chapter “9 - Naming Conventions”*. Oracle. 1999. URL: <https://www.oracle.com/java/technologies/javase/codeconventions-namingconventions.html> (visited on 2022-11-13).
- [116] *Configuration. Apache Log4j*. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/manual/configuration.html> (visited on 2022-12-17).
- [117] Douglas Crockford. *Code Conventions for the JavaScript Programming Language*. 2019-05-15. URL: <https://www.crockford.com/code.html> (visited on 2026-06-14).
- [118] *Date of Easter*. en. 2022-11-18. URL: https://en.wikipedia.org/wiki/Date_of_Easter#Gauss's_Easter_algorithm (visited on 2022-11-20).
- [119] Kyzer R. Davis, Brad G. Peabody, and Robert Kieffer. *Next Generation UUID Formats*. IETF. 2021-10. URL: <https://github.com/uuid6/uuid6-ietf-draft> (visited on 2022-12-15).



- [120] *Default Methods*. Version Java 8. Oracle. 2022. URL: <https://docs.oracle.com/javase/tutorial/java/IandI/defaultmethods.html> (visited on 2022-12-20).
- [121] *Documentation Comment Specification for the Standard Doclet (JDK 17)*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/javadoc/doc-comment-spec.html> (visited on 2022-11-13).
- [122] *Effective Go*. go.dev. 2009. URL: https://go.dev/doc/effective_go (visited on 2026-06-14).
- [123] *Enum java.lang.annotation.RetentionPolicy*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/RetentionPolicy.html> (visited on 2023-03-25).
- [124] Attila Fejér. *What does it mean to program to Interfaces?* Baeldung. 2022-11-11. URL: <https://www.baeldung.com/cs/program-to-interface> (visited on 2022-12-26).
- [125] *Filter (software)*. en. 2022-02-24. URL: [https://en.wikipedia.org/wiki/Filter_\(software\)](https://en.wikipedia.org/wiki/Filter_(software)) (visited on 2022-12-03).
- [126] Martin G. Fowler. *Refactoring: Improving the Design of Existing Code*. 1999.
- [127] Erich Gamma et al. *Design Patterns. Elements of Reusable Object-Oriented Software*. 1995. ISBN: 0201633612.
- [128] *Gaußsche Osterformel*. de. 2022-04-15. URL: https://de.wikipedia.org/wiki/Gau%C3%9Fsche_Osterformel (visited on 2022-11-20).
- [129] *Go Style*. Google. URL: <https://google.github.io/styleguide/go> (visited on 2026-06-14).
- [130] Brian Goetz. *JEP 325: Switch Expressions (Preview)*. OpenJDK. 2017-12-04. URL: <https://openjdk.org/jeps/325> (visited on 2023-04-02). last modified 2022-03-11. closed/delivered.
- [131] *Google C++ Style Guide*. Google. URL: <https://google.github.io/styleguide/cppguide.html> (visited on 2026-06-14).
- [132] *Google Guava*. Version 31.2. The Guava Authors. 2022-02-28. URL: <https://github.com/google/guava> (visited on 2022-12-01).



- [133] James Gosling et al. *The Java Language Specification. Java SE 25 Edition*. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/index.html> (visited on 2026-06-13).
- [134] James Gosling et al. *The Java Language Specification, chapter “11. Exceptions”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-11.html> (visited on 2022-12-03).
- [135] James Gosling et al. *The Java Language Specification, chapter “11.1.1. The Kinds of Exceptions”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-11.html#jls-11.1.1> (visited on 2022-12-03).
- [136] James Gosling et al. *The Java Language Specification, chapter “12.1.4 Invoke ‘Test.main’”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-12.html#jls-12.1.4> (visited on 2022-11-08).
- [137] James Gosling et al. *The Java Language Specification, chapter “14.10. The assert Statement”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.10> (visited on 2022-11-19).
- [138] James Gosling et al. *The Java Language Specification, chapter “14.11. The switch Statement”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.11> (visited on 2023-04-02).
- [139] James Gosling et al. *The Java Language Specification, chapter “14.19. The synchronized Statement”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.19> (visited on 2023-01-19).
- [140] James Gosling et al. *The Java Language Specification, chapter “14.20. The try statement”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.20> (visited on 2023-03-26).



-
- [141] James Gosling et al. *The Java Language Specification, chapter “14.20.3. try-with-resources”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.20.3> (visited on 2023-03-26).
 - [142] James Gosling et al. *The Java Language Specification, chapter “14.20.3.2. Extended try-with-resources”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.20.3.2> (visited on 2022-12-20).
 - [143] James Gosling et al. *The Java Language Specification, chapter “14.21. The yield Statement”*. Java SE 25 Edition. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html#jls-14.21> (visited on 2026-06-28).
 - [144] James Gosling et al. *The Java Language Specification, chapter “14.4. Local Variable Declarations”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.4> (visited on 2023-03-01).
 - [145] James Gosling et al. *The Java Language Specification, chapter “15.13 Method Reference Expressions”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html#jls-15.13> (visited on 2022-11-24).
 - [146] James Gosling et al. *The Java Language Specification, chapter “15.27 Lambda Expressions”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html#jls-15.27> (visited on 2022-11-24).
 - [147] James Gosling et al. *The Java Language Specification, chapter “15.28. switch Expressions”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html#jls-15.28> (visited on 2023-04-02).
 - [148] James Gosling et al. *The Java Language Specification, chapter “15.28.1. The Switch Block of a switch Expression”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html#jls-15.28.1>



- com/javase/specs/jls/se17/html/jls-15.html#jls-15.28.1 (visited on 2023-04-02).
- [149] James Gosling et al. *The Java Language Specification, chapter “3.10.6. Text Blocks”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-3.html#jls-3.10.6> (visited on 2023-03-26).
- [150] James Gosling et al. *The Java Language Specification, chapter “3.8 Identifiers”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-3.html#jls-3.8> (visited on 2022-11-10).
- [151] James Gosling et al. *The Java Language Specification, chapter “4.5. Parameterized Types”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-4.html#jls-4.4> (visited on 2022-11-13).
- [152] James Gosling et al. *The Java Language Specification, chapter “6.1. Declarations”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-6.html#jls-6.1> (visited on 2022-11-26).
- [153] James Gosling et al. *The Java Language Specification, chapter “6.2 Names and Identifiers”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-6.html#jls-6.2> (visited on 2022-11-10).
- [154] James Gosling et al. *The Java Language Specification, chapter “7.3. Compilation Units”*. *Java SE 25 Edition*. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html#jls-7.3> (visited on 2026-06-20).
- [155] James Gosling et al. *The Java Language Specification, chapter “7.4.2. Unnamed Packages”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-14.html#jls-14.20.3> (visited on 2022-12-20).
- [156] James Gosling et al. *The Java Language Specification, chapter “7.7 Module Declarations”*. *Java SE 17 Edition*. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-7.html#jls-7.7> (visited on 2022-11-10).



-
- [157] James Gosling et al. *The Java Language Specification, chapter “8.1.1.2. sealed, non-sealed, and final Classes”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-8.html#jls-8.1.1.2> (visited on 2022-12-20).
 - [158] James Gosling et al. *The Java Language Specification, chapter “8.10 Record Classes”*. Java SE 25 Edition. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-8.html#jls-8.10> (visited on 2026-06-22).
 - [159] James Gosling et al. *The Java Language Specification, chapter “8.4. Method Declarations”*. Java SE 25 Edition. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-8.html#jls-8.4> (visited on 2026-06-28).
 - [160] James Gosling et al. *The Java Language Specification, chapter “9.1.1.4. sealed and non-sealed Interfaces”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html#jls-9.1.1.4> (visited on 2022-12-20).
 - [161] James Gosling et al. *The Java Language Specification, chapter “9.4. Method Declarations”*. Java SE 25 Edition. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html#jls-9.4> (visited on 2026-06-28).
 - [162] James Gosling et al. *The Java Language Specification, chapter “9.6 Annotation Interfaces”*. Java SE 25 Edition. Version Java 25. Oracle. 2025-07-29. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-9.html#jls-9.6> (visited on 2026-06-23).
 - [163] James Gosling et al. *The Java Language Specification, chapter “9.6.3. Repeatable Annotation Interfaces”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html#jls-9.6.3> (visited on 2022-12-08).
 - [164] James Gosling et al. *The Java Language Specification, chapter “9.6.4.9. @FunctionalInterface”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html#jls-9.6.4.9> (visited on 2022-12-06).



- [165] James Gosling et al. *The Java Language Specification, chapter “9.8. Functional Interfaces”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html#jls-9.8> (visited on 2022-12-06).
- [166] James Gosling et al. *The Java Language Specification, chapter “Chapter 18. Type Inference”*. Java SE 17 Edition. Version Java 17. Oracle. 2021-08-09. URL: <https://docs.oracle.com/javase/specs/jls/se17/html/jls-18.html> (visited on 2023-03-01).
- [167] *Graphviz*. URL: <https://graphviz.org/> (visited on 2023-03-28).
- [168] *GSON. A Java serialization/deserialization library to convert Java Objects into JSON and back*. Version 2.10.1. 2023-01-06. URL: <https://github.com/google/gson> (visited on 2023-03-25).
- [169] *H2 Database Engine*. Version 2.1.214. 2022-06-13. URL: <http://www.h2database.com/html/main.html> (visited on 2022-12-01).
- [170] Dimitri van Heesch. *Doxygen. Generate documentation from source code*. URL: <https://doxygen.nl/> (visited on 2022-11-13).
- [171] *Hibernate ORM*. Version Hibernate ORM 5.6.14.Final. 2022-11-04. URL: <https://hibernate.org/orm/> (visited on 2022-12-01).
- [172] *How to Write Doc Comments for the Javadoc Tool*. Oracle. URL: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html> (visited on 2022-11-17).
- [173] *Hungarian notation*. en. 2023-02-10. URL: https://en.wikipedia.org/wiki/Hungarian_notation (visited on 2023-01-28).
- [174] *IBM WebSphere Application Server. A flexible, security-rich Java server runtime environment for enterprise applications*. IBM. URL: <https://www.ibm.com/products/websphere-application-server> (visited on 2022-12-01).
- [175] *Interface jakarta.jms.Connection*. Version Jakarta Messaging API 3.0.0. Eclipse Foundation. URL: <https://jakarta.ee/specifications/messaging/3.0/apidocs/jakarta/jms/connection> (visited on 2023-03-27).



- [176] *Interface java.awt.event.KeyListener*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/awt/event/KeyListener.html> (visited on 2023-02-18).
- [177] *Interface java.beans.beancontext.BeanContext*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/beans/beancontext/BeanContext.html> (visited on 2023-02-20).
- [178] *Interface java.io.Closeable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Closeable.html> (visited on 2022-11-04).
- [179] *Interface java.lang.annotation.Annotation*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Annotation.html> (visited on 2023-03-25).
- [180] *Interface java.lang.annotation.Retention*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Retention.html> (visited on 2023-03-25).
- [181] *Interface java.lang.Appendable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Appendable.html> (visited on 2022-12-04).
- [182] *Interface java.lang.AutoCloseable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/AutoCloseable.html> (visited on 2022-11-03).
- [183] *Interface java.lang.CharSequence*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/CharSequence.html> (visited on 2022-11-24).
- [184] *Interface java.lang.Cloneable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Cloneable.html> (visited on 2022-11-04).
- [185] *Interface java.lang.Comparable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Comparable.html> (visited on 2023-03-02).



- [186] *Interface java.lang.Iterable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Iterable.html> (visited on 2023-02-20).
- [187] *Interface java.lang.Iterator*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Iterator.html> (visited on 2023-03-01).
- [188] *Interface java.lang.Runnable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runnable.html> (visited on 2022-12-06).
- [189] *Interface java.lang.Thread.UncaughtExceptionHandler*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.UncaughtExceptionHandler.html> (visited on 2022-12-06).
- [190] *Interface java.nio.file.Path*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/file/Path.html> (visited on 2023-02-20).
- [191] *Interface java.sql.Connection*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/Connection.html> (visited on 2023-03-27).
- [192] *Interface java.sql.ResultSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/ResultSet.html> (visited on 2023-03-27).
- [193] *Interface java.sql.Statement*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/Statement.html> (visited on 2023-03-27).
- [194] *Interface java.util.Collection*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html> (visited on 2022-12-21).
- [195] *Interface java.util.Comparator*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Comparator.html> (visited on 2023-01-20).



- [196] *Interface java.util.concurrent.Callable*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/Callable.html> (visited on 2022-11-04).
- [197] *Interface java.util.concurrent.locks.Lock*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/locks/Lock.html> (visited on 2023-03-27).
- [198] *Interface java.util.concurrent.ThreadFactory*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ThreadFactory.html> (visited on 2022-12-06).
- [199] *Interface java.util.Formatter*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.javadoc/jdk/javadoc/doclet/Doclet.html> (visited on 2022-11-24).
- [200] *Interface java.util.function.Supplier*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/function/Supplier.html> (visited on 2023-02-19).
- [201] *Interface java.util.List*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html> (visited on 2022-12-03).
- [202] *Interface java.util.logging.LoggingMXBean*. Version Java 17. Oracle. URL: <https://docs.oracle.com/javase/7/docs/api/java/util/logging/LoggingMXBean.html> (visited on 2022-12-18).
- [203] *Interface java.util.Map*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html> (visited on 2022-12-03).
- [204] *Interface java.util.Map.Entry*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.Entry.html> (visited on 2022-03-01).
- [205] *Interface java.util.Set*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Set.html> (visited on 2022-12-21).



- [206] *Interface java.util.SortedMap*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/SortedMap.html> (visited on 2023-02-20).
- [207] *Interface java.util.SortedSet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/SortedSet.html> (visited on 2022-12-21).
- [208] *Interface jdk.javadoc.doclet.Doclet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.javadoc/jdk/javadoc/doclet/Doclet.html> (visited on 2022-11-19).
- [209] *Interface jdk.javadoc.doclet.Taglet*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.javadoc/jdk/javadoc/doclet/Taglet.html> (visited on 2022-11-19).
- [210] *Interface org.apache.logging.log4j.Logger*. *Apache Log4j*. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/log4j-api/apidocs/org/apache/logging/log4j/Logger.html> (visited on 2023-02-16).
- [211] *Interface org.slf4j.Logger*. Version 2.0.5. QOS. URL: <https://www.slf4j.org/apidocs/org/slf4j/Logger.html> (visited on 2023-02-16).
- [212] *ISO/IEC 8859-1*. en. 2026-06-13. URL: https://en.wikipedia.org/wiki/ISO/IEC_8859-1 (visited on 2026-06-22).
- [213] Bobby Jack. *What Is Hungarian Notation and Should I Use It?* MUO. 2021-04-03. URL: <https://www.makeuseof.com/what-is-hungarian-notation-and-should-i-use-it/> (visited on 2023-02-28).
- [214] *Jackson Project Home @github*. Version 2.13.4. 2022. URL: <https://github.com/FasterXML/jackson> (visited on 2023-03-25).
- [215] *Jakarta Persistence*. Version 3.0. Eclipse Foundation. URL: <https://jakarta.ee/specifications/persistence/3.0/> (visited on 2023-03-25).



-
- [216] *Jakarta XML Binding*. Version 3.0. Replacement for JAXB. Eclipse Foundation. URL: <https://eclipse-ee4j.github.io/jaxb-ri/> (visited on 2023-03-25).
 - [217] *Jakarta XML Web ServicesTM*. Version 4.0. Eclipse Foundation. URL: <https://projects.eclipse.org/projects/ee4j.jaxws> (visited on 2023-03-25).
 - [218] *jAlbum*. Version 30. jAlbum Team. 2023-02-09. URL: <https://jalbum.net/> (visited on 2023-02-12).
 - [219] *Java Language Updates - 4 Records. Java 14*. Oracle. URL: <https://docs.oracle.com/en/java/javase/14/language/records.html> (visited on 2026-06-22).
 - [220] *Java Language Updates - Pattern Matching for switch Expressions and Statements. Java 17*. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/language/pattern-matching-switch-expressions-and-statements.html> (visited on 2022-11-02).
 - [221] *Java Language Updates - Switch Expressions. Java 17*. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html> (visited on 2022-12-24).
 - [222] *Java Logging Overview. Java Platform, Standard Edition*. Version Java 17. Oracle. 2022-10. URL: <https://docs.oracle.com/en/java/javase/17/core/java-logging-overview.html> (visited on 2022-12-08).
 - [223] *Java Logging Overview - Configuration File. Java Platform, Standard Edition*. Version Java 17. Oracle. 2022-10. URL: https://docs.oracle.com/en/java/javase/17/core/java-logging-overview.html#GUID-B83B652C-17EA-48D9-93D2-563AE1FF8EDA_CONFIGURATIONFILE-4C48BDCE (visited on 2022-12-17).
 - [224] *Java Logging Overview - Default Configuration. Java Platform, Standard Edition*. Version Java 17. Oracle. 2022-10. URL: https://docs.oracle.com/en/java/javase/17/core/java-logging-overview.html#GUID-B83B652C-17EA-48D9-93D2-563AE1FF8EDA_DEFAULTCONFIGURATION-4D023EDA (visited on 2022-12-17).



- [225] *Java Object Serialization Specification, chapter “1.6 Documenting Serializable Fields and Data for a Class”*. Version Java 17. Oracle. 2022. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/serialization/serial-arch.html#documenting-serializable-fields-and-data-for-a-class> (visited on 2022-11-19).
- [226] *Java Persistence API (JPA)*. Version 2.1. IBM. 2023-02-07. URL: <https://www.ibm.com/docs/en/was-liberty/base?topic=overview-java-persistence-api-jpa> (visited on 2023-03-25).
- [227] *JavaBeansTM. JavaBeansTM API specification*. Version 1.01-A. SUN Microsystems. 1997. URL: <https://download.oracle.com/otn-pub/jcp/7224-javabeans-1.01-fr-spec-oth-JSpec/beans.101.pdf> (visited on 2022-11-07).
- [228] *JavaBeansTM, chapter 8.3 “Design Patterns for Properties” on page 55. JavaBeansTM API specification*. Version 1.01-A. SUN Microsystems. 1997. URL: <https://download.oracle.com/otn-pub/jcp/7224-javabeans-1.01-fr-spec-oth-JSpec/beans.101.pdf#page=55> (visited on 2022-11-07).
- [229] *JavaBeansTM, chapter 8.3.2 “Boolean properties” on page 55. JavaBeansTM API specification*. Version 1.01-A. SUN Microsystems. 1997. URL: <https://download.oracle.com/otn-pub/jcp/7224-javabeans-1.01-fr-spec-oth-JSpec/beans.101.pdf#page=55> (visited on 2022-11-13).
- [230] *JavaDoc Guide*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/javadoc/javadoc.html> (visited on 2022-11-13).
- [231] *JAX-WS*. Version 2.2. IBM. 2023-01-26. URL: <https://www.ibm.com/docs/en/was/9.0.5?topic=services-jax-ws> (visited on 2023-03-25).
- [232] *jEdit – Programmer’s Text Editor*. Version 5.6.0. jedit.org. 2020-09-03. URL: <http://www.jedit.org/> (visited on 2023-02-12).
- [233] *JMX. Apache Log4j*. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/manual/jmx.html> (visited on 2022-12-18).



- [234] *JUnit 5*. URL: <https://junit.org/junit5/> (visited on 2022-11-24).
- [235] Kohsuke Kawaguchi. *args4j*. 2022. URL: <https://github.com/kohsuke/args4j>.
- [236] Kohsuke Kawaguchi. *args4j parent. args4j: Java command line arguments parser*. Version 2.33. 2016-01-31. URL: <http://args4j.kohsuke.org/>.
- [237] Donald E. Knuth. "Structured Programming with Goto Statements". In: *ACM Journals. ACM Computing Surveys* 6.4 (1974-12), pp. 261–301. URL: <https://dl.acm.org/doi/10.1145/356635.356640> (visited on 2022-11-23).
- [238] Jim Laskey and Gavin Bierman. *JEP 511: Module Import Declarations*. OpenJDK. 2024-11-21. URL: <https://openjdk.org/jeps/511> (visited on 2026-06-22). last modified 2025-07-22. closed/delivered.
- [239] Jim Laskey and Stuart Marks. *Programmer's Guide to Text Blocks*. Oracle. 2020-09-15. URL: <https://docs.oracle.com/en/java/javase/15/text-blocks/index.html> (visited on 2023-03-27).
- [240] Paul J. Leach, Michael H. Mealling, and Rich Salz. *A Universally Unique Identifier (UUID) URN Namespace*. IETF. 2005-07. URL: <https://www.ietf.org/rfc/rfc4122.txt> (visited on 2022-12-15).
- [241] Paul J. Leach et al. *Draft: A Universally Unique Identifier (UUID) URN Namespace*. Version expires 2023-04-13. IETF. 2022-10-10. URL: <https://ietf-wg-uuidrev.github.io/rfc4122bis/draft-00/draft-ietf-uuidrev-rfc4122bis.html> (visited on 2022-12-15). DRAFT.
- [242] *LibreOffice*. Version 7.5. The Document Foundation. URL: <https://www.libreoffice.org/> (visited on 2023-02-12).
- [243] Fabio Lima. *TSID Creator. A Java library for Time Sortable Identifier (TSID)*. 2022. URL: <https://github.com/f4b6a3/tsid-creator> (visited on 2022-12-10).
- [244] *Logback*. Version 1.3.5/1.4.5. QOS. URL: <https://logback.qos.ch/> (visited on 2022-12-08).



- [245] *Markers. Apache Log4j*. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/manual/markers.html> (visited on 2023-02-16).
- [246] Stuart W. Marks. *Local Variable Type Inference – Style Guidelines. P1. Reading code is more important than writing code*. 2018-03. URL: <https://openjdk.org/projects/amber/guides/lvti-style-guide#P1> (visited on 2026-06-13).
- [247] Stuart W. Marks. *Local Variable Type Inference – Style Guidelines. P3. Code readability shouldn't depend on IDEs*. 2018-03. URL: <https://openjdk.org/projects/amber/guides/lvti-style-guide#P3> (visited on 2022-12-27).
- [248] Stuart W. Marks. *Local Variable Type Inference – Style Guidelines*. 2018-03. URL: <https://openjdk.org/projects/amber/guides/lvti-style-guide> (visited on 2022-12-24).
- [249] Dustin Marx. *JDK 10's Summary Javadoc Tag*. DZone. 2018-02-21. URL: <https://dzone.com/articles/jdk-10s-summary-javadoc-tag> (visited on 2022-11-16).
- [250] Cameron McKenzie. *Java assert*. TheServerSide. 2017-09. URL: <https://www.theserverside.com/definition/Java-assert#:~:text=The%20term%20assert%20is%20a,debugging%20routines%20on%20production%20systems>. (visited on 2023-02-16).
- [251] *Method java.io.PrintStream::format*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#format\(java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#format(java.lang.String,java.lang.Object...)) (visited on 2023-03-26).
- [252] *Method java.io.PrintStream::printf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#printf\(java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#printf(java.lang.String,java.lang.Object...)) (visited on 2023-03-26).
- [253] *Method java.io.PrintWriter::format*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintWriter.html#format\(java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintWriter.html#format(java.lang.String,java.lang.Object...)) (visited on 2023-03-26).



- [254] *Method java.io.PrintWriter::printf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintWriter.html#printf\(java.lang.String, java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintWriter.html#printf(java.lang.String,java.lang.Object...)) (visited on 2023-03-26).
- [255] *Method java.lang.AutoCloseable::close*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/AutoCloseable.html#close\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/AutoCloseable.html#close()) (visited on 2023-03-26).
- [256] *Method java.lang.Class::forName*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#forName\(java.lang.String\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#forName(java.lang.String)) (visited on 2023-02-27).
- [257] *Method java.lang.Class::getComponentType*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getComponentType\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getComponentType()) (visited on 2023-03-24).
- [258] *Method java.lang.Class::getDeclaredMethod*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getDeclaredMethod\(java.lang.String, java.lang.Class...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getDeclaredMethod(java.lang.String,java.lang.Class...)) (visited on 2023-02-19).
- [259] *Method java.lang.Class::getMethod*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getMethod\(java.lang.String, java.lang.Class...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Class.html#getMethod(java.lang.String,java.lang.Class...)) (visited on 2023-02-19).
- [260] *Method java.lang.Comparable::compareTo*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Comparable.html#compareTo\(T\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Comparable.html#compareTo(T)) (visited on 2023-03-02).
- [261] *Method java.lang.Object::clone*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#clone\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#clone()) (visited on 2022-11-04).



- [262] *Method java.lang.Object::equals*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#equals\(java.lang.Object\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#equals(java.lang.Object)) (visited on 2022-11-24).
- [263] *Method java.lang.Object::finalize*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#finalize\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#finalize()) (visited on 2022-11-24).
- [264] *Method java.lang.Object::hashCode*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#hashCode\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#hashCode()) (visited on 2022-11-24).
- [265] *Method java.lang.Object::toString*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#toString\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html#toString()) (visited on 2022-11-24).
- [266] *Method java.lang.reflect.Array::newInstance*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Array.html#newInstance\(java.lang.Class,int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/reflect/Array.html#newInstance(java.lang.Class,int)) (visited on 2023-03-24).
- [267] *Method java.lang.Runnable::run*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runnable.html#run\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runnable.html#run()) (visited on 2022-12-06).
- [268] *Method java.lang.String::format*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#format\(java.util.Locale,java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#format(java.util.Locale,java.lang.String,java.lang.Object...)) (visited on 2023-03-26).
- [269] *Method java.lang.String::formatted*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#formatted\(java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#formatted(java.lang.Object...)) (visited on 2023-03-26).



-
- [270] *Method java.lang.String::indexOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#indexOf\(java.lang.String\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#indexOf(java.lang.String)) (visited on 2023-02-24).
- [271] *Method java.lang.String::replace*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#replace\(java.lang.CharSequence,java.lang.CharSequence\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#replace(java.lang.CharSequence,java.lang.CharSequence)) (visited on 2023-03-21).
- [272] *Method java.lang.System::exit*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/System.html#exit\(int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/System.html#exit(int)) (visited on 2022-12-03).
- [273] *Method java.lang.Thread::run*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#run\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#run()) (visited on 2022-12-06).
- [274] *Method java.lang.Thread::setDefaultUncaughtExceptionHandler*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#setDefaultUncaughtExceptionHandler\(java.lang.Thread.UncaughtExceptionHandler\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#setDefaultUncaughtExceptionHandler(java.lang.Thread.UncaughtExceptionHandler)) (visited on 2022-12-06).
- [275] *Method java.lang.Thread::setUncaughtExceptionHandler*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#setUncaughtExceptionHandler\(java.lang.Thread.UncaughtExceptionHandler\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#setUncaughtExceptionHandler(java.lang.Thread.UncaughtExceptionHandler)) (visited on 2022-12-06).
- [276] *Method java.lang.Thread::sleep*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#sleep\(long\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html#sleep(long)) (visited on 2023-02-23).
- [277] *Method java.lang.Thread.UncaughtExceptionHandler::uncaughtException*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.UncaughtExceptionHandler.html#uncaughtException\(java.lang.Thread,java.lang.Throwable\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.UncaughtExceptionHandler.html#uncaughtException(java.lang.Thread,java.lang.Throwable)) (visited on 2022-12-06).
-



- [278] *Method java.lang.ThreadGroup:uncaughtException*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ThreadGroup.html#uncaughtException\(java.lang.Thread,java.lang.Throwable\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ThreadGroup.html#uncaughtException(java.lang.Thread,java.lang.Throwable)) (visited on 2022-12-06).
- [279] *Method java.lang.Throwable::addSuppressed*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#addSuppressed\(java.lang.Throwable\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#addSuppressed(java.lang.Throwable)) (visited on 2023-03-26).
- [280] *Method java.lang.Throwable::addSuppressed*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#getSuppressed\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#getSuppressed()) (visited on 2023-03-26).
- [281] *Method java.lang.Throwable::initCause*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#initCause\(java.lang.Throwable\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#initCause(java.lang.Throwable)) (visited on 2022-11-03).
- [282] *Method java.lang.Throwable::printStackTrace*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#printStackTrace\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html#printStackTrace()) (visited on 2023-03-27).
- [283] *Method java.math.BigInteger::clearBit*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigInteger.html#clearBit\(int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigInteger.html#clearBit(int)) (visited on 2023-03-21).
- [284] *Method java.sql.JDBCType::valueOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/JDBCType.html#getVendorTypeNumber\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/JDBCType.html#getVendorTypeNumber()) (visited on 2023-02-23).
- [285] *Method java.sql.JDBCType::valueOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/JDBCType.html#valueOf\(int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/JDBCType.html#valueOf(int)) (visited on 2023-02-23).



-
- [286] *Method java.util.Arrays::copyOfRange*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html#copyOfRange\(byte%5C%5B%5C%5D,int,int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html#copyOfRange(byte%5C%5B%5C%5D,int,int)) (visited on 2023-02-23).
- [287] *Method java.util.Collection.toArray(IntFunction<T[]>)*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray\(java.util.function.IntFunction\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray(java.util.function.IntFunction)) (visited on 2023-03-24).
- [288] *Method java.util.Collection.toArray(T[])*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray\(T%5B%5D\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray(T%5B%5D)) (visited on 2023-03-24).
- [289] *Method java.util.Collection.toArray()*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html#toArray()) (visited on 2023-03-24).
- [290] *Method java.util.Collections::unmodifiableCollection*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableCollection\(java.util.Collection\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableCollection(java.util.Collection)) (visited on 2023-02-23).
- [291] *Method java.util.Collections::unmodifiableList*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableList\(java.util.List\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableList(java.util.List)) (visited on 2023-02-23).
- [292] *Method java.util.Collections::unmodifiableMap*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableMap\(java.util.Map\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableMap(java.util.Map)) (visited on 2023-02-23).
- [293] *Method java.util.Collections::unmodifiableSet*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableSet\(java.util.Set\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html#unmodifiableSet(java.util.Set)) (visited on 2023-02-23).



- [294] *Method java.util.List::copyOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html#copyOf\(java.util.Collection\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html#copyOf(java.util.Collection)) (visited on 2023-02-22).
- [295] *Method java.util.List::of*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html#of\(E...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html#of(E...)) (visited on 2023-03-21).
- [296] *Method java.util.logging.Logger::setParent*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/Logger.html#setParent\(java.util.logging.Logger\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/Logger.html#setParent(java.util.logging.Logger)) (visited on 2023-02-16).
- [297] *Method java.util.logging.LogManager::readConfiguration*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/LogManager.html#readConfiguration\(java.io.InputStream\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/LogManager.html#readConfiguration(java.io.InputStream)) (visited on 2023-02-16).
- [298] *Method java.util.Map::copyOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#copyOf\(java.util.Map\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#copyOf(java.util.Map)) (visited on 2023-02-22).
- [299] *Method java.util.Map::get*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#get\(java.lang.Object\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#get(java.lang.Object)) (visited on 2023-02-24).
- [300] *Method java.util.Map::put*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#put\(K,V\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html#put(K,V)) (visited on 2022-11-10).
- [301] *Method java.util.Objects::isNull*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#isNull\(java.lang.Object\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#isNull(java.lang.Object)) (visited on 2022-12-18).



-
- [302] *Method java.util.Objects::nonNull*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#nonNull\(java.lang.Object\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#nonNull(java.lang.Object)) (visited on 2022-12-18).
- [303] *Method java.util.Objects::requireNonNull*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#requireNonNull\(T,java.lang.String\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html#requireNonNull(T,java.lang.String)) (visited on 2023-02-17).
- [304] *Method java.util.Queue::peek*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html#peek\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html#peek()) (visited on 2022-11-10).
- [305] *Method java.util.Set::copyOf*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Set.html#copyOf\(java.util.Collection\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Set.html#copyOf(java.util.Collection)) (visited on 2023-02-22).
- [306] *Method java.util.stream.Stream::peek*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html#peek\(java.util.function.Consumer\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html#peek(java.util.function.Consumer)) (visited on 2022-11-10).
- [307] *Method java.util.UUID::randomUUID*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/UUID.html#randomUUID\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/UUID.html#randomUUID()) (visited on 2022-12-15).
- [308] *Method org.apache.logging.log4j.Logger::always*. Apache Log4j. Version 2.19.0. The Apache Software Foundation. 2022-09-13. URL: <https://logging.apache.org/log4j/2.x/log4j-api/apidocs/org/apache/logging/log4j/Logger.html#always--> (visited on 2023-02-16).
- [309] MichaelCymerman. *Smarter Java development. Use interfaces to effectively enforce development contracts while maintaining loose coupling of code*. InfoWorld. 1999-08-01. URL: <https://www.infoworld.com/article/2076468/smarter-java-development.html#:~:text=Coding%20to%20interfaces%20is%20a,coding%20to%20the%20object%20itself>. (visited on 2022-12-26).
-



- [310] Vlad Mihalcea. *The best UUID type for a database Primary Key*. 2022-12-08. URL: <https://vladmihalcea.com/uuid-database-primary-key/> (visited on 2022-12-09).
- [311] *Minecraft*. URL: <https://www.minecraft.net/> (visited on 2022-12-02).
- [312] Adam Murdoch, Daz DeBoer, and Bo Zhang. *Gradle Build Tool*. Version 8.0. Gradle Inc. 2023-02-08. URL: <https://gradle.org/> (visited on 2023-02-26).
- [313] *Nassi-Shneiderman Diagram Generator*. GreenLightning. 2014-09-29. URL: <https://github.com/GreenLightning/nsdg/releases/tag/v1.0> (visited on 2026-06-14).
- [314] *Nassi-Shneiderman Diagram Generator*. GreenLightning. URL: <https://greenlightning.eu/nsdg/> (visited on 2026-06-14).
- [315] *Nassi-Shneiderman diagram*. en. 2025-07-30. URL: https://en.wikipedia.org/wiki/Nassi%E2%80%93Shneiderman_diagram (visited on 2026-06-14).
- [316] *Oracle WebLogic Server*. Oracle. URL: <https://www.oracle.com/java/weblogic/> (visited on 2022-12-01).
- [317] Ed Ort and Bhakti Mehta. *Java Architecture for XML Binding (JAXB)*. Deprecated for the JDK since Java 9, removed from the JDK with Java 11. Oracle. URL: <https://www.oracle.com/java/technologies/jaxb.html> (visited on 2023-03-25).
- [318] *Package java.time*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/package-summary.html> (visited on 2022-11-13).
- [319] *Package java.util.concurrent*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html> (visited on 2022-11-04).
- [320] *Package java.util.logging*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.logging/java/util/logging/package-summary.html> (visited on 2022-12-08).



-
- [321] Dmitri Pavlutin. *Programming to Interface vs. to Implementation*. 2022-01-24. URL: <https://dmitripavlutin.com/interface-vs-implementation/> (visited on 2022-12-26).
 - [322] Tim Peters. *"The Python Way"*. python.org. 1999-06-04. URL: <https://mail.python.org/pipermail/python-list/1999-June/001951.html> (visited on 2026-06-13).
 - [323] *Programming With Assertions*. Version Java 7. Oracle. URL: <https://docs.oracle.com/javase/7/docs/technotes/guides/language/assert.html> (visited on 2022-12-19).
 - [324] *Project Jigsaw*. openjdk.org. 2017-09-22. URL: <https://openjdk.org/projects/jigsaw/> (visited on 2023-01-19).
 - [325] *Python Indentation*. *Python Glossary*. W3Schools. URL: https://www.w3schools.com/python/gloss_python_indentation.asp#:~:text=Indentation%20refers%20to%20the%20spaces,indicate%20a%20block%20of%20code. (visited on 2022-12-28).
 - [326] *Red Hat JBoss Enterprise Application Platform*. Red Hat. URL: <https://www.redhat.com/en/technologies/jboss-middleware/application-platform> (visited on 2022-12-01).
 - [327] *Separation of concerns*. en. 2026-06-12. URL: https://en.wikipedia.org/wiki/Separation_of_concerns (visited on 2026-06-14).
 - [328] Robert Simmons Jr. *Hardcore Java - Secrets of the Java Masters*. Sebastopol, CA: O'Reilly Media, 2004. ISBN: 0596005687.
 - [329] *SLF4J. Simple Logging Facade for Java*. Version 2.0.5. QOS. URL: <https://www.slf4j.org/> (visited on 2022-12-03).
 - [330] *Specification for JEP 325: Switch Expressions*. Oracle. 2018. URL: <https://docs.oracle.com/javase/specs/jls/se12/preview/switch-expressions.html> (visited on 2023-04-02). last modified January 2019.
 - [331] *Stack Overflow*. Stack Overflow. URL: <https://stackoverflow.com/> (visited on 2026-06-14).
 - [332] *Standard charsets*. Version Java 17. Oracle. URL: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java.nio/charset/Charset.html#standard](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java.nio.charset/Charset.html#standard) (visited on 2022-12-04).



- [333] *The Eclipse Jetty Project*. Version Jetty 11.0.x. Eclipse Foundation. URL: <https://www.eclipse.org/jetty/> (visited on 2022-12-01).
- [334] *The javadoc Command*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/man/javadoc.html> (visited on 2022-11-13).
- [335] *The javadoc Command - Standard doclet Options*. Version Java 17. Oracle. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/man/javadoc.html#standard-doclet-options> (visited on 2022-11-17).
- [336] *The Unicode Standard*. Version 17.0.0. Unicode. 2025. URL: <https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-1/> (visited on 2026-06-22).
- [337] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.BUG.tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/BUG.html> (visited on 2022-12-08).
- [338] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.ClassVersion.tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/ClassVersion.html> (visited on 2023-02-08).
- [339] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.FixList.tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/FixList.html> (visited on 2022-12-08).
- [340] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.MountPoint.tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/MountPoint.html> (visited on 2022-12-20).



-
- [341] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.PlaygroundClass. tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/PlaygroundClass.html> (visited on 2023-02-08).
 - [342] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.ProgramClass. tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/ProgramClass.html> (visited on 2023-02-08).
 - [343] Thomas Thrien. *Annotation org.tquadrat.foundation.annotation.UtilityClass. tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/annotation/UtilityClass.html> (visited on 2023-02-08).
 - [344] Thomas Thrien. *Class oorg.tquadrat.foundation.lang.ThreadGroupExt. tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/ThreadGroupExt.html> (visited on 2022-12-08).
 - [345] Thomas Thrien. *Class org.tquadrat.foundation.exception.BlankArgumentException. tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/BlankArgumentException.html> (visited on 2022-12-18).
 - [346] Thomas Thrien. *Class org.tquadrat.foundation.exception.EmptyArgumentException. tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/EmptyArgumentException.html> (visited on 2022-12-18).
 - [347] Thomas Thrien. *Class org.tquadrat.foundation.exception.ImpossibleExceptionError. tquadrat's Foundation Library*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/>



- `org.tquadrat.foundation.base/org/tquadrat/foundation/exception/ImpossibleExceptionError.html` (visited on 2022-12-04).
- [348] Thomas Thrien. *Class `org.tquadrat.foundation.exception.NullArgumentException`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/NullArgumentException.html> (visited on 2022-12-18).
- [349] Thomas Thrien. *Class `org.tquadrat.foundation.exception.PrivateConstructorForStaticClassCalledError`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/PrivateConstructorForStaticClassCalledError.html> (visited on 2022-11-18).
- [350] Thomas Thrien. *Class `org.tquadrat.foundation.exception.UnexpectedExceptionError`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/UnexpectedExceptionError.html> (visited on 2022-12-04).
- [351] Thomas Thrien. *Class `org.tquadrat.foundation.exception.UnsupportedEnumError`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/UnsupportedEnumError.html> (visited on 2022-11-03).
- [352] Thomas Thrien. *Class `org.tquadrat.foundation.exception.ValidationException`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/exception/ValidationException.html> (visited on 2022-12-18).
- [353] Thomas Thrien. *Class `org.tquadrat.foundation.lang.Lazy`. `tquadrat's Foundation Library`*. `tquadrat.org`. 2022. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Lazy.html> (visited on 2022-12-17).



-
- [354] Thomas Thrien. *Class org.tquadrat.foundation.lang.Objects. tquadrat's Foundation Library*. tquadrat.org. 2020. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html> (visited on 2022-12-18).
 - [355] Thomas Thrien. *Class org.tquadrat.foundation.util.UniqueIdUtils. tquadrat's Foundation Library*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-util/javadoc/org.tquadrat.foundation.util/org/tquadrat/foundation/util/UniqueIdUtils.html> (visited on 2022-12-15).
 - [356] Thomas Thrien. *foundation-base. The base for all the Foundation Libraries*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base> (visited on 2022-11-20).
 - [357] Thomas Thrien. *foundation-javacomposer. A fork of JavaPoet*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-javacomposer> (visited on 2026-06-13).
 - [358] Thomas Thrien. *foundation-javadoc. An extension to the Javadoc tool*. tquadrat.org. 2021. URL: <https://tquadrat.github.io/foundation-javadoc/> (visited on 2022-11-17).
 - [359] Thomas Thrien. *foundation-util. The core library of the Foundation suite*. tquadrat.org. 2022. URL: <https://tquadrat.github.io/foundation-util> (visited on 2022-12-01).
 - [360] Thomas Thrien. *Interface org.tquadrat.foundation.lang.AutoLock*. tquadrat.org. 2026. URL: <https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/AutoLock.html> (visited on 2022-11-03).
 - [361] Thomas Thrien. *Method org.tquadrat.foundation.lang.Objects::checkFromIndexSize. tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkFromIndexSize\(int,int,int\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkFromIndexSize(int,int,int)) (visited on 2023-02-23).



- [362] Thomas Thrien. *Method `org.tquadrat.foundation.lang.Objects::checkFromToIndex`*. *tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkFromToIndex\(int,int,int\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkFromToIndex(int,int,int)) (visited on 2023-02-23).
- [363] Thomas Thrien. *Method `org.tquadrat.foundation.lang.Objects::checkIndex`*. *tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkIndex\(int,int\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#checkIndex(int,int)) (visited on 2023-02-23).
- [364] Thomas Thrien. *Method `org.tquadrat.foundation.lang.Objects::require`*. *tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#require\(T,java.lang.String,java.util.function.Predicate\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#require(T,java.lang.String,java.util.function.Predicate)) (visited on 2023-02-23).
- [365] Thomas Thrien. *Method `org.tquadrat.foundation.lang.Objects::requireNonNull`*. *tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#requireNonNull\(T\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#requireNonNull(T)) (visited on 2023-02-23).
- [366] Thomas Thrien. *Method `org.tquadrat.foundation.lang.Objects::requireValidArgument`*. *tquadrat's Foundation Library*. tquadrat.org. 2020. URL: [https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#requireValidArgument\(T,java.lang.String,java.util.function.Predicate\)](https://tquadrat.github.io/foundation-base/javadoc/org.tquadrat.foundation.base/org/tquadrat/foundation/lang/Objects.html#requireValidArgument(T,java.lang.String,java.util.function.Predicate)) (visited on 2023-02-23).
- [367] Thomas Thrien. *My Projects on GitHub*. tquadrat.org. 2026. URL: <https://tquadrat.github.io> (visited on 2026-06-13).
- [368] Thomas Thrien. *Repository for the Document Source*. LaTeX Sources. tquadrat.org. 2026. URL: <https://github.com/tquadrat/documents> (visited on 2026-06-13).



-
- [369] Raoul-Gabriel Urma and Richard Warburton. *Java 10 Local Variable Type Inference*. Oracle. 2022-05-10. URL: <https://developer.oracle.com/learn/technical-articles/jdk-10-local-variable-type-inference> (visited on 2023-03-01).
 - [370] *UTF-8*. en. 2026-06-17. URL: <https://en.wikipedia.org/wiki/UTF-8> (visited on 2026-06-22).
 - [371] Deepak Vohra. *How to Create an Oracle Database Sequence: Explained with Examples*. Toad. 2022-03-29. URL: <https://blog.toadworld.com/oracle-create-sequence-explained-with-examples> (visited on 2022-12-15).
 - [372] *Zen of Python*. en. 2026-03-11. URL: https://en.wikipedia.org/wiki/Zen_of_Python (visited on 2026-06-13).





Index

A

Active MQ .. *see* Apache ActiveMQ
 Apache ActiveMQ 195
 Apache Commons 195
 Lang 195
 Apache James 196
 Apache Log4j 195
 Apache Maven 197, 198
 Apache MQ 196
 Apache OpenOffice 197
 Apache Tomcat 196
 ARGS4J 356
 arithmetical round brackets 43
 ASCII 55

B

block statement ... *see* compound
 statement 87
 boolean 293
 BSD style 39
 build.gradle 58
 builder pattern 91

C

C 25, 132, 335, 343, 347, 407
 C++ . 25, 132, 313, 335, 343, 347,
 384, 385
 STL 385
 C# 25

call chaining *see* method chaining
 checked exception .. *see* exception
 code block *see* compound
 statement
 Code Conventions for the Java Pro-
 gramming Language ... 23,
 39, 41-43, 45, 47, 48
 Coding Guardrails for Java in a Nut-
 shell 15
 compilation unit 55, 57, 62
 compound statement 87
 constructor 83
 copy constructor 367, 369, 371
 copyright 58

D

doxygen 25, 144

E

Eclipse 41
 Eclipse Jetty 195, 196
 enum 66
 exception
 checked 204
 unchecked 204
 exit point *see* return point

F

feature library 195
 function 388



function library 195
functional interface 60

G

Gauß, Carl Friedrich 52, 175
GitHub 25, 28
github 21
GNU style 39
go 25
golang *see* go
Google 25
Google Guava 195
Gradle 58, 198
graphviz 21
Groovy 58, 132
GSON 356

H

H2 195, 196
helper class *see* utility class
Hibernate 195, 356
HTML 24, 59, 73
Hungarian Notation 132, 324

I

IBM WebSphere 196
immutable type 136, 288, 303, 308,
311, 336, 364, 365, 371
indentation 39
 base unit 40
 character 41
 style 40
IntelliJ IDEA 41
introspection *see* reflection
ISO 25
ISO-8859 55
ISO-8859-1 55

J

Jackson 356
jakarta.jms
 Connection 383
jAlbum 196
Java Platform Module System .. 62,
111, 345
java.awt
 BorderLayout
 NORTH 277
 Container
 add() 277
java.awt.event
 KeyAdapter 277
 keyTyped() 277
 KeyEvent 277, 279
 getKeyChar() 277
 getKeyCode() 277
 getKeyLocation() 277
 getKeyText() 277
 getModifiersEx() 277
 getModifiersExText() 277
 isActionKey() 277
 KEY_LOCATION_LEFT ... 277
 KEY_LOCATION_NUMPAD 277
 KEY_LOCATION_RIGHT . 277
 KEY_LOCATION_STANDARD 277
 KeyListener 277, 279
 keyPressed() 279
 keyReleased() 279
 keyTyped() 277, 279
java.awt.peer
 CanvasPeer 61
java.beans.beancontext
 BeanContext 290
java.io 383
 BufferedInputStream 292



- BufferedOutputStream 292
- BufferedReader 326, 333
 - readLine() 333
- ByteArrayOutputStream .. 303, 326
- Closeable 380, 383, 384
- File 361
- FileInputStream 268, 270, 273, 281, 292, 379, 380, 382
 - read() 382
- FileOutputStream ... 292, 380, 382
 - write() 382
- InputStream 61, 289, 292, 333, 379, 380, 382
 - close() 379
 - read() 380
- InputStreamReader 333
- IOException 205, 268–270, 272, 273, 281, 353, 379–382
- OutputStream .. 289, 292, 303, 380
 - write() 380
- PrintStream 393
 - format() 377
 - print() 393
 - printf() 362, 377
- PrintWriter
 - format() 377
 - printf() 377
- Writer 92
- java.lang 60
 - annotation
 - Annotation 82
 - Documented 82
 - Retention 82
 - Target 82
- ArithmeticException . 201, 205
- AssertionError . 264, 348, 353, 355
- AutoCloseable .. 380, 382–384, 386
 - close() 380–382, 386, 387
- Autocloseable 104
- Boolean 293
- CharSequence ... 81, 254, 257, 289, 293, 303, 352, 353, 390
 - isEmpty() 254, 257
 - toString() 257
- Class 199, 285, 343, 355
 - class 199
 - forName() 199, 200
 - getAnnotation() 355
 - getClass() 320
 - getComponentType() 351
 - getDeclaredField() 320
 - getDeclaredMethod() ... 285, 353
 - getMethod() ... 285, 348, 369
 - getResourceAsStream() . 259
 - isArray() 254
- ClassCastException 353
- ClassNotFoundException .. 199
- Cloneable .. 307, 364–367, 369
- CloneNotSupportedException 307, 337, 340, 364–367, 369
 - initCause() 369
- Comparable 337, 339, 340
 - compareTo() ... 337, 339, 340
- Double
 - compare() 340
- Error . 202, 204, 205, 369, 380
- Exception .. 204, 205, 380, 381
- ExceptionInInitialiserError 353



FunctionalInterface....82, 281
IllegalAccessException ... 348,
353, 369
IllegalArgumentException . 52,
76, 77, 175, 179, 180, 251,
253, 260, 270, 273, 348, 349,
369
IllegalStateException.....287
Integer.....360-362
 toHexString().....360
 toString() 360, 372, 374, 378
 valueOf() 316, 317
InterruptedException 298, 300
Iterable.....290
Module344
NoSuchMethodException.348,
353, 369
NullPointerException205, 252,
258
Number.....331, 332
Object289, 303, 330, 331, 338,
348, 355, 357, 364, 365
 clone.....369
 clone()...301, 305, 307, 337,
338, 340, 357, 363-367, 369-
371
 equals()..319, 338, 357-360,
392
 finalize().....357, 385
 getClass().....254, 348, 351,
352, 355, 358, 360, 392
 hashCode()....316-319, 338,
357-360
 hashCode() 359
 toString()293, 316-318, 337,
339, 340, 357, 360-363, 378
Override...269, 272, 273, 276,
277, 279, 316, 317, 319, 346,
347, 364, 366, 367
Package344
Runnable 349, 386, 387
 run() 349, 387
RuntimeException...204, 205,
369
Serializable.....81
String 61, 76, 77, 81,
83, 84, 293, 303, 308, 309,
315, 316, 353, 371, 372, 390
 CASE_INSENSITIVE_ORDER304
 compareTo() 337
 format() 61, 377
 formatted() ... 353, 362, 377,
393
 getBytes().....382
 indexOf() 308, 309
 isBlank() 93, 251, 257
 isEmpty()251, 253, 254, 257,
375, 390
 length() 93
 replace().....276, 315, 316
 split()390
 startsWith() 93
 substring() 308
 toLowerCase()259
 toUpperCase()259
StringBuffer.....254, 361,
372-375, 378
 append()372-374
 toString() 372
StringBuilder 92, 254, 288,
289, 293, 303, 360, 361, 373-
376, 378, 387, 390
 add().....390
 append().....375, 378



- setLength() 390
 - toString() 360, 375
- StringJoiner 378
- SuppressWarnings 199
- System 61
 - arraycopy() 301, 307
 - exit() 202
 - out 61, 362
- Thread
 - sleep() 298, 300
- Throwable 191, 204, 205, 208, 381
 - addSuppressed() 381
 - getCause() 348, 353
 - getSuppressed() 381
 - printStackTrace() .. 221, 381
- UnsupportedOperationException 287
- java.lang.annotation
 - Annotation 344, 355
- java.lang.reflect
 - AnnotatedElement 355
 - getAnnotation() 355
 - Array 344
 - getLength() 254
 - Constructor 344
 - Field 320, 344
 - get() 320
 - setAccessible() 320
 - InvocationTargetException 348, 353, 369
 - getCause() 369
 - Method 344, 348
 - invoke() 348, 353, 369
 - setAccessible() 353
- java.math
 - BigInteger 315, 316
 - clearBit() 315, 316
- java.net
 - ServerSocket 383
 - Socket 383
 - getInputStream() 333
 - URL 269, 272
 - openStream() 269, 272
- java.nio.file
 - Files
 - newBufferedReader() 326
 - Path 290
- java.sql
 - Connection 383
 - JDBCType 298, 299
 - BOOLEAN 299
 - getVendorTypeNumber() 298
 - valueOf() 298
 - SQLException 205
 - Statement 383
 - Types 298
 - BOOLEAN 299
- java.time 308, 315, 361
 - Instant 361
 - LocalDate 52, 175
 - minusDays() 52, 175
 - of() 52, 175
 - plusDays() 52, 175
 - Month
 - MARCH 52, 175
 - Year 52, 175
 - getValue() 52, 175
 - isAfter() 52, 175
 - isBefore() 52, 175
 - ZonedDateTime 362
- java.time.format
 - DateTimeFormatterBuilder . 92
- java.util



- ArrayList... 287-291, 304-306,
313, 321, 325, 327, 328, 346,
349, 366, 367, 369
 - add()..... 325
 - ensureCapacity()..... 328
 - trimToSize()..... 328
- Arrays
 - copyOf()..... 301, 307
 - copyOfRange()..... 258, 314
 - sort()..... 339, 342
 - stream()... 93, 301, 307, 339
- Collection.. 254, 289, 290, 303
 - add()..... 337, 339
 - isEmpty()..... 254
 - stream()..... 333, 334
 - toArray()..... 352
- Collections..... 306
 - unmodifiableCollection() 306
 - unmodifiableList().. 305, 306
 - unmodifiableMap()..... 306
 - unmodifiableSet()..... 306
- Comparator..... 331
- Date .. 308, 314, 315, 319, 320
 - clone()..... 319
 - equals()..... 319
 - getTime()..... 315, 320
 - setTime()..... 315, 320
- Enumeration..... 254
 - hasMoreElements()..... 254
- EnumMap..... 291
- EnumSet... 290, 291, 294, 296
 - noneOf()..... 294, 296
 - of()..... 294
- EventListener..... 279, 346
- EventObject..... 346
- Formatter..... 377
 - format()..... 378
- function
 - IntFunction..... 78
- HashMap 46, 77, 291, 306, 367
- HashSet.... 290, 291, 306, 325
 - add()..... 325
- Hashtable..... 291
- IdentityHashMap..... 348
- Iterator 331, 332, 350, 351
 - hasNext()..... 350, 351
 - next()..... 350, 351
 - remove()..... 331, 332
- LinkedHashMap..... 291
- LinkedHashSet..... 290, 291
- LinkedList. 290, 291, 303, 387
- List.... 81, 287, 288, 290, 292,
303-305, 313, 315, 324-328,
330, 331, 346, 349, 362, 366,
367, 369, 387
 - add()..... 288, 304, 305, 346,
349, 366, 367, 369, 387
 - addAll()..... 288
 - clear()..... 304
 - copyOf()..... 288, 305, 306
 - of()..... 315, 326, 330, 331
 - remove()..... 346
 - sort()..... 304, 305, 387
 - toArray()..... 315
- Locale
 - ROOT..... 259
- Map..... 46, 76,
77, 92, 254, 291, 306, 308,
309, 331-333, 335, 348, 362,
367
 - copyOf()..... 306
- Entry..... 92
- entrySet() 331-334, 348, 367
- get()..... 308, 309



- isEmpty().....254
- put() 348, 367
- Map.Entry.....331, 332
 - comparingByValue()333, 334
 - getKey() 333, 334, 348
 - getValue().....348
- Objects 252, 258, 260
 - isNull()260
 - nonNull().....260
 - requireNonNull().....252
 - toString() 362
- Optional 92, 254, 309, 310, 333, 355
 - ifPresent().....309, 310
 - isEmpty().....254
 - map() 333, 334
 - ofNullable() 310, 355
- OptionalDouble.....309
- OptionalInt.....309
 - ifPresent().....309
- OptionalLong 309
- PriorityQueue.....330, 331
- Set 290, 306, 325
 - add().....296
 - contains() 294, 296
 - copyOf()306
 - iterator() 331, 332
- SortedMap.....291
- SortedSet.....290
- Stream.....92, 254
- StringJoiner 254, 360, 362, 375, 376
 - add().....362, 375
 - length().....254
 - setEmptyValue().....254
 - toString().....360, 362, 375
- TreeMap 291
- TreeSet.....290, 291
- UUID 339, 361
- Vector.....290, 291
- java.util.concurrent.locks
 - Lock 385, 386
 - lock()385
 - unlock() 385
 - ReentrantLock.....386
- java.util.function
 - BiFunction281
 - apply().....281
 - Supplier.....285
- java.util.reflect
 - Array
 - newInstance().....351, 352
- java.util.stream
 - Collectors
 - counting() 333, 334
 - groupingBy().....333, 334
 - joining() 93
 - Stream 390
 - builder().....390
 - collect().....93
 - filter() 93, 339
 - forEach().....339
 - max() 333, 334
 - stream() 333, 334
 - toArray() 301, 307
 - Stream.Builder 390
 - add().....390
 - build()390
- javadoc25, 144, 196
- JavaScript.....25, 132
- javax.annotation
 - Generated 26
- javax.naming



 OperationNotSupportedExcep-
 tion 205
javax.servlet
 ServletException 205
javax.swing
 JFrame 277
 getContentPane() 277
 pack() 277
 setVisible() 277
 JTextField 277
 addKeyListener() 277
JAX_WS 356
JAXB 356
JBoss *see* Redhat JBoss
jEdit 196
Jetty. *see* Eclipse Jetty, *see* Eclipse
 Jetty
JPA 356
JPMS ... *see* Java Platform Module
 System
JUnit 195, 355

K
K&R style .. *see* Kernighan-Ritchie
 style
Kernighan-Ritchie style 39
Kotlin 58

L
label 88
lambda 44, 78
LaTeX 21, 59
LaTex 24
LEGO Mindstorms 468
LibreOffice 197
license 58
Lichtenberg, Heiner 52, 175

line length 48
 articles and documentation. 48
 comments 48
lint 27
Log4j *see* Apache Log4j

M

magic number 136, 138, 199, 298,
 308, 309
maillet 196
maintainable code 17
marshalling 357
Maven *see* Apache Maven
method chaining 91
method reference 350, 394
Minecraft 196
 mod 196
module-info.java 55, 57

N

Nassi-Shneiderman diagram... *see*
 structogram
Nassi-Shneiderman Diagram Gen-
 erator 21
NSDG *see* Nassi-Shneiderman
 Diagram Generator

O

Oracle WebLogic 196
org.apache.logging.log4j
 Logger
 always() 229
 log() 229
 withLocation() 229
 withThrowable() 229
org.junit.jupiter.api
 Assertions
 assertTrue() 353



- org.tquadrat.foundation.annotation
 - MountPoint.....270, 273
- org.tquadrat.foundation.exception
 - BlankArgumentException . 253
 - EmptyArgumentException 253, 254, 257
 - NullArgumentException....93, 253, 254, 257, 258, 349
 - UnexpectedExceptionError 337, 340, 348, 364, 365, 369
 - UnsupportedEnumError 93
 - ValidationException . 253, 259, 390
- org.tquadrat.foundation.lang . 258
 - AutoLock..... 385, 386
 - lock() 386
 - of() 386
 - Objects 52, 175, 252, 258
 - checkFromIndexSize()... 258
 - checkFromToIndex() 258
 - checkIndex()..... 259
 - equals().....358, 392
 - hash() 316, 317, 319, 358
 - isNull() .. 253, 254, 260, 281, 316, 317, 369
 - nonNull() 260, 339, 366, 367
 - require() 259
 - requireNonNull()..... 259
 - requireNonNullArgument()52, 175, 253, 254, 258, 259, 294, 296, 310, 314, 315, 319, 337, 339, 340, 353, 355, 369, 390
 - requireNotBlankArgument()252, 253, 257, 259, 337, 339
 - requireNotEmptyArgument()253, 254
 - requireValidArgument... 337
 - requireValidArgument() 259, 337
 - requireValidDoubleArgument() 260, 339, 340, 342
 - requireValidIntArgument()260
 - requireValidLongArgument()260
 - org.tquadrat.foundation.util
 - StringUtils
 - breakText 390
 - format().....259, 337, 340
 - isEmptyOrBlank() 390
 - isNotEmpty() 390
 - splitString() 293, 390
 - org.tquadrat.foundation.utils
 - StringUtils
 - abbreviate() 93

P

 - package 60
 - package-info.java 57
 - parameter round brackets 43
 - parentheses... *see* round brackets
 - PASCAL.....343, 388
 - permits 80, 82
 - Peters, Tim 24
 - PHP 132
 - PL/SQL.....25
 - PoC *see* Proof of Concept
 - procedure 388
 - programming to the interface 289, 303, 327, 406
 - Project Jigsaw... *see* Java Platform Module System
 - Proof of Concept.....31, 32
 - properties file 73
 - property file 55
 - Python 39



R

readable code 18
record 68, 123, 125, 146, 153,
155, 335
Redhat JBoss 196
reflection 198-200, 312, 314, 315,
318, 320, 321, 343-345, 347,
350-352, 369
resource file 55, 58, 73
retention policy 355
return 93
return point 388, 390, 392, 396
round brackets 42
runtime exception .. *see* exception

S

scope 87
sealed 80
Separation of concerns 17
shell script 59
SoC ... *see* Separation of concerns
struct 335
structogram 390, 392
SUN 23, 41, 48
SUN Java Coding Conventions *see*
Code Conventions for the Java
Programming Language
suppressed exception 381

T

TeX 21, 59
Tex 24
Tomcat *see* Apache Tomcat
tquadrat Foundation
JavaComposer 195
Util 195
try-catch *see* try-catch-finally

try-catch-finally 43, 103, 104, 106,
379, 381, 382, 385
try-finally *see* try-catch-finally
try-with-resources .. 103-105, 378,
379, 381-384, 386

U

unchecked exception *see*
exception
unit test 15, 207, 345, 353
unmarshalling 357
unnamed variable 44
UTF-8 55
utility class 56, 195, 250, 337

W

WebLogic *see* Oracle WebLogic
WebSphere ... *see* IBM WebSphere

X

XML 24, 59, 73

Y

yield 93, 398

Z

Zen of Python 15, 24